

Machine Learning: The Quest for Information and Meaning

Weimao Ke

Table of contents

Preface	1
I. Data, Information, and Mathematical Foundations	3
1. From Data to Information and Meaning	5
1.1. From Data to Meaning	6
1.2. Concept Generalizability	10
1.3. Final Observation	13
2. Data Types and Representations	15
2.1. Datum	15
2.1.1. Numerical	15
2.1.2. Categorical	17
2.2. Data	18
2.2.1. Sets	18
2.2.2. Vectors	24
3. Vectors, Matrices, and Geometric Thinking	29
3.1. Basic Matrix Operation	29
3.2. Sum of Squares	35
3.3. Vector Magnitude (Length)	37
3.4. Vector Distance	39
3.5. Vector Angle	41
3.6. Vector Cross Product	45
3.7. Optimization	45

Table of contents

3.8. Kernel Functions	46
3.9. Graph Analysis	46
4. Probability and Statistical Thinking	47
4.1. Probability	47
4.2. Probability Basics	49
4.3. Joint probability	51
4.4. Independent events	53
4.5. Dependent Events	54
4.6. The Bayesian Theorem	56
4.7. Naive Bayes' Model	57
4.8. Probability Estimators	61
4.9. Probability Distributions and Models	65
4.10. Continuous Probability Distribution	68
4.11. Probability Estimators	73
4.11.1. Discrete Probability Estimation	73
4.11.2. MLE for a Continuous Variable	77
4.12. Summary	78
5. Information, Entropy, and Divergence	81
5.1. Shannon Entropy	82
5.1.1. Properties of Shannon Entropy	83
5.1.2. Entropy in Thermodynamics?	87
5.1.3. Applications	88
5.2. Limits and Other Measures	92
5.2.1. Relative Entropy	94
5.3. Jensen-Shannon Divergence	97
5.4. IDF and KL Divergence	100
5.5. Least Information Theory (LIT)	102
6. Data Preparation and Feature Engineering	111
6.1. Data Quality and Provenance	111
6.2. Missing, Noisy, and Inconsistent Data	111
6.3. Scaling, Normalization, and Transformation	112

6.4. Encoding Categorical and Structured Variables	112
6.5. Feature Selection and Dimensionality Reduction	112
6.6. Leakage and Reproducible Pipelines	113
6.7. Summary and Exercises	113
II. Learning from Data	115
7. Classification: From Neighbors to Neural Networks	117
7.1. Lazy Learning	118
7.1.1. K Nearest Neighbors (kNN)	119
7.1.2. Other Distance Metrics	122
7.2. Linear Classifiers	123
7.2.1. Between Centroids	125
7.2.2. Support Vector Machines	130
7.2.3. Perceptron: Toward a Neural Network	133
7.3. Non-Linear Models	138
7.3.1. Kernel SVM	139
7.3.2. Multi-Layer Neural Networks	144
8. Multiclass and Structured Decisions	161
8.1. From Binary to Multiple Classes	161
8.2. One-vs-Rest and One-vs-One Strategies	161
8.3. Scores, Probabilities, and Softmax	162
8.4. Decision Trees and Rule-Based Decisions	162
8.5. Multilabel and Ordinal Outcomes	162
8.6. Error Analysis and Calibration	163
8.7. Summary, Exercises, and Lab	163
9. Numeric Prediction and Regression	165
9.1. Prediction with Continuous Outcomes	165
9.2. Linear Regression	165
9.3. Loss Functions and Fitting	166
9.4. Regularization	166

Table of contents

9.5. Nonlinear Regression	166
9.6. Uncertainty and Diagnostics	167
9.7. Summary, Exercises, and Lab	167
10. Convolutional Neural Networks: Learning Spatial Structure	169
10.1. Why Image Structure Changes the Model	170
10.2. Images as Tensors	171
10.3. A Kernel as a Small Shared Detector	172
10.3.1. A Complete Small Example	173
10.4. Padding, Stride, and Output Size	175
10.5. From One Channel to Many Features	176
10.6. Pooling and Growing Receptive Fields	177
10.7. From Feature Maps to a Class Prediction	177
10.8. Looking Inside and Knowing the Limits	180
10.9. Summary	182
10.10 Exercises	182
11. Clustering and Organization	185
11.1. Example Data	188
11.2. Hierarchical Clustering	191
11.2.1. Issues of HAC	194
11.3. K-means Partitioning	195
11.3.1. Issues of K-means	198
11.4. Probabilistic Clustering	199
11.4.1. Initialization	201
11.4.2. Expectation	202
11.4.3. Maximization	205
11.4.4. Iterations of Expectation-Maximization	208
11.4.5. EM with Higher Dimensionality	210

III. Language, Retrieval, and Structure	211
12. Text and Human Language	213
12.1. Text Vectorization	213
12.1.1. Tokenization	214
12.1.2. Term Weighting	215
12.2. Similarity Measures	220
12.3. Probabilities and Implications	230
12.3.1. Zipf's Law	230
12.3.2. Feature Selection	232
12.4. Text Classification	233
12.4.1. Naive Bayes with Binary Term Occurrences (Bernoulli)	234
12.4.2. Naive Bayes with Term Frequencies (Multinomial) .	240
Summary	245
13. Search and Retrieval	247
13.1. Basic Paradigm	248
13.2. Indexing	250
13.3. Matching	252
13.4. Ranking	255
13.4.1. Content-based Ranking	256
13.4.2. Popularity-based Ranking	260
13.4.3. PageRank	265
13.5. Information Filtering	269
14. Graphs and Structural Analysis	271
14.1. Graphs as Data	271
14.2. Paths, Connectivity, and Traversal	271
14.3. Centrality and Influence	272
14.4. Communities and Structural Roles	272
14.5. Link Prediction	272
14.6. From Graph Features to Graph Learning	273
14.7. Summary, Exercises, and Lab	273

IV. Trustworthy and Practical Machine Learning	275
15. Evaluation and Experimentation	277
15.1. Precision	279
15.2. Recall	280
15.3. Sensitivity and Specificity	282
15.4. Kappa and Chance Agreement	284
15.5. Averaging	288
15.6. Rank Evaluation	289
15.7. Evaluation of Numeric Predictions	291
15.7.1. Error Metrics	291
15.7.2. Correlation	293
15.8. Experimentation	295
15.9. On Efficiency	296
16. Generalization, Model Selection, and Fit	299
16.1. Training Performance and Generalization	299
16.2. Bias, Variance, and Model Capacity	299
16.3. Validation and Cross-Validation	300
16.4. Regularization	300
16.5. Learning Curves and Diagnostics	300
16.6. Hyperparameter Selection Without Leakage	301
16.7. Summary, Exercises, and Lab	301
17. Scaling, Deployment, and Responsible Use	303
17.1. Time, Space, Data, and Compute	303
17.2. Batch, Streaming, and Distributed Workflows	303
17.3. From Experiment to Reproducible Pipeline	304
17.4. Deployment, Monitoring, and Drift	304
17.5. Privacy, Security, and Misuse	304
17.6. Energy, Sustainability, and Proportionality	305
17.7. Responsible Decisions and Human Oversight	305
17.8. Summary and Exercises	305

V. Practice and Projects: A Companion Volume	307
Practice and Projects	309
How to use this volume	309
Computational Toolkit	311
Environment setup	311
Python programming primer	311
Control flow, randomness, and input/output	311
File processing	311
Core data types and structures	312
Python and Jupyter Setup	313
Recovered activity	313
Purpose	313
Your Options	314
Use CCI's JupyterHub (recommended)	314
Setup on Your Computer (optional)	315
Result	316
Python Programming Primer	319
Recovered activity	319
Programming	321
Introduction to Python	323
Python Language	323
Working Environment	324
Jupyter Notebook (or Hub)	324
Command Line	324
First Interactions	324
Variables	325
Python Input and Output	327
Recovered activity	327

Table of contents

Output	327
Input	328
Example Program: Knock-knock	329
Python Program	329
The Program Code	330
Python Selection	331
Recovered activity	331
IF... Structure	332
IF... ELIF... ELSE... Structure	333
Python Randomness	335
Recovered activity	335
Example: Addition Game	336
Python Repetition	339
Recovered activity	339
Loop Basics	340
Loop for a Pattern	341
Nested Loop	341
Python File Processing	345
Recovered activity	345
Outline	345
Text Files	346
What text files?	346
What is not a text file?	347
Reading a file	347
Reading a file by smaller pieces	348
Reading a file by line	348
Reading all lines in a file	349
Writing to files	349
Writing to files	350
More examples	350

Table of contents

Strings Processing and Statistics	352
Example problem	352
Reading survey	353
On list and split	353
Counting votes	354
Counting ALL Votes	355
Output	356
Data Cleaning	357
String methods	358
CSV Data Files	359
CSV	359
Python CSV	359
Other Formats and Tools	362
Pandas	362
NumPy	363
SciPy	363
References	363
Practice 1: From Data to Information and Meaning	365
From Data to Meaning	365
Concept Generalizability	365
Practice 2: Data Types and Representations	367
Datum	367
Numerical	367
Categorical	368
Data	368
Sets	368
Vectors	368
Python Data Types	371
Recovered activity	371
Programming – Data Types	371
Data abstraction	372

Table of contents

Basic Python: Atomic Data Types	372
Basic Python – Atomic Data Types	373
Basic Python – Atomic Data Types	374
Basic Python – Boolean	374
Basic Python - Collections	375
Basic Python - List	376
Basic Python - List Methods	377
Basic Python - List functions	378
Basic Python - String	379
Basic Python - Tuple	380
Basic Python – Set	381
Basic Python – Dictionary	382
References	384
Python Data Structures	385
Recovered activity	385
Basic data structures	385
Stacks	386
Stack Design	386
Stack Implementation	387
Stacks in Action	388
Stack Applications	389
Example Application	389
Queue	394
Queue Design	394
Queue Implementation	395
Deque	396
Deque Design	397
Deque class implementation	398
Deque in Action	398
References	399
Python Data Structures Assignment	401
Recovered activity	401

Table of contents

Objectives	401
0. Academic Honesty Statement	402
1. Create a Stack abstract data type (2 points)	403
2. Create a check_html function (5 points)	403
3. Function Calls (0.5 point)	405
4. Markdown and Comments (2 point)	406
Bonus (+1 point)	406
Important Notes	407
Practice 3: Vectors, Matrices, and Geometric Thinking	409
Basic Matrix Operation	409
Sum of Squares	409
Vector Magnitude (Length)	410
Vector Distance	410
Vector Angle	410
Vector Cross Product	411
Optimization	411
Kernel Functions	411
Graph Analysis	411
Data, Vectors, and Cosine Assignment	413
Recovered activity	413
Preparation	413
1. Basic Statistics (2 points)	414
2. IRIS Data Statistics	415
2.1. Attribute Types (1 point)	415
2.2. Data Distributions (1.5 points)	416
2.3. Subset Data Distributions (2 points)	417
2.4. Boxplots vs. Target Levels (1.5 points)	418
3. Correlation, Similarity and Distance	419
3.1. Scatter Plots (0.25 point)	419
3.2. Visual Examination (1 point)	419
3.3. Pearson Correlation (1 point)	419

Table of contents

3.4. Euclidean Distance (0.25 point)	420
3.5. Cosine 1 with Original Values (0.25 point)	420
3.6. Cosine 2 with Vectors from Mean (0.25 point)	420
3.7. Summary and Observation (1 point)	421
Practice 4: Probability and Statistical Thinking	423
Probability	423
Probability Basics	423
Joint probability	424
Independent events	424
Dependent Events	424
The Bayesian Theorem	424
Naive Bayes' Model	425
Probability Estimators	425
Probability Distributions and Models	425
Continuous Probability Distribution	425
Probability Estimators	426
Discrete Probability Estimation	426
MLE for a Continuous Variable	426
Probability and Linearity in Data	427
Recovered activity	427
A. Probabilities in Language	428
A.1. Load Data	428
A.2. Zipf's Law	428
A.3. Probabilities in Training Data	431
A.4. Probabilistic Binary Model (Bernoulli Naive Bayes)	434
A.5. Probabilistic Term Frequency Model (Multinomial Naive Bayes)	435
B. Linear Classification Model	437
B.1. Linear Classification (Perceptron) with Idealized Data	438
B.2. Linear Classification with Not-So-Ideal Data	440
C. Non-Linear Model with Multi-layer Neural Network	442
D. Further Research and Exercise	447

Practice 5: Information, Entropy, and Divergence	449
Shannon Entropy	449
Properties of Shannon Entropy	449
Entropy in Thermodynamics?	450
Applications	450
Limits and Other Measures	450
Relative Entropy	450
Jensen-Shannon Divergence	451
IDF and KL Divergence	451
Least Information Theory (LIT)	451
Practice 6: Data Preparation and Feature Engineering	453
Data Quality and Provenance	453
Missing, Noisy, and Inconsistent Data	453
Scaling, Normalization, and Transformation	454
Encoding Categorical and Structured Variables	454
Feature Selection and Dimensionality Reduction	454
Leakage and Reproducible Pipelines	454
Python Data Preprocessing	455
Recovered activity	455
Data Quality	455
Applying descriptive analysis	457
Outlier Detection	459
Data Integration	460
Data Integration 2	461
Dimensionality Reduction + Normalization	462
Outliers versus the Normal	465
Recovered activity	465
A. A NORMAL Spender Model	466
Mean and deviation	466
Normal Distribution (Model)	466
Normal Density Distribution	466

Table of contents

B. Outliers in Belgium International Phone Calls	467
Data	468
Statistics	468
Distribution	468
Potential Correlation	468
Linear Regression	468
C. Parametric Detection: What is a NORMAL temperature? . .	472
1. Estimate Mean and Deviation from Sample	473
2. Build the Normal Model with Estimated Parameters . .	474
3. Compute the Likelihood of Temperature Ranges	474
D. NORMAL Age in Context	474
1. Census Income Data	475
2. Working Age of Male vs. Female	476
3. NORMAL Age with a Graduate Degree	476
References	477
Practice 7: Classification	479
Lazy Learning	479
K Nearest Neighbors (kNN)	479
Other Distance Metrics	480
Linear Classifiers	480
Between Centroids	480
Support Vector Machines	480
Perceptron: Toward a Neural Network	481
Non-Linear Models	481
Kernel SVM	481
Multi-Layer Neural Networks	481
Practice 8: Multiclass and Structured Decisions	483
From Binary to Multiple Classes	483
One-vs-Rest and One-vs-One Strategies	483
Scores, Probabilities, and Softmax	484
Decision Trees and Rule-Based Decisions	484
Multilabel and Ordinal Outcomes	484

Table of contents

Error Analysis and Calibration	485
Practice 9: Numeric Prediction and Regression	487
Prediction with Continuous Outcomes	487
Linear Regression	487
Loss Functions and Fitting	488
Regularization	488
Nonlinear Regression	488
Uncertainty and Diagnostics	489
Classification Patterns and Evaluation Assignment	491
Recovered activity	491
Preparation	491
1. Data Cleaning	492
1.1. The first problem (1 point)	493
1.2. The second problem (1 point)	493
1.3. The third problem (1 point)	494
2. Data Normalization + Reduction	494
2.1. Performance (1 point)	498
2.2. Insight and Explanation (2 points)	499
3. Association Rule Mining: Theory	499
3.1. Rule Calculation	500
3.2. Pattern Evaluation Method	503
4. Association Rule Mining: Practice	504
4.1. Discretization (1 point)	505
4.2. Implementation (1 point)	506
4.3. Insight and Explanation (1 point)	506
Practicum: From Pixels to CNN Features	509
Learning Question and Evidence Rules	510
Setup: One Dataset, Three Roles	510
Part I: Establish an MLP Baseline	512
Flatten the Grid	512
Build and Train the MLP	513

Table of contents

Part II: Make Convolution Visible	515
One Transparent Operation	515
Kernels Produce Evidence, Not Labels	516
Pooling and Parameter Counting	519
Part III: Replace the MLP with a CNN	520
Preserve the Grid and Add a Channel Axis	520
Build a Nearly Parameter-Matched CNN	520
Compare Outcomes, Then Explain	522
Use Loss to Diagnose Overfitting	524
Part IV: Return to Kernels: Learned Activations for 1 and 7 . .	526
Final Investigation	529
What to Submit	529
Practice 10: Clustering and Organization	531
Example Data	531
Hierarchical Clustering	531
Issues of HAC	532
K-means Partitioning	532
Issues of K-means	532
Probabilistic Clustering	532
Initialization	533
Expectation	533
Maximization	533
Iterations of Expectation-Maximization	533
EM with Higher Dimensionality	534
Text and Cluster Analysis	535
Recovered activity	535
Introduction	535
Text vectorization	536
K-means clustering	539
Bottom-up method	542
References:	543

Practice 11: Text and Human Language	545
Text Vectorization	545
Tokenization	545
Term Weighting	546
Similarity Measures	546
Probabilities and Implications	546
Zipf's Law	547
Feature Selection	547
Text Classification	547
Naive Bayes with Binary Term Occurrences (Bernoulli) . .	547
Naive Bayes with Term Frequencies (Multinomial)	548
 Text Vectorization	 549
Recovered activity	549
Objectives	549
1. Data	549
2. Create a <i>Course</i> class	550
NOW OUTSIDE THE COURSE CLASS	551
3. Create a <code>print_dictionary</code> function	551
4. Steps to Vectorize the Course Titles	551
 Text Vectorization Programming Assignment	 555
Recovered activity	555
Objectives	555
A. Data Files (1 point)	556
B. Create a <i>Document</i> abstract data type/class (2 points) . . .	556
C. NOW OUTSIDE THE DOCUMENT CLASS	557
C1. Create a <code>save_dictionary</code> function (1 points)	557
C2. Create a <code>vectorize</code> function (5 points)	558
C3. Finally, call the vectorize function (1 point)	560
Bonus (optional, +1 point)	560
Submission	560

Table of contents

Integrated Data Learning Assignment	561
Recovered activity	561
Preparation	561
A. Data Preparation (1 points)	561
B. Probabilities and Zipf (3 points)	562
B.1. Class Distributions and Probabilities (0.5 point)	562
B.2. Term Probabilities and Zipf's Law (2.5 points)	562
C. Text Vectorization (2 points)	564
C.1. Training and Test Data	564
C.2. Text Vectorization	564
C.3. Terms and Conditional Probabilities	564
D. Classification (5 points)	565
D.1. Probabilistic Naive Bayes Model (2 points)	565
D.2. Linear Model (2 points)	565
D.3 Non-linear Classification or Alternative (1 points)	566
E. Conclusion (1 point)	567
 Practice 12: Search and Retrieval	 569
Basic Paradigm	569
Indexing	569
Matching	570
Ranking	570
Content-based Ranking	570
Popularity-based Ranking	570
PageRank	571
Information Filtering	571
Recovered retrieval questions and exercise	571
Recovered questions	571
Recovered retrieval calculation	573
 SQLite and Data Retrieval Assignment	 575
Recovered activity	575
A. Data	575
B. SQLite Database Schema (2 points)	576

Table of contents

C. Load Data into SQLite (2 points)	577
D. In-database Query and Analytics with SQLite Data (6 points)	578
Bonus (+1 point)	579
Submission	579
SQLite Practice	581
Recovered activity	581
SQLite	581
SQLite Interactive	581
SQLite Interactive	582
Load SQL Magic	582
Database college.db	582
Load SQLite Database	583
programs table	583
students table	584
Select and Filter	585
Group Aggregates	586
Update Student Data	586
Join Tables	587
References	589
SQLite with Python	591
Recovered activity	591
SQLite with Python	591
Python Code for SQLite	592
Import SQLite	592
Load SQLite Database	592
programs table	592
students table	593
Group Aggregates	595
Update Student Data	595
Join Tables	596

Table of contents

References	597
Practice 13: Graphs and Structural Analysis	599
Graphs as Data	599
Paths, Connectivity, and Traversal	600
Centrality and Influence	600
Communities and Structural Roles	600
Link Prediction	601
From Graph Features to Graph Learning	601
Practice 14: Evaluation and Experimentation	603
Precision	603
Recall	603
Sensitivity and Specificity	604
Kappa and Chance Agreement	604
Averaging	604
Rank Evaluation	605
Evaluation of Numeric Predictions	605
Error Metrics	605
Correlation	605
Experimentation	606
On Efficiency	606
Association Rule Mining	607
Recovered activity	607
Introduction	607
Preprocessing	608
Implementation	610
Visualization	610
Practice 15: Generalization, Model Selection, and Fit	613
Training Performance and Generalization	613
Bias, Variance, and Model Capacity	613
Validation and Cross-Validation	614

Table of contents

Regularization	614
Learning Curves and Diagnostics	614
Hyperparameter Selection Without Leakage	615
Practice 16: Scaling, Deployment, and Responsible Use	617
Time, Space, Data, and Compute	617
Batch, Streaming, and Distributed Workflows	617
From Experiment to Reproducible Pipeline	618
Deployment, Monitoring, and Drift	618
Privacy, Security, and Misuse	618
Energy, Sustainability, and Proportionality	618
Responsible Decisions and Human Oversight	619
Data Mining Project	621
Recovered activity	621
I. Data Mining and Report	621
II. Algorithm Implementation	622
III. Literature Review	622
Group Submission	623
Reinforcement Learning Extension	625
Recovered activity	625
Motivation	625
Reinforcement Learning	626
Q-Learning	628
Q-Table	628
Exploration vs. Exploitation	629
Taxi Cab	629
Problem	635
Action without Learning	636
Explore and Learn	637
Act on Knowledge	641
How did the agent learn?	642
Branches	642

Table of contents

Comparison	647
Conclusion	647
References	648
References	649

List of Figures

1.1.	Scatter plots: V_4 vs. V_1 , V_2 , and V_3	8
1.2.	Pairwise scatter plots with the value of V_4 encoded by symbol and color. Red circles represent 0; blue plus signs represent 1.	8
1.3.	Separating 0 vs. 1 in V_4 with the linear equation $V_3 - V_2 = 0$	10
1.4.	Shapes of different numbers of sides, widths, and heights, based on the shapes data. Gray shapes are classified as <i>standing</i>	11
1.5.	A <i>standing</i> architecture.	12
2.1.	Intersection $A \cap B$	19
2.2.	Union $A \cup B$	20
2.3.	Union of three sets.	21
2.4.	Intersection of three sets.	21
2.5.	Subset $A \subseteq B$	23
2.6.	Set subtraction $A - B$	23
2.7.	Two-dimensional vector space.	25
3.1.	Row vectors of matrix X	30
3.2.	Row vectors of matrix Y	31
3.3.	Subtraction geometry.	32
3.4.	Geometry of $g_3 \times 2$, as part of scalar multiplication $G \times 2$	33
3.5.	Geometry of addition $z_3 = x_3 + g_3 \times 2$	34
3.6.	Vector magnitude.	38
3.7.	Vector distance.	40
3.8.	Normalization to unit vectors.	43
3.9.	Cosine values for special angles.	44

List of Figures

4.1. Head versus tail.	48
4.2. Mutually exclusive events.	50
4.3. Events that are not mutually exclusive.	51
4.4. Ace $\cap$ spade: a joint (AND) event.	52
4.5. Blind men and elephant. If you have not seen an elephant, does it mean there is no elephant? If you only touch an elephant on its leg, does it mean the elephant is like a tree trunk? These misperceptions are due to our sample data (where you touch the elephant) and should be corrected with external knowledge.	63
4.6. Frequency distribution of flipping a coin: 500 heads versus 500 tails after 1,000 trials in the ideal world.	66
4.7. Probability distribution of flipping a coin: heads and tails each have probability 0.5.	66
4.8. Probability distribution of rolling a die.	67
4.9. Probability density of a continuous variable x	68
4.10. Distribution of adult male heights, approximately a normal distribution (bell curve).	70
4.11. Word frequency distribution	72
5.1. Shannon entropy of two mutually exclusive events.	84
5.2. Shannon entropy of m equally likely events.	85
5.3. Triangle inequality: the sum of two distances is never less than the third, $d(A, B) + d(B, C) \geq d(A, C)$	96
5.4. Least Information vs. Shannon Entropy vs. KL Divergence. This shows the amount of information by reducing two mutually exclusive inferences to certainty. The X axis is p_1 , the probability of the 1 st inferences. The Y axis is the amount of information (I) in terms of each information measure. The 1 st inference is assumed to be the ultimate outcome ($q_1 = 1$) in the figure.	105
5.5. Sum of distances in the same direction, $a + b = c$	107

7.1.	Nearest neighbors $k = 1$, $+$ denotes <i>standing</i> (positive) and $\circ$ denotes <i>lying</i> (negative). $\otimes$ is the test instance.	120
7.2.	Nearest neighbors $k = 3$, $+$ denotes <i>standing</i> (positive) and $\circ$ denotes <i>lying</i> (negative). $\otimes$ is the test instance.	121
7.3.	Linear classification with centroids, $+$ denotes <i>standing</i> (positive) and $\circ$ denotes <i>lying</i> (negative). $\bullet$ represents a centroid of the respective class. $\otimes$ is a test instance.	127
7.4.	Linear classification with centroids, with smaller values in the negative class	129
7.5.	Linear classification with SVM, one candidate margin . . .	131
7.6.	Linear classification with SVM, support vectors and maximum margin	132
7.7.	Linear classification with a Perceptron, where the output is a step function based on the weighted sum of input variables	134
7.8.	Example of a step function. Output is 0 for any value below zero, and is 1 if above zero.	135
7.9.	Car evaluation data. $+$ is the positive (acceptable) class and $\circ$ is the negative (unacceptable).	140
7.10.	Car evaluation data, with transformed v_1^2 and v_2^2	142
7.11.	Car evaluation data, mapped to higher dimensions with v_1^2 , v_2^2 , and $\sqrt{2}v_1v_2$. The two planes are closest to the opposite class. The projected shadows are for reference only. (a) shows all data points in the three dimensional space, and (b) is a closer view of the two planes with support vectors. Dashed line is the gap (margin) between the two classes. Circled data points are support vectors.	143
7.12.	Car evaluation data, transformed by a step function. A step function with a threshold 2.5 converts the original data ($+$ and $\circ$ in faded colors) to those with 0 and 1 values ($+$ and $\circ$ in bright colors). The solid line at $v_1 + v_2 = 0.5$ separate the two classes.	145
7.13.	Multi-layer neural network with step functions. $\sum$ represents the weighted sum of input values leading to the present node.	146

List of Figures

7.14. Sigmoid function	148
7.15. Multi-layer neural network with the sigmoid function. s_1 and s_2 are the weighted sum of input values leading to the respective hidden nodes whereas s is the weighted sum of input values leading to the output layer.	149
7.16. Backpropagation step 1 with 0 initial weights.	152
7.17. Backpropagation step 2 with updated weights	156
7.18. Neural network training iterations and convergence	156
7.19. Neural network final model. Compare to the referenced figure. Note that we transform the bias node on the hidden layer with the sigmoid as well, i.e. $h_0 = f(1) = \frac{1}{1+e^{-1}} = 0.73$. In the final model, $w_0 h_0 = -1.92 \times 0.73 = -1.4$	158
7.20. Visualization of the final model on car evaluation. Data transformed by the sigmoid function $h_i = f(s_i) = \frac{1}{1+e^{-s_i}}$, where s_i is the weighted sum of each hidden node: $s_1 = 2.5 - v_1$ and $s_2 = 2.5 - v_2$. The solid line at $h_1 + h_2 - 1.92f(1) = 0$ is where the sigmoid of the output layer makes the cut. . .	159
10.1. Flattening preserves every pixel value but converts two-dimensional neighborhoods into positions along a one-dimensional vector.	170
10.2. Hand-designed kernels applied to MNIST examples of 1 and 7. Bright positions are positive local responses after ReLU, not class probabilities.	174
10.3. A compact MNIST CNN preserves spatial structure through two convolution and pooling blocks, then flattens the learned feature maps for dense classification.	178
10.4. A matched comparison of MNIST test accuracy, test errors, and parameter count.	179
10.5. Two learned first-layer kernels and their activation maps for representative handwritten 1 and 7 examples.	181
11.1. Hard, flat partitioning	186

List of Figures

11.2. Illustration of soft partitioning, where data may belong to multiple clusters	187
11.3. Hierarchical clustering. A nested circle represents child (branch) of the enclosing cluster.	189
11.4. Tree representation of the hierarchical structure in the referenced figure	189
11.5. IRIS flowers data: Petal Length vs. Petal Width	191
11.6. Step 1 of HAC clustering on IRIS data. The circle indicates a new cluster formed by the closest pair of instances. The centroid of the cluster is marked $\times$ in (b).	192
11.7. Further iterations of HAC clustering on IRIS data	193
11.8. Result of HAC clustering on IRIS data	194
11.9. K-means initial steps. A $\times$ represents an initial centroid. Undirected lines represent cluster membership. Directed lines $\nearrow$ indicates how the centroid will move (change) after re-computation based on assigned members.	196
11.10 K-means further steps. A $\times$ represents an centroid before it is updated. Undirected lines represent cluster membership. Directed lines $\nearrow$ indicates how the centroid will move (change) after re-computation based on assigned members.	197
11.11 K-means final clusters and steps	198
11.12 Initial parameters of normal distributions. $\circ$ are IRIS data instances. $\times$ marks each an initial (random) mean μ (similar to a centroid), around which a normal distribution curve is plotted. The shaded area under curve shows the range of deviation from mean, i.e. $[\mu - \delta, \mu + \delta]$	201
11.13 First expectation result. Color of a data instance indicates the cluster membership with the highest probability.	205
11.14 First Maximization result. A bell curve with with a shaded area ($\pm\delta$) is the new model with updated parameters for the cluster.	207

List of Figures

11.15 Iterations of Expectation and Maximization. Note that there are 5 data instances in each iris species. But because their petal width values overlap on the only dimension, there appear to be 3 or 4 instances in each cluster, which, in fact, has 5.	209
12.1. Documents in a two-dimensional space	223
12.2. Thought experiment with two documents	224
12.3. Thought experiment a third document, where d_3 doubles the content of d_2	225
12.4. Cosine of an angle	227
12.5. Cosine of a small angle, i.e. vectors of close directions . . .	229
12.6. Cosine of a large angle, i.e. vectors with nearly perpendicular directions	229
12.7. Zipf law function, roughly for the top 1,000 in English . . .	231
13.1. Basic paradigm of an <i>Information Retrieval</i> system	249
13.2. Two postings from the inverted index. Highlighted document IDs with double border indicate matched documents that satisfy <i>Declaration AND Independence</i>	253
13.3. Simultaneous walking to merge postings, starting at the first document ID. The posting with the smaller value will move to the next ID.	254
13.4. Page references via hyperlinks. Page A links to several other pages, including B and C.	262
13.5. Two links being equal. Without more data, the two links are assumed to contribute equally to the target page's impact. 263	
13.6. Two links being different. When we know more about the pages linking to the target page, we can no longer assume their contributions are equal.	264
15.1. Precision-recall curve	281
15.2. ROC curve and area under curve (AUC)	283

List of Figures

15.3. Pearson correlation as cosine of vectors originated from means, with $N = 2$ for a 2-dimensional visualization	294
17.1. First Screen	317
17.2. Second Screen	318
17.3. One example from each MNIST class. The task is simple enough to train quickly, but the handwriting contains varying strokes, slants, and loops.	511
17.4. A digit 7 represented as a two-dimensional image and as the same 784 values in row-major vector order.	513
17.5. A digit 1 and a digit 7 passed through vertical, horizontal, and diagonal hand-designed kernels. Bright regions mark strong positive responses.	518
17.6. Test accuracy, error count, and parameter count for the matched MLP and CNN run.	523
17.7. Training and validation loss and accuracy for the MLP and CNN. Dashed lines mark the minimum validation loss restored by early stopping.	525
17.8. Two learned first-layer kernels and their feature maps for a representative 1 and 7. The channels were selected by their average relative activation difference across all test examples of the two classes.	528

List of Tables

1.1. Raw data	6
1.2. Raw data, with patterns identified and numbers converted.	7
1.3. Shapes data with column information	10
5.1. All possible sets of answers to a quiz of 3 binary questions. Each answer set has a probability of 1/8 to be the correct one.	81
5.2. Four clusters of shapes	89
7.1. Shapes data	117
7.2. Car evaluation data. Buying price and maintenance cost are on a scale of 1 to 4, with 1 meaning the lowest price (cost) and 4 the highest.	138
7.3. Car evaluation, training data for multi-layer neural network. Evaluation value 1 for <i>acceptable</i> and 0 for unacceptable.	152
11.1. IRIS flowers sample data	190
12.1. A new email to be classified	237
15.1. A confusion matrix	277
15.2. Confusion matrix based on the zero-effort solution, by clas- sifying all transactions to non-fraud.	278

Preface

Numerous valuable reference materials exist on the topics of data mining and machine learning, making it relatively easy for researchers in the field to locate relevant articles, online tutorials, and specialized reference books on particular algorithms and methods.

Despite this, as an educator, I have faced challenges in finding appropriate textbooks for my students enrolled in information retrieval, data mining, and machine learning courses. A significant number of these textbooks are excessively technical or rapidly delve into mathematical equations, making them unsuitable as introductory texts. Consequently, students—both undergraduate and graduate—often become overwhelmed by the intricacies of technicality and mathematics, causing them to lose sight of the overarching concepts and ideas.

I have long desired to create a straightforward book for my students, one that maintains a balance between overarching concepts and essential technical specifics; a book that is both technically nuanced and easily accessible.

This book is not intended to serve as a reference guide, as there are already numerous resources available. Instead, it aims to rigorously introduce related subjects with the necessary details, sparking students' interest and encouraging them to delve deeper. The presentation will guide them through the field's landscape, helping them understand particular aspects within context.

My aspiration is for this book to foster imagination and critical thinking while showcasing the state of the art. As educators, our role goes beyond

Preface

transmitting knowledge; we must also prepare future researchers and practitioners for innovation. I endeavor to discuss academic topics within the context of real-world applications while valuing thought experiments involving questions and contemplation. This approach has been appreciated by my students, and I frequently employ it in the book to convey crucial ideas or thought processes.

The book's organization will be driven by significant challenges in the search for pertinent information and methods of addressing data using computational techniques. Beginning with fundamental questions and their (competing) solutions, the book will progressively delve into related issues, removing certain assumptions and considering additional factors to prompt the reader to seek alternatives and innovative approaches.

Ultimately, each chapter will explore the necessary technical details of the state-of-the-art, linking them back to the initially raised issues and discussing what has been addressed or not. Readers will be left with critical questions and directions for further study and research.

I hope that reading this book will help the audience gain insights into developing big data solutions for today and the future. On a personal level, I anticipate that it will aid my students in understanding, questioning, thinking, and innovating within the dynamic field of data science and engineering.

Weimao Ke
Philadelphia, 2023

Part I.

Data, Information, and Mathematical Foundations

1. From Data to Information and Meaning

With great power comes great responsibility.

Uncle Ben, Spider-Man

We encounter data and information every day. We live in a world where information appears to be constantly generated and consumed. We have research disciplines where the subject of study is on information. Yet when it comes to the basic definition of what information is, there is hardly any agreement.

Information is ubiquitous and conveyed in all sorts of media. It can be communicated on news papers, radio, or television. It exists in books that can be read and touched, in records that can be played and listened to. Yet information is not the physical medium being used nor the symbols and signals being transmitted. It is a seemingly intangible concept of a tangible reality. With a conscious mind, information can be internalized into knowledge, which in turn gives rise to new ideas and, as it may go on, new information.

Information is power, in so many tangible ways.

The biological coding of DNA dictates the physical growth, development and functioning of living organisms. The very existence of *mass* dictates the curvature of space-time and the physics of moving objects. Is it not a manifestation of information itself?

1. From Data to Information and Meaning

Information appears to be so universal that a school of prominent physicists have proposed information to be the fundamental element of the universe, with matter and energy as its incidents. "Everything is information," as the late John A. Wheeler put it.

Will it turn out to be true that information is indeed the essence behind "everything," a reality more fundamental than matter and energy, or quantum and light? In other words, is the universe simply a manifestation of its inherent information? If this is the case, we can perhaps regard the natural world as a giant hologram projected from its information core.

In this view, everything we encounter originates from the fundamental reality of information. The pitches of sound we hear, the rays of light we observe, and the forces that push us together or pull us apart... all are signals of information from deep within. When we record the signals and events, they become our data. They are traces of the underlying information we are yet to discover.

And from here we embark on our quest for *information*, following the footprints of *data* with the vehicle of *computing*.

1.1. From Data to Meaning

Suppose we receive the following data:

Table 1.1.: Raw data

011 0100 0010 0 100 0101 1001 1 011 1001 0101 0
011 0010 0100 1 100 0001 1000 1 100 0101 0011 0

Clearly, the data arrive in the binary form but one has to find out what the numbers actually stand for. If metadata¹ are provided, the values can be

¹Metadata is simply data about data.

1.1. From Data to Meaning

properly interpreted and used. Otherwise, one may examine the patterns in the data and identify potential structures.

For example, one may discover that number of digits between each pause (space) repeats in a 3-4-4-1 pattern. If we break each combination of 3-4-4-1 into a line and convert the bits into decimals, the data become:

Table 1.2.: Raw data, with patterns identified and numbers converted.

3	4	2	0
4	5	9	1
3	9	5	0
3	2	4	1
4	1	8	1
4	5	3	0
...			.

Now in Table 1.2, we are yet to find out what the data mean. There appear to be 4 columns for each row we received. The 4th column is binary, with 0 vs. 1 indicating a boolean answer for one way or the other. We suspect that values of the first 3 columns might have something to do with the 4th.

When examining their relations, it is often helpful to use visualizations. We may start with a scatter plot for the 4th column vs. each of the first 3 columns, as shown in Figure 1.1.

Now that the scatter plots hardly reveal any obvious relation, we may start to look at the interplay of multiple variables. We may ask, for example, "is the boolean value of the 4th column in any way associated with the relation between the first two columns?" To answer this question, we may create a scatter plot of the 1st and 2nd columns while color-coding the 4th. When we do the same for 2nd vs. 3rd and 1st vs. 3rd as well, we get Figure 1.2.

1. From Data to Information and Meaning

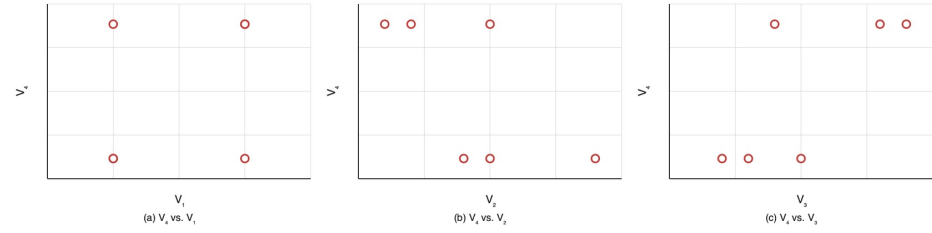


Figure 1.1.: Scatter plots: V_4 vs. V_1 , V_2 , and V_3 .

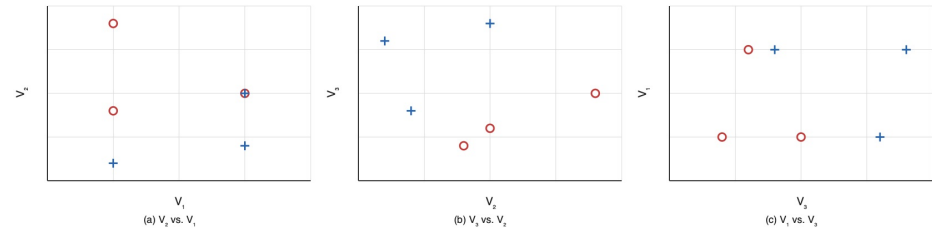


Figure 1.2.: Pairwise scatter plots with the value of V_4 encoded by symbol and color. Red circles represent 0; blue plus signs represent 1.

1.1. From Data to Meaning

It appears the data are nicely separated in terms of 4th column values in Figure 1.2 (b), showing the scatter plot of the 3rd vs. 2nd column. When the value of the 3rd column is greater than the value of the 2nd, it is 1 in the 4th column; otherwise, it gives 0. This can be written as:

$$V_4 = \begin{cases} 1 & \text{if } V_3 > V_2 \\ 0 & \text{otherwise} \end{cases} \quad (1.1)$$

This appears to be the pattern so far and can be further verified when we receive more data. Ultimately, if this is consistent with the underlying relation in the data – as far as we can tell from sufficient data we can receive – we can claim to have discovered a pattern or new “knowledge”, which is essentially what we have learned in the process of analyzing the data.

In the machine learning context, we refer to the result of learning, i.e. the new “knowledge”, as the *concept*. Equation 1.1 is a rule-based approach to concept representation (descriptor). Sometimes, a (linear) equation may also be used to represent the result of learning. In this example, equation $V_3 - V_2 = 0$ is the straight line that separates the 0 and 1 values of column 4, as shown in Figure 1.3.

Whether the concept representation is based on the rules in Equation 1.1 or the linear equation in Figure 1.3, it can be identified by computing and comparing values in the data. Normally such concept representation is discovered through *training data* and to be applied on testing data, where decisions or predictions will be made.



Put it into practice

Continue with the corresponding practice activity to turn **From Data to Meaning** into a calculation, implementation, visualization, diagnostic, or interpretation task.

1. From Data to Information and Meaning

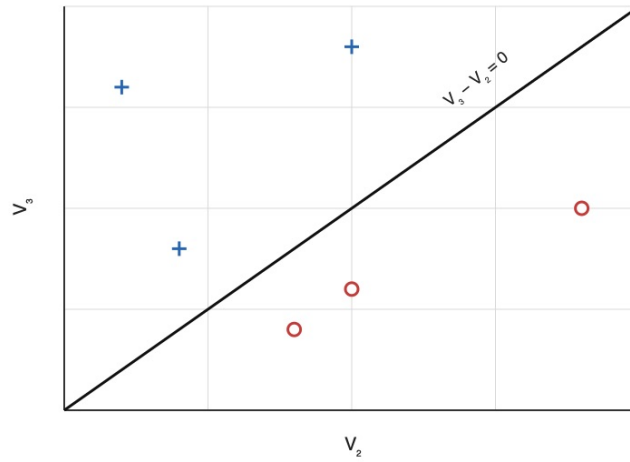


Figure 1.3.: Separating 0 vs. 1 in V_4 with the linear equation $V_3 - V_2 = 0$.

1.2. Concept Generalizability

The example shows that we can potentially identify a concept and gain new knowledge by crunching numbers in the data. This could be done without understanding what the data mean in context. It is important, however, to examine the representativeness of training data and the generalizability of concept representation thus learned in the application domain.

To give the above example some context, the data are in fact about a collection of shapes, each of which has a number of sides, a width, a height, and a code indicating whether the shape is *standing* or *lying*.

Table 1.3.: Shapes data with column information

# Sides	Width	Height	Standing?
3	4	2	0
4	5	9	1

1.2. Concept Generalizability

# Sides	Width	Height	Standing?
3	9	5	0
3	2	4	1
4	1	8	1
4	5	3	0
...	...	...	.

Data in Table 1.3 represent the shapes shown in Figure 1.4. Intuitively, the *concept* of standing can be understood as describing a human-like object – when a person stands, her height is greater than the width; when one lies down, the horizontal measurement (width) becomes relatively greater.

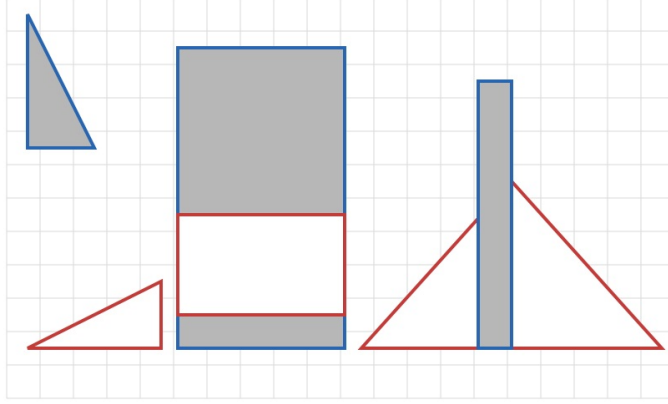


Figure 1.4.: Shapes of different numbers of sides, widths, and heights, based on the shapes data. Gray shapes are classified as *standing*.

It is in this context of human-like shapes that the concept of standing is injected into the data and can be discovered by mining. It is also in this context – again the assumption of human-like shapes – that the identified relation of width vs. height can be applied to related data for predictions, decision making, etc.

1. From Data to Information and Meaning

With related assumptions attached, the concept does not represent universal truth. The same notion of *standing* might be very different with a more complex shape or structure. In Figure 1.5, for example, the shape of a building may be flat in terms of its height-width ratio but we see it *standing as an architecture*.

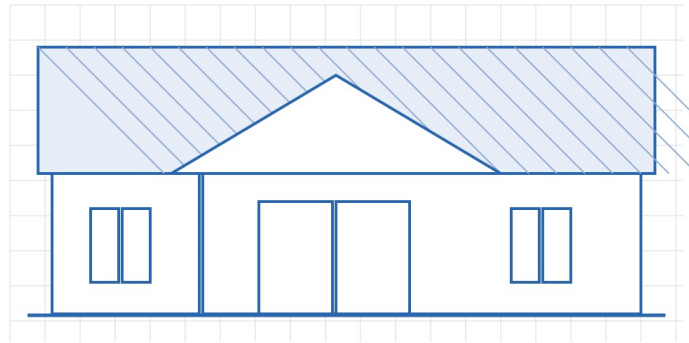


Figure 1.5.: A *standing* architecture.

It takes an in-depth analysis with proper evaluation and interpretation to assess the learned concept, its generalizability, and applicability. This cannot be accomplished by crunching the numbers alone. It requires human intelligence along with machine learning.

1.3. Final Observation

For the concept of *standing*, it turns out that the number of sides of a shape has no impact on the classification. Variables (features) as such can be safely removed for the analysis. As far as the specific concept goes, these variables are part of the noise in the data and can be identified through feature selection.

The example also shows that it is sometimes (often) insufficient to model a *concept* on *individual* variables. In the shapes data example, the concept of *standing* is a relation between the *width* and *height*. For now, it is a simple, linear function. Nonetheless, a more complex concept with messier real-world data may require a non-linear model.² In the following chapters, we will be looking at approaches by which relations of various complexity can be discovered.

Ultimately, machine learning on massive data may help us identify patterns and relations that cannot be eye-spotted, which will then lead to new insight and meaning. And the gained knowledge, if understood properly, will hopefully result in better decision making and improved productivity.

²For example, a non-linear model can be a polynomial function. Sometimes, it can also be a linear function based on a non-linear transformation of the original data.

2. Data Types and Representations

As we employ computers to crunch the data to find meaning and insight, we need to understand the basic form of data, their basic types and implications.

2.1. Datum

2.1.1. Numerical

To compute is to reckon with numbers. Ultimately, all data should be provided as or transformed into numbers: real numbers, integers or decimal values.

Consider the mileage of a car A , such as 96 thousand miles or, if one demands to be more precise, 96,123.5 miles. Compared to another car B at 64 thousand miles, one can reasonably make the following statements:

1. Car A has a greater mileage than B has;
2. Car A has 32 thousand more miles than B has;
3. The mileage of car B is $\frac{2}{3}$ of that of A .

The 1st statement can be made because all numbers can be compared, e.g. $96 > 64$. However, the 2nd and 3rd statements make sense because of special characteristics of the *mileage* variable here.

2. Data Types and Representations

In fact, we can make the 3rd statement because mileage is a *ratio-scaled* variable, for which the zero point is well defined. In this example, a 0 mileage corresponds to the fact that the car has not been driven at all since its production, and is therefore measured absolutely zero. On such a ratio-scaled variable, many mathematical operations such as subtraction, addition, and division can be performed.

Conversely, a temperature variable in Fahrenheit or Celsius is not ratio-scaled, because a 0 value in either $^{\circ}F$ or $^{\circ}C$ is no indication of "zero temperature" or no heat at all. In this case, we cannot perform addition on its values to get a "total temperature" or division to obtain a ratio. Nonetheless, related temperature values are *interval-scaled* because they are measured in equal units and a distance (difference) between two values is meaningful.

It is reasonable to say that $100^{\circ}F$ is 40 degrees higher than $60^{\circ}F$. As an interval-scaled variable, one can perform subtraction to compute the distance (interval). The sum (addition), however, does not have a clear meaning (except as an intermediary step to calculate the average).

There are also variables for which the unit of measurement is not necessarily equal or consistent. On a likert scale, for example, rating numbers from 1 to 5 may be associated with levels of customer satisfaction, e.g. extremely unsatisfied to extremely satisfied. Arguably, the difference between 1 (extremely unsatisfied) and 2 (unsatisfied) may not be the same as that between 3 (moderately satisfied) and 4 (satisfied). In this case, one can compare the satisfaction values and rank them. But it is not as meaningful – at least not consistent – to compute and compare their intervals (differences).

Such a variable is considered *ordinal* as it can be arranged in a meaningful order but lacks an equal unit of measurement to be interval-scaled. While we can compare its values, one should refrain from performing subtraction on them. Neither is addition meaningful with an ordinal variable.

 Put it into practice

Continue with the corresponding practice activity to turn **Numerical** into a calculation, implementation, visualization, diagnostic, or interpretation task.

2.1.2. Categorical

Ordinal variables are in fact categorical. The values are discrete numbers to be compared and ranked; but other than that, no mathematical operations can be performed on them. In fact, they do not have to be numbers. For example, an *Education* variable with values such as *high school*, *college*, and *graduate* can be compared and ranked. These are named values with an order.

A categorical variable can have named values (labels or categories) without an order. We refer to such variables as *nominal*, of which values are from a set of *names* or *labels*. The very basic form of a nominal variable is a binary one, where there are two possible named values: true or false, yes or no, 0 or 1, etc.

For many variables, these labels are discrete, mutually exclusive choices without an explicit relation. For example, a *State* variable can have values such as *PA* and *NY*, and there is no relation between *PA* and *NY*. One cannot establish such relation as $PA > NY$ or $NY > PA$ unless another variable such as state population is considered. This type of variable is purely categorical, not ordinal. With a categorical variable, we can only use the equality operator to determine whether two values are equal or not, e.g. is *PA* the same or different from *NY*.

2. Data Types and Representations

Put it into practice

Continue with the corresponding practice activity to turn **Categorical** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **Datum** into a calculation, implementation, visualization, diagnostic, or interpretation task.

2.2. Data

2.2.1. Sets

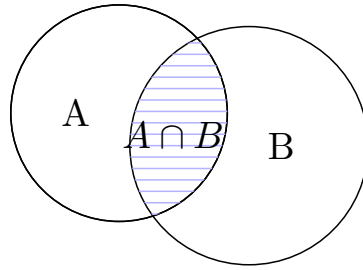
We have discussed categorical variables with values from a set of labels. A set is a collection of member objects or elements. The elements can be any unique entities, symbols, names, and/or numbers. The number of elements in a set is referred to as its *cardinality*.

Consider the skill sets of two data science teams A and B:

$$\begin{aligned} A &= \{AI, Math, Python, SQL\} \\ B &= \{InfoVis, Math, R, SQL, Stats\} \end{aligned}$$

It is easy to count the number of skills in each team and identify their cardinalities: $|A| = 4$ and $|B| = 5$.

The common skill set of the two teams is their *intersection*:

Figure 2.1.: Intersection $A \cap B$.

$$\begin{aligned} A \cap B &= \{Math, SQL\} \\ |A \cap B| &= 2 \end{aligned}$$

which is illustrated by the shaded area in Figure 2.1. The intersection operation is communicative, i.e. $A \cap B = B \cap A$, because the order does not matter.

Now, if we are to put the two team together on one project, the overall skill set of the merged project team will be the *union* of the two:

$$A \cup B = \{AI, InfoVis, Math, Python, R, SQL, Stats\}$$

In this case, the cardinality of the union is:

$$\begin{aligned} |A \cup B| &= |A| + |B| - |A \cap B| \\ &= 4 + 5 - 2 \\ &= 7 \end{aligned}$$

2. Data Types and Representations

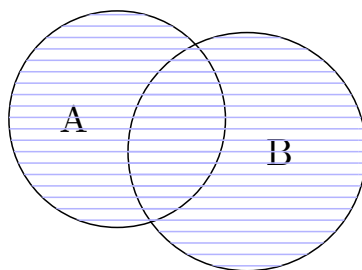


Figure 2.2.: Union $A \cup B$.

Because a set contains only *unique* elements, those in the intersection $A \cap B$ will have duplicates when two sets are combined and should be removed with $-|A \cap B|$.

The union operation is also commutative, i.e. $A \cup B = B \cup A$, because it does not matter whether we add B to A or A to B. In addition, both the union and intersection operations have the associative property, that is:

$$\begin{aligned}(A \cup B) \cup C &= A \cup (B \cup C) \\ (A \cap B) \cap C &= A \cap (B \cap C)\end{aligned}$$

for any sets A, B, and C. As illustrated in Figure 2.3, we will obtain the same union set (the shaded circles) regardless of which two are combined first. The same is true with intersection, as illustrated in Figure 2.4.

The union ($\cup$) operation is similar to *addition* ($+$), whereas the intersection $\cap$ operation can be likened to *multiplication* ($\times$). For demonstration, let's look at a couple of special cases. Let 0 denote an empty set, with no elements at all; 1 be the the universal set, containing all elements in the universe.

For a regular set A, we can show the similarity between union ($\cup$) and addition ($+$):

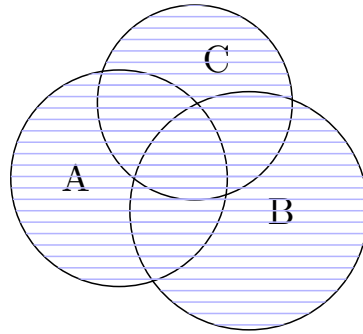


Figure 2.3.: Union of three sets.

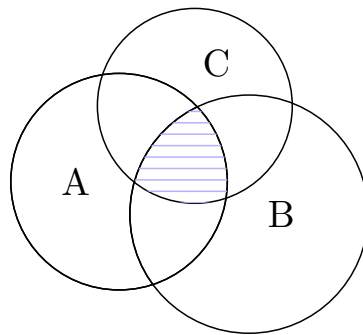


Figure 2.4.: Intersection of three sets.

2. Data Types and Representations

$$\begin{aligned}
 A \cup 0 &= A + 0 \\
 &= A \\
 1 \cup 0 &= 1 + 0 \\
 &= 1 \\
 1 \cup A &= 1 + A \\
 &\approx 1
 \end{aligned}$$

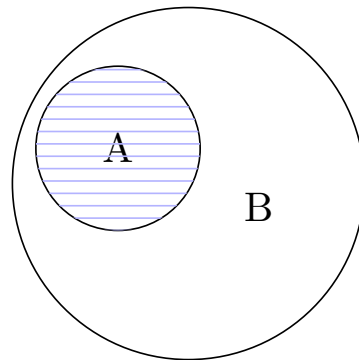
The union of a set A and an empty set 0 is the A itself: $A \cup 0 = A$. The same is true about adding an empty set 0 to the universal set 1 . $1 \cup A$ is not exactly $1 + A$, but they are roughly the same as long as the universal set 1 is far greater than A .

Likewise, intersection ($\cap$) and multiplication ($\times$) operations are similar in that:

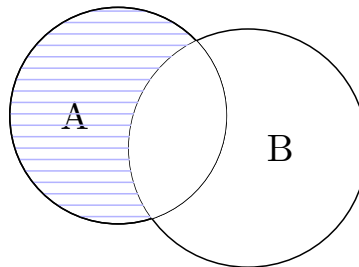
$$\begin{aligned}
 A \cap 0 &= A \times 0 \\
 &= 0 \\
 1 \cap 0 &= 1 \times 0 \\
 &= 0 \\
 A \cap 1 &= A \times 1 \\
 &= A
 \end{aligned}$$

The intersection between any set (A or 1) and an empty set 0 is always an empty set: $A \cap 0 = 0$ and $1 \cap 0 = 0$. The intersection between a set A and the universal set 1 is A , because the universal includes A as a subset. In fact, for any set A that is a subset of B (see Figure 2.5), their intersection is A and their union is B :

$$\begin{aligned}
 A \cap B &\stackrel{\forall A \in B}{=} A \\
 A \cup B &\stackrel{\forall A \in B}{=} B
 \end{aligned}$$

Figure 2.5.: Subset $A \subseteq B$.

Besides union and intersection operators that can be likened to addition and multiplication, there is also the subtraction operator on sets. For two sets A and B , $A - B$ computes the difference between A and B , or the elements of A that do not appear in B . This is essentially the removal of their intersection from set A , as shown in Figure 2.6.

Figure 2.6.: Set subtraction $A - B$.

The cardinality of the subtracted set can be computed by:

2. Data Types and Representations

$$|A - B| = |A| - |A \cap B|$$

If A and B have no intersection, $A - B$ is the same as A. If A is a subset of B, then the result is an empty set. Subtraction is not communicative, as $A - B \neq B - A$.

Put it into practice

Continue with the corresponding practice activity to turn **Sets** into a calculation, implementation, visualization, diagnostic, or interpretation task.

2.2.2. Vectors

In geometrical terms, a *vector* is an entity having both magnitude (length) and direction, in a vector space with multiple dimensions.

Figure 2.7 shows data v_1 and v_2 in a space with X and Y dimensions. In the vector space, each data point is a vector from the origin $(0, 0)$, as shown as directed lines in Figure 2.7, and can be represented by its dimensional values:

$$v_1 = (2, 3)$$

$$v_2 = (3, 1)$$

So in this representation, a vector is simply a list of its coordinate values, each of which represents a feature dimension (variable). The greater the number of dimensions, the longer the list of the values.

A *set* can be represented by a vector. For example, suppose $\{\text{AI, InfoVis, Math, Python, R, SQL, Stats}\}$ is a comprehensive set of skills for data science teams. Given the A and B teams and their skills we discussed earlier, we can have the following tabulated binary representation:

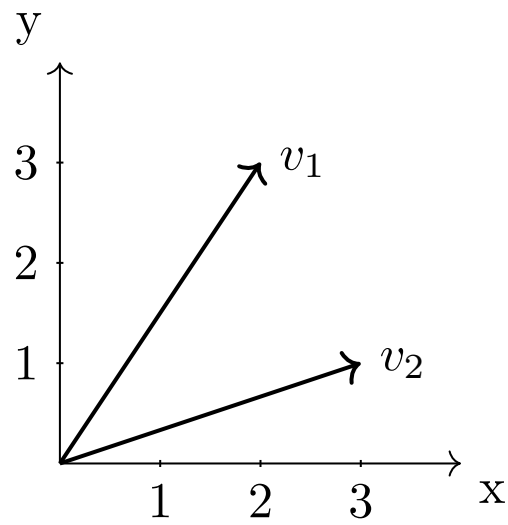


Figure 2.7.: Two-dimensional vector space.

2. Data Types and Representations

Team	AI	InfoVis	Math	Python	R	SQL	Stats
A	1	0	1	1	0	1	0
B	0	1	1	0	1	1	1

where 0 represents absence of a skill and 1 denotes presence of the skill. By populating the 0 and 1 values in a fixed order of the skills, we obtain the vector representations for teams A and B identical to Table [tab-num-teams]:

$$\begin{aligned}A^T &= (1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0) \\B^T &= (0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1)\end{aligned}$$

where the vector space has 7 dimensions, which correspond to seven skills in the comprehensive set, to represent instances of data science teams. These vectors are *row vectors* as they represent data rows.

Note we use superscripted A^T and B^T , where T is the *transpose* operator, because of the convention of using column vectors by default. By transposing the column vectors A and B – with a 90° counterclockwise rotation – we obtain the row vectors A^T and B^T :

$$A^T = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}^T = (1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0)$$

$$B^T = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{pmatrix}^T = (0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1)$$

Here, each column vector is a column with 7 rows of *real* numbers. Hence, $A \in \mathbb{R}^m$ and $B \in \mathbb{R}^m$, where $m = 7$. By transposing the column vectors, we obtain row vectors, each of which has m columns of *real* numbers.

 Put it into practice

Continue with the corresponding practice activity to turn **Vectors** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Data** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3. Vectors, Matrices, and Geometric Thinking

When we have multiple row vectors of data instances, they form a data matrix. For example, a data matrix of the two teams A^T and B^T can be represented as D :

$$D = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}$$

Here D is an $n \times m$ matrix, with $n = 2$ rows and $m = 7$ columns. That is, $D \in \mathbb{R}^{n \times m}$, where $n = 2$ and $m = 7$. In D , we not only have row vectors for the teams but also column vectors for the skills:

$$\begin{array}{ccccccc} AI & InfoVis & Math & Python & R & SQL & Stats \\ = \begin{pmatrix} 1 \\ 0 \end{pmatrix} & = \begin{pmatrix} 0 \\ 1 \end{pmatrix} & = \begin{pmatrix} 1 \\ 1 \end{pmatrix} & = \begin{pmatrix} 1 \\ 0 \end{pmatrix} & = \begin{pmatrix} 0 \\ 1 \end{pmatrix} & = \begin{pmatrix} 1 \\ 1 \end{pmatrix} & = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \end{array}$$

3.1. Basic Matrix Operation

Consider another data set about the annual revenue and profit of three companies. Let X be last year's data (in million dollars):

3. Vectors, Matrices, and Geometric Thinking

$$X = \begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix} \begin{pmatrix} 3 & 0.5 \\ 20 & 1.5 \\ 12 & 1 \end{pmatrix}$$

where each row is a 2-dimensional vector and can be visualized in Figure 3.1.

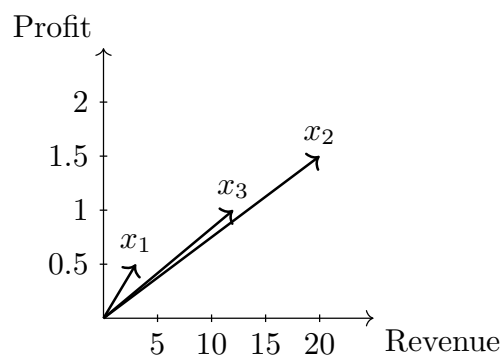


Figure 3.1.: Row vectors of matrix X .

Let Y be this year's data as follow, which, likewise, can be visualized as 3 row vectors in Figure 3.2:

$$Y = \begin{matrix} y_1 \\ y_2 \\ y_3 \end{matrix} \begin{pmatrix} 6 & 1 \\ 15 & 1 \\ 16 & 2 \end{pmatrix}$$

Apparently, X and Y have the same dimensionality – $X \in \mathbb{R}^{3 \times 2}$ and $Y \in \mathbb{R}^{3 \times 2}$ – as they are about the same three companies (rows) and two variables (columns). We can perform simple computation such as addition and subtraction on the matrices.

3.1. Basic Matrix Operation

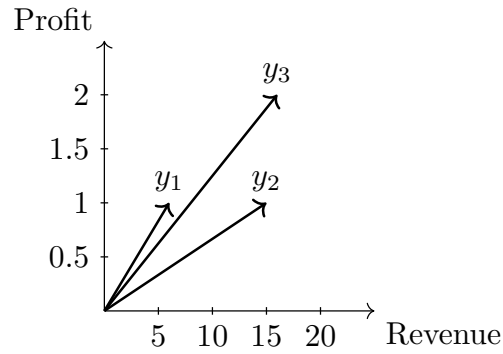


Figure 3.2.: Row vectors of matrix Y .

For example, it is easy to compute the increase or decrease in revenue and profit with subtraction:

$$\begin{aligned}
 G &= Y - X \\
 &= \begin{pmatrix} 6 & 1 \\ 15 & 1 \\ 16 & 2 \end{pmatrix} - \begin{pmatrix} 3 & 0.5 \\ 20 & 1.5 \\ 12 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} 6 - 3 & 1 - 0.5 \\ 15 - 20 & 1 - 1.5 \\ 16 - 12 & 2 - 1 \end{pmatrix} \\
 &= \begin{pmatrix} g_1 & 3 & 0.5 \\ g_2 & -5 & -0.5 \\ g_3 & 4 & 1 \end{pmatrix}
 \end{aligned}$$

Matrix G , the result of the subtraction, contains data about growth (increases and decreases). The rows of matrix G are vectors connecting respective X vectors to Y vectors. Let's single out the 3^{rd} company

3. Vectors, Matrices, and Geometric Thinking

from the data, i.e. x_3 , y_3 , and g_3 . As shown in Figure 3.3, subtraction $g_3 = y_3 - x_3$ is the vector that starts at x_3 and ends at y_3 .

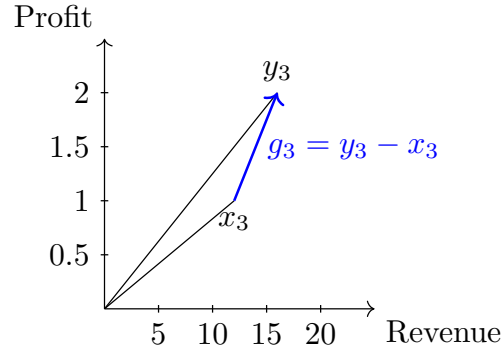


Figure 3.3.: Subtraction geometry.

Suppose the rate of growth in G is projected to continue (the same) next year. This will double the growth, which can be computed by:

$$\begin{aligned}
 G \times 2 &= \begin{pmatrix} 3 & 0.5 \\ -5 & -0.5 \\ 4 & 1 \end{pmatrix} \times 2 \\
 &= \begin{pmatrix} 3 \times 2 & 0.5 \times 2 \\ -5 \times 2 & -0.5 \times 2 \\ 4 \times 2 & 1 \times 2 \end{pmatrix} \\
 &= \begin{pmatrix} 6 & 1 \\ -10 & -1 \\ 8 & 2 \end{pmatrix}
 \end{aligned}$$

Here every value in matrix G is doubled; so is every vector – row or column – in the matrix. For example, Figure 3.4 shows when g_3 , the growth vector of the 3rd company (row), multiplies 2, the vector doubles its length

3.1. Basic Matrix Operation

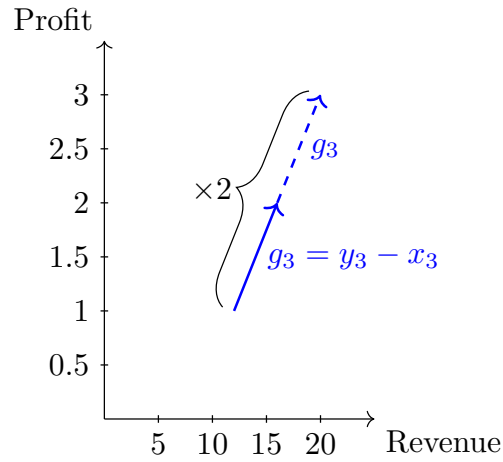


Figure 3.4.: Geometry of $g_3 \times 2$, as part of scalar multiplication $G \times 2$.

(magnitude) in the same direction. Multiplying a matrix with a scalar (single number) results in a matrix of the same dimensionality, in which every value is multiplied by the same scalar. Note the multiplication can be written as $2 \cdot G$, $2G$ or, more generally, aG given a scalar a and a matrix G .

Now with the projected growth (or decrease) data in the $2G$ matrix, we can add them to the last year's data matrix X to compute the final projection for next year Z :

3. Vectors, Matrices, and Geometric Thinking

$$\begin{aligned}
 Z &= Y + 2G \\
 &= \begin{pmatrix} 3 & 0.5 \\ 20 & 1.5 \\ 12 & 1 \end{pmatrix} + \begin{pmatrix} 6 & 1 \\ -10 & -1 \\ 8 & 2 \end{pmatrix} \\
 &= \begin{pmatrix} 3+6 & 0.5+1 \\ 20-10 & 1.5-1 \\ 12+8 & 1+2 \end{pmatrix} \\
 &= \begin{matrix} z_1 \\ z_2 \\ z_3 \end{matrix} \begin{pmatrix} 9 & 1.5 \\ 10 & 0.5 \\ 20 & 3 \end{pmatrix}
 \end{aligned}$$

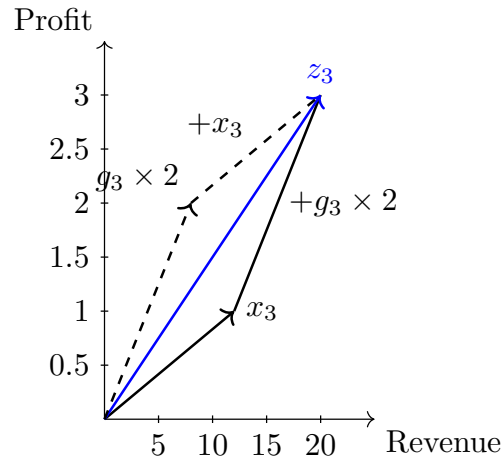


Figure 3.5.: Geometry of addition $z_3 = x_3 + g_3 \times 2$.

Note that the data matrix Z projected from last year's numbers in X should be the same as that projected based on this year's data in Y , i.e. $Z = X + 2G = Y + G$ given that $G = Y - X$.

Geometrically, for corresponding vectors in the matrices, the addition

3.2. Sum of Squares

operation is to connect them head to tail. For example, as shown by the solid lines in Figure 3.5, for the 3rd row vector, $x_3 + g_3 \times 2$ is to connect vectors x_3 and $2g_3$ and the result is z_3 , which draws from the tail of x_3 to the head of $2g_3$. Vector and matrix addition is communicative, the same result can be obtained by $2g_3 + x_3$, as shown by dashed lines in Figure 3.5.

💡 Put it into practice

Continue with the corresponding practice activity to turn **Basic Matrix Operation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3.2. Sum of Squares

Suppose x is a m -dimensional column vector, which is essentially a matrix of m rows and 1 column: $x \in \mathbb{R}^{m \times 1}$.

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_m \end{pmatrix}$$

Now we can transpose the column vector x into a row vector x^T , which is a matrix of 1 row and m columns, i.e. $x^T \in \mathbb{R}^{1 \times m}$:

$$x^T = (x_1 \quad x_2 \quad \dots \quad x_m)$$

We can multiply x^T with x and get their dot product:

3. Vectors, Matrices, and Geometric Thinking

$$\begin{aligned}x^T x &= (x_1 \ x_2 \ \dots \ x_m) \times \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_m \end{pmatrix} \\&= x_1^2 + x_2^2 + \dots + x_m^2 \\&= \sum_{i=1}^m x_i^2\end{aligned}$$

The result is the total sum of squares, a scalar. Sum of squares is often performed on error terms in statistical analysis. For example, given the following errors of three instances:

$$e = \begin{pmatrix} 0.5 \\ 0.2 \\ 0.7 \end{pmatrix}$$

We can compute the Error Sum of Squares (ESS) by:

$$\begin{aligned}e^T e &= (0.5 \ 0.2 \ 0.7) \times \begin{pmatrix} 0.5 \\ 0.2 \\ 0.7 \end{pmatrix} \\&= 0.5^2 + 0.2^2 + 0.7^2 \\&= 0.78\end{aligned}$$

 Put it into practice

Continue with the corresponding practice activity to turn **Sum of Squares** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3.3. Vector Magnitude (Length)

Any vector has two components: a direction and a magnitude (length). Taking the square root of the sum of squares, we can compute a vector's magnitude:

$$\begin{aligned}\|x\| &= \sqrt{x^T x} \\ &= \sqrt{\sum_{i=1}^m x_i^2}\end{aligned}$$

which is the length from the origin to the data point x . For example, given a two-dimensional vector v :

$$v = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

Its magnitude can be computed by:

$$\begin{aligned}\|v\| &= \sqrt{3^2 + 4^2} \\ &= 5\end{aligned}$$

which is the length shown in Figure 3.6.

Put it into practice

Continue with the corresponding practice activity to turn **Vector Magnitude (Length)** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3. Vectors, Matrices, and Geometric Thinking

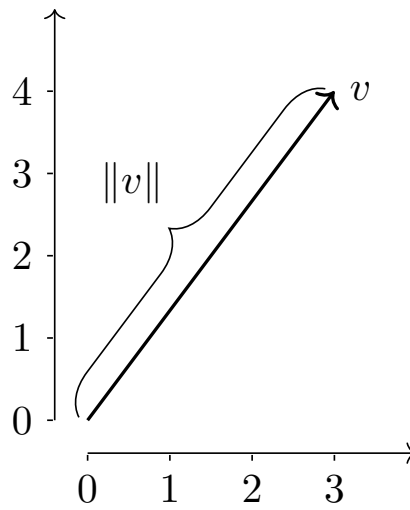


Figure 3.6.: Vector magnitude.

3.4. Vector Distance

Let y be another column vector with the same dimensionality $y \in \mathbb{R}^{m \times 1}$:

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{pmatrix}$$

The difference between x and y can be computed by:

$$\begin{aligned} x - y &= \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_m \end{pmatrix} - \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{pmatrix} \\ &= \begin{pmatrix} x_1 - y_1 \\ x_2 - y_2 \\ \dots \\ x_m - y_m \end{pmatrix} \end{aligned}$$

which, geometrically, is the vector connecting the head of x to that of y . The magnitude of this difference vector can be computed using Equation [eq-matrix-magnitude]:

$$\begin{aligned} \|x - y\| &= \sqrt{(x - y)^T (x - y)} \\ &= \sqrt{\sum_{i=1}^m (x_i - y_i)^2} \end{aligned}$$

Consider two two-dimensional vectors:

$$u = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad v = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

3. Vectors, Matrices, and Geometric Thinking

Their difference can be computed by:

$$\begin{aligned}v - u &= \begin{pmatrix} 3 \\ 4 \end{pmatrix} - \begin{pmatrix} 2 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 3 \end{pmatrix}\end{aligned}$$

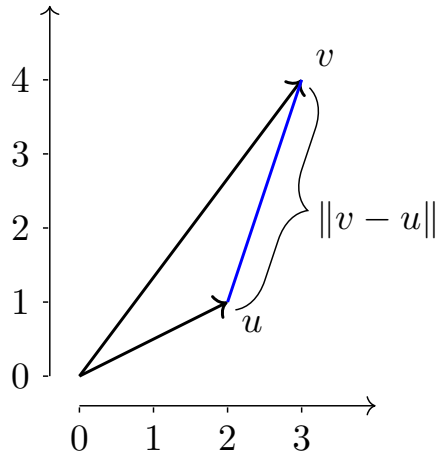


Figure 3.7.: Vector distance.

Their Euclidean distance can be computed by the magnitude of their difference:

$$\begin{aligned}\|v - u\| &= \sqrt{1^2 + 3^2} \\ &= \sqrt{10}\end{aligned}$$

Euclidean distance is symmetric as a metric, i.e. $\|v - u\| = \|u - v\|$. The distance between the two vectors is illustrated in Figure 3.7.

 Put it into practice

Continue with the corresponding practice activity to turn **Vector Distance** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3.5. Vector Angle

For any vector x , we can factor in its magnitude with $\frac{1}{\|x\|}$, we obtain its unit vector:

$$\hat{x} = \frac{x}{\|x\|}$$

of which the length is normalized to 1. We can do the same to another vector y to obtain its unit vector:

$$\hat{y} = \frac{y}{\|y\|}$$

If x and y are both column vectors of the same dimensionality, $x \in \mathbb{R}^m$ and $y \in \mathbb{R}^m$, we can compute their dot product by:

$$\begin{aligned} x^T y &= (x_1 \ x_2 \ \dots \ x_m) \times \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{pmatrix} \\ &= x_1 y_1 + x_2 y_2 + \dots + x_m y_m \\ &= \sum_{i=1}^m x_i y_i \end{aligned}$$

Likewise, we can compute the dot product of their unit vectors:

3. Vectors, Matrices, and Geometric Thinking

$$\begin{aligned}\hat{x}^T \hat{y} &= \left(\frac{x}{\|x\|} \right)^T \left(\frac{y}{\|y\|} \right) \\ &= \sum_{i=1}^m \frac{x_i}{\|x\|} \frac{y_i}{\|y\|}\end{aligned}$$

This turns out to be the cosine of the angle between the two vectors x and y :

$$\begin{aligned}\cosine(x, y) &= \left(\frac{x}{\|x\|} \right)^T \left(\frac{y}{\|y\|} \right) \\ &= \frac{x^T y}{\|x\| \|y\|} \\ &= \frac{\sum_{i=1}^m x_i y_i}{\|x\| \|y\|} \\ &= \frac{\sum_{i=1}^m x_i y_i}{\|x\| \|y\|}\end{aligned}$$

Consider the same two vectors discussed earlier:

$$u = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad v = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

Their magnitudes are:

$$\begin{aligned}\|u\| &= \sqrt{2^2 + 1^2} \\ &= \sqrt{5} \\ \|v\| &= \sqrt{3^2 + 4^2} \\ &= 5\end{aligned}$$

The two vectors can be normalized to unit vectors:

$$\begin{aligned}\hat{u} &= \frac{1}{\sqrt{5}} \cdot \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.894 \\ 0.447 \end{pmatrix} \\ \hat{v} &= \frac{1}{5} \cdot \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}\end{aligned}$$

3.5. Vector Angle

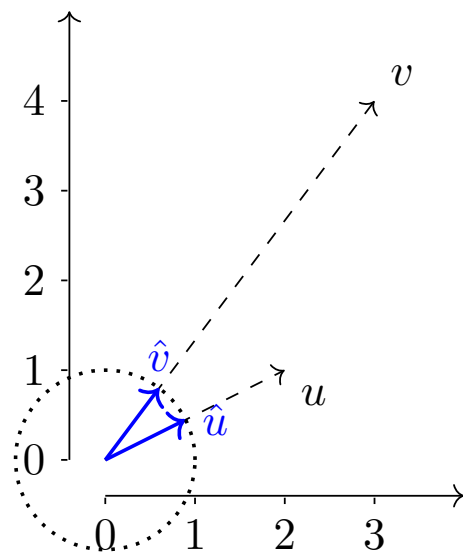


Figure 3.8.: Normalization to unit vectors.

3. Vectors, Matrices, and Geometric Thinking

As shown in Figure 3.8, the normalized unit vectors $\hat{u}$ and $\hat{v}$ point to the same directions of their original vectors u and v respectively. However, the normalization produce vectors of equal magnitudes. As the dotted circle shows, the unit vectors are part of a unit circle with radius 1.

Please note that we can use a circle to visualize the unit vectors on the 2-dimensional space (for the 2-dimensional vectors here). For 3 dimensions, we can use a unit sphere for such visualization. For higher dimensionality, however, we can hardly visualize it but the same idea holds true mathematically with the notion of *hypersphere*.

Now the dot product of the two unit vectors is:

$$\begin{aligned}\hat{u}^T \hat{v} &= (0.894 \quad 0.447) \cdot \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix} \\ &= 0.894 \times 0.6 + 0.447 \times 0.8 \\ &= 0.894\end{aligned}$$

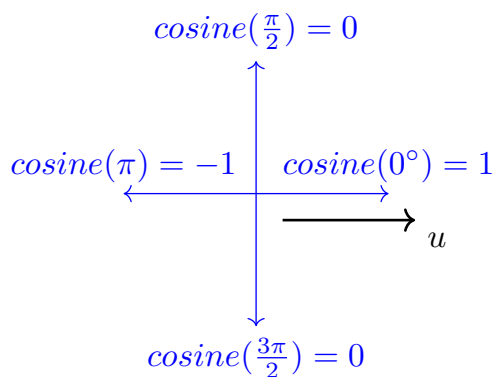


Figure 3.9.: Cosine values for special angles.

which is the cosine of the angle between u and v , shown in Figure 3.8. The arc cosine of 0.894 corresponds to an angle of roughly 27° out of a 360° full

3.6. Vector Cross Product

circle, or 0.465 on a 2π scale. Cosine values range between -1 and 1 : it is 1 when two vector are in the exact same direction (parallel) and -1 when they are in the opposite direction (a 180° or π angle). Cosine is 0 when two vectors are orthogonal (perpendicular) to each other (right angle). See Figure 3.9.

 Put it into practice

Continue with the corresponding practice activity to turn **Vector Angle** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3.6. Vector Cross Product

 Put it into practice

Continue with the corresponding practice activity to turn **Vector Cross Product** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3.7. Optimization

 Put it into practice

Continue with the corresponding practice activity to turn **Optimization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3. Vectors, Matrices, and Geometric Thinking

3.8. Kernel Functions

 Put it into practice

Continue with the corresponding practice activity to turn **Kernel Functions** into a calculation, implementation, visualization, diagnostic, or interpretation task.

3.9. Graph Analysis

 Put it into practice

Continue with the corresponding practice activity to turn **Graph Analysis** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4. Probability and Statistical Thinking

Probability theory is nothing but common sense reduced to calculation.

Pierre-Simon Laplace

In 2012, a national retailer store broke the news by identifying a teen pregnancy before her father knew about it. After seeing coupons of baby clothes and cribs sent to his daughter, the enraged father went to the store to complain about the incident, only to discover later that he was the one who needed to apologize for his ignorance.

While this story unfolded a spectrum of ethical and privacy-related issues, we will focus on the technical question for now – how did the store figure out the girl was pregnant and it is therefore relevant to send her baby coupons?

As one can imagine, this was based on the evidence of related purchases in the girl's shopping history. But even with that, the store could not be certain about the pregnancy and had to guess how *likely* that was the case. Computing the *likelihood* or *probability* is at the core of the technical problem here. We shall look at the basic theory and rules of *probability* before we can use them to reduce all this to calculation.

4. Probability and Statistical Thinking

4.1. Probability

Probability, according to the Oxford Dictionaries, is "the quality or state of being probable; the extent to which something is likely to happen or be the case." While this definition appears common sense and easy to understand, in reality we use the term *probability* or *likelihood* in two very different ways. In the *Frequentist* view, probability is equivalent to the long-term frequency of an event's occurrence. Flipping a *perfect* coin, for example, has a 50% probability to turn up a head and 50% tail because each side has equal chances (frequencies) if it is repeated for *sufficient* times.



Figure 4.1.: Head versus tail.

On the other hand, the *Bayesian* view of probability is associated with the degree of belief without complete information or knowledge. Each coin flipping has nothing to do with a long-term repetition and which side it turns up is a result of many factors. Likewise, how likely you are going to get an A on an upcoming test is a belief based on the known and the

4.2. Probability Basics

unknown – for example, how well you are prepared and what questions the test will be comprised of.

These views are rather divided and may lead to different approaches of modeling and interpretation. Nonetheless, many methods and tools of great practical values have been derived from both views. And we can focus on their application without digging deep into the theoretical subtlety.

 Put it into practice

Continue with the corresponding practice activity to turn **Probability** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.2. Probability Basics

We denote the probability of an event A 's occurrence as $P(A)$. A probability is a numeric value between 0 and 1:

$$0 \leq P(A) \leq 1$$

where 0 probability means the event will never occur and 1, or 100%, means it will always occur.

Be A the event of turning up a head in a perfect coin flipping, $P(A) = \frac{1}{2}$; if A is the event of getting number 3 by rolling a dice, we may estimate $P(A)$ to be $\frac{1}{6}$ if we assume six equal sides without additional information.

The probability of the complement of event A , or that A will NOT occur, is denoted by $P(A')$,¹ which can be computed by:

¹The complementary probability can also be denoted by $P(\bar{A})$, $P(-A)$ or $P(A^c)$.

4. Probability and Statistical Thinking

$$P(A') = 1 - P(A)$$

Head and tail on a coin complement each other. Hence:

$$\begin{aligned} P(Tail) &= P(Head') \\ &= 1 - P(Head) \end{aligned}$$

For mutually exclusive events A and B, their union probability, namely the probability that *either one* of them will occur is the sum of their individual probabilities:

$$P(A \cup B) = P(A) + P(B)$$

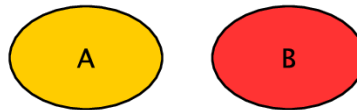


Figure 4.2.: Mutually exclusive events.

where the symbol $\cup$ (union) is equivalent to logical OR and implies addition of the events. When two events are mutually exclusive, they do not occur simultaneously and, as visualized in Figure 4.2, have *no overlap* or intersection in their probability space. Because of that, we can simply add the two together.

In the case of flipping a coin, Head and Tail are mutually exclusive. By the same rule, we have:

$$P(Head \cup Tail) = P(Head) + P(Tail)$$

4.3. Joint probability

which is 1 because Head and Tail complete the entire probability space. Likewise, numbers 1 through 6 on a dice are complete, mutually exclusive events. The same is true that you always get *one* of the six numbers on a dice:

$$\begin{aligned} & P(1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6) \\ = & P(1) + P(2) + P(3) + P(4) + P(5) + P(6) \\ = & 1 \end{aligned}$$

💡 Put it into practice

Continue with the corresponding practice activity to turn **Probability Basics** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.3. Joint probability

If events A and B are not mutually exclusive, their union probability should be the sum of their probabilities subtracted by the probability of them occurring simultaneously – that is the joint probability:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

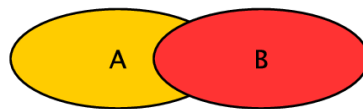


Figure 4.3.: Events that are not mutually exclusive.

4. Probability and Statistical Thinking

The reason for subtracting the joint probability $P(A \cap B)$ can be illustrated in Figure 4.3. As shown in the figure, when A and B can occur simultaneously, they have an overlap (intersection) which should be discounted so it is not counted *twice* in the addition.

Here is an example of not mutually exclusive events – what is the probability of drawing a card from a deck (52 cards) that is *either* an Ace (4 out of 52) *or* a Spade (13 out of 52). The probability of having each can be estimated by:

$$\begin{aligned}P(Ace) &= \frac{4}{52} \\P(Spade) &= \frac{13}{52}\end{aligned}$$



Figure 4.4.: $Ace \cap spade$: a joint (AND) event.

Because there is 1 card that is both an Ace and a Spade, the joint probability is $P(Ace \cap Spade) = \frac{1}{52}$ and the union probability can be computed by:

$$\begin{aligned}&P(Ace \cup Spade) \\&= P(Ace) + P(Spade) - P(Ace \cap Spade) \\&= \frac{4}{52} + \frac{13}{52} - \frac{1}{52}\end{aligned}$$

 Put it into practice

Continue with the corresponding practice activity to turn **Joint probability** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.4. Independent events

Again, the joint probability of A and B, denoted by $P(A \cap B)$,² is the probability that both A and B will occur.

If A and B are independent, meaning the occurrence of A has nothing to do with the occurrence of B, then the joint probability can be computed by the simple multiplication:

$$P(A \cap B) = P(A)P(B)$$

For example, if you flip a coin and roll a dice, how likely will you get a Head *and* number 3. Because the two events are independent, their joint probability can be computed by:

$$\begin{aligned} P(\text{Head} \cap 3) &= P(\text{Head})P(3) \\ &= \frac{1}{2} \times \frac{1}{6} \end{aligned}$$

The joint probability is certainly not limited to two events. You can apply the same rule to three or more independent events. For a set of independent events $E = \{E_1, E_2, \dots, E_m\}$, their joint probability is computed by the product of their probabilities:

²Joint probability can also be denoted by $P(A, B)$ or P(A AND B).

4. Probability and Statistical Thinking

$$P(E_1 \cap E_2 \cap \dots \cap E_m) = \prod_{i=1}^m P(E_i)$$

Again, the equation holds as long as the events are independent – that is, one event’s occurrence does not influence the other. While this is often unrealistic, event independence is often assumed for model simplicity.



Put it into practice

Continue with the corresponding practice activity to turn **Independent events** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.5. Dependent Events

Now let’s take a step further and look at joint probability of dependent events. If the occurrence of A is dependent on the occurrence of B, we can use $P(A|B)$ to denote the probability of event A *on the condition that* event B has already been observed. This conditional probability is referred to as the *posterior probability* because it is estimated after the fact (here the observation of event B).

The joint probability that both events A and B will occur is then computed by:

$$P(A \cap B) = P(A|B)P(B)$$

where $P(B)$ is the *prior probability*, which is a priori before the further observation of data. Again, $P(A|B)$ is the *posterior probabilities* conditioned on the observation of the other event and is about the probability

4.5. Dependent Events

that event A will occur if we know event B occurs. Likewise, the joint probability can also be computed by:

$$P(A \cap B) = P(B|A)P(A)$$

This is based on the prior probability of and the posterior probability conditioned on event A. So far the discussion on probability may look quite abstract. So let's take a look at an example to make sense of this.

Suppose you are running a restaurant and you look at your transaction data, and try to learn about the statistics on how to improve your business. Suppose you find out 60% of the customers order burgers, 54% order burgers and fries. Now you want to know – if a customer already orders a burger, how likely he will also order fries. And this question is certainly relevant to your business.

First let's setup proper notation. Let:

- F be the event that a customer orders *fries*, and
- B be the event that a customer orders *burgers*.

In this case, you know 60% of the chance a customer orders burgers, which is $P(B) = 0.60$. With 54% of customers ordering both burgers AND fries, the joint probability of the two events $P(F \cap B) = 0.54$.

Now the question is about the probability of a customer ordering fries (F), *on the condition* that she has already ordered a burger (B), which is the posterior probability $P(F|B)$. By replacing A and B with F and B respectively in Equation [eq-prob-joint_dep], we have:

$$P(B)P(F|B) = P(F \cap B)$$

Divide both sides by $P(B)$, we get:

4. Probability and Statistical Thinking

$$\begin{aligned} P(F|B) &= \frac{P(F \cap B)}{P(B)} \\ &= 0.54/0.60 \\ &= 0.90 \end{aligned}$$

So it appears the customer will probably (with 90% probability) order fries as well if she is ordering a burger.

We can also apply these probability rules to the retailer store story we discussed at the beginning of the chapter. We can ask questions like – if a customer has purchased a crib, how likely will she be interested in coupons of baby clothes? With other related evidence in the shopping history, can we predict whether the customer is pregnant (or how likely she is) and, if so, follow up with customized marketing efforts?

Put it into practice

Continue with the corresponding practice activity to turn **Dependent Events** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.6. The Bayesian Theorem

To attack the above questions, we need to take one step further and look at the Bayes Theorem.

From equations [eq-prob-joint_dep] and [eq-prob-joint_dep2], we have:

$$P(A|B)P(B) = P(B|A)P(A)$$

Divide both sides by $P(B)$, we get:

4.7. Naive Bayes' Model

$$P(A|B) = \frac{P(A)P(B|A)}{P(B)}$$

And with this, we arrive at a very important probability rule, the *Bayes' Theorem* (rule) on computing posterior probabilities. It is named after Rev. Thomas Bayes³ and further developed by Pierre-Simon Laplace. The rule provides a foundation for various applications with Bayesian inference and probabilistic modeling.

 Put it into practice

Continue with the corresponding practice activity to turn **The Bayesian Theorem** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.7. Naive Bayes' Model

Let us look at an application of the Bayes Theorem to get a sense of how useful the theory is in solving practical problems. One such application is the Naive Bayes model, which is widely adopted in machine learning, data mining, and information retrieval. Pay attention to the two keywords – *Naive* and *Bayes* – the model is *Naive* because it has naive assumptions about variable *independence*, and is *Bayesian* because the core of the model is the Bayes' Theorem.

Ultimately, the Naive Bayes model is to estimate the probability of an inference (hypothesis H) given observed data (evidence E), that is, in the binary scenario, for example, $P(H|E)$ (true) vs $P(H'|E)$ (false). We can compute these probabilities according to the Bayes' rule:

³Richard Price helped publish Bayes' essay with the formula in 1763, after he passed away. Price, a moral philosopher, regarded Bayes Theorem as a means to reason on the probability of miracles and to prove the existence of God.

4. Probability and Statistical Thinking

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$
$$P(H'|E) = \frac{P(E|H')P(H')}{P(E)}$$

where prior probability $P(E)$ is the common denominator that does not differentiate the two and can be ignored for now. $P(H)$ and $P(H')$ can be estimated a priori without evidence E , whereas $P(E|H)$ and $P(E|H')$ can be computed using data observed, in cases when the hypothesis holds or when it does not.

We shall use a specific example to show how it works. In clinical diagnosis, a doctor observes a patient's symptoms (evidence) and determines whether the patient has certain disease (inference / hypothesis). Chickenpox, for example, exhibits symptoms such as fever and blisters (rash). Let:

- E_1 denote the symptom of fever, and
- E_2 denote the symptom of blister (rash).

The question is how likely the patient has developed chickenpox (hypothesis H) if there are symptoms of both fever E_1 and blisters E_2 .

Suppose⁴ the prevalence of chickenpox is 20 in every 1,000 individuals, that is $P(H) = 0.02$ and $P(H') = 0.98$; of those *who develop chickenpox*, 90% have fever ($P(E_1|H) = 0.90$) and 55% have blisters ($P(E_2|H) = 0.55$); of those *without chickenpox*, the chances of have these individual symptoms are $P(E_1|H') = 0.10$ for fever and $P(E_2|H') = 0.05$ for blisters.

The first probability we need to attack is $P(E|H)$, the probability that a patient exhibiting the observed symptoms *if* she indeed has chickenpox. Because we consider two symptoms here, this is about the joint probability when fever and blisters appear simultaneously. With the *Naive* assumption

⁴Numbers in the chickenpox example are hypothetical and only intended for the sake of computational exercise here.

4.7. Naive Bayes' Model

that the symptoms' occurrences are statistically independent, $P(E|H)$ can be computed by:

$$P(E|H) = P(E_1|H)P(E_2|H)$$

Put this in Equation [eq-prob-bayes_he] and we get:

$$\begin{aligned} P(H|E) &= \frac{P(E_1|H)P(E_2|H)P(H)}{P(E)} \\ &= \frac{0.90 \times 0.55 \times 0.02}{P(E)} \\ &= \frac{0.0099}{P(E)} \end{aligned}$$

Likewise, we can compute the probability of a false hypothesis given the same evidence (symptoms) by:

$$\begin{aligned} P(H'|E) &= \frac{P(E_1|H')P(E_2|H')P(H')}{P(E)} \\ &= \frac{0.25 \times 0.02 \times 0.98}{P(E)} \\ &= \frac{0.0049}{P(E)} \end{aligned}$$

The result shows $P(H|E) > P(H'|E)$, meaning that it is more likely than not the patient has developed chickenpox. Again, this model is based on the Bayes' theorem and the assumption that evidential events (symptoms) observed are statistically independent. In another word, the occurrence of one symptom has nothing to do with the other. In reality we know this independence assumption is rarely true. For example, one who has blisters is more likely to have fever than those without.

4. Probability and Statistical Thinking

In the above example, we only use two pieces of evidence (symptom observation) to model the likelihood of chickenpox. In reality, this can be extended to any number of predictor variables where E is a set of variables $E_1, E_2, \dots, E_m$ and m is the size of the set. In this general case, the Naive Bayesian Model in equation [eq-prob-bayes_he] can be extended to:

$$\begin{aligned} P(H|E) &= \frac{P(E|H)P(H)}{P(E)} \\ &= \frac{P(E_1|H)P(E_2|H)\dots P(E_m|H)P(H)}{P(E)} \\ &= P(H) \prod_{i=1}^m P(E_i|H) / P(E) \end{aligned}$$

Likewise, on the probability of a false hypothesis:

$$P(H'|E) = P(H') \prod_{i=1}^m P(E_i|H') / P(E)$$

where variables $E_1, E_2, \dots, E_m$ are assumed to be statistically independent. The main purpose of having this naive assumption is for mathematical simplicity when we compute joint probabilities. Although this assumption is at odds with reality, related models have turned out to perform quite well in experiments. Practically the number of variables (features) m can be very large. For example, in text analysis, it is common to treat unique words as individual features (variables) and, with even a moderate text collection, there can be tens of thousands, if not millions, of variables in the model.

The Bayes Model is not limited to a binary case, e.g. chickenpox vs. not chickenpox. It can also be applied to any classification problem on a number of discrete outcomes. In that case, we simply need to compute the probability of each discrete outcome (hypothesis) and find out which one has the greatest likelihood. It can also be extended to a continuous

case where numeric predictions can be made based on certain probability distribution models, which we will discuss later in the chapter.

 Put it into practice

Continue with the corresponding practice activity to turn **Naive Bayes' Model** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.8. Probability Estimators

Again, consider Equation [eq-prob-bayes_hem], in order to compute the probability of each hypothesis, we need to first estimate the probability of each evidence E_i in $\prod_{i=1}^m P(E_i|H)$. Where do these probabilities come from? Essentially they are learned from the training data where we can count their frequencies, or the number of times each unique situation occurs. For now, the probability $P(E_i|H)$ is computed by:

$$P(E_i|H) = \frac{F(E_i|H)}{F(H)}$$

where $F(E_i|H)$ is the number of instances with E_i where hypothesis H is true and $F(H)$ is the total number of instances with the true hypothesis.

Imagine you are working on the problem of email spam detection. You have a training data set of 1,000 emails and 100 have been identified spams (where hypothesis H_{spam} is true). Suppose *prize* is one of the keywords (evidence E_{prize}) that can be used to model the spam detection problem and, of the 100 spam emails (H_{spam}) there are 20 containing that keyword *prize*. In this case, $F(H_{spam}) = 100$ and $F(E_{prize}|H_{spam}) = 20$ so, according to the probability estimator in equation [eq-prob-pf], the probability $P(E_{prize}|H_{spam})$ can be computed by:

4. Probability and Statistical Thinking

$$\begin{aligned}
 P(E_{prize}|H_{spam}) &= \frac{F(E_{prize}|H_{spam})}{F(H_{spam})} \\
 &= \frac{20}{100} \\
 &= 0.2
 \end{aligned}$$

This will work just fine as long as the relative frequencies (occurrences) of related keywords in the training data (*sample*) are proportional to those of the same keywords in all spam emails in the world (*population*). In this case, we assume what we observe in the sample data is what will most likely occur in the population.

In reality, however, we may observe data in samples with varied departures from the entire population. Imagine we want to use *prize* as one feature in the Naive Bayes but observe no occurrence at all of this term in the training spam data, that is $F(E_{prize}|H_{spam}) = 0$.

In this case, are we to simply conclude the probability of having *prize* in *any* spam email $P(E_{prize}|H_{spam})$ is 0? This is obviously a premature conclusion – the fact that you have not seen an *elephant* does not lead you to conclude on the non-existence of that very creature. There is variance in the data and we do not want to be misled by data in its departure (skewness) from the overall reality.

Data may be localized, just as the story of *blind men and elephant* shows (Figure 4.5). But we should aim to see the whole picture.

In addition, there is also an undesired consequence for having a zero probability in the Naive Bayes model. With E_{prize} as one of the evidence features, its probability $P(E_{prize}|H_{spam})$ is one of the factors in computing the product in the Bayes equation above. When $P(E_{prize}|H_{spam}) = 0$, it leads to $P(H|E) = 0$ regardless of the values of other variables.

In short, we have observed that zero probabilities are not desirable, neither theoretically nor practically. One simple remedy to zero probability estimates, as suggested by Pierre Laplace, is to add 1 to each frequency

4.8. Probability Estimators

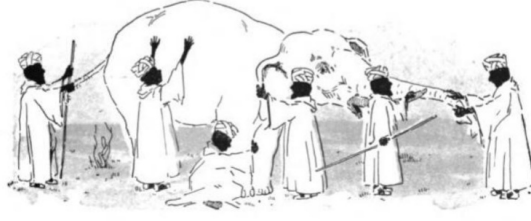


Figure 4.5.: Blind men and elephant. If you have not seen an elephant, does it mean there is no elephant? If you only touch an elephant on its leg, does it mean the elephant is like a tree trunk? These misperceptions are due to our sample data (where you touch the elephant) and should be corrected with external knowledge.

(count) in the sample. So the frequency of having term *prize* in the spams becomes:

$$F(E_{prize}|H_{spam}) + 1$$

To be fair, we also need to add 1 to other frequency counts, including the frequency of NOT having the term *prize* in the spam subset:

$$F(E'_{prize}|H_{spam}) + 1$$

The above two are complete and mutually exclusive cases. Hence, the total number of instances in the spams becomes:

$$\begin{aligned} & F(E_{prize}|H_{spam}) + 1 + F(E'_{prize}|H_{spam}) + 1 \\ = & F(H_{spam}) + 2 \end{aligned}$$

Note that we add 2 to the total number because we have a binary variable, namely having term *prize* (true) vs. not having it (false). If a variable

4. Probability and Statistical Thinking

has k discrete values, then we need to add k to the total. For example, a categorical variable that marks an email's importance may have three different levels such as *low*, *normal*, and *high*. If we add 1 to the frequency of each level, in the end we will have added 3 to the total.

But the question then is why add 1? In fact the idea is that we can add any *constant* c to each count so zero values can be avoided. And the probability of observing a variable value v within the sample where the hypothesis is true can be computed by:

$$P(E_i = v|H) = \frac{F(E_i = v|H) + c}{F(H) + kc}$$

where k is the number of discrete values variable E_i can have, e.g. $k = 3$ for the email priority variable, and c is a chosen constant which represents some prior knowledge about the chance of any value. A great c value will make the probability more predetermined and a lesser value gives relatively more weight to frequencies observed in the sample data. When a discrete value occurs less frequently in the sample, c will change its probability estimate more dramatically.

In a sense, an added constant c favors the rare values and penalizes the frequent ones. Other than simplicity, is this a rational strategy to estimate the probabilities? The answer depends on how much we know about the probability distributions with or without the sample data. If in fact we know some values are going to occur more frequently regardless – for example, we can safely assume most emails would have been marked as *normal* – then we should add more to its count proportionately in the sample data. The probability can be estimated by:

$$P(E_i = v|H) = \frac{F(E_i = v|H) + cp_v}{F(H) + kc}$$

where p_v is the *a priori* probability of each discrete value v ($v \in [v_1, v_2, \dots, v_k]$) and they add up to $\sum_v p_v = 1$. The assignment

4.9. Probability Distributions and Models

of p_v values depends on *prior knowledge*. Practically it is not always clear how they can be obtained. Besides domain knowledge, training samples are often the major source where we gather related statistics so the simple transformation of adding 1 or another constant is what works in many practical applications.

💡 Put it into practice

Continue with the corresponding practice activity to turn **Probability Estimators** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.9. Probability Distributions and Models

Think about this. The very fact that we have to *normalize* the obtained frequencies – to avoid zeros or for other purposes – indicates that we do not totally trust what observed data (samples) can tell us about. The underlying assumption is that we have certain understanding (prior knowledge) about how data should behave and we will need to *correct* the data whenever they *drift* from that expected behavior. By doing the normalization or smoothing, we impose a *model* of expected *probability distributions* on the analysis.

We shall now prepare us with an introductory discussion of probability distributions, from discrete to continuous cases.

First let's look at some discrete cases. Imagine you toss a coin for 1000 times and in the perfect world you get 500 heads and 500 tails. And you can create a simple bar plot to show the frequency distribution of 500 for both heads and tails. This is a uniform distribution because every event, head vs. tail, has an equal chance.

4. Probability and Statistical Thinking

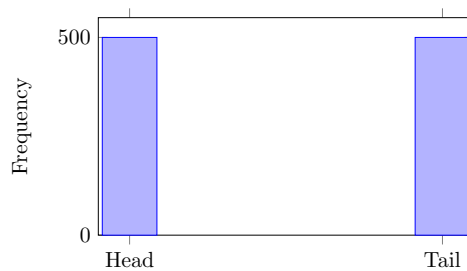


Figure 4.6.: Frequency distribution of flipping a coin: 500 heads versus 500 tails after 1,000 trials in the ideal world.

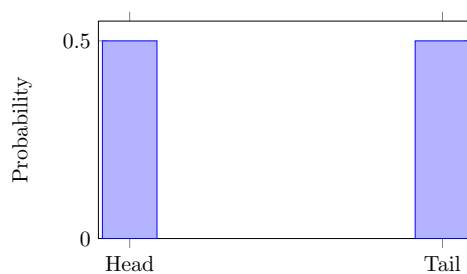


Figure 4.7.: Probability distribution of flipping a coin: heads and tails each have probability 0.5.

4.9. Probability Distributions and Models

Now instead of talking about frequencies or counts, we can describe the distribution in terms of chance (probabilities). And in this case, head and tail have equal chances (probabilities) of 50% or 0.5 in the distribution, as Figure 4.7 shows.

For the sake of discussion, let's extend the binary case, head vs. tail, to a larger number of exclusive outcomes. For example, imagine you roll a dice. If you roll it for a sufficient number of times, the sides will have equal chances to appear, that is, each side from 1 to 6 has a probability of $1/6$ to occur, as shown in Figure 4.8.

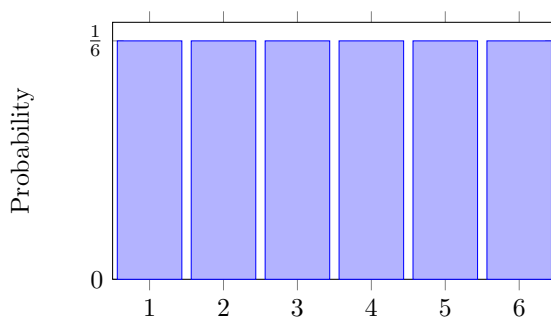


Figure 4.8.: Probability distribution of rolling a die.

Two simple but important observations here:

1. The probabilities of the six sides add up to one because they are exhaustive, mutually exclusive choices;
2. The probability distribution is a uniform distribution as they are all equal. The heights of the bars form a flat, horizontal line that characterizes the uniform distribution.

4. Probability and Statistical Thinking

💡 Put it into practice

Continue with the corresponding practice activity to turn **Probability Distributions and Models** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.10. Continuous Probability Distribution

So far these are discrete distributions. But we can extend this to a continuous distribution. If the outcome x is no longer limited to such integers or specific labels. Perhaps the outcome of the question at hand can be any number in the range of 1 and 6, and you are to predict that number. If it is a uniform distribution in with the continuous variable, it remains a flat line in the range of 1 - 6, shown in Figure 4.9.

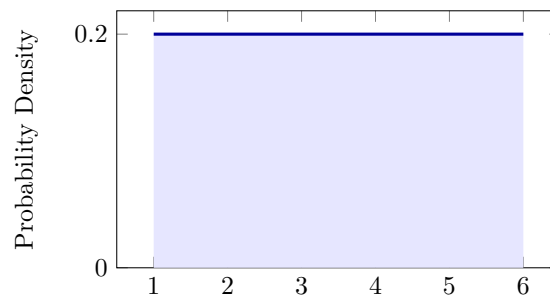


Figure 4.9.: Probability density of a continuous variable x .

Again, when we add all probabilities up in the distribution, the total should still be 1. But because this is a continuous space, we are talking about infinite amount of numbers. The outcome can be number 3 or any decimal number such as 3.14159. To add the infinite number of heights (some sort of "probability" values) in the distribution is essentially *integration* of

4.10. Continuous Probability Distribution

the distribution function (a flat line in this case), that is, the area under the distribution line or curve. The result of the integration or the entire area will *always* be 1, as long as the plotted range includes all possible outcomes.

With a uniform (flat) distribution, the area is a rectangle as shown in Figure 4.9 and it is easy to calculate the area. In the above case, the area of the rectangle between 1 and 6 can be computed by width times height, which is $(6 - 1) \times 0.2 = 1$. So this meets the expectation that the sum of the entire probability distribution is 1.

Although the above is discussed as a *probability* distribution, we shall clarify on an important concept. The uniform height 0.2 in Figure 4.9 is NOT a probability value per se. It is what we refer to as a *probability density*, but not exactly a probability. For now, let us move on from the uniform distribution to a more common distribution in the real world before we come back to elaborate on the concept of *probability density* again.

In the real world, you do not often see uniform distributions⁵. Normal (Gaussian) distributions are much more common. For example, if we collect statistics about adult male heights in the U.S., you will see the heights are not equally likely. Instead, the overwhelming majority of (average) people have the average height around 70 inches. And much rarer are shorter than 62 or taller than 78. The whole distribution is a bell curve as shown in the figure here.

This is again a probability density distribution so if you integrate the curve, the total area under it will be 1 or 100% probability. When you pick a particular number on the x axis such as 70 inches in the middle, you notice it is $y = 0.1$ on the distribution curve. Now be careful that this value 0.1 does NOT mean that there is 10% chance to be 70 inches height (exactly).

⁵Even in an equal-opportunity society, there is rarely any even (uniform) distribution of goods, wealth, etc.

4. Probability and Statistical Thinking

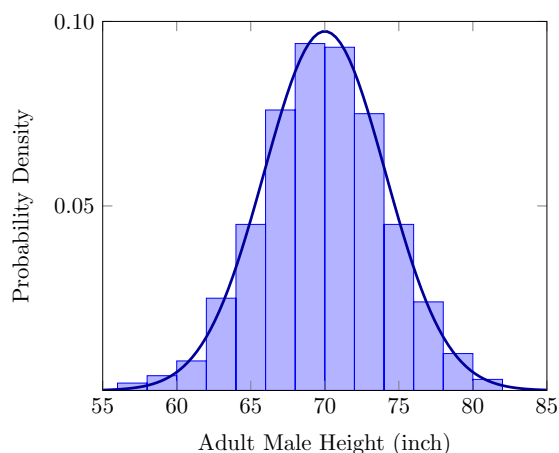


Figure 4.10.: Distribution of adult male heights, approximately a normal distribution (bell curve).

Because there are an infinite number of values in the range, each exact height value in fact has a 0 probability to occur. The reason is when we say 70 inches, we mean exactly 70.000000... with an infinite number of decimal zeros. And it is close to *impossible* to get that exact value on a continuous scale.

Back to the question. So what does a value like 0.1 mean on the y axis? It means 0.1 probability density – that is, 0.1 probability *per inch* (range) on the adult height scale. And this value will change if you change the unit of height, for example by switching to centimeters.

What does this probability density entail now that it is not probability and it varies on the measuring unit? Probability density does allow you to compare probabilities of different continuous *values* and *value ranges*. For example, in the normal distribution here, you can still tell that the probability of a height *around* 70 inches is greater than that in the proximity

4.10. Continuous Probability Distribution

of 80. But to tell the exact probability, you will need to specify a range. You can ask the question of how likely an adult height is 70 *something* (i.e. between 70 and 71) inches. And you can integrate the area under the bell curve in the range to find out.

To reiterate this point – on the simpler example of the uniform distribution presented in Figure 4.9, the 0.2 value on y is not probability but probability density or probability per unit. You can still tell that all probabilities are equal by looking at the flat line. And you can ask the question about how likely you will get a value between 2 and 4 by multiplying the density which is 0.2 and the range which is 2 and the result turns out to be 0.4 – that is 40% of the chance you will get a value *in that range*.

As always, all probabilities add up to 1 as you can verify by $0.2 \times (6 - 1) = 1$. So if you ask how likely a value occurs in the range between 1 and 6, given the distribution in Figure 4.9, the answer is that it will *always be true*.

There are several major categories of probability distributions beyond the uniform and normal distributions we discussed here. Some of these are asymmetric, exhibiting a form of imbalance between the lower and higher ends. Power-law distributions and their variations, for example, are very common in many natural and social structures. They are often associated with basic *counts* in these structures and activities.

The distribution of wealth or income is an example of the power-law distribution, where the overwhelming majority is *poor* (with relatively little wealth) and a small percent at the top are extremely wealthy. This is a result commonly regarded as due to the *Matthew Effect*⁶, in which the rich get rich. In citation and network analysis, this effect is also known as the *cumulative advantage*(Price 1976) or *preferential attachment*(Barabási and Albert 1999).

⁶The term *Matthew Effect* is named after the Gospel of Matthew, in which Jesus is recorded as saying, "for to every one that hath shall be given, and he shall abound: but from him that hath not, that also which he seemeth to have shall be taken away" with the parable of the talents.

4. Probability and Statistical Thinking

Spoken language (word frequencies) follows the Zipf law, which can be regarded as a special case of the power-law distribution. Figure 4.11 shows the distribution of word frequencies taken from the novel *Moby Dick*. As shown in Figure 4.11 (a), there is a great divide between the rich and the poor – a large number of words have only been used once or twice in the novel (tall bars on the left of the distribution) where very few words have a frequency higher than 100 (the long tail).

It is difficult, however, to examine the long tail on normal coordinates as the values are very small (low) within a long range. Logarithmic transformation is often conducted to better visualize the distribution. The result is Figure 4.11 (b), where both x (words' occurrences) and y (frequencies of the occurrences) have been transformed with logarithm.⁷

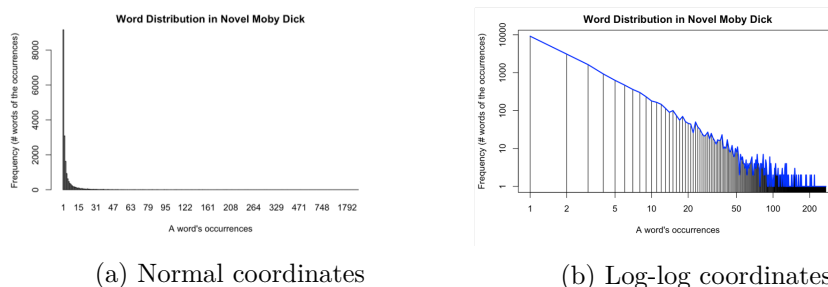


Figure 4.11.: Word frequency distribution

One prominent characteristic of a power-law distribution is that it follows a straight line on log-transformed coordinates spanning across a vast range of numeric orders and is therefore considered *scale free*. One should caution though that many distributions may roughly look linear with log-transformation when they are in fact the result of a different distribution. In reality, there are various constraints and capacity limits on related structures where the outcome is not ideally power-law. In social networks, for example, even those who are super active can only maintain up to a

⁷Essentially log-transformation reduces a number to its order, e.g. 1000 to 3 and 100 to 2 with $\log_{10}$.

certain number of friendly contacts and therefore the outcome of friendship distribution will be subject to such human limits and not scale-free.

Put it into practice

Continue with the corresponding practice activity to turn **Continuous Probability Distribution** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.11. Probability Estimators

The first approach we used to compute probability is to follow whatever the sample data tell us. If we take a sample of 1000 emails and 300 of them are spams, the estimated probability is given by $300/1000 = 0.3$. If we analyze 100 emails and *NONE* of them have the term *prize* in them, then we are bound to conclude the probability that we will ever encounter the term is 0. This naive method of trusting whatever in the sample data is part of a spectrum of methods referred to as the *Maximum Likelihood Estimator* (MLE).

Simply put, an MLE chooses as estimates the values that maximize the likelihood of the observed sample. In the case of observing 0 emails with term *prize* in the sample, we shall thus estimate the term never occurs in any email in the whole world (population) so that NOT observing it is the most likely outcome, which is the case of the observed sample. In general, for discrete distributions, the result of an MLE is given by frequencies of related values in the sample without further transformation.

4.11.1. Discrete Probability Estimation

Now if we have additional knowledge about the domain and can assume probabilities follow a particular distribution model, then the question

4. Probability and Statistical Thinking

becomes how we can specify parameters of that model to maximize the probability of the observed data. Suppose among all emails you receive, you find out for three particular days there are 5, 3, and 7 emails containing the term *prize* on each day. We assume that:

- The number of such emails you receive one day is independent of that on another day. That is, receiving more or less such email today does not affect the number you will receive tomorrow and in the future.
- During a small time interval, the probability of receiving such an email is proportional to the length of the time interval. That is, the longer the time, proportionally greater the probability.

Under these conditions, the probability of receiving x number of *prize* emails can be modeled by a Poisson probability distribution and the probability mass function is:

$$P(x) = e^{-\mu} \frac{\mu^x}{x!}$$

where:

- e is the Euler constant which is approximately 2.71828;
- x is the number of such emails in question, and $x!$ is the factorial $x! = x \times (x - 1) \times (x - 2) \times \dots \times 2 \times 1$;
- μ is the only parameter, the mean number of *prize* emails, to be estimated.

With an MLE, we are to maximize the likelihood of obtaining the 3-day sample, that is the joint probability of having 5, 3, and 7 emails $P(x = 5 \cap x = 3 \cap x = 7)$ on three separate days. Now that we assume the days are independent, the joint probability can be computed by:

4.11. Probability Estimators

$$\begin{aligned} & P(x = 5 \cap x = 3 \cap x = 7) \\ = & P(x = 5)P(x = 3)P(x = 7) \end{aligned}$$

With the Poisson probability function in Equation [eq-prob-poisson], this becomes:

$$\begin{aligned} & P(x = 5 \cap x = 3 \cap x = 7) \\ = & e^{-\mu} \frac{\mu^5}{5!} \times e^{-\mu} \frac{\mu^3}{3!} \times e^{-\mu} \frac{\mu^7}{7!} \\ = & \prod_{x \in [5, 3, 7]} e^{-\mu} \frac{\mu^x}{x!} \end{aligned}$$

Now MLE of the Poisson distribution is to estimate the only parameter, the value of mean μ , so that the above probability in Equation [eq-prob-point_poisson] is maximized. Now instead of maximizing the product (multiplications) of the probabilities, we can take the logarithm of Equation [eq-prob-point_poisson], which becomes:

$$\begin{aligned} l(\mu) &= \ln \left(\prod_{x \in [5, 3, 7]} e^{-\mu} \frac{\mu^x}{x!} \right) \\ &= \sum_{x \in [5, 3, 7]} \ln \left(e^{-\mu} \frac{\mu^x}{x!} \right) \end{aligned}$$

and try to maximize the sum of log likelihood⁸. We can do this by taking the derivative of Equation [eq-prob-point_poisson_log]:

$$l'(\mu) = \frac{1}{\mu} \sum_{x \in [5, 3, 7]} x - n$$

⁸With a positive variable such as a probability, when its value p increases, its $\log(p)$ always increases. So when $\log(p)$ reaches its maximum, so does the p value. Because of this, we can maximize $\log(p)$ in order to maximize p .

4. Probability and Statistical Thinking

where n is the number of observations of the sample, which is 3 for 3 days. The maximum can be reached with the derivative at 0, which is:

$$\begin{aligned} & \frac{1}{\mu} \sum_{x \in [5, 3, 7]} x - 3 \\ = & \frac{5 + 3 + 7}{\mu} - 3 \\ = & 0 \end{aligned}$$

And we can conclude that:

$$\begin{aligned} \mu &= (5 + 3 + 7)/3 \\ &= 5 \end{aligned}$$

maximizes the probability of observing the sample data of 3, 5, 7 on three days following the Poisson distribution. With the estimate for parameter $\mu = 5$, we can now compute the probability for any discrete value with Equation [eq-prob-poisson]. For example, we can find out that it is very unlikely on one day to receive 10 emails containing the term *prize*:

$$\begin{aligned} P(x = 10) &= e^{-\mu} \frac{\mu^x}{x!} \\ &= e^{-5} \frac{5^{10}}{10!} \\ &\approx 0.01 \end{aligned}$$

We can also make it a general case that, a Poisson MLE shall estimate the parameter μ based on the sample mean:

$$\begin{aligned} \mu &= \bar{x} \\ &= \frac{\sum_{i=1}^n x_i}{n} \end{aligned}$$

4.11. Probability Estimators

where n is the number of observations in the sample data.

Put it into practice

Continue with the corresponding practice activity to turn **Discrete Probability Estimation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.11.2. MLE for a Continuous Variable

For a continuous variable, we can also assume a specific functional form of its distribution such as a normal distribution and the objective of an MLE is to estimate related parameters (e.g. mean μ and variance δ^2) of the distribution function, from which the sample data are most likely to be observed. These parameter estimates are then used to compute probability densities at specific values or probabilities within certain ranges.

Suppose X is a numeric variable with a normal distribution, the probability of any value x is given by the following function:

$$f(x) = \frac{1}{\sqrt{2\pi}\delta} e^{-\frac{(x-\mu)^2}{2\delta^2}}$$

This is one of the most recognized formulas in statistics. Although it looks daunting, there are only two parameters and the rest are constants, including π the ratio of a circular circumference to its diameter which is about 3.14159. From a sample data of n instances, the two parameters, namely mean μ and variance δ^2 , can be estimated by the sample mean and variance:

4. Probability and Statistical Thinking

$$\hat{\mu} = \frac{\sum_{i=1}^n x_i}{n}$$
$$\hat{\sigma}^2 = \frac{\sum_{i=1}^n (x_i - \hat{\mu})^2}{n - 1}$$

where x_i is each observed value of x in the sample data.

Normal distributions are symmetric with the highest probability density at the very center of the bell curve. Given the objective of MLE to maximize the likelihood of observed sample, it makes sense that the parameter estimates should ultimately put the sample data at the center (so they are more likely to occur). Now that the center is also the mean of the symmetric distribution, we can now estimate the population mean using directly the mean of the sample, which is what Equation [eq-prob-norm_paras] does.

 Put it into practice

Continue with the corresponding practice activity to turn **MLE for a Continuous Variable** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Probability Estimators** into a calculation, implementation, visualization, diagnostic, or interpretation task.

4.12. Summary

Because it is probabilistic, we assume there is no absolute certainty. So any probability estimate is a guess based on the data you have and best

4.12. Summary

evidence you can identify. There is always a chance it may go wrong. And so we are not to blindly trust the outcome/prediction of a model (neither probabilistic nor any other models). In many critical settings, human machine collaboration is key – in the medical environment, there have been retrieval and AI systems that can provide necessary assistance to technicians and physicians, but they are not there to take over and do everything for us. That said, we also have to recognize there are human errors as well. When we think and make decisions, even as professionals/experts, we cannot be 100 In short, the point is – models are not perfect, as humans are. So the best way is to understand what they are really good at and in what ways we can use their assistance.

5. Information, Entropy, and Divergence

It from bit.

John Archibald Wheeler

Imagine taking a quiz consisting of 3 true/false questions. If one has not studied the subject, she can take a guess on each of the questions and there are 8 (i.e. $2 \times 2 \times 2 = 2^3$) possible sets of answers to the entire quiz. Without the ultimate knowledge about what is correct, each set of answers has a likelihood of $p = 1/8$ to be the correct one.

Table 5.1.: All possible sets of answers to a quiz of 3 binary questions. Each answer set has a probability of $1/8$ to be the correct one.

T	T	T	T	F	F	F	F
T	T	F	F	T	T	F	F
T	F	T	F	T	F	T	F
1	2	3	4	5	6	7	8

Now if we assume the 3 questions are independent (without an overlap of knowledge), one needs 3 piece of information to answer them. Therefore, the amount of information required to find the correct answer is proportional to:

5. Information, Entropy, and Divergence

$$\begin{aligned} H &= 3 \\ &= \log_2 2^3 \\ &= -\log_2 \frac{1}{8} \\ &= -\log_2 p \end{aligned}$$

where p is the probability of one out of 8 possible outcomes and the log base 2 reflects that fact that each question is binary (true or false). In the above 8 mutually exclusive answer sets, each represents a choice and adds to the uncertainty of getting the correct set of answers. Of the overall $-\log_2 \frac{1}{8}$, each answer set i of the 8, if treated equally likely, requires the following amount to be confirmed or eliminated as the ultimate outcome:

$$H_i = -\frac{1}{8} \log_2 \frac{1}{8}$$

where $\frac{1}{8}$ can be regarded as the probability of i^{th} outcome p_i :

$$H_i = -p_i \log_2 p_i$$

This quantity can be linked to Shannon's information theory, also known as the Mathematical Theory of Communication, in which the amount of (missing) information can be measured by the *Entropy* of a probability distribution. (Shannon 1948)

5.1. Shannon Entropy

Indeed, Shannon views communications as a process through which a sequence of symbols are produced by a random (unpredictable) means. The degree of unpredictability or uncertainty is due to the lack of information. Thus information can be measured by the reduction of entropy.

5.1. Shannon Entropy

We can equate the process of communicating a message from a set of symbols with predicting on a set of events or inferences. Given a set of m mutually exclusive events, its Shannon entropy can be computed by:

$$H = \sum_{i=1}^m -p_i \log p_i$$

where p_i is the probability of the i^{th} event (inference). The Entropy represents the amount of missing information in the probability distribution. We may discuss the amount in terms of *bits* if 2 is used for the log base. The example of Equation [eq-info-h_simple] is a special case of its application, where there are 3 pieces (bits) of missing information for 3 binary questions:

$$\begin{aligned} H &= \sum_{i=1}^8 -\frac{1}{8} \log_2 \frac{1}{8} \\ &= 3 \end{aligned}$$

5.1.1. Properties of Shannon Entropy

One may question why this has to be *the* equation for entropy. In fact, Shannon started with several desired properties about such an *entropy* measure before he concluded that this is the only function that meets the expected properties. We shall discuss some of the important properties.

Figure 5.1 shows that the entropy function is smooth with regard to changes in the probability p_1 . When the two mutually exclusive events are equally likely $p_1 = p_2 = 0.5$, the entropy of the system is at its maximum (the peak in the middle).

It has been proved that, with any number of mutually exclusive events, the greatest entropy is achieved when all events are equally possible. Imagine a number of applicants compete for the same position. When everyone has

5. Information, Entropy, and Divergence

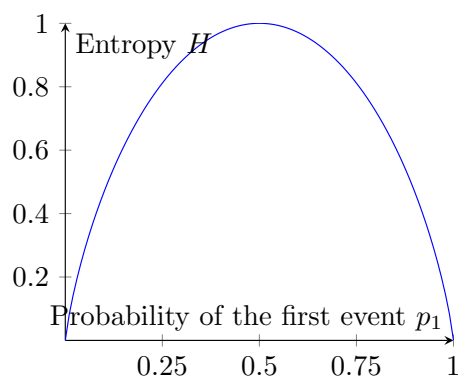


Figure 5.1.: Shannon entropy of two mutually exclusive events.

an equal chance and there is no leading candidate, it is the most uncertain situation with the greatest entropy.

Figure 5.2 plots entropy H vs. the number of equally likely events m . When the number of events (choices) increases, so does the entropy. With an increasingly larger pool of applicants for a position – supposedly with equal chances – the harder it is to predict who will be hired in the end.

A third property of the Shannon entropy is related to how the entropy of one system can be computed based on entropies of its sub-systems. Let us look at a simple example: Three candidates running for office, with candidate 1 from party A and candidates 2 and 3 from party B . Suppose their probabilities of winning the election are estimated as follow:

$$\begin{aligned} p_1 &= 0.3 \\ p_2 &= 0.2 \\ p_3 &= 0.5 \end{aligned}$$

We can easily compute the overall entropy by:

5.1. Shannon Entropy

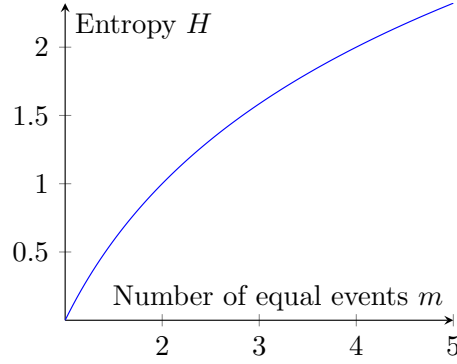


Figure 5.2.: Shannon entropy of m equally likely events.

$$\begin{aligned}
 H_{overall} &= -0.3 \log_2 0.3 - 0.2 \log_2 0.2 - 0.5 \log_2 0.5 \\
 &= 1.485
 \end{aligned}$$

We can also look at the sub-systems, i.e. parties A and B , and their entropies. The probability of party A is the same as that of candidate 1, $p_A = p_1 = 0.3$, whereas the probability of party B is the sum of its candidates' probabilities $p_B = 0.2 + 0.5 = 0.7$. Therefore, the two-party system has an entropy of:

$$\begin{aligned}
 H_{AB} &= -0.3 \log_2 0.3 - 0.7 \log_2 0.7 \\
 &= 0.881
 \end{aligned}$$

Party A only has one candidate, which has a 100% chance to "win" within the party, and there is no uncertainty:

$$\begin{aligned}
 H_A &= -1 \times \log_2 1 \\
 &= 0
 \end{aligned}$$

5. Information, Entropy, and Divergence

Of party B , there are two candidates with overall probabilities $p_2 = 0.2$ and $p_3 = 0.5$, which can be translated into the following probabilities *within* the party:

$$\begin{aligned} p_{B2} &= \frac{0.2}{0.2 + 0.5} \\ &= \frac{2}{7} \\ p_{B3} &= \frac{0.5}{0.2 + 0.5} \\ &= \frac{5}{7} \end{aligned}$$

Therefore, the sub-system entropy of party B is:

$$\begin{aligned} H_B &= -\frac{2}{7} \log_2 \frac{2}{7} - \frac{5}{7} \log_2 \frac{5}{7} \\ &= 0.863 \end{aligned}$$

In the end, the entropy of the entire system overall can be computed by the weighted sum of its sub-systems:

$$\begin{aligned} H_{overall} &= H_{AB} + p_A H_A + p_B H_B \\ &= 0.881 + 0.3 \times 0 + 0.7 \times 0.863 \\ &= 1.485 \end{aligned}$$

This is exactly what we got in Equation [eq-info-h__overall], where we looked at the overall of three candidates regardless of their parties (sub-systems). It can be shown that the overall entropy of any probability distribution is the same as the weighted sum of entropies of its sub-systems.

Put it into practice

Continue with the corresponding practice activity to turn **Properties of Shannon Entropy** into a calculation, implementation, visualization, diagnostic, or interpretation task.

5.1.2. Entropy in Thermodynamics?

The notion of *entropy* predated Shannon's information theory, and had been used in thermodynamics and statistical mechanics. Historically the information-theoretic entropy has been taken as a completely separated development and is only mathematically analogous to the same notion in thermodynamics. Many regarded the identical name as an accident.¹

Nonetheless, several prominent theoretical physicists, including the late John A. Wheeler, have argued that the two are equivalent if given the same dimensionality. (Bekenstein 2003) While it is tempting to elaborate on the debate, for now we shall avoid the unsettled issues but emphasize a few useful points drawn from statistical mechanics.

In thermodynamics, entropy measures the degree of uncertainty in the distribution of microscopic states, e.g. the distribution of temperature in a body of gas. This uncertainty, sometimes referred to as some *disorder* – not of the system itself, but according to the human "eye" – can be attributed to the lack of information or knowledge about the states.

The second law of thermodynamics asserts that the *entropy* of a closed system, without external energy input, will never decrease. That is, it requires energy to reduce entropy and restore *order*. While the thought experiment of *Maxwell's demon* appears to suggest that this second law

¹According to one story, John von Neumann advised Shannon to use the term *entropy* for two reasons: first because it is something already used in statistical mechanics, and second, "as nobody knows what entropy is, whenever you use the term you will always be at an advantage."

5. Information, Entropy, and Divergence

can be violated, some physicists had resorted to the notion of *information* in refuting the paradox between energy and entropy.² One can argue that obtaining information (e.g. about "active" particles) requires energy and the outcome of Maxwell's demon does not violate the second law of thermodynamics. In this reasoning, there appears to be some form of connection between information and entropy even in the physical reality of thermodynamics.

In statistical mechanics, entropy is a measure of uncertainty given the potential number of arrangements or configurations. For example, thermal entropy of gas is due to the lack of knowledge about its particle positions and velocities. It is a macroscopic measure of probability distributions on related microscopic states. Mathematically, this is consistent with quantifying entropy in the arrangements of coding and communication given a probability distribution.

Put it into practice

Continue with the corresponding practice activity to turn **Entropy in Thermodynamics?** into a calculation, implementation, visualization, diagnostic, or interpretation task.

5.1.3. Applications

Shannon's mathematical theory of communication, commonly known as the information theory, has been used in a wide spectrum of areas including digital coding, communication, and information technology applications.

On binary media such as digital computer systems, for example, Shannon entropy offers the basic unit of *bit* to quantify the amount of information and

²The Maxwell's demon goes like this: If we place a demon at the gate between two chambers of gas and it only allows active particles to pass the gate in one direction, then the overall entropy of the two chambers as one closed system will decrease

5.1. Shannon Entropy

the necessary space to store and process it. It also enables a fundamental paradigm shift in the study of communication channel capacity, from signal power (energy) to bandwidth (frequency).

Modeling information as reduction of entropy (uncertainty) provides a valuable vehicle in the design and engineering of digital information systems. In information retrieval and text mining, for example, information and probability theories have provided important guidance to the development of classic methods for representation, ranking, and classification. (Robertson and Zaragoza 2009) For example, computing information amounts of words based on their probability distributions enables the weighing and ranking of terms for text representation.

Let's look at the following example where entropy offers a powerful evaluation tool –

Data mining algorithms often require certain evaluations conducted and decisions made during related training or modeling processes. For example, decision tree construction needs a method to evaluate how good a tree branch is given its membership composition. Data clustering aims to put like entities together and one needs to measure how similar or dissimilar entities are within each cluster. In any of these applications, a good metric of internal homogeneity or heterogeneity (within each decision tree branch or cluster) is critical for the success of related processes.

Suppose we are to group a set of shapes (of types $\triangle$, $\heartsuit$, $\circ$) into related clusters or branches, and Table 1.2 shows four clusters with different membership compositions.

Table 5.2.: Four clusters of shapes

$\triangle$	$\triangle$	$\triangle$	$\triangle$	$\triangle$	$\heartsuit$	$\triangle$	$\triangle$	$\triangle$	$\circ$	$\circ$	$\circ$
$\heartsuit$	$\heartsuit$	$\heartsuit$	$\heartsuit$	$\circ$	$\circ$	$\triangle$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$
$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$
(1)			(2)			(3)			(4)		

5. Information, Entropy, and Divergence

One classic metric is *purity*, which measures the size of the dominant type (class) relative to the entire cluster. That is:

$$Purity(c) = \frac{1}{n_c} \max_t c_t$$

where n_c is the size of cluster c (the total number of members) and c_t is the number of members in c that belong to type t . For cluster (1) in Table 1.2, the members are equally distributed among three types, each has 3 and the purity is:

$$\begin{aligned} Purity(c_1) &= \frac{1}{9} \max(3, 3, 3) \\ &= \frac{3}{9} \end{aligned}$$

For cluster (4), there is only one type, namely $\bigcirc$, and its purity is $9/9 = 100\%$. So far, purity appears consistent with our intuitive observation that cluster (4) is pure and cluster (1) is much less so because it is consisted of three equal types.

If we apply purity to clusters (2) and (3) in Table 1.2, we get:

$$\begin{aligned} Purity(c_2) &= \frac{1}{9} \max(2, 2, 5) \\ &= \frac{5}{9} \\ Purity(c_3) &= \frac{1}{9} \max(4, 0, 5) \\ &= \frac{5}{9} \end{aligned}$$

Both have the same purity score. However, it is clear that cluster (2) has three types (2-2-5) and is more heterogeneous than cluster (3), which only

5.1. Shannon Entropy

has two types (4-5). Because purity only relies on the dominant type, it ignores the overall distribution and does not differentiate between the two clusters here.

Now that we know Shannon entropy is a property of a probability distribution, it can be applied to the problem here to measure member distribution in the types. By converting the counts (frequencies) into probabilities, we can compute the entropy of each cluster by:

$$H = - \sum_t p_t \log p_t$$

where p_t is the probability of type t and can be estimated by c_t/n_c , i.e. the proportion of cluster c that is in type t . Hence:

$$\begin{aligned} H(c_1) &= -3 \times \frac{3}{9} \log \frac{3}{9} \\ &= 1.585 \\ H(c_2) &= -\frac{2}{9} \log \frac{2}{9} - \frac{2}{9} \log \frac{2}{9} - \frac{5}{9} \log \frac{5}{9} \\ &= 1.436 \\ H(c_3) &= -\frac{4}{9} \log \frac{4}{9} - \frac{5}{9} \log \frac{5}{9} \\ &= 0.991 \\ H(c_4) &= -\frac{9}{9} \log \frac{9}{9} \\ &= 0 \end{aligned}$$

From clusters (1) through (4), the entropy decreases. Cluster (1) has an even distribution among the three types and the most heterogeneous with greatest entropy (disorder). Cluster (4), on the other hand, is the ideal and purest with 0 entropy. Furthermore, cluster (3) is not as broadly distributed as cluster (2) and has less entropy. In terms of the examples in

5. Information, Entropy, and Divergence

Table 1.2, Shannon entropy measures the overall distribution better than purity.



Put it into practice

Continue with the corresponding practice activity to turn **Applications** into a calculation, implementation, visualization, diagnostic, or interpretation task.



Put it into practice

Continue with the corresponding practice activity to turn **Shannon Entropy** into a calculation, implementation, visualization, diagnostic, or interpretation task.

5.2. Limits and Other Measures

Despite its broad use, there are assumptions that define the boundary of the classic information theory, beyond which its application requires careful examination of the domain context. For one, the basic assumption that information entails the reduction of entropy does not always hold true. There are abundant examples where having more information may lead to confusion and increased uncertainty. Sometimes, less is more.

The original purpose of Shannon’s theory, as noted in his master piece, was for engineering communication systems where the “meaning of information was considered irrelevant”. It is about technical problems that can be treated as independent the semantic content of messages.(RAPOPORT 1953)

Shannon entropy is best used in applications where uncertainty or the amount of (missing) information is indeed a *property* of a distribution, no less and no more. In domains where receiving information may lead to

5.2. Limits and Other Measures

revised probabilities (change of beliefs) but not necessarily reduction of uncertainty, Shannon entropy as a function solely based on *a* probability distribution becomes insufficient.

To clarify on this point, let us look at a real world example –

Consider the 2016 presidential election of the U.S.³ Poll data and analytics had led many people to believe, up until the election night, that Hillary Clinton would win a landslide against Donald Trump. A common prediction would put 90% for Clinton and 10% for Trump⁴, resulting in an entropy of:

$$\begin{aligned} H_{2016} &= -0.9 \log 0.9 - 0.1 \log 0.1 \\ &= 0.469 \end{aligned}$$

which is much less than the entropy of a 50-50 case ($H = 1$). So with this belief, there was very little uncertainty (doubt) about the outcome given one was highly favored (likely) to win.

In terms of Shannon entropy, there was *little* missing information in that belief (probability distribution). This would make sense if the likely candidate eventually won.

However, when the election turned out in the opposite direction, many would immediately ask questions like "what went wrong" and "what did the polls miss?" Clearly there was in fact lots of missing information in the polls. And the surprising result does require tremendous amount of explanation (with additional information).

But the Shannon entropy does not "care" as it only models the estimated distribution, regardless of the outcome or what turns out to be correct.

³While this is an example from politics, the analysis here is apolitical.

⁴The data are hypothetical, rough estimates of the election day sentiment. For a simplified discussion, we only focus on candidates of the two major parties rather than the actual four.

5. Information, Entropy, and Divergence

We can draw from the above example that different amounts of information are needed to explain different (and sometimes opposite) outcomes. The amount of information should depend not only on the uncertainty of inferences but also on the ultimate outcome (the correct inference or the true probability distribution).

We reason that while uncertainty is a property of a specified probability distribution, the amount of information required to explain the outcome and more generally to explain a probability distribution change is more complex than a linear function of uncertainty and should depend on both distributions, i.e. a belief and a changed belief. We will discuss a couple of models that account for this relative entropy change.

5.2.1. Relative Entropy

Relative Entropy, or KL Divergence, models the amount of information it loses when one uses certain *estimates* to approximate a true probability distribution. (Kullback and Leibler 1951)

Given a probability distribution P and an estimated distribution Q of the same variable, the relative entropy, or information loss due to the estimation, can be computed by:

$$\begin{aligned} KL(P||Q) &= (-\sum_i p_i \log q_i) - (-\sum_i p_i \log p_i) \\ &= \sum_i p_i \log \frac{p_i}{q_i} \end{aligned}$$

where p_i and q_i are, respectively, the (true) probability and estimate of the i^{th} inferences. As shown in the formula, the KL divergence measures the difference in the number of arrangements between Q and P , weighted by the *true* probabilities in P . This is the amount of information (in *bits* with log base 2) lost in the estimates.

5.2. Limits and Other Measures

We apply this to the 2016 election example. Let Q denote the estimates according to polls: $q_{clinton} = 0.9$ and $q_{clinton} = 0.1$. P denotes the true probabilities (actual result).

Suppose the election went as predicted, i.e. with a Clinton win $p_{clinton} = 1$. KL can be computed by:

$$\begin{aligned} KL(P_{clinton}||Q) &= 1 \times \log \frac{1}{0.9} + 0 \times \log \frac{0}{0.1} \\ &= 0.152 \end{aligned}$$

Now that the election actually turned out the opposite, i.e. with a Trump win $p_{trump} = 1$, KL is in fact:

$$\begin{aligned} KL(P_{trump}||Q) &= 0 \times \log \frac{0}{0.9} + 1 \times \log \frac{1}{0.1} \\ &= 3.322 \end{aligned}$$

which indicates a much greater (about 20 times) information loss, compared to the *true* distribution (outcome), in the estimated chances based on poll data. The relative entropy does appear to capture the *surprise* factor.

KL divergence has a few useful properties. KL is non-negative for any distributions. It increases with increasing differences in the P and Q values and can approach infinity with an infinite $\frac{p_i}{q_i}$ ratio, e.g. with a $q_i \rightarrow 0$ – that is, when something is mistakenly believed to be impossible when it is in fact possible.

KL divergence cannot, however, be referred to as a distance measure or metric. To qualify as a metric distance, a function has to satisfy the following conditions: 1) it is non-negative, 2) it returns zero for two identical sets of values, 3) it is symmetric, and 4) it satisfies the triangular inequality.

5. Information, Entropy, and Divergence

KL does meet the first two requirements on non-negativity and identify of indiscernibles – KL is zero, $KL(P||Q) = KL(Q||P) = 0$, only when the two distributions are identical – but does not satisfy the triangular inequality.

For any distance function d , the triangle inequality requires that, given three points (sets of values or probability distributions) A , B , and C , the sum of two distances (sides) is no less than the third. This can be illustrated in Figure 5.3, where any two distances (sides) $d(A, B) + d(B, C) \geq d(A, C)$.

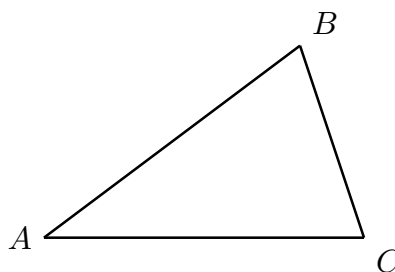


Figure 5.3.: Triangle inequality: the sum of two distances is never less than the third, $d(A, B) + d(B, C) \geq d(A, C)$.

KL divergence does not satisfy the triangle inequality. That is, it does allow $KL(A||B) + KL(B||C) < KL(A||C)$ to happen under certain conditions. In addition, KL is not symmetric and the divergence from Q to P is not the same as that from P to Q , i.e. $KL(P||Q) \neq KL(Q||P)$, where P and Q are not identical.

KL divergence extends the classic Shannon entropy and offers an alternative to measuring information as a linear reduction of entropy. However, the fact that it is not symmetric and not a metric disqualifies it from certain applications where a geometric distance is fitting.

One important consequence of not being a metric is that one cannot add up the amounts of divergence in the *course* of probability changes. It is

5.3. Jensen-Shannon Divergence

meaningless to add $KL(A||B)$ and $KL(B||C)$, which has little to do with $KL(A||C)$. In other words, one cannot discuss the sum of information in KL terms. In addition, as discussed earlier, KL divergence can approach infinity and there is no upper bound.

KL divergence has enabled important development in statistical analysis. Mutual information, for example, is the KL relative entropy between the joint distribution of two random variables and the product of their marginal distributions.

In information retrieval and text mining, the classic IDF (inverse document frequency) term weighting scheme in can be regarded as a function derived from KL divergence. Later in the chapter, we will discuss related problems due to undesirable properties of KL and a remedy developed and tested recently.

 Put it into practice

Continue with the corresponding practice activity to turn **Relative Entropy** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Limits and Other Measures** into a calculation, implementation, visualization, diagnostic, or interpretation task.

5.3. Jensen-Shannon Divergence

One approach to make KL divergence symmetric is to compute the sum of its scores in both ways. For example:

5. Information, Entropy, and Divergence

$$J(P, Q) = KL(P||Q) + KL(Q||P)$$

However, this does not resolve the problem of no upper bound; nor does it satisfy other desirable properties such as triangle inequality. Another alternative is the Jensen-Shannon divergence, which measures the average of information losses using the mixture of two probability distributions:

$$JS(P||Q) = \frac{KL(P||M) + KL(Q||M)}{2}$$

where M is the mixture probability, i.e. $M = \frac{P+Q}{2}$. It is easy to show that $JS(P||Q) = JS(Q||P)$.

A metric can be derived by taking the square root of Jensen-Shannon divergence:

$$JSR(P||Q) = \sqrt{\frac{KL(P||M) + KL(Q||M)}{2}}$$

Using the discussed 2016 election, for example, JSR can be computed for a Trump win, i.e. $P_{trump} = 1$ and $P_{clinton} = 0$. Given polling data shows chances before election night, $Q_{trump} = 0.1$ and $Q_{clinton} = 0.9$, the mixture probability distribution M is:

$$\begin{aligned} M_{trump} &= \frac{1 + 0.1}{2} \\ &= 0.55 \\ M_{clinton} &= \frac{0 + 0.9}{2} \\ &= 0.45 \end{aligned}$$

The Jensen-Shannon divergence is:

5.3. Jensen-Shannon Divergence

$$\begin{aligned}
 JS(P||Q) &= \frac{KL(P||M) + KL(Q||M)}{2} \\
 &= \frac{1 \times \log \frac{1}{0.55} + 0 \times \log \frac{0}{0.45} + 0.1 \times \log \frac{0.1}{0.55} + 0.9 \times \log \frac{0.9}{0.45}}{2} \\
 &= 0.758
 \end{aligned}$$

Then JSR is the square root of JS:

$$\begin{aligned}
 JSR(P||Q) &= \sqrt{0.758} \\
 &= 0.871
 \end{aligned}$$

In the evening of the election, data and the outcome of some states led to an increased likelihood of a Trump win. Suppose the changed probability estimates became Q' where $Q'_{trump} = 0.9$ and $Q'_{clinton} = 0.1$, the opposite of the earlier estimates Q .

We can compute JSR between Q and Q' :

$$\begin{aligned}
 JSR(Q'||Q) &= \frac{0.9 \times \log \frac{0.9}{0.5} + 0.1 \times \log \frac{0.1}{0.5} + 0.1 \times \log \frac{0.1}{0.5} + 0.9 \times \log \frac{0.9}{0.5}}{2} \\
 &= 0.729
 \end{aligned}$$

We can also compute JSR between Q' and P :

$$\begin{aligned}
 JS(P||Q') &= \frac{1 \times \log \frac{1}{0.95} + 0 \times \log \frac{0}{0.05} + 0.9 \times \log \frac{0.9}{0.95} + 0.1 \times \log \frac{0.1}{0.05}}{2} \\
 &= 0.228
 \end{aligned}$$

5. Information, Entropy, and Divergence

Clearly, the sum of $JS(P||Q')$ and $JSR(Q'||Q)$ is $0.228 + 0.729 = 0.957$, which is greater than $JS(P||Q) = 0.871$. This does not violate the triangular inequality.

It has been shown that the square root of Jensen-Shannon divergence (JSR) satisfies triangular inequality. (Endres and Schindelin 2006) And because JS measures the average information loss to the mixture probability, the amount is the same for $JS(P||Q)$ and $JS(Q||P)$ – that is, JS is a symmetric function and so is its square root JSR.

JSR does satisfy all the requirements for a distance function and can be used where related metric properties are desired. Whereas JS divergence can still be regarded as measuring the amount of information in bits (at least with log base 2), it is not clear what its square root (JSR) actually means.



Put it into practice

Continue with the corresponding practice activity to turn **Jensen-Shannon Divergence** into a calculation, implementation, visualization, diagnostic, or interpretation task.

5.4. IDF and KL Divergence

Before we get to an alternative to the divergence measures, we shall discuss an important connection between the classic IDF (inverse document frequency) method for text processing and KL divergence. The theoretical connection will enable us to see why certain treatments based on IDF may or may not be a good idea.

As we discussed in chapter , one important step in text processing is to determine how informative each term is and how much weight should be given. To do this, let's look at probabilities associated with a term.

5.4. IDF and KL Divergence

Suppose we randomly draw a document from a collection of N documents, the likelihood of observing term t in the document can be estimated by:

$$Q(t) = \frac{n_t}{N}$$

where n_t is the number of documents containing term t . Its complementary probability, i.e. the probability that term t does not occur, can be computed by $\bar{Q}(t) = \frac{N-n_t}{N}$.

Now with a specific document where term t actually appears, its probability is $P(t) = 1$ and the complementary $\bar{P}(t) = 0$. We can compute the amount of information loss due to the probability estimate Q using KL divergence:

$$\begin{aligned} KL(P||Q) &= P(t) \log \frac{P(t)}{Q(t)} + \bar{P}(t) \log \frac{\bar{P}(t)}{\bar{Q}(t)} \\ &= 1 \log \frac{1}{\frac{n_t}{N}} + 0 \log \frac{0}{\frac{N-n_t}{N}} \\ &= \log \frac{N}{n_t} \end{aligned}$$

which is exactly the IDF (inverse document frequency) term weighting scheme, where n_t is document frequency (DF).

Here we see that the classic IDF can be regarded as an application of KL divergence in the text processing context. In related applications, it is common to compute the sum of IDF weights for processes such as document ranking (in information retrieval). However, this might not be the proper treatment for a non-metric.

As we observed earlier, KL divergence does not have an upper bound and can reach infinity. In particular, IDF as an information amount of a term can be extremely large when the term is very rare. When term IDF weights are combined to determine ranking, for example, the rare term will

5. Information, Entropy, and Divergence

dominate the ranking score and the weights of other terms will no longer matter.



Put it into practice

Continue with the corresponding practice activity to turn **IDF and KL Divergence** into a calculation, implementation, visualization, diagnostic, or interpretation task.

5.5. Least Information Theory (LIT)

The Least Information Theory (LIT), a very recent development, has attempted to resolve the problems associated with divergence-related information measures.(Ke 2015) In particular, its major objective is to create an information quantity that: 1) is a metric, 2) can be interpreted properly such as in terms of bits, and 3) has an upper bound and does not produce infinite values even with extreme probabilities.

Based on Shannon entropy, LIT does not assume a linear relation between information and the reduction of entropy. Rather, we acknowledge that receiving information can lead to either an increase or decrease of entropy, and measures the amount of information as the sum of microscopic, absolute entropy changes. Given two probability distributions P and Q on m discrete inferences of the same variable, their Shannon entropies can be computed by:

$$H(P) = - \sum_{i=1}^m p_i \log p_i$$
$$H(Q) = - \sum_{i=1}^m q_i \log q_i$$

5.5. Least Information Theory (LIT)

Let dH_i be the amount of entropy change due to a very small change in the probability of the i^{th} inference dp_i . We can compute dH_i based on Shannon entropy:

$$dH_i = -\log p_i dp_i$$

This microscopic entropy represents the change of the weighted (p_i) number of arrangements ($-\log p_i$ or $\log \frac{1}{p_i}$). It is the change in the number of configurations due to a change weight (probability). By integrating (aggregating) the microscopic changes, we can compute the amount of entropy due to a greater, macroscopic probability change from P to Q in the i^{th} inference:

$$\begin{aligned} I_i(P \rightarrow Q) &= \int_{p_i}^{q_i} -\log p dp \\ &= |p_i(1 - \ln p_i) - q_i(1 - \ln q_i)| \end{aligned}$$

The overall entropy change is the sum of entropy changes of all inferences:

$$I(P \rightarrow Q) = \sum_{i=1}^m |p_i(1 - \ln p_i) - q_i(1 - \ln q_i)|$$

where m is the total number of mutually exclusive inferences, i.e. the number of discrete values of the probability distribution. Note that in the final formula, the logarithm is always with a natural base ($\ln$). This is the result of the integral of any log function.

The Least Information measure (LIT) in Equation [eq-info-lit] can be interpreted as the minimum amount of information, in Shannon entropic terms, *necessary* for the probability distribution shift, or a change of belief. In this model, information does not always reduce entropy – in fact, it can lead to entropy change in any direction. In addition, it does not consider

5. Information, Entropy, and Divergence

the removal of information, which can be regarded as new information that refutes prior knowledge.

Figure 5.4 compares least information (LIT) with Shannon entropy and KL divergence, using a binary example (of two mutually exclusive inferences). The figure assumes the 1st inference to be the ultimate outcome – that is, it starts with a probability distribution P (with some p_1 and p_2) and ends with changed distribution Q , where $q_1 = 1$ and $q_2 = 0$. For example, the reading of (0.5, 1) on the LIT curve indicates the amount of least information is 1 that changes the probability distribution from $P(0.5, 0.5)$ to $Q(1, 0)$.

The symmetry of Shannon entropy indicates that the amount is the same regardless of the outcome. KL divergence can approach infinity with $p_1 \rightarrow 0$. Least Information (LIT), however, is limited to the range of $[0, 2]$ in the binary case.

Further examination of Equation [eq-info-lit] reveals that the Least Information Theory (LIT) possesses several important properties:

1. Non-negativity: $I(P, Q) \geq 0$, the amount of least information is no less than zero.
2. Identity of indiscernibles: $I(P, Q) = 0$, if and only if P and Q are identical, i.e. no change at all in the probability distribution.
3. Symmetry: $I(P, Q) = I(Q, P)$, the amount of *least information* required for a probability change from P to Q is the same as that from Q to P .
4. Triangular inequality: $I(P, Q) + I(Q, R) \geq I(P, R)$, among three probability distributions, the sum of two least information amount is no less than the third.
5. Addition of continuous change: $I(P, Q) + I(Q, R) = I(P, R)$, when $P \rightarrow Q$ and $Q \rightarrow R$ are in the same direction of probability changes (i.e. for each individual inference, they either both increase or both

5.5. Least Information Theory (LIT)

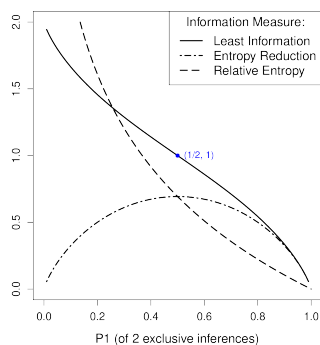


Figure 5.4.: Least Information vs. Shannon Entropy vs. KL Divergence.

This shows the amount of information by reducing two mutually exclusive inferences to certainty. The X axis is p_1 , the probability of the 1st inferences. The Y axis is the amount of information (I) in terms of each information measure. The 1st inference is assumed to be the ultimate outcome ($q_1 = 1$) in the figure.

5. Information, Entropy, and Divergence

decrease its probability). In other words, amounts of *least information* for small, continuous probability changes in the same directions add linearly to the amount responsible for the overall change.

6. Unit Information: In the special case when there are two equally possible inferences, the amount of *least information* needed to explain/interpret an outcome (certainty) is one: $I(p_1 = p_2 = \frac{1}{2} \rightarrow q_1 = 1, q_2 = 0) = 1$ (see data point $(\frac{1}{2}, 1)$ in Figure 5.4).
7. In the special case of reducing uncertain inferences to certainty (the ultimate case):
 - With equally likely inferences, when there are more choices, the least information needed to explain an outcome is larger.
 - The less likely the outcome, the larger the amount of *least information* needed to explain it.

The first four properties listed above mean that the Least Information (LIT) function is a distance metric. The 5th property can be likened to a function such as euclidean distance, where small distances in the same direction can add up to the overall distance. This is illustrated in Figure 5.5, where $a + b = c$. This is the only situation in which they are equal with the triangle inequality.

We can test the Least Information Theory (LIT) on the example of 2016 election. Given the probability change from election day estimates of $Q_{trump} = 0.1$ and $Q_{clinton} = 0.9$ to adjusted estimates of $Q'_{trump} = 0.9$ and $Q'_{clinton} = 0.1$ to the final election outcome of $P_{trump} = 1$ and $Q_{clinton} = 0$, one can compute the the amounts of LIT on the two stages:

5.5. Least Information Theory (LIT)

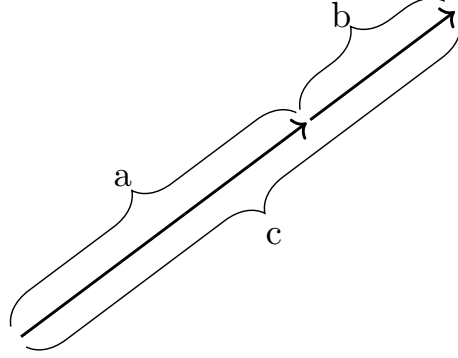


Figure 5.5.: Sum of distances in the same direction, $a + b = c$.

$$\begin{aligned}
 I(Q \rightarrow Q') &= |0.9(1 - \ln 0.9) - 0.1(1 - \ln 0.1)| \\
 &\quad + |0.1(1 - \ln 0.1) - 0.9(1 - \ln 0.9)| \\
 &= 1.329 \\
 I(Q' \rightarrow P) &= |1(1 - \ln 1) - 0.9(1 - \ln 0.9)| \\
 &\quad + |0(1 - \ln 0) - 0.1(1 - \ln 0.1)| \\
 &= 0.335
 \end{aligned}$$

The overall LIT from early estimates Q to the final result P is:

$$\begin{aligned}
 I(Q \rightarrow P) &= |1(1 - \ln 1) - 0.1(1 - \ln 0.1)| \\
 &\quad + |0(1 - \ln 0) - 0.9(1 - \ln 0.9)| \\
 &= 1.664 \\
 &= 1.329 + 0.335
 \end{aligned}$$

which is exactly the sum of the two stages' amounts, i.e. $I(Q \rightarrow P) =$

5. Information, Entropy, and Divergence

$I(Q \rightarrow Q') + I(Q' \rightarrow P)$. This equality occurs because $Q \rightarrow Q'$ and $Q' \rightarrow P$ represent probability changes in the same directions – that is, for each and every inference of the variable, they either both increase or both decrease in that probability.

In the context of text processing and information retrieval, the least information measure has produced superior results. Earlier we have shown that the classic IDF (inverse document frequency) term weighting scheme is a derived method from KL divergence. We can derive a similar method based the least information. Again, the likelihood of having term t in a random document from a text collection is $Q(t) = \frac{n_t}{N}$, where n_t is document frequency (DF) of term t and N the total number of documents in the collection. The amount of least information of actually observing term t in a specific document, with $P(t) = 1$ and $\bar{P}(t) = 0$, can be computed by:

$$\begin{aligned} I(t) &= I(Q_t, P_t) \\ &= |1(1 - \ln 1) - Q_t(1 - \ln Q_t)| \\ &\quad + |0(1 - \ln 0) - \bar{Q}_t(1 - \ln \bar{Q}_t)| \\ &= \underbrace{[1 - Q_t(1 - \ln Q_t)]}_{Q_t \text{ terms}} + [\bar{Q}_t(1 - \ln \bar{Q}_t)] \end{aligned}$$

We may focus on the Q_t terms (the underbraced in Equation [eq-info-lib2]) and neglect $\bar{Q}_t$ terms. We can define $LIB(t, d)$, or least information binary, to measure the informativeness of term t for document d when it appears ($t \in d$) and when it does not ($t \notin d$). Given that $Q_t = \frac{n_t}{N}$, we have:

$$LIB(t, d) = \begin{cases} 1 - \frac{n_t}{N} \left(1 - \ln \frac{n_t}{N}\right) & t \in d \\ -\frac{n_t}{N} \left(1 - \ln \frac{n_t}{N}\right) & t \notin d \end{cases}$$

where n_t is the document frequency of term t and N is the total number of documents. The larger the LIB, the more information the term contributes

5.5. Least Information Theory (LIT)

to the document and should be weighted more heavily in the document representation.

LIB is similar in spirit to IDF and its value represents the discriminative power of the term when it appears in a document. Nonetheless, LIB is formulated based on *least information* of query terms whereas IDF quantifies their KL divergence or the loss of information due to estimates from the entire collection.

Research has shown that these methods derived from LIT worked extremely well for text mining. In a range of experiments conducted on several benchmark data sets, LIT-based methods outperformed their classic counterparts such as TF*IDF in text clustering, classification, and information retrieval.(Ke 2017)

Put it into practice

Continue with the corresponding practice activity to turn **Least Information Theory (LIT)** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6. Data Preparation and Feature Engineering

Trust, but verify.

Ronald Reagan

6.1. Data Quality and Provenance

 Put it into practice

Continue with the corresponding practice activity to turn **Data Quality and Provenance** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6.2. Missing, Noisy, and Inconsistent Data

 Put it into practice

Continue with the corresponding practice activity to turn **Missing, Noisy, and Inconsistent Data** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6. Data Preparation and Feature Engineering

6.3. Scaling, Normalization, and Transformation

Put it into practice

Continue with the corresponding practice activity to turn **Scaling, Normalization, and Transformation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6.4. Encoding Categorical and Structured Variables

Put it into practice

Continue with the corresponding practice activity to turn **Encoding Categorical and Structured Variables** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6.5. Feature Selection and Dimensionality Reduction

Put it into practice

Continue with the corresponding practice activity to turn **Feature Selection and Dimensionality Reduction** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6.6. Leakage and Reproducible Pipelines

 Put it into practice

Continue with the corresponding practice activity to turn **Leakage and Reproducible Pipelines** into a calculation, implementation, visualization, diagnostic, or interpretation task.

6.7. Summary and Exercises

There is no perfect data. In fact, a Kaggle 2017 survey showed that *dirty data* "is the most common problem for workers in the data science realm." (Kaggle, n.d.) About 50% of survey responses cited it as a barrier at work.

Part II.

Learning from Data

7. Classification: From Neighbors to Neural Networks

To be, or not to be,
that is the question...

Hamlet, William Shakespeare

A binary question represents the basic decision making: yes or no, true or false, etc. So how do we build a model to answer basic questions of such? In particular, what does a model *learn* from training data to make predictions?

We shall continue to use a simple data set we encountered earlier, namely the instances of shapes on the concept of *standing* vs. *lying*. Suppose we have identified *width* and *height* as relevant attributes to the concept and Table 1.1 shows the data:

Table 7.1.: Shapes data

Width	Height	Standing?
4	2	No
5	9	Yes
9	5	No
2	4	Yes
1	8	Yes
5	3	No

7. Classification: From Neighbors to Neural Networks

If the shape instances here can serve as training data, how can an algorithm identify the relation between width and height as a predictor for the concept of standing?

Given this simple problem – simple to the human eye – it may be tempting to hardcode a selection structure such as *height* > *width*? into an algorithm and make predictions without the training data. This, however, defeats the purpose of data-driven machine learning. While we can easily identify the classification rule with this small data set, it is impossible to do so with massive data and more complex relations.

We should instead design a general strategy by which related rules and relations can be captured from training data and then used in predictions. What potential strategies can we propose? Let's look at a few proposals.

7.1. Lazy Learning

The easiest approach to learning is not to learn at all, or simply memorize all given materials without trying to understand them. We can refer to this as *lazy learning*, which is similar to what some "lazy" students may do in reality.

But in the real world, how does a student perform on a test (examination) if she has not understood anything at all? The solution is to search, whether it is searching one's memory or materials and notes. Without understanding, one will try to find notes and sections *most closely related* to a test question and develop an answer based on the search result.

Likewise, in machine learning, a lazy learning method does not build any *model* at all but simply remembers (stores) training instances. To predict on a test instance, the method searches its memory (training data), finds the *most closely related* instances to the test, and predict the answer based on related instances it has found.

7.1.1. K Nearest Neighbors (kNN)

We normally refer to the most closely related instances as the *nearest neighbors*. (Cover and Hart 1967) A classic lazy learning algorithm is the *k nearest neighbors* (kNN) approach, which makes predictions based on *k* closest training instances.

But how does the algorithm find out which ones are the nearest? It needs a function to measure the closeness or distance between two instances. An example of such a distance function is the *Euclidean distance*, which the length a straight line connecting the endpoints of two instances *u* and *v* in dimensional space:

$$D(u, v) = \sqrt{\sum_{i=1}^m (u_i - v_i)^2}$$

where *m* is the number of dimensions (features) to represent each data instance. Figure 7.1 shows the shapes data on two dimensions, i.e. *width* and *height*. Suppose a new instance at (4, 7), i.e. *width* = 4 and *height* = 7, is to be tested. The dashed lines are the euclidean distances from the test instance to the training instances.

If we set *k* = 1 for the kNN algorithm, we will use only 1 nearest neighbor in prediction. The instance at (5, 9), which belongs to the *standing* class, has a distance from the test instance:

$$\begin{aligned} D[(4, 7), (5, 9)] &= \sqrt{(4 - 5)^2 + (7 - 9)^2} \\ &= \sqrt{5} \end{aligned}$$

This turns out to be the nearest in terms of Euclidean distance and the test instance will be predicted *standing* (positive) as well.

If we change *k* = 3, the 3 nearest neighbors will all be positive (+) instances and the prediction for the above test instance will remain positive.

7. Classification: From Neighbors to Neural Networks

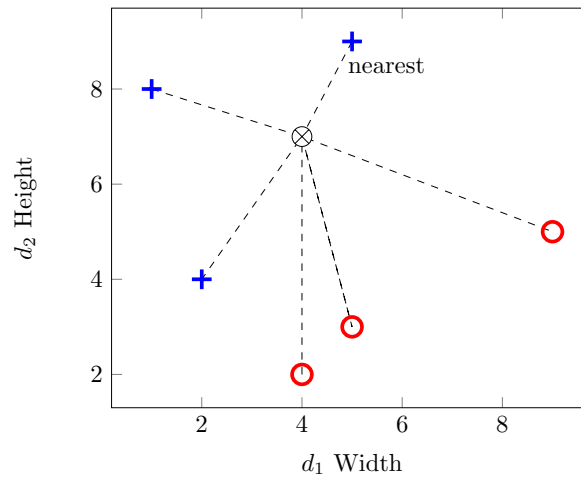


Figure 7.1.: Nearest neighbors $k = 1$, + denotes *standing* (positive) and $\circ$ denotes *lying* (negative). $\otimes$ is the test instance.

7.1. Lazy Learning

For a different test instance, the k nearest neighbors might have conflicting answers. For binary classification, one strategy is to pick an odd number for k and let the nearest neighbors vote for the final prediction.

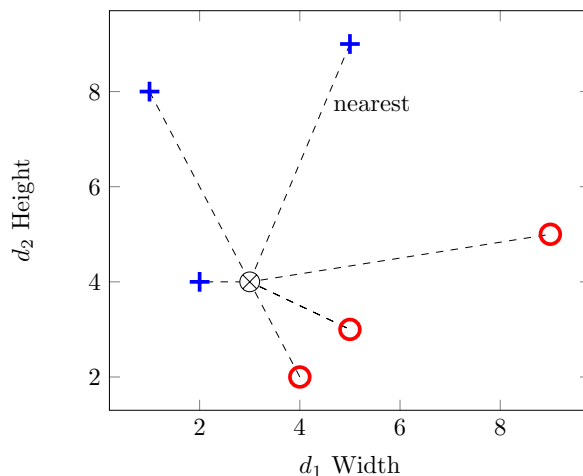


Figure 7.2.: Nearest neighbors $k = 3$, + denotes *standing* (positive) and $\circ$ denotes *lying* (negative). $\otimes$ is the test instance.

Figure 7.2 shows another test instance at $(3, 4)$. With $k = 3$, it turns out that the three nearest neighbors include 1 positive and 2 negatives. Using a majority vote strategy, the test instance will be mistakenly classified into the negative (lying) class.

Two observations on the misclassification. First, the selection of k does have an impact on the classification result. This is especially true when training instances are sparse. With $k = 1$, the method relies on one single instance for each test and is hardly robust. A larger k can reduce the likelihood of predicting with atypical neighbors but it introduces noise, e.g. instances from across the class boundary as illustrated in Figure 7.2. Finding the optimal k may require experimentation with data.

7. Classification: From Neighbors to Neural Networks

Second, more important, a lazy learner such as kNN does not have a general function (model) to represent a concept of learning. It purely relies on individual training instances for predictions and has *no generalizability*. Its success or failure depends on where the test instance hit and how training data distribute around it. This is analogous to a *lazy* student who memorizes everything without understanding – she will not be able to answer a question for which no similar examples have been provided in the materials.



Put it into practice

Continue with the corresponding practice activity to turn **K Nearest Neighbors (kNN)** into a calculation, implementation, visualization, diagnostic, or interpretation task.

7.1.2. Other Distance Metrics

Euclidean distance is a special case of a category of distance metrics known as the *Minkowski distance*, which is defined as:

$$MD(u, v) = \left(\sum_{i=1}^m |u_i - v_i|^p \right)^{1/p}$$

where $p \geq 1$ and m is the number of dimensions. When $p = 2$, this is the Euclidean distance. When $p = 1$, it becomes the Manhattan distance:

$$MD_1(u, v) = \sum_{i=1}^m |u_i - v_i|$$

which is similar to counting the number of blocks in a city grid.

Another metric is similar to (but not exactly) Minkowski distance with $p = 0$:

$$\begin{aligned}
 MD_0(u, v) &= \sum_{i=1}^m (u_i - v_i)^0 \\
 &= \sum_{u_i \neq v_i} 1
 \end{aligned}$$

which counts the number of dimensions where there is a difference and is often referred to as *edit distance*. This is equivalent to treating every dimension as a binary variable and the overall distance is the amount of change in the bits.

 Put it into practice

Continue with the corresponding practice activity to turn **Other Distance Metrics** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Lazy Learning** into a calculation, implementation, visualization, diagnostic, or interpretation task.

7.2. Linear Classifiers

As we observed, a lazy learner such as the k Nearest Neighbors (kNN) does not actually *learn* and build a model (function) that can be generalized. Although kNN has decent performances in many applications, it suffers in domains where, for example, test data appear on a different scale from that of training data.

If we assume the classes can be separated by a *function* (model) with certain parameters, then building the model requires identification of those

7. Classification: From Neighbors to Neural Networks

parameters from training data. Surely there are way too many different functional forms one can choose the model from. We start with the simplest, a linear function.

With the shapes data in Figure 7.1, there are two variables *width* (v_1 for the x axis) and *height* (v_2 for y axis). y as a linear function of x can be written as $f(x) = a + bx$ or $a + bx - y = 0$, with two parameters a and b for two variables.

To make it extendable, We can rewrite this two-variable linear equation as:

$$w_0 + w_1 v_1 + w_2 v_2 = 0$$

where the w parameters (instead of a and b) are to be estimated. When there are more variables (dimensions), the function can be extended to:

$$\begin{aligned} 0 &= w_0 + w_1 v_1 + w_2 v_2 + \dots + w_m v_m \\ &= w_0 + \sum_{i=1}^m w_i v_i \end{aligned}$$

where m is the number of variables (dimensions).

With $m = 2$ for the shapes data, how do we estimate the parameters w_0 , w_1 , and w_2 in Equation [eq-bin-lin2] to linearly separate the two classes *standing* vs. *lying*? Note that a linear equation in a 2-dimensional space is a straight line. It is a plane (flat surface) in a 3-dimensional space and a *hyperplane* in a higher-dimensional space.¹ We will discuss several approaches of the linear model.

¹A hyperplane in more three dimensions is hard to visualize. Mathematically, it shares the same functional form of Equation [eq-bin-lin] and is a natural extension of the lower-dimensional representation.

7.2.1. Between Centroids

We start with finding the *centroid* for instances in each class. A centroid is the center of the mass for a group of objects and can be computed as the average of values on each dimension. The j^{th} dimension of the centroid for class c is computed by:

$$\mu_j(c) = \frac{\sum_{i=1}^{n_c} v_{ij}}{n_c}$$

where v_{ij} is the value of the j^{th} dimension of the i^{th} instance in class c . Following this equation, the centroids for the positive (standing) and negative (lying) classes can be obtained:

$$\begin{aligned} \mu(\oplus) &= (\mu_1(\oplus), \mu_2(\oplus)) \\ &= \left(\frac{5+2+1}{3}, \frac{9+4+8}{3} \right) \\ &= \left(\frac{8}{3}, 7 \right) \\ \mu(\ominus) &= \left(\frac{4+5+9}{3}, \frac{2+3+5}{3} \right) \\ &= \left(6, \frac{10}{3} \right) \end{aligned}$$

Figure 7.3 shows the two centroids and a dashed line with an arrowhead connecting them. The arrow represents a vector in the direction $\vec{\mu} = (6 - \frac{8}{3}, \frac{10}{3} - 7) = (\frac{10}{3}, -\frac{11}{3})$, whereas the middle of the dashed line is $\bar{\mu} = (\frac{13}{3}, \frac{31}{6})$.

Now if we can find a linear equation, i.e. another straight line, that: 1) passes through the middle of the dashed line (equal to each centroid), and 2) is perpendicular to the dashed line, then any point on the linear equation (line) should have an equal distance from both centroid. In this case, the line can serve as the *boundary* between the two classes.

7. Classification: From Neighbors to Neural Networks

Now, identifying the linear equation turns out to be very straightforward. For any point on the line (v_1, v_2) , it can be seen as a vector originated from the middle point $\bar{\mu} = (\frac{13}{3}, -\frac{31}{6})$:

$$\vec{v} = (v_1 - \frac{13}{3}, v_2 - \frac{31}{6})$$

For the vector to be perpendicular to the dashed vector $\vec{\mu}$, their dot product should be 0. That is:

$$\begin{aligned}\vec{v} \cdot \vec{\mu} &= \frac{10}{3}(v_1 - \frac{13}{3}) - \frac{11}{3}(v_2 - \frac{31}{6}) \\ &= 0\end{aligned}$$

which is:

$$-1 - \frac{20}{27}v_1 + \frac{22}{27}v_2 = 0$$

or:

$$v_2 = 1.227 + 0.909v_1$$

Comparing this to Equation [eq-bin-lin2], we have identified the parameters $w_0 = -1$, $w_1 = -\frac{20}{27}$, and $w_2 = \frac{22}{27}$ for the linear classification function. This is the solid straight line separating the two classes (centroids) in Figure 7.3.

With the identified linear function $f(v) = 1 + \frac{20}{27}v_1 - \frac{22}{27}v_2$, one can plug in values of a test instance and make a prediction. If the result is positive, it will predict positive (standing); otherwise, it will predict negative (lying). For example, given a test instance at $(4, 7)$ ($\otimes$ on Figure 7.3), we can put the values in the linear function and get:

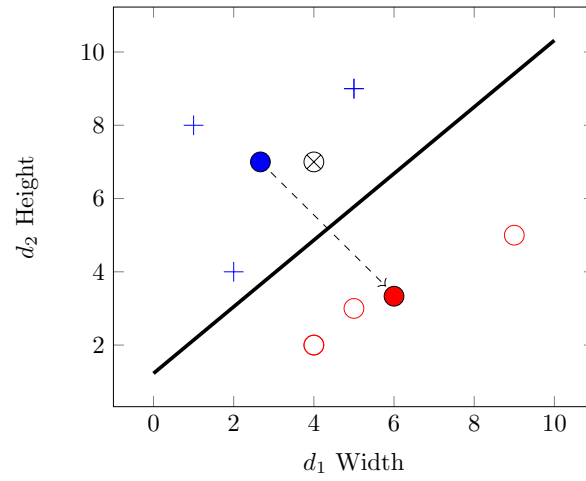


Figure 7.3.: Linear classification with centroids, + denotes *standing* (positive) and o denotes *lying* (negative). • represents a centroid of the respective class. ⊗ is a test instance.

7. Classification: From Neighbors to Neural Networks

$$\begin{aligned} f(4, 7) &= -1 - \frac{20}{27} \times 4 + \frac{22}{27} \times 7 \\ &= \frac{47}{27} \\ &> 0 \end{aligned}$$

The final result is positive – the same positivity with the positive class centroid – and hence the test instance $\otimes$ will be classified into the *standing* class.

Theoretically, we understand that the concept of *standing* is on the relation of *height* – *width* > 0 or, in terms of the linear function, $0 - v_1 + v_2 > 0$. The identified equation is a rough approximation of this theoretical function. The model assumption about a linear separation holds true in this example. But how precise we can establish the exact linear function depends largely on the training data. The data distribution influences the identification of related parameters.

In particular, the centroid approach is greatly impacted by the density of the training set and where most data instances are. For example, having more data instances with smaller width and height in the negative class change its centroid, which will, in turn, lead to a very different linear function, illustrated in Figure 7.4 (compare to Figure 7.3).

The centroid-based approach to linear classification is greatly influenced by all data points in the training set and how they are distributed. The basic assumption underlying the above modeling is that the two classes are of the same size roughly and the "border" lies halfway between their centers. This is often untrue and its performance suffers.



Put it into practice

Continue with the corresponding practice activity to turn **Between Centroids** into a calculation, implementation, visualization, diagnos-

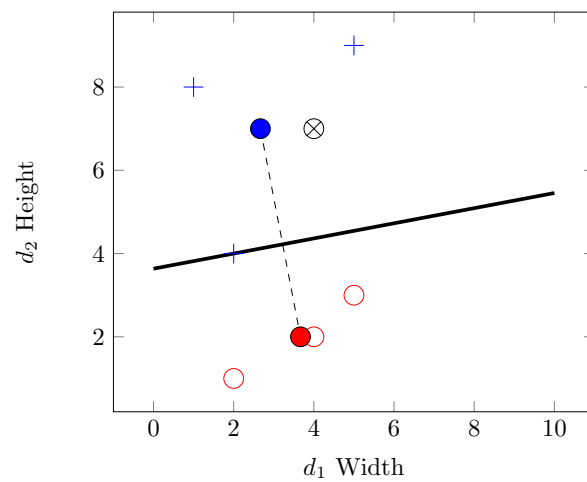


Figure 7.4.: Linear classification with centroids, with smaller values in the negative class

tic, or interpretation task.

7.2.2. Support Vector Machines

Imagine settling the border between two countries, for example, the United States and Canada. It does not matter where the centers of the two countries are. Even if the two countries are of the same size, the border is rarely halfway between the centers.

Rather, it is much more relevant to find out where cities, landmarks, and populations are *on the border*. For U.S. and Canada, cities such as Seattle vs. Detroit vs. Vancouver, Detroit vs. Toronto help define their border.

For classification, it matters not where the centroids are but where the *borderline* data are. The Support Vector Machines, or SVM, is a method focused on the border populations and draws the "border" based on the so-called *support vectors*, data points that are located on the boundary and together form the border.

We start by connecting a number of data points within the same class to construct a linear function, i.e. a line on a 2-dimensional space, a plane for 3 dimensions, and a hyperplane for higher dimensions. On the 2-dimensional shapes data, this means using any two points in each class to form a straight line that, when used as the linear classification function, satisfies two conditions: 1) all data points in the same class fall on *one side* of the linear function, and 2) most if not all data points of the other class reside on the other side of the line. In other words, if a line is drawn in the positive (standing class), then 1) data in it will test all positive (or all negative), and 2) data in the other class will test the opposite, negative (or conversely positive).

Figure 7.5 illustrates the idea with the shapes data.² Pairwise lines drawn in the figure all satisfy the 1st condition and collectively form a *convex hull*

²Please note additional data points than previously discussed in the illustration.

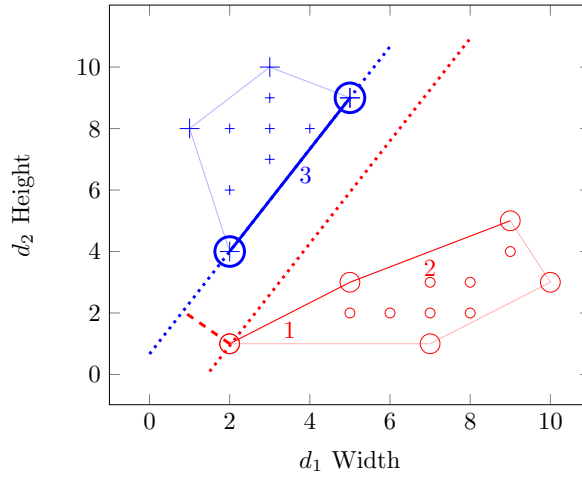


Figure 7.5.: Linear classification with SVM, one candidate margin

for each class. However, not all lines meet the 2nd condition. In fact, only the lines marked 1, 2, and 3 meet both conditions. They are candidate *support vectors*.

For each of the three candidate lines, we can draw a line perpendicular to it from any point in the other class. From this process, we can find out the margin between that candidate support vector line and the other class (convex hull), which is where the boundary can be defined. For example, in Figure 7.5, the two dotted lines show the margin between line #3 and the negative class.

Yet a greater margin – in fact the greatest margin of all – can be found between line #1 and point (2, 4) in the positive class, as the two dotted lines in Figure 7.6 show. The greatest margin will be chosen as the decision boundary, which is defined by a few data points known as the *support vectors*. As shown as circled data points in Figure 7.6, support vectors are

7. Classification: From Neighbors to Neural Networks

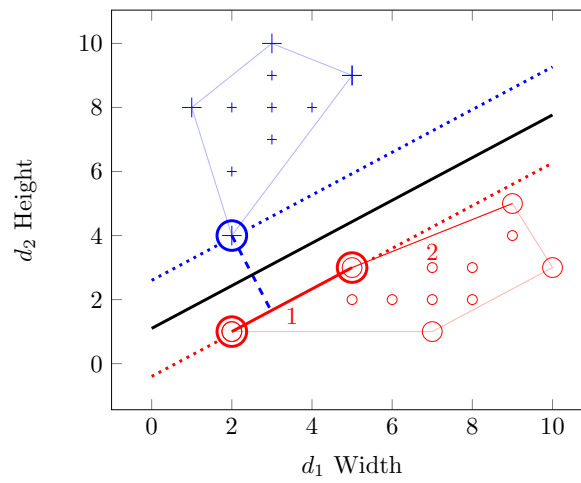


Figure 7.6.: Linear classification with SVM, support vectors and maximum margin

those that determine the maximum margin between the two *convex hulls* of the two classes.

With this, it is now easy to identify the linear equation in the margin to separate the two classes. In Figure 7.6, for example, the solid line at $v_2 = 1.1 + 0.667v_1$ can be used as the linear classifier. One can certainly use the linear functions on the support vectors for classification as well, e.g. to predict one class vs. another if a test instance is within the support vector boundary or to estimate a probability if it is on the margin.

Note that with the shapes data example, we only have to deal with two dimensions and any linear equation is a straight line. With a higher dimensionality, the process remains the same mathematically but a linear function becomes a plane or hyperplane. On three dimensions, for example, three data points should be connected to construct a linear equation (plane) to identify the convex hulls and support vectors. Chapter has details on the mathematics for SVM.



Put it into practice

Continue with the corresponding practice activity to turn **Support Vector Machines** into a calculation, implementation, visualization, diagnostic, or interpretation task.

7.2.3. Perceptron: Toward a Neural Network

The linear function $f(v) = w_0 + w_1v_1 + w_2v_2$ can be visualized as a *perceptron* in Figure 7.7, where the node at $\sum$ computes the weighted sum of input variables. Obviously, this can be extended to include any number of variables m for which $f(v) = \sum_{i=1}^m w_i v_i$.

A perceptron is a simple, one-layer neural network – besides the input variables, there is only one layer of nodes (only one $\sum$ node) in the model, as shown in Figure 7.7. The final output of the perceptron is binary 0

7. Classification: From Neighbors to Neural Networks

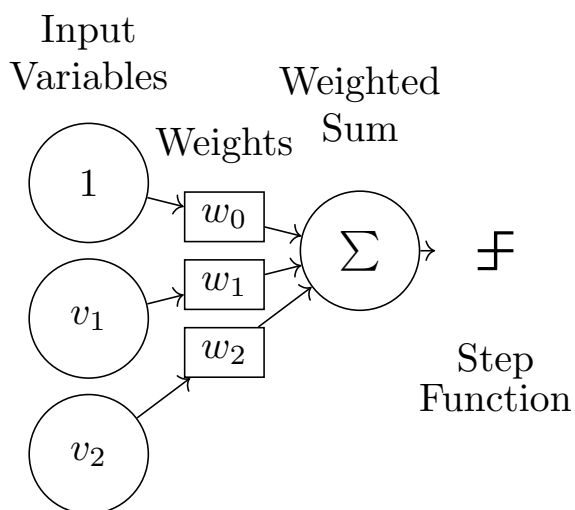


Figure 7.7.: Linear classification with a Perceptron, where the output is a step function based on the weighted sum of input variables

or 1 by using a step function, which simply converts values higher than a threshold to 1 and those lower to 0. For example, one can use 0 as the threshold for a step function $f(s)$:

$$f(s) = \begin{cases} 1 & \text{if } s > 0 \\ 0 & \text{otherwise} \end{cases}$$

which is visualized in Figure 7.8.

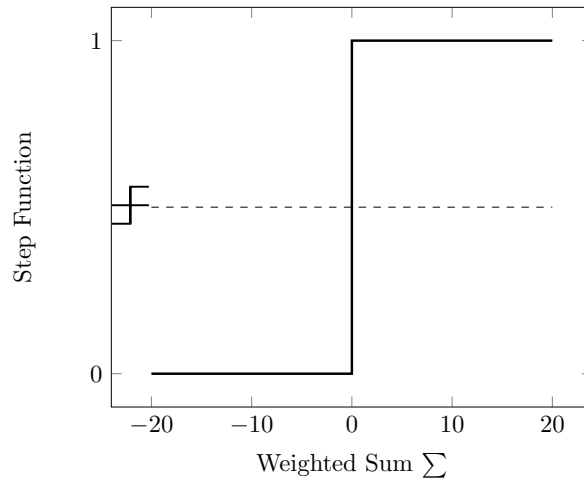


Figure 7.8.: Example of a step function. Output is 0 for any value below zero, and is 1 if above zero.

Before training, the weights are unknown and should be initialized with certain values, e.g. all 0. For each training instance, its values v_1 and v_2 will be fed into the input layer and multiplied with their weights w_1 and w_2 respectively.

7. Classification: From Neighbors to Neural Networks

Note that in Figure 7.7, the input with weight w_0 is always 1 for every training instance. This is referred to as the *bias*, which adds the constant w_0 to the linear function. Once every input is multiplied by its weight, the output computes the sum of the weighted values.

For the shapes data, we can decide that the model will predict positive (standing) if the output is greater than 0; negative (lying) if otherwise. During training, if a correct prediction is made, the model will have no need to adjust its weights (for now) and will simply move to the next training instance.

If a misclassification occurs, however, the model will have to adjust its weights so it can potentially make more correct predictions in the future. When the model predicts negative on an actual positive instance, the weights should be adjusted so the overall output (sum) will move toward the positive side; and vice versa.

For example, for a specific shape instance with v_1 (width) and v_2 (height) values, which is $(1, v_1, v_2)$ with the bias input included. Suppose $v_2 > v_1$ and this instance actually belongs to the positive (standing) class. Now if the model finds out $w_0 + w_1v_1 + w_2v_2 < 0$ and predicts negative (lying). We will add the input values to their current weights:

$$\begin{aligned}w'_0 &\leftarrow w_0 + 1 \\w'_1 &\leftarrow w_1 + v_1 \\w'_2 &\leftarrow w_2 + v_2\end{aligned}$$

where w' is the updated weight. Because $v_2 > v_1$, the weight for height w_2 will increase more than the weight for width w_1 .

Conversely, for a training instance with $v_1 > v_2$ that actually belongs to the negative (lying) class, if it is misclassified as positive, then the current weights will be reduced by the input values:

7.2. Linear Classifiers

$$\begin{aligned}w'_0 &\leftarrow w_0 - 1 \\w'_1 &\leftarrow w_1 - v_1 \\w'_2 &\leftarrow w_2 - v_2\end{aligned}$$

Now that $v_1 > v_2$, the weight for width w_1 will be reduced more than w_2 . The training continues for a number of iterations or until the weights stabilize. With more training instances and misclassifications, the weights will be adjusted further, leading to relatively a greater (positive) value of w_2 and a smaller (negative) value of w_1 .

Theoretically, we know that the classification function for *standing* vs. *lying* should be in the form of $f(v) = 0 - v_1 + v_2$, with w_1 and w_2 having opposite signs. Overall, the adjustment of weights discussed above is moving in the correct direction. As for the *bias*, it will simply be adjusted back and forth with positive and negative misclassifications. For the shapes data, w_0 will be more or less canceled out and its absolute value will become much smaller than those of w_1 and w_2 .

The *perception* model is learning by *mistakes*, as it updates the weights only when there is misclassification. For classes that are linearly separable, the updating strategy is guaranteed to converge on a set of weights.

Put it into practice

Continue with the corresponding practice activity to turn **Perceptron: Toward a Neural Network** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **Linear Classifiers** into a calculation, implementation, visualization, diagnostic, or interpretation task.

7.3. Non-Linear Models

Many machine learning problems in the real world cannot be solved by a linear equation. Consider the car evaluation data in Table 1.2.

Table 7.2.: Car evaluation data. Buying price and maintenance cost are on a scale of 1 to 4, with 1 meaning the lowest price (cost) and 4 the highest.

Buying Price	Maintenance Cost	Evaluation
2	2	Acceptable
2	1	Acceptable
1	2	Acceptable
1	1	Acceptable
4	4	Unacceptable
4	3	Unacceptable
4	2	Unacceptable
4	1	Unacceptable
3	4	Unacceptable
3	3	Unacceptable
3	2	Unacceptable
3	1	Unacceptable
2	4	Unacceptable
2	3	Unacceptable
1	4	Unacceptable
1	3	Unacceptable

In the data, the concept is about whether a car is *acceptable* or *unacceptable* given its *buying price* and *maintenance cost*.³ The price (cost) variables are

³The tiny data set was inspired by the UCI's Car Evaluation collection. However, this simplified version is intended for demonstration only and does not represent the actual data in the UCI repository.

7.3. Non-Linear Models

on an ordinal scale: low, medium, high, and very high, which we convert into an equal-interval numeric scale from 1 (low) to 4 (very high).

While it is possible to include other variables such as the number of doors and safety features, we will focus on the cost factors in the discussion here. Figure 7.9 plots *maintenance cost* vs. *buying price*, with the + indicating an *acceptable* car and ◦ unacceptable. The two classes appear not separable linearly on the two-dimensional space – that is, there is no straight line that can be drawn to separate the two.

Again, if one can introduce other predictor variables and the classes might be separable linearly on a higher-dimensional space. But even with the only two variables, namely *maintenance cost* and *buying price*, the plot seems to suggest that a car will be evaluated *unacceptable* with either a high buying price or a high maintenance cost.

Perhaps one can draw two straight lines, one to separate the classes on the price dimension and the other on the maintenance cost. However, this is no longer one linear equation of the two variables that can fit in with a linear model. Alternatively, we can still draw a curve (a non-linear function) to separate them. For example, as shown as the dashed line in Figure 7.9, a circular function such as $(v_1 - 1)^2 + (v_2 - 1)^2 = 3$ can set the boundary between the two classes. The question is: how can we identify such a non-linear function from the training data?

7.3.1. Kernel SVM

We have discussed the *support vector machines*, which, in its original form, is a linear model. How can we apply SVM to non-linear data? Now instead of finding out a non-linear function, we can think about how the data can be transformed in such a way that they are linearly separable.

In particular, we can map the data onto a higher dimensional space with richer features, where a linear classifier is applicable. Now that we do not have additional variables, how can we create a higher-dimensional space?

7. Classification: From Neighbors to Neural Networks

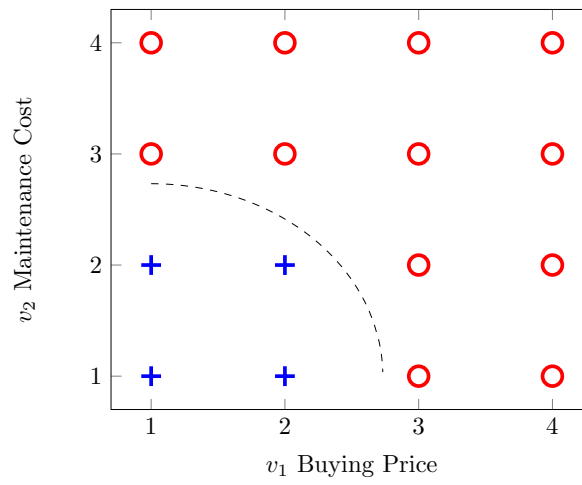


Figure 7.9.: Car evaluation data. + is the positive (acceptable) class and o is the negative (unacceptable).

7.3. Non-Linear Models

The basic idea is to find non-linear combinations of existing variables, via a so-called *kernel* function. We will have an in-depth discussion of *kernel* functions in Chapter . But in the nutshell, a *kernel* function can be computed by a dot product with expanded features and makes it extremely efficient to identify the decision hyperplane in the higher dimensional space.

Among families of kernel functions, a polynomial function is often used. For example, we may consider a quadratic transformation with the car evaluation data. Given the two existing variables, any data instance with $v = (v_1, v_2)$ (a two-dimensional vector) can be expanded onto a vector with more dimensions:

$$\phi(v) = (v_1^2, v_2^2, \sqrt{2}v_1v_2)$$

Note that the quadratic expansion should result in 5 dimensions including the original dimensions of v_1 and v_2 . We focus on the 3 additional features and thus limit the number of dimensions to three, which can be visualized.

With the v_1^2 and v_2^2 transformation, data in the two classes already appear to be linearly separable. As shown in Figure 7.10, the solid line at $v_1 + v_2 - 9 = 0$ can separate the data we have so far. However, this does not necessarily represent the general idea of car evaluation and new data may violate the rule (cross the decision boundary).

Whether a car is acceptable or not may depend on some form of interaction between the buying price and maintenance cost. With the quadratic expansion, the additional dimension of $\sqrt{2}v_1v_2$ can be seen as an interaction variable in the form of multiplication.

When all three expanded features are considered, the data are mapped to a three-dimensional space shown in Figure 7.11, where they may be separated linearly. Of all linear equations (planes) formed by any three points in each class, we have identified two planes closest to their opposite classes.

7. Classification: From Neighbors to Neural Networks

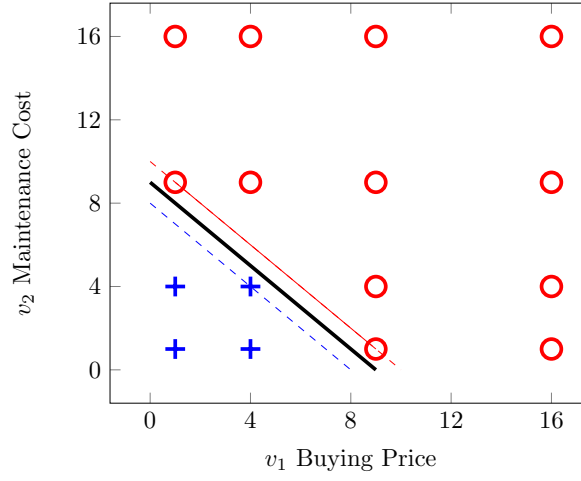


Figure 7.10.: Car evaluation data, with transformed v_1^2 and v_2^2 .

Figure 7.11 (a) shows all data points in the expanded dimensional space with quadratic mapping. The two planes formed by data points are the closest to the opposite class, which are candidate support vectors. Shadows of the planes are projected to the "ground" (i.e. v_2^2 vs. v_1^2 dimensions) for easier reading of related data points.

Figure 7.11 (b) zooms in on the planes formed by candidate support vectors in the three-dimensional space, where a decision boundary can be identified to maximize the margin between the two classes (convex hulls). The dashed line connects the closest points of data in the two classes (convex hulls), by drawing a line from the circled positive point perpendicular to the plane formed by three circled negative points. The four circled points (one positive and three negative) are the identified support vectors, based on which the margin is established and classification can be performed.

Besides polynomial kernels, other popular kernel functions for SVM include

7.3. Non-Linear Models

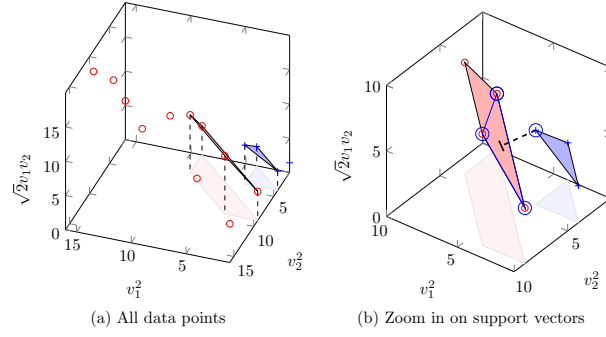


Figure 7.11.: Car evaluation data, mapped to higher dimensions with v_1^2 , v_2^2 , and $\sqrt{2}v_1v_2$. The two planes are closest to the opposite class. The projected shadows are for reference only. (a) shows all data points in the three dimensional space, and (b) is a closer view of the two planes with support vectors. Dashed line is the gap (margin) between the two classes. Circled data points are support vectors.

7. Classification: From Neighbors to Neural Networks

radial basis and string kernels. A string kernel, for example, operates directly on symbol sequences without vectorized features and is useful in applications such as text classification and gene sequence analysis.

Put it into practice

Continue with the corresponding practice activity to turn **Kernel SVM** into a calculation, implementation, visualization, diagnostic, or interpretation task.

7.3.2. Multi-Layer Neural Networks

We know the single perceptron can only be applied to linearly separable data. The car evaluation data, as shown in Figure 7.9, cannot be separated by one linear function. However, with a step function, one can transform the data into something linearly separable. Specifically, for a value s on the v_1 or v_2 dimension, we use the following step function:

$$f(s) = \begin{cases} 1 & \text{if } s > 2.5 \\ 0 & \text{otherwise} \end{cases}$$

So it returns 1 for a value above 2.5, and 0 for a value below. Applying the step function to both v_1 (buying price) and v_2 (maintenance cost), we have transformed data in Figure 7.12. All positive (acceptable) data have been mapped to the (0,0) point whereas the negative (unacceptable) ones are at (0,1), (1,1), and (1,0). As the solid line in Figure 7.12 shows, the transformed data are now linearly separable.

This non-linear classification with step function transformation can be represented with a multi-layer neural network. As shown in Figure 7.13, each of the input variables v_1 and v_2 is normalized by a step function and then multiplied by -1 before feeding into the final output, which is

7.3. Non-Linear Models

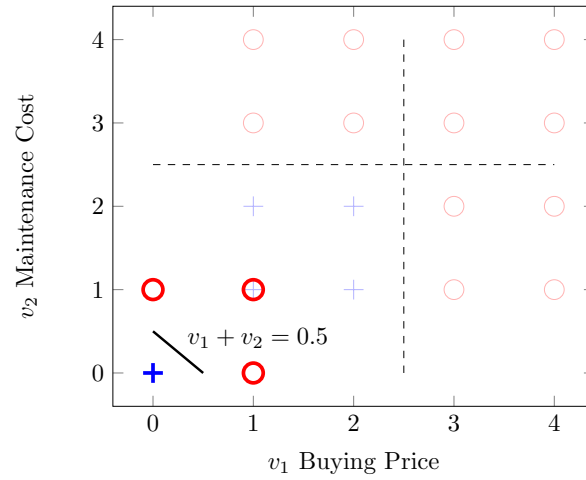


Figure 7.12.: Car evaluation data, transformed by a step function. A step function with a threshold 2.5 converts the original data (+ and o in faded colors) to those with 0 and 1 values (+ and o in bright colors). The solid line at $v_1 + v_2 = 0.5$ separate the two classes.

7. Classification: From Neighbors to Neural Networks

combined with the bias weighted 0.5. This is equivalent to $0.5 \times 1 - 1 \times f(v_1) - 1 \times f(v_2)$, where f is the step function for v_1 and v_2 values.

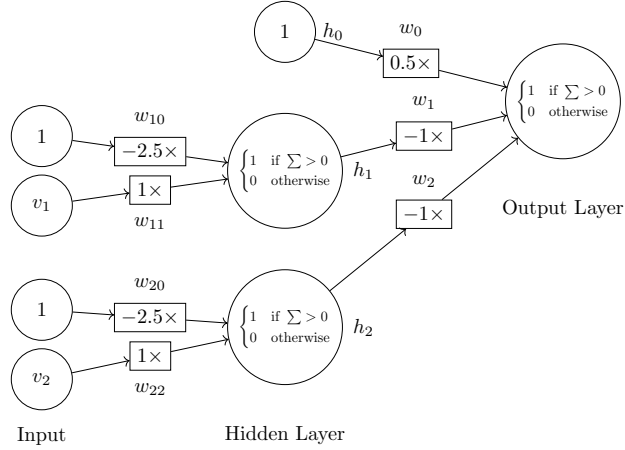


Figure 7.13.: Multi-layer neural network with step functions. $\sum$ represents the weighted sum of input values leading to the present node.

For example, with data point $(2, 2)$, the step function will convert them into $(0, 0)$. With the bias 0.5×1 , the weighted sum is $0.5 \times 1 - 1 \times 0 - 1 \times 0 = 0.5$ and it will be classified positive (acceptable). With data point $(3, 1)$, it is transformed into $(1, 0)$ by the step function and the final weighted sum is $0.5 \times 1 - 1 \times 1 - 1 \times 0 = -0.5$, which will be classified negative (unacceptable).

Using the multi-layer neural network, based on the additional hidden layer with step functions between the input and final output, it is now possible to establish a non-linear model such as the one in Figure 7.13.

But the question remains – how can the weights of the model, together with the step function, be identified properly? While it is tempting to use a similar strategy as with the single perceptron, the non-linearity of the model rules out such a linear strategy for weight adjustment. That is,

7.3. Non-Linear Models

simply adding or subtracting weights (linear combinations) does not help when the model is non-linear.

Can we adjust the weights in a non-linear manner during training? In other words, can we adjust the weights based on how much each variable contributes to the final output via the hidden layer? The general optimization technique called *gradient descent* makes it possible to find optimal values (weights) by searching in the derivatives of related functions. However, a step function, as illustrated in Figure 7.8, has a sudden jump (not smooth transition) between 0 and 1 and is not differentiable.

If we are to retain the idea of a step function, can we develop one that functions in the like manner and yet smooth and differentiable? A candidate is the *sigmoid* function, which is defined by:

$$f(v) = \frac{1}{1 + e^{-v}}$$

for a value v . Figure 7.14 shows the functional curve of sigmoid, which transforms the weighted sum of a node to a value approaching 0 or 1. The threshold is 0 by default but we can use any threshold for the transformation. For example, with a threshold of 5, the sigmoid function becomes:

$$f(v) = \frac{1}{1 + e^{-(v-5)}}$$

which shifts the functional curve on the x coordinates by +5 (dashed line in Figure 7.14). $v - 5 = 1 \times v + (-5) \times 1$ is equivalent to the weighted sum of variable v (with weight $w = 1$) and bias 1 (with weight $w_0 = -5$). It can be generalized as a linear function of any number of input variables with bias in the form of $\sum = w_0 + \sum_i w_i v_i$.

Now if we replace the step function with the sigmoid function in Figure 7.13, we have the multi-layer neural network in Figure 7.15, in which each layer (node) is differentiable and the weight can be learned by gradient descent.

7. Classification: From Neighbors to Neural Networks

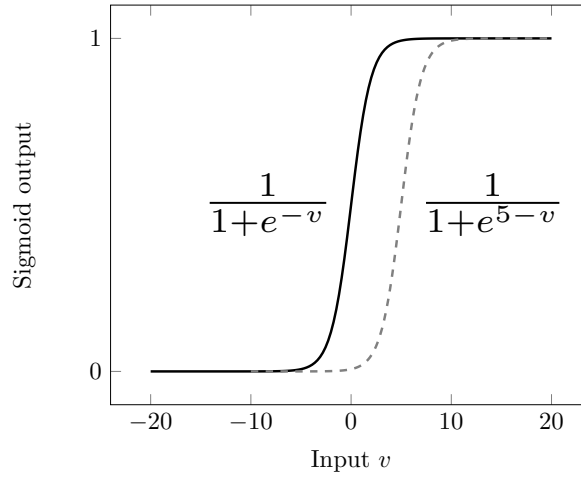


Figure 7.14.: Sigmoid function

The weighted sum for each hidden node i can be computed by:

$$s_i = \sum_{j=0}^m w_{ij} v_j$$

where m is the number of variables and v_0 is the bias, which is always 1. Given the sigmoid function f , the output of each node in the hidden layer is:

$$\begin{aligned} h_i &= f(s_i) \\ &= f\left(\sum_{j=0}^m w_{ij} v_j\right) \end{aligned}$$

In Figure 7.15, this can be written as:

7.3. Non-Linear Models

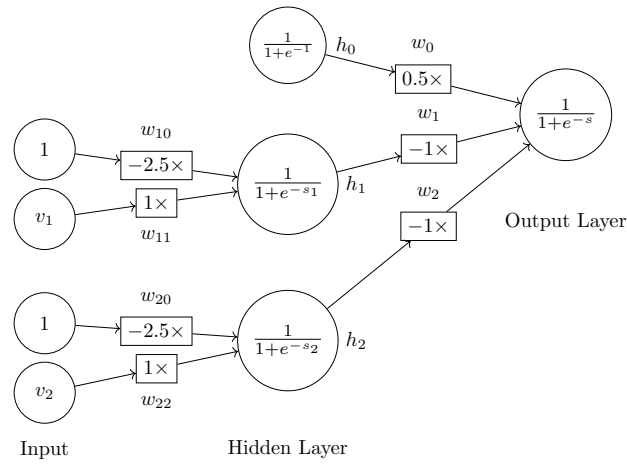


Figure 7.15.: Multi-layer neural network with the sigmoid function. s_1 and s_2 are the weighted sum of input values leading to the respective hidden nodes whereas s is the weighted sum of input values leading to the output layer.

7. Classification: From Neighbors to Neural Networks

$$\begin{aligned}
 h_1 &= f\left(\sum_{j=0}^m w_{1j}v_j\right) \\
 &= f(w_{10}v_0 + w_{11}v_1 + w_{12}v_2) \\
 &= f(-2.5 \times 1 + 1 \times v_2 + 0 \times v_2) \\
 h_2 &= f\left(\sum_{j=0}^m w_{2j}v_j\right) \\
 &= f(w_{20}v_0 + w_{21}v_1 + w_{22}v_2) \\
 &= f(-2.5 \times 1 + 0 \times v_2 + 1 \times v_2)
 \end{aligned}$$

The final output, predicted value $\hat{y}$, is the sigmoid of the weighted sum of values from the hidden layer:

$$\begin{aligned}
 \hat{y} &= f(s) \\
 &= f\left(\sum_{i=0}^m w_i h_i\right) \\
 &= f\left(\sum_{i=0}^m w_i f\left(\sum_{j=0}^m w_{ij}v_j\right)\right)
 \end{aligned}$$

which, for the model in Figure 7.15, can be computed by:

$$\begin{aligned}
 \hat{y} &= f(w_0h_0 + w_1h_1 + w_2h_2) \\
 &= f\left(w_0f(1) \right. \\
 &\quad \left. + w_1f(w_{10} + w_{11}v_1 + w_{12}v_2) \right. \\
 &\quad \left. + w_2f(w_{20} + w_{21}v_1 + w_{22}v_2)\right)
 \end{aligned}$$

Let y be the actual outcome (class 0 or 1) for input from the hidden layer h . If we define the error E as a squared function:

7.3. Non-Linear Models

$$E(h) = \frac{1}{2}(y - \hat{y})^2$$

For a prediction based on the sigmoid function f , it can be shown that the differential with respect to a weight w_i to the output layer is:

$$\begin{aligned} \frac{dE(h)}{dw_i} &= (f(s) - y)f'(s)f(s_i) \\ &= (f(s) - y)f'(s)h_i \end{aligned}$$

where s is the weighted sum at the output layer and $f'(s)$ is the derivative of the sigmoid function $f(s)$, which is as simple as $f(s)(1 - f(s))$. This gives the differentials needed to optimize w_i weights from the hidden layer to the output, i.e. w_0 , w_1 , and w_2 weights in Figure 7.15.

Now that values in the hidden layer h are based on weighted sums of the input layer, we also need to optimize w_{ij} weights connecting the input to the hidden layer. Given input values v , the differential with respect to a weight w_{ij} to hidden node h_i can be computed by:

$$\frac{dE(v)}{dw_{ij}} = (f(s) - y)f'(s)f'(s_i)v_j$$

The differentials will be used to update weights to both layers during the training process. In particular, the differential $\frac{dE(h)}{dw_i}$ in Equation [eq-bin-dwi] will be used to subtract from existing weights of w_0 , w_1 , and w_2 . The differential $\frac{dE(v)}{dw_{ij}}$ in Equation [eq-bin-dwij] will be used to subtract from existing weights of w_{10} and w_{11} for h_1 ; w_{20} and w_{22} for h_2 in Figure 7.15. This approach of revising weights to the two layers can thought of as propagating differentials of squared errors *backward* in the feed-forward neural network, and is referred to as *backpropagation*.

To understand how backpropagation actually operates, let's walk through the training process with the first data instance in Table 1.3.

7. Classification: From Neighbors to Neural Networks

Table 7.3.: Car evaluation, training data for multi-layer neural network.
Evaluation value 1 for *acceptable* and 0 for *unacceptable*.

Buying Price	Maintenance Cost	Evaluation
2	2	1
..	..	..

Before training, we don't know the weights and may initialize them with all 0 values. Note that we use 0 initial weights for easy demonstration. This is not necessarily the best strategy for initialization.

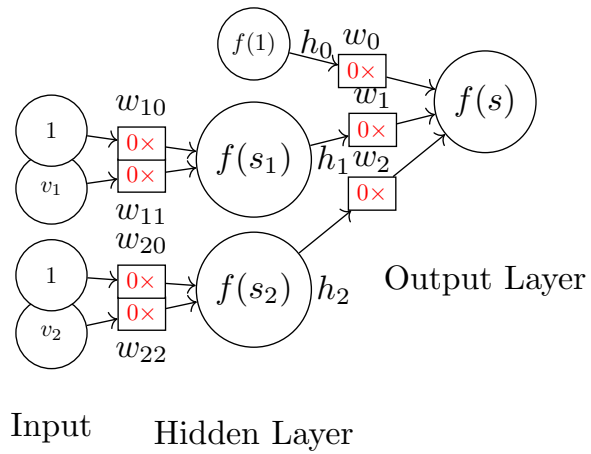


Figure 7.16.: Backpropagation step 1 with 0 initial weights.

Now with the model and initial weights in Figure 7.16, given the first training instance of $(2, 2)$, we can compute:

7.3. Non-Linear Models

$$\begin{aligned}
 h_0 &= f(1) \\
 &= \frac{1}{1 + e^{-1}} \\
 &= 0.731 \\
 h_1 &= f(s_1) \\
 &= f(0 \times 1 + 0 \times 2) \\
 &= 0.5 \\
 h_2 &= f(s_2) \\
 &= f(0 \times 1 + 0 \times 2) \\
 &= 0.5
 \end{aligned}$$

The final prediction $\hat{y}$ is computed by:

$$\begin{aligned}
 f(s) &= f(0 \times 0.731 + 0 \times 0.5 + 0 \times 0.5) \\
 &= f(0) \\
 &= 0.5
 \end{aligned}$$

We get:

$$\begin{aligned}
 (f(s) - y)f'(s) &= (f(s) - y)f(s)(1 - f(s)) \\
 &= (0.5 - 1) \times 0.5 \times (1 - 0.5) \\
 &= -0.125
 \end{aligned}$$

With the actual class being 1, we can compute the differentials. For each w_i weight leading to the output layer:

7. Classification: From Neighbors to Neural Networks

$$\begin{aligned}\frac{dE(h)}{dw_0} &= (f(s) - y)f'(s)h_0 \\ &= -0.125 \times h_0 \\ &= -0.125 \times 0.73 \\ &= -0.0914 \\ \frac{dE(h)}{dw_1} &= -0.125 \times 0.5 \\ &= -0.0625 \\ \frac{dE(h)}{dw_2} &= -0.0625\end{aligned}$$

Subtracting these (negative) values from their current weights respectively, we obtain the new weights $w_0 = 0.0914$, $w_1 = 0.0625$, and $w_2 = 0.625$.

Now we can further back-propagate the error to the w_{ij} weights from the input layer:

7.3. Non-Linear Models

$$\begin{aligned}
\frac{dE(v)}{dw_{10}} &= (f(s) - y)f'(s)f'(s_1)v_0 \\
&= -0.125 \times f(s_1)(1 - f(s_1))v_0 \\
&= -0.125 \times 0.5 \times (1 - 0.5) \times 1 \\
&= -0.0313 \\
\frac{dE(v)}{dw_{11}} &= (f(s) - y)f'(s)f'(s_1)v_1 \\
&= -0.125 \times 0.5 \times (1 - 0.5) \times 2 \\
&= -0.0625 \\
\frac{dE(v)}{dw_{20}} &= -0.0313 \\
\frac{dE(v)}{dw_{22}} &= -0.0625
\end{aligned}$$

Subtracting these from current 0 values, we obtain new weights for of all w_{ij} , namely $w_{10} = 0.0313$ and $w_{11} = 0.0625$ to hidden node h_1 , and $w_{20} = 0.0313$ and $w_{22} = 0.0625$ for hidden node h_2 .

Figure 7.17 shows the model with updated weights after training by the first instance (2, 2) with a known class label 1. We can continue to train the model with additional data instances in Table 1.3. This can be repeated with the same data until weights are stabilized and the error is reduced to a certain level.

Figure 7.18 shows relative weights and error over time, via iterations of repeated training with data in Table 1.3. Note that in linear equations, absolute values of related weights (coefficients) are not important. Rather, it is their relative weight to one another that matters. Hence, we present weights relative to w_0 , i.e. divided by the *bias* weight.

Overall, the figures show the weights and error all appear to converge. However, they seem to differ in the number of iterations needed for their convergence. In Figure 7.18 (a), the relative weight of w_{11} (the weight

7. Classification: From Neighbors to Neural Networks

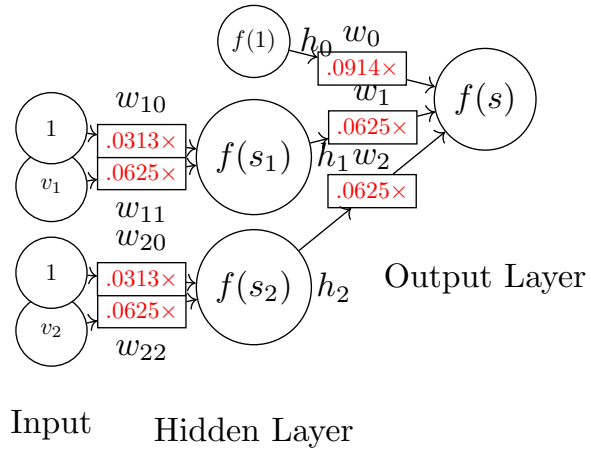


Figure 7.17.: Backpropagation step 2 with updated weights

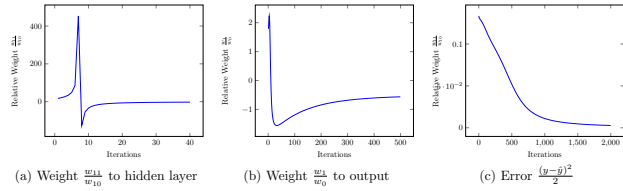


Figure 7.18.: Neural network training iterations and convergence

7.3. Non-Linear Models

of v_1 to the hidden node h_1) fluctuates a lot in the first 10 iterations but stabilizes quickly after a few dozen iterations. In Figure 7.18 (b), w_1 – the weight from the hidden node h_1 to the final output – appears to take hundreds of iterations to converge. The squared error, as shown in Figure 7.18 (c), takes even even more iterations to be minimized.

In the end, we obtain the weights in Table [tab-bin-car_nn_result] with minimized squared error.

Weights	w_0	w_1	w_2	w_{10}	w_{11}	w_{20}	w_{22}	Error
Absolute	-18.4	9.47	9.44	7.35	-2.93	7.43	-2.96	0.0027
Relative	-1	0.52	0.52	1	-0.40	1	-0.40	0.0027
Relative	-1.92	1	1	2.5	-1	2.5	-1	0.0027

The result of Table [tab-bin-car_nn_result] can be represented as the final (converged) model in Figure 7.19, which is highly similar to the one in Figure 7.15. For example, the weighted sum for hidden node h_1 is $2.5 - v_1$ in the model here, which is simply the negative of what is in Figure 7.15, where the sum is $v_1 - 2.5$. We observe the same with the hidden node h_2 . This leads to the different signs of w_1, w_2 vs. w_0 to the output layer.

Figure 7.20 visualizes how the model transforms the original data into something separable. A sigmoid function with a weighted sum of $2.5 - v$ (for both v_1 and v_2) converts the original data (+ and $\circ$ in faded colors) to values close to 0 (if $v > 2.5$) and 1 values (if $v < 2.5$).

Given the limited training data we have, there are many non-linear solutions to the classification problem. In the end, we obtain the model in Figure 7.19 which is different from what we proposed in Figure 7.15. Both models will make perfect classifications on the training data presented in Table 1.3. However, a model thus obtained has not necessarily found the global optimal. In fact, the back-propagation mechanism via gradient descent can converge to a local optimum, which depends on many factors such as the starting point (initial values) and the sequence of training data. The

7. Classification: From Neighbors to Neural Networks

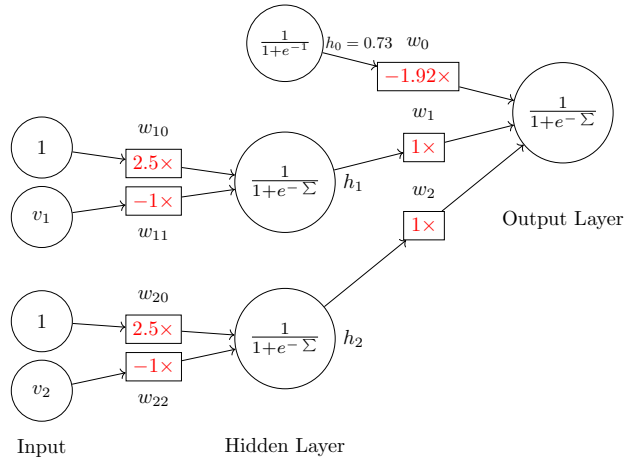


Figure 7.19.: Neural network final model. Compare to the referenced figure. Note that we transform the bias node on the hidden layer with the sigmoid as well, i.e. $h_0 = f(1) = \frac{1}{1+e^{-1}} = 0.73$. In the final model, $w_0 h_0 = -1.92 \times 0.73 = -1.4$.

7.3. Non-Linear Models

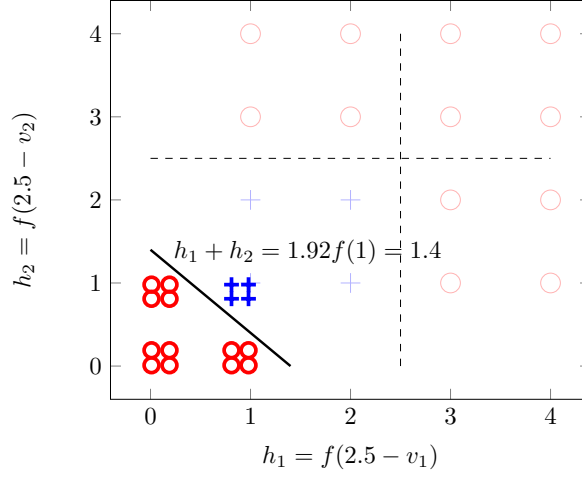


Figure 7.20.: Visualization of the final model on car evaluation. Data transformed by the sigmoid function $h_i = f(s_i) = \frac{1}{1+e^{-s_i}}$, where s_i is the weighted sum of each hidden node: $s_1 = 2.5 - v_1$ and $s_2 = 2.5 - v_2$. The solid line at $h_1 + h_2 - 1.92f(1) = 0$ is where the sigmoid of the output layer makes the cut.

7. Classification: From Neighbors to Neural Networks

learning rate also impacts the convergence of weights and, occasionally, may also influence where they converge.

Put it into practice

Continue with the corresponding practice activity to turn **Multi-Layer Neural Networks** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **Non-Linear Models** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8. Multiclass and Structured Decisions

8.1. From Binary to Multiple Classes

 Put it into practice

Continue with the corresponding practice activity to turn **From Binary to Multiple Classes** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8.2. One-vs-Rest and One-vs-One Strategies

 Put it into practice

Continue with the corresponding practice activity to turn **One-vs-Rest and One-vs-One Strategies** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8.3. Scores, Probabilities, and Softmax

 Put it into practice

Continue with the corresponding practice activity to turn **Scores, Probabilities, and Softmax** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8.4. Decision Trees and Rule-Based Decisions

 Put it into practice

Continue with the corresponding practice activity to turn **Decision Trees and Rule-Based Decisions** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8.5. Multilabel and Ordinal Outcomes

 Put it into practice

Continue with the corresponding practice activity to turn **Multilabel and Ordinal Outcomes** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8.6. Error Analysis and Calibration

 Put it into practice

Continue with the corresponding practice activity to turn **Error Analysis and Calibration** into a calculation, implementation, visualization, diagnostic, or interpretation task.

8.7. Summary, Exercises, and Lab

9. Numeric Prediction and Regression

9.1. Prediction with Continuous Outcomes

 Put it into practice

Continue with the corresponding practice activity to turn **Prediction with Continuous Outcomes** into a calculation, implementation, visualization, diagnostic, or interpretation task.

9.2. Linear Regression

 Put it into practice

Continue with the corresponding practice activity to turn **Linear Regression** into a calculation, implementation, visualization, diagnostic, or interpretation task.

9. Numeric Prediction and Regression

9.3. Loss Functions and Fitting

 Put it into practice

Continue with the corresponding practice activity to turn **Loss Functions and Fitting** into a calculation, implementation, visualization, diagnostic, or interpretation task.

9.4. Regularization

 Put it into practice

Continue with the corresponding practice activity to turn **Regularization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

9.5. Nonlinear Regression

 Put it into practice

Continue with the corresponding practice activity to turn **Nonlinear Regression** into a calculation, implementation, visualization, diagnostic, or interpretation task.

9.6. Uncertainty and Diagnostics

 Put it into practice

Continue with the corresponding practice activity to turn **Uncertainty and Diagnostics** into a calculation, implementation, visualization, diagnostic, or interpretation task.

9.7. Summary, Exercises, and Lab

10. Convolutional Neural Networks: Learning Spatial Structure

An image is numerical data, but it is not merely a list of numbers. A pixel's meaning depends partly on the pixels around it. A short dark stroke beside another short dark stroke may form a corner; several corners may form a digit; and several parts may form an object. A useful image model should preserve this spatial structure long enough to learn from it.

The classification chapter introduced multilayer neural networks as nonlinear classifiers. This chapter changes the *architecture* of that network to match image data. A **convolutional neural network** (CNN) uses small, shared sets of weights to find local patterns wherever they occur. The same ideas apply to other grid-like data, but images provide the clearest introduction.

By the end of the chapter, you should be able to:

- explain why immediately flattening an image discards useful assumptions;
- calculate a two-dimensional cross-correlation with a small kernel;
- track height, width, and channels through convolution and pooling layers;
- calculate output sizes and parameter counts; and
- explain what a CNN learns and how its learned features support classification.

10.1. Why Image Structure Changes the Model

Consider an MNIST image: a 28×28 grid of grayscale values. A multilayer perceptron (MLP) reshapes that grid into a vector of length 784 before applying a dense layer. This is a valid baseline, but the reshaping removes the model's direct knowledge that two values were neighbors in the original image.

Flattening does not erase pixel values. It erases the *geometry encoded by their arrangement*. The dense layer can still learn spatial relations, but it must learn each relation through separate weights. A stroke near the upper-left and the same stroke near the lower-right reach different input positions and therefore different weights.

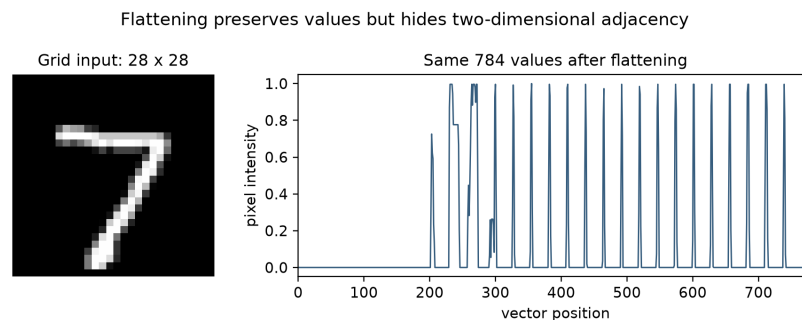


Figure 10.1.: Flattening preserves every pixel value but converts two-dimensional neighborhoods into positions along a one-dimensional vector.

A CNN introduces two useful assumptions:

1. **Locality.** Nearby values are especially likely to form a meaningful pattern, so an early unit can examine a small neighborhood instead of the entire image.
2. **Weight sharing.** A useful pattern can matter in more than one location, so the same detector can scan across the image.

These assumptions are an **inductive bias**: a deliberate restriction that helps the learner when it agrees with the data. A dense network asks, “Which input position connects to which hidden unit?” A convolutional layer asks, “Does this small pattern occur here?” CNNs were developed into effective systems for handwritten-document recognition well before the present deep-learning era (LeCun et al. 1998), and locality and weight sharing remain their defining ideas (Zhang et al. 2023).

 Put it into practice

Establish the comparison point in Part I of the CNN practicum, then explain what information the flattening step retains and what structural assumption it removes.

10.2. Images as Tensors

We will describe shapes in **channels-last** order because that is the default in the accompanying Keras labs:

Object	One example	A batch of B examples
Grayscale image	$H \times W \times 1$	$B \times H \times W \times 1$
RGB color image	$H \times W \times 3$	$B \times H \times W \times 3$
Feature maps inside a CNN	$H \times W \times C$	$B \times H \times W \times C$

Here H is height, W is width, and C is the number of channels. The batch axis says how many examples are processed together; it does not describe one image.

The word *channel* needs care. An RGB image has three input channels because each pixel has red, green, and blue values. A convolutional layer

10. Convolutional Neural Networks: Learning Spatial Structure

may produce dozens of output channels, each recording the response of a different learned filter. Channels coexist within one layer; they are not separate network layers.

For MNIST, one raw example has shape 28×28 . We explicitly add a channel axis to obtain $28 \times 28 \times 1$. The numerical values have not changed; the shape now states that there is one value at each spatial position.

💡 Put it into practice

Use the CNN practicum setup to diagnose several common shape errors before a framework reports them.

10.3. A Kernel as a Small Shared Detector

A **kernel** or **filter** is a small array of weights. It moves across an input, one local window at a time. At each location, we multiply corresponding input and kernel entries and add the products. The resulting number reports how strongly that local window matches the kernel.

For a single-channel input $\mathbf{X}$ and a kernel $\mathbf{K}$, the operation used by most deep-learning libraries is

$$Y_{i,j} = b + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} K_{u,v} X_{i+u,j+v}, \quad (10.1)$$

where b is a learned bias. Strict mathematical convolution flips the kernel before sliding it. Equation 10.1 does not, so it is formally **cross-correlation**. Deep-learning practice nevertheless calls the layer a convolutional layer. Because its weights are learned, the distinction does not limit what pattern the layer can learn.

10.3.1. A Complete Small Example

Let the 4×4 input be

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}.$$

The kernel gives a large positive response where a light region changes to a dark region from left to right. At the first position, the window is all ones, so the response is

$$1(1) + 1(-1) + 1(1) + 1(-1) = 0.$$

One step to the right, the window crosses the boundary:

$$1(1) + 0(-1) + 1(1) + 0(-1) = 2.$$

Sliding over every valid position produces the **feature map**

$$\mathbf{Y} = \begin{bmatrix} 0 & 2 & 0 \\ 0 & 2 & 0 \\ 0 & 2 & 0 \end{bmatrix}.$$

The vertical line of twos marks the detected boundary. Notice the economy: the same four weights were used at all nine output positions. In a trained CNN we usually do not hand-design these values. Backpropagation adjusts them to reduce the task loss, just as it adjusts dense-layer weights. An activation such as ReLU, $\max(0, Y_{i,j})$, is then commonly applied to introduce nonlinearity.

10. Convolutional Neural Networks: Learning Spatial Structure

Hand-designed kernels are nevertheless useful for learning. In Figure 10.2, vertical, horizontal, and diagonal kernels produce bright responses where a local stroke agrees with their weights. The same kernel responds at multiple positions because its weights are shared.

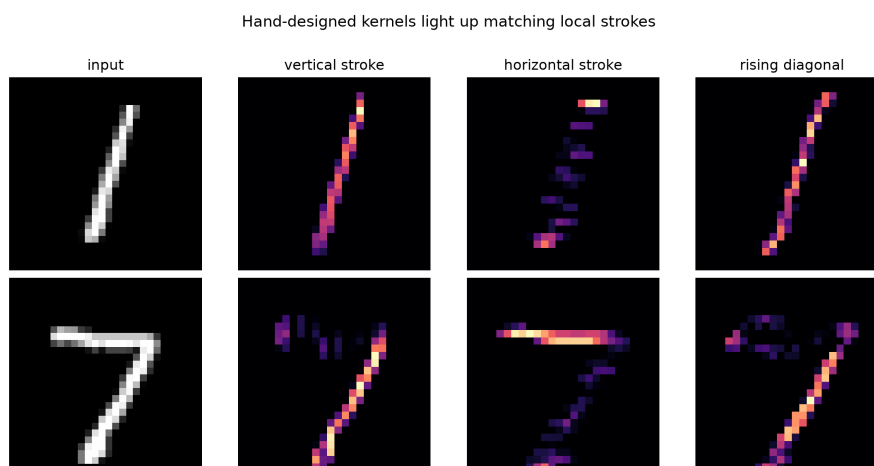


Figure 10.2.: Hand-designed kernels applied to MNIST examples of 1 and 7. Bright positions are positive local responses after ReLU, not class probabilities.

A crucial caution follows from the figure: several kernels respond to parts of both digits. One filter reports local evidence; it does not decide the class. Classification emerges from combining many feature maps across layers.

💡 Put it into practice

Reproduce every number, alter the input and filter, and write a general NumPy implementation in Part II of the CNN practicum.

10.4. Padding, Stride, and Output Size

Three choices determine how the kernel moves and how large the feature map is:

- **Kernel size** (K_h, K_w) controls the local window.
- **Padding** (P_h, P_w) adds values, usually zeros, around the boundary.
- **Stride** (S_h, S_w) controls how many positions the window moves at a time.

With unit dilation, the output height and width are

$$H_{out} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1. \quad (10.2)$$

For a 28×28 image, a 3×3 kernel, no padding, and stride 1, the result is 26×26 . Adding one row or column of padding on each side gives 28×28 . Frameworks call these choices `padding="valid"` and `padding="same"`, respectively, when stride is one.

A larger stride reduces resolution because the layer visits fewer windows. For the same 28×28 input, 3×3 kernel, padding 1, and stride 2, Equation 10.2 gives 14×14 . This reduction saves computation, but it also discards spatial detail. Output-shape arithmetic is therefore part of model design, not merely bookkeeping.

 Put it into practice

Predict the output before running the code in Part II of the CNN practicum, then use assertions to check the formula.

10.5. From One Channel to Many Features

A filter must span all input channels. For an input with C_{in} channels, one filter has shape $K_h \times K_w \times C_{in}$. Its responses across those channels are summed, so one filter produces one two-dimensional feature map. Using C_{out} different filters produces C_{out} output channels:

$$Y_{i,j,d} = b_d + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c=0}^{C_{in}-1} K_{u,v,c,d} X_{i+u,j+v,c}. \quad (10.3)$$

The full kernel bank therefore has shape $K_h \times K_w \times C_{in} \times C_{out}$. Including one bias for each output channel, its parameter count is

$$(K_h K_w C_{in} + 1) C_{out}. \quad (10.4)$$

Suppose a $28 \times 28 \times 1$ image enters a layer with 16 same-padded 3×3 filters. The output is $28 \times 28 \times 16$, and the layer learns $(3 \cdot 3 \cdot 1 + 1)16 = 160$ parameters. A following layer with 32 filters sees all 16 incoming feature maps. Its output has 32 channels, and it learns $(3 \cdot 3 \cdot 16 + 1)32 = 4,640$ parameters.

This resolves a common confusion:

- the number of **input channels** determines each filter's depth;
- the number of **filters** determines the number of output channels; and
- the number of **layers** tells us how many transformations are composed.



Put it into practice

Track channels and verify parameter counts in Part II of the CNN practicum, then verify the parameter count against a Keras model summary.

10.6. Pooling and Growing Receptive Fields

A pooling layer summarizes each local window without learning kernel weights. **Max pooling** keeps the largest activation; **average pooling** keeps the mean. With a 2×2 window and stride 2, a 28×28 feature map becomes 14×14 .

Pooling has two practical effects. It lowers the cost of later layers, and it makes their representations less sensitive to small changes in feature location. This is useful stability, but it is not perfect invariance: moving or changing an input can still change the result, especially near a pooling boundary. Aggressive pooling can also erase small but important details.

The **receptive field** of an activation is the region of the original input that could affect it. One 3×3 layer sees a 3×3 neighborhood. Two stride-one 3×3 layers compose information from a 5×5 region. Deeper layers can therefore combine short strokes into corners, corners into parts, and parts into larger patterns. This hierarchy is the main conceptual reason to stack convolutional layers.

Put it into practice

Compare convolution and max pooling on the same feature map in Part II of the CNN practicum, including a case where pooling removes useful detail.

10.7. From Feature Maps to a Class Prediction

A small image classifier usually has two conceptual blocks:

1. a **feature extractor** made of convolution, activation, and downsampling;

10. Convolutional Neural Networks: Learning Spatial Structure

2. a **classification head** that converts the learned representation into class scores.

The following compact MNIST model will be used in the companion labs. **same** padding keeps the spatial size during convolution, and each 2×2 pooling layer halves it.

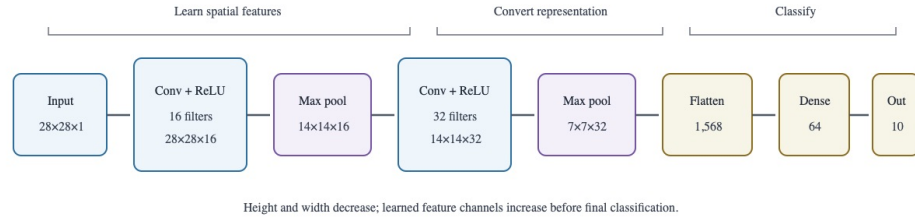


Figure 10.3.: A compact MNIST CNN preserves spatial structure through two convolution and pooling blocks, then flattens the learned feature maps for dense classification.

Stage	Output shape	Trainable parameters
Input	$28 \times 28 \times 1$	0
16 filters, 3×3 , ReLU	$28 \times 28 \times 16$	160
Max pool, 2×2	$14 \times 14 \times 16$	0
32 filters, 3×3 , ReLU	$14 \times 14 \times 32$	4,640
Max pool, 2×2	$7 \times 7 \times 32$	0
Flatten	1,568	0
Dense, 64 units, ReLU	64	100,416
Dense, 10 units, softmax	10	650
Total		105,866

The final dense layer produces one score per digit. Softmax converts the scores to a probability distribution, and multiclass cross-entropy measures the loss. Backpropagation carries the resulting error through the dense

10.7. From Feature Maps to a Class Prediction

head and the convolutional feature extractor. Thus a filter is not rewarded for looking like an edge detector; it is rewarded when its responses help the whole model predict the correct label.

The corresponding MLP lab uses $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$, or 109,386 parameters. The two models are similar in size. If their behavior differs, the comparison highlights architectural bias rather than simply giving one model a much larger parameter budget.

In one controlled full-MNIST run of the companion practicum, the MLP reached 97.56% test accuracy and made 244 errors; the CNN reached 98.75% and made 125 errors. The CNN therefore reduced the observed error count by 119, or about 49%, while using slightly fewer parameters. This is a case study, not a universal performance guarantee, but it is consistent with the benefit of matching the architecture to image structure.

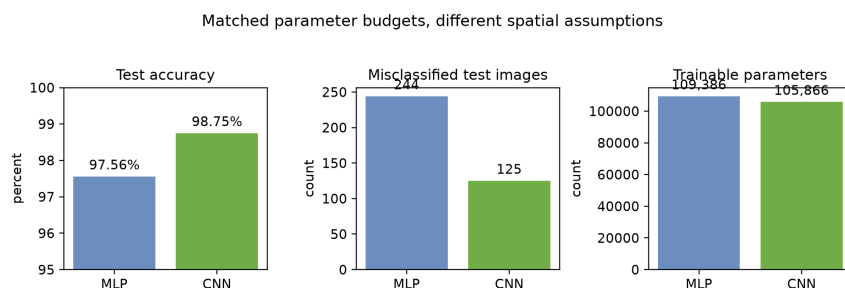


Figure 10.4.: A matched comparison of MNIST test accuracy, test errors, and parameter count.

Dropout, early stopping, and validation curves help diagnose overfitting, but they do not replace a final evaluation on untouched test data. These issues are developed further in Generalization, Model Selection, and Fit. In the matched run above, training loss kept falling after validation loss reached its minimum for both models. That divergence is stronger evidence of overfitting than training accuracy by itself; the companion practicum plots and interprets the complete curves.

10. Convolutional Neural Networks: Learning Spatial Structure

Put it into practice

Train both models under a shared protocol in Part I and Part III of the CNN practicum, then compare accuracy, parameter count, learning curves, errors, and training cost.

10.8. Looking Inside and Knowing the Limits

The output of each filter is observable. We can pass an image through a trained model and plot selected feature maps. Early maps often respond to simple local changes; deeper maps combine earlier responses and are harder to name. Such plots are useful evidence about model behavior, but a bright activation is not automatically a human-interpretable explanation.

Figure 10.5 compares two learned first-layer kernels on examples of 1 and 7. The channels were selected using their *average relative activation* across all test examples of those two classes, rather than because one image looked persuasive. Even then, neither map is a literal “1 detector” or “7 detector.” Each is one component of a distributed representation.

CNNs are also not universally appropriate. Their local, shared filters work well when the same kind of nearby pattern can matter across locations. They are less natural when every feature position has a permanently different meaning, as in some tabular problems. Standard convolution is translation *equivariant*: shifting an input tends to shift its feature map. Pooling and final aggregation may add some stability, but the complete classifier is not guaranteed to ignore all translations, rotations, scale changes, or background shifts.

Useful extensions include data augmentation, residual connections, and transfer learning from a pretrained feature extractor. These can greatly expand a CNN’s reach, but they do not change the central lesson of this

10.8. Looking Inside and Knowing the Limits

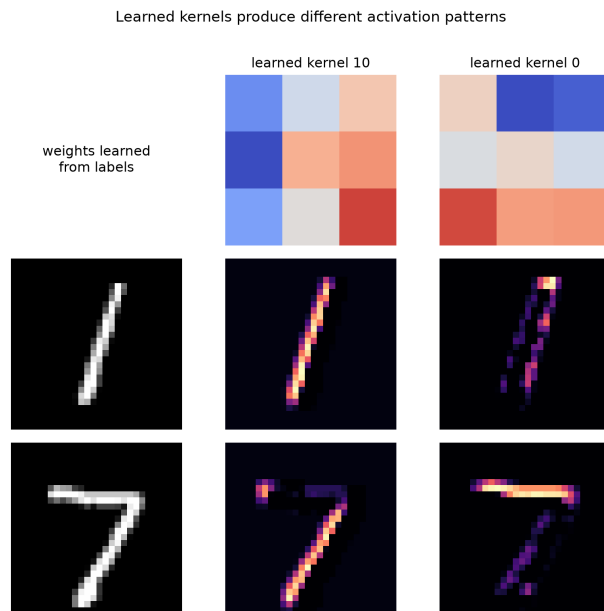


Figure 10.5.: Two learned first-layer kernels and their activation maps for representative handwritten 1 and 7 examples.

10. Convolutional Neural Networks: Learning Spatial Structure

chapter: architecture encodes assumptions about the structure of the data.



Put it into practice

Inspect hand-designed kernels, learned feature maps, and representative mistakes in Part IV of the CNN practicum. State what each visualization supports and what it does not prove.

10.9. Summary

- An MLP can classify images, but immediate flattening does not directly exploit spatial neighborhoods.
- A convolutional layer applies a small shared kernel across locations. The library operation is usually cross-correlation, though it is called convolution.
- Padding, stride, and kernel size determine spatial output dimensions.
- Each filter spans all input channels and produces one output channel.
- Pooling reduces resolution, while stacked layers grow receptive fields and combine simple local patterns into larger ones.
- CNN filters are learned end to end from the task loss; they are not normally prescribed by hand.
- A fair MLP-versus-CNN comparison controls the data split, training protocol, evaluation procedure, and approximate parameter budget.

10.10. Exercises

1. Apply the kernel in Section 10.3.1 after transposing only the input. Then transpose only the kernel. Explain why these are different experiments.

10.10. Exercises

2. Find the output shape for a $32 \times 32 \times 3$ input, 24 filters of size 5×5 , padding 2, and stride 2. Count the trainable parameters.
3. A layer receives 12 input channels and produces 20 output channels. How many separate two-dimensional kernel slices are learned for a 3×3 layer?
4. Compare two successive 3×3 stride-one convolutions with one 5×5 convolution. What receptive field do they share, and how do their nonlinearities and parameter counts differ?
5. Give one task for which translation equivariance is useful and one for which absolute position should matter. Explain how that changes your model design.

11. Clustering and Organization

We as human organize things. An important aspect of human development is to recognize the commonality of objects and group them together, for example, in terms of shapes and colors.

A related category of machine learning algorithms is called *clustering*, which aims to bring like entities together and, by doing so, discovers major groups, patterns, and themes in the data. Clustering is often referred to as unsupervised learning because its outcome is not predetermined (unsupervised) but whatever the data shall reveal.

Clustering helps in the organization (grouping) of information and makes it easier to access and navigate. For example, when a student organizes course materials in folders of subject areas, it becomes more efficient for her to locate related materials when she needs to review them.

Clustering is also an important step toward information aggregation and pattern recognition of data at hand. By discovering major groups and themes, a high-level summarization and overview can be identified with data instances representative of these groups. This can often be integrated with interactive information visualization for graphical representation and navigation of the data space.

We often think of clustering as *hard* and *flat partitioning* – *hard* because cluster membership is concrete and every data instance belongs to only one group and *flat* because there is only one level of clusters. Figure 11.1 shows an example of two flat partitions, where data belong to either one of them. However, the result of clustering can be a more complex structure than that.

11. Clustering and Organization

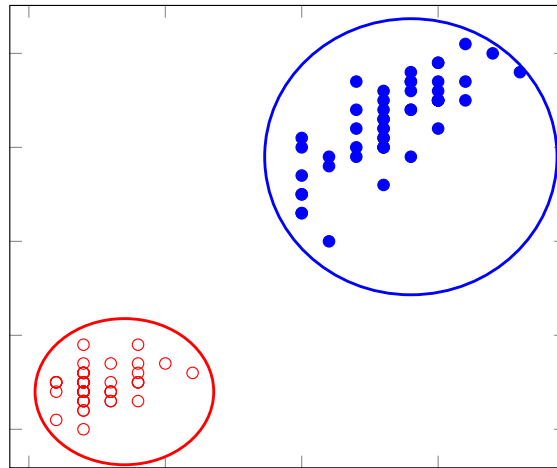


Figure 11.1.: Hard, flat partitioning

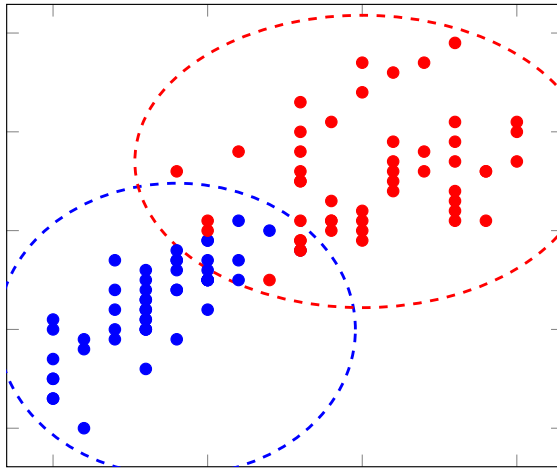


Figure 11.2.: Illustration of soft partitioning, where data may belong to multiple clusters

11. Clustering and Organization

There are such methods for *soft* or *fuzzy* clustering, where each data instance can belong to multiple clusters. For example, a book on quantum computing can group with other books on quantum physics as well as those in computer science/engineering. The membership is not a concrete assignment but a degree of association. One can certainly view such cluster associations as probabilities. Soft clustering can be very helpful in applications where one may need different *routes*, so to speak, to data of interest.

In addition, there are clustering algorithms that produce a hierarchical structure with levels of nested partitions, i.e. a tree with branches and branches of branches. Figure 11.4 illustrates hierarchical clustering, where a circle represents a cluster and a nested circle is a child (branch) of the one containing it. This can be achieved by either recursively merging data instances until the whole structure has emerged (the bottom-up approach) or recursively dividing the entire data into subsets until they can no longer be broken down (the top-down approach).

The result of hierarchical clustering in Figure 11.3 can also be represented as a tree structure in Figure 11.4. A hierarchy or dendrogram thus constructed may reveal the underlying taxonomy of themes in the data. It is a great tool for information organization and data exploration.

11.1. Example Data

Now that we have had an overview of what clustering does and what it can produce, we shall examine how related algorithms perform clustering on data. We will look at a few classic methods and illustrate the clustering process with a well-known dataset, namely the IRIS flower data created by British statistician and geneticist Ronald Fisher. Table shows a small sample of the data set:

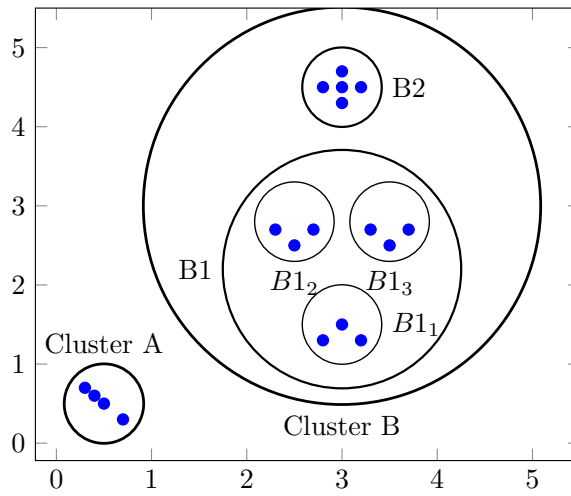


Figure 11.3.: Hierarchical clustering. A nested circle represents child (branch) of the enclosing cluster.

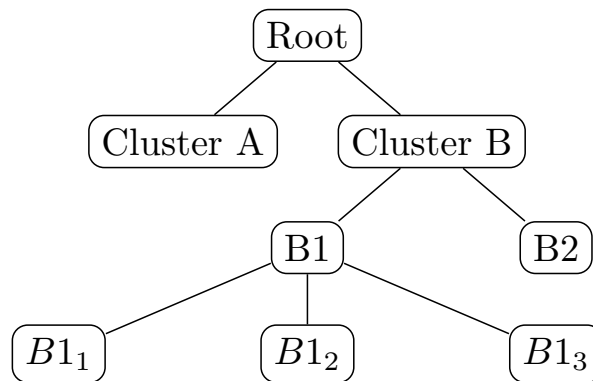


Figure 11.4.: Tree representation of the hierarchical structure in the referenced figure

11. Clustering and Organization

Table 11.1.: IRIS flowers sample data

Sepal Length	Sepal Width	Petal Length	Petal Width	Class
5.1	3.5	1.4	0.2	Setosa
7.0	3.2	4.7	1.4	Versicolor
6.3	3.3	6.0	2.5	Virginica
..	..	..	..	..

Each instance is an Iris flower observation, with measurements of its sepal length, sepal width, petal length, and petal width in centimeters, as well as classification of three Iris species (setosa, versicolor, or virginica). For demonstration, we use only 5 data instances from each species (15 total) and focus on two variables in the analysis, namely *petal length* vs. *petal width*. In Figure 11.5, we plot *petal length* vs. *petal width* and color code data in each class (species).

There appear to be three clusters, as shown in Figure 11.5, consistent with the three species. But how can we design algorithms to identify these clusters? Bear in mind that any clustering method, as unsupervised learning, should be data driven and should aim to discover inherent patterns from rather than imposing a structure on the data.

Although clustering does not predict on the class label, we will use the classes of the IRIS data to help evaluate (at least visually) how well a clustering method has performed. In the end, if each identified cluster can be related to one of the species, then the clustering process must have done something right with regard to the data.



Put it into practice

Continue with the corresponding practice activity to turn **Example Data** into a calculation, implementation, visualization, diagnostic, or

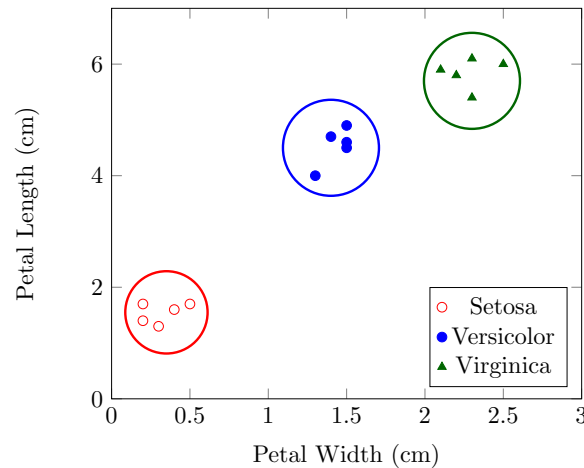


Figure 11.5.: IRIS flowers data: Petal Length vs. Petal Width

interpretation task.

11.2. Hierarchical Clustering

Hierarchical clustering is primarily used to create a tree structure of the data. Nonetheless, one can cut the tree at a specific level to obtain a desired number of clusters. As we discussed, there are two ways to construct a hierarchical structure: 1) the top-down divisive approach and 2) the bottom-up approach. We will first discuss a classic bottom-up method, namely Hierarchical Agglomerative Clustering (HAC), and come back to the top-down approach in the next section on .

In the bottom up approach, we start with each data instance as a separate cluster and repeatedly merge the closest pair of clusters. Given N instances,

11. Clustering and Organization

the initial number of clusters will be N and this number will be reduced by 1 whenever two clusters are joined together as one. The process will continue until the number of clusters have been reduced to a (predefined) desired number k , or, if the objective is to build the entire hierarchical structure, until all clusters have been merged into one (the root).

This process is simple but we should clarify on two aspects before a demonstration with the IRIS data. First, how do we measure distances in order to determine which two are the *closest*? There are many distance and similarity functions one can use. Among others, we discuss the Euclidean distance as a commonly used measure in Chapter and cosine similarity for text analysis in Chapter [ch-text]. One should carefully evaluate how suitable a measure is with the data before adopting it.

Second, when two clusters are merged to become one, how do we represent the new cluster so that we can measure its distance (or similarity) to other clusters? A basic strategy is to compute the centroid of the new cluster using vector values of member data instances. And this is the strategy that we will use in the following demonstration as it is easy to visualize. We will discuss several other strategies after that.

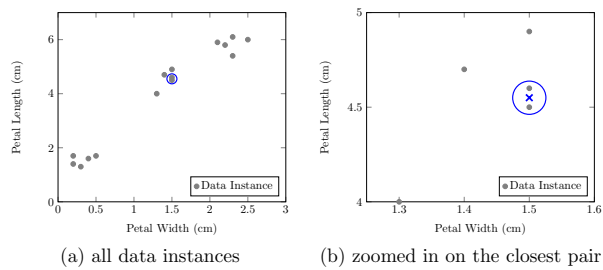


Figure 11.6.: Step 1 of HAC clustering on IRIS data. The circle indicates a new cluster formed by the closest pair of instances. The centroid of the cluster is marked $\times$ in (b).

Figure 11.6 (a) plots the IRIS data and visualizes the 1st step of HAC

11.2. Hierarchical Clustering

clustering. Using Euclidean distance, the closest pair of instances can be identified, as circled on the plot, and joined as a new cluster.

Figure 11.6 (b) offers a closer look at the new cluster. Once the new cluster is formed, it is represented by the average values of its members, i.e. the geometric center of the two data instances on the plot (see $\times$ in the circled cluster). Now this new cluster with a centroid at $\times$ is simply treated as another data instance that can be compared to and merged with others.

Now we can continue to find the next closest pairs. In Figure 11.7 (a), the 1st merged cluster is found to be closest to yet another data instance, and they are joined to form a new cluster again. This continues to merge with another instance, as shown in Figure 11.7 (b).

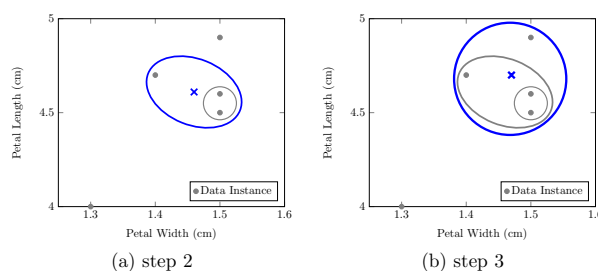


Figure 11.7.: Further iterations of HAC clustering on IRIS data

Repeat the merging process, we can get results in Figure 11.8. If the desired number of clusters k is predefined, one can stop the iterative process once k clusters have been obtained. In Figure 11.8 (a), for example, the clustering process stops at 3 clusters ($k = 3$). If one desires to build the entire hierarchy of the data, as Figure 11.8 (b) shows, the process can continue until all clusters have been merged into the root ($k = 1$).

11. Clustering and Organization

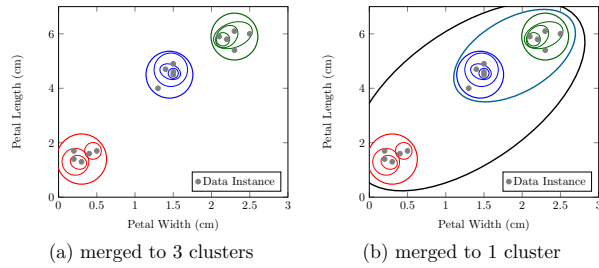


Figure 11.8.: Result of HAC clustering on IRIS data

11.2.1. Issues of HAC

In the above demonstration, we have relied on the notion of *centroid* (with average values of cluster members) when we compute the distance between clusters. Regardless of the distance or similarity function being used, there are several alternative strategies for measuring intra-cluster distances or similarities.

Average link, for example, compare each data instance in one cluster to another instance in the other cluster, and computes the average distance (or similarity) of all pair-wise distances.

Single link does the same pair-wise comparisons between members of two clusters, but singles out the closest or most similar pair. The shortest distance (or the greatest similarity) is then used as the distance (or similarity) of the two clusters. *Complete link*, on the other hand, identifies the farthest pair and uses the greatest distance (or the least similarity) as the distance (or similarity) of the two clusters.

Hierarchical Agglomerative Clustering (HAC) is not an efficient method and does not scale well with large data sets. Consider the amount of computation during each iteration of HAC. Given N data instances – which will later include merged clusters as pseudo-instances – it requires $N \times (N - 1)/2$ pair-wise comparison to identify the closest pair, which is

11.3. K-means Partitioning

$O(N^2)$ complexity. When we factor in the number of iterations, which is proportional to $\log_2 N$ as each iteration reduces two clusters into one, the time complexity of the entire process is of $O(N^2 \log N)$.

Suppose we can manage to perform HAC on $N = 1000$ data instances, say in 1 minute. For $N = 10^6$ data instances, it will take $(\frac{10^6}{10^3})^2 \log \frac{10^6}{10^3} \approx 10,000,000$ minutes (about 19 years). On large volumes of data, HAC can be costly to perform and one has to find an alternative to hierarchical clustering in a cost-effective way.

 Put it into practice

Continue with the corresponding practice activity to turn **Issues of HAC** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Hierarchical Clustering** into a calculation, implementation, visualization, diagnostic, or interpretation task.

11.3. K-means Partitioning

We shall now look at *k-means*, a more efficient method for flat partitioning but, as we will discuss later, can also be used for hierarchical clustering as well. In the nutshell, k-means is a cluster optimization technique that improves on initial (random) clusters through iterations of cluster membership assignment and adjustment.

In the beginning when clusters are unknown, it picks a number of random cluster centroids and assigns every data instance to a centroid depending on its proximity (similarity). Once all cluster membership has been determined,

11. Clustering and Organization

the centroid (center) of each cluster can be re-computed based on its assigned member instances. With the re-computed centroids, data instances will be reassigned, which will lead to further change of cluster centroids. So the process will continue on with centroid adjustment and cluster membership re-assignment, until the centroids no longer move (or move very little) or it has gone through a sufficient number of iterations.

Now let's look at how k-means may identify clusters from the IRIS data. Again, we set the number of desired clusters $k = 3$ to see whether it may produce clusters consistent with the three Iris species.

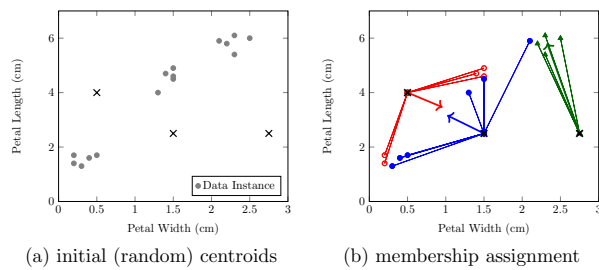


Figure 11.9.: K-means initial steps. A $\times$ represents an initial centroid. Undirected lines represent cluster membership. Directed lines $\nearrow$ indicates how the centroid will move (change) after re-computation based on assigned members.

With data plotted in Figure 11.9 (a), we start with 3 random cluster centroids, shown as $\times$ in the plot. For each data instance, we compute its distance to each of the three initial centroids and identify the closest centroid, to which it will be assigned as a member. Figure 11.9 (b) shows lines connecting data instances to their corresponding closest centroids.

Once membership of each cluster is assigned, the center of cluster can be re-computed based on averaging member instances. After this computation, the centroids will likely change (move). Directed lines (arrows) in Figure 11.9 (b) represent the changes of cluster centroids.

11.3. K-means Partitioning

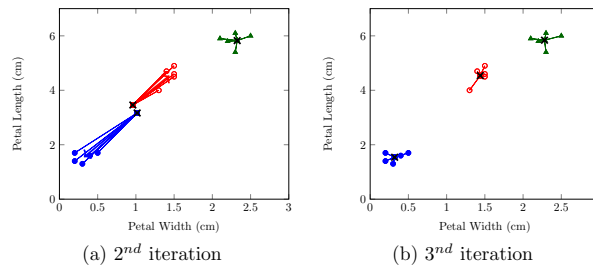


Figure 11.10.: K-means further steps. A $\times$ represents an centroid before it is updated. Undirected lines represent cluster membership. Directed lines $\nearrow$ indicates how the centroid will move (change) after re-computation based on assigned members.

With the new cluster centroids, the distance of each data instance has also changed and should be measured again to find out which cluster one belongs to now. By doing so, some data instances may move from one cluster to another, as shown in the 2nd iteration in Figure 11.10 (a), which, in turn, leads to further changes of cluster centroids. See the 3rd iteration in Figure 11.10 (b). By repeating the iterations of cluster member assignment and centroid re-computation, the cluster centroids will continue to move and gradually converge. One may stop the k-means optimization process when cluster centroid does not move or moves very little, or after a certain number of iterations.

For the small IRIS data set here, it only takes less than 5 iterations for the cluster centroids to converge (with little further movement). Figure 11.11 (a) shows the final 3 clusters thus identified and they are consistent with the three species of Iris flowers in the data. Figure 11.11 (b) gives an overview of the movement of cluster centroids through the iterations. Even though we started with random centroids, k-means gradually moves the centroids toward more representative segments of the data and, in the end, reaches an optimal outcome.

11. Clustering and Organization

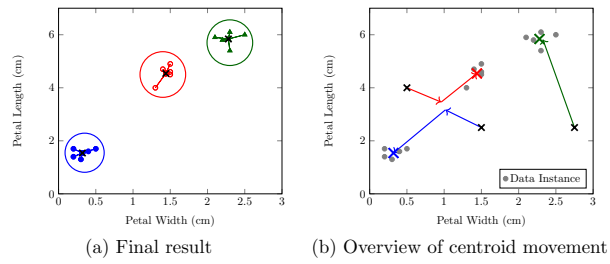


Figure 11.11.: K-means final clusters and steps

11.3.1. Issues of K-means

K-means can be interpreted as to minimize member instances' mean squared distance from their centroid. The iterative process can always converge to an optimum, but there is no guarantee that it will be globally optimal. In each iteration, it compares k clusters to N data instances, which is $O(kN)$. Because k is usually much smaller than N , k-means is much more efficient and scalable than HAC.

Clustering is understood as unsupervised learning. Theoretically, it is data driven without human input. In practice, however, it is not a clear cut. There are many ways in which one can influence the outcome of clustering, making it somewhat supervised. With the k-means method, for example, one has to predefine the number of clusters to be generated, which makes the outcome rather arbitrary. The data at hand does not necessarily have that supposed k number of clusters. One may have to test and evaluate clustering with different numbers of clusters to find out what an ideal k may be.

The distance function also plays an important role in clustering. For text clustering, feature (term) distributions matter more than the absolute weights of terms. In that domain of applications, cosine (angle) similarity works better than Euclidean distance and like measures. Furthermore,

11.4. Probabilistic Clustering

even with the same distance function, the weights and scales of variables (features) make a major difference. If all variables are to be treated equally, then they should be normalized to the same scale, e.g. within $[0, 1]$ for positive values. Otherwise, they should be properly weighted by their discriminative power based on domain theory and/or statistics.

Put it into practice

Continue with the corresponding practice activity to turn **Issues of K-means** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **K-means Partitioning** into a calculation, implementation, visualization, diagnostic, or interpretation task.

11.4. Probabilistic Clustering

While k-means discovers cluster centroids, the Euclidean distance, for example, essentially defines a cluster boundary as a sphere of a certain radius in a dimensional space, e.g. circles on two dimensions in Figure 11.11 (a). This implies two basic assumptions: 1) the distribution of member instances in a cluster is spherical, and 2) each data instance only belongs to one and only one cluster.

Is it possible to assume a different data distribution for clustering? In the same time, can there be soft (fuzzy) associations between data instances and clusters? The soft cluster membership can perhaps be thought of as probabilities.

11. Clustering and Organization

Indeed, there is a general probabilistic approach to clustering, namely a model-based clustering. The term *model* means that data are assumed to follow a distribution function with certain parameters (model) to be discovered. So based on the model, one can estimate the probabilities of cluster membership and try to maximize the probabilities in the process of identifying the clusters.

For simpler illustration and explanation, let's first consider only one variable, *petal width*, in the IRIS data. Suppose data instances with petal width values v follows the normal distribution, based on which, as we discussed in Chapter , the probability density of value v is computed by:

$$\begin{aligned} f(v) &= f(v|\mu, \delta^2) \\ &= \frac{1}{\sqrt{2\pi\delta^2}} e^{-\frac{(v-\mu)^2}{2\delta^2}} \end{aligned}$$

In the context of clustering, however, we assume the same normal distribution *within* each cluster c . The above density function can be extended to include terms related to the cluster c and the probability of a value v in cluster c can be computed by:

$$\begin{aligned} P(v|c) &\propto f_c(v) \\ &= f(v|\mu_c, \delta_c^2) \\ &= \frac{1}{\sqrt{2\pi\delta_c^2}} e^{-\frac{(v-\mu_c)^2}{2\delta_c^2}} \end{aligned}$$

where μ_c and δ_c^2 are the mean and variance of values associated with cluster c , the only two parameters of the model for each cluster.

The above formula shows that, in order to compute related probabilities, we need to know parameter values, namely mean μ_c and variance δ_c^2 , for each cluster c . The problem is, we are yet to identify these values, which, in fact, are based on related probabilities.

11.4. Probabilistic Clustering

With the chicken or the egg dilemma, we shall start with some initial values for the μ_c and δ_c^2 so that we can feed them into the calculation of probabilities, which then help in the identification of better μ_c and δ_c^2 values. This is similar to k-means where we start with initial centroids, which, through membership assignment and reassignment, change and improve.

11.4.1. Initialization

With the IRIS data on only one dimension, i.e. petal width, we randomize μ_c and δ_c for $k = 3$ clusters: $\mu_1 = 1$ and $\delta_1 = 0.25$ for cluster c_1 , $\mu_2 = 1.6$ and $\delta_2 = 0.25$ for cluster c_2 , and $\mu_3 = 2.8$ and $\delta_3 = 0.25$ for cluster c_3 . Figure 11.12 shows the distribution of IRIS data instances on the petal width v dimension, with a normal (bell) curve based on each set of initial parameters.

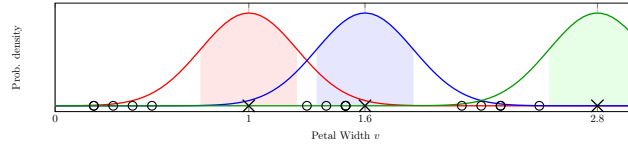


Figure 11.12.: Initial parameters of normal distributions. $\circ$ are IRIS data instances. $\times$ marks each an initial (random) mean μ (similar to a centroid), around which a normal distribution curve is plotted. The shaded area under curve shows the range of deviation from mean, i.e. $[\mu - \delta, \mu + \delta]$.

In addition, we don't know the prior probability $P(c)$ of each cluster c . If we assume equal chances, we have for the 3 clusters in the IRIS data: $P(c_1) = P(c_2) = P(c_3) = 1/3$.

11. Clustering and Organization

Put it into practice

Continue with the corresponding practice activity to turn **Initialization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

11.4.2. Expectation

Given each cluster model with the initial parameter, we can estimate $P(v|c)$, how likely an instance v can be observed – generated from the model – in that cluster c .

For now, however, we are more interested in finding the probability of an instance's membership association with a cluster. That is, given an instance v , how likely it is associated with each cluster c , $P(c|v)$. Following the Bayes Theorem, $P(c|v)$ can be estimated using $P(v|c)$:

$$\begin{aligned} P(c|v) &= \frac{P(v|c)P(c)}{P(v)} \\ &= \frac{f(v|\mu_c, \delta_c^2)P(c)}{\sum_{i=1}^k P(c_i)f(v|\mu_{c_i}, \delta_{c_i}^2)} \end{aligned}$$

where k is the number of clusters and the denominator is the sum of probabilities for the data instance v to be generated by distribution models of all k clusters.

Computing $P(c|v)$ is similar to the membership assignment of data instances to clusters in k-means. But instead of a *hard* membership, we have a probabilistic (soft) assignment here. We refer to the process of estimating the posterior probabilities of cluster membership as *Expectation*.

Let's apply this to the IRIS data. We use the first data instance with petal width $v = 0.2$ (cm) (see the ◦ closest to the zero in Figure 11.12). First,

11.4. Probabilistic Clustering

we compute $P(v|c)$, probability that each cluster model will generate the instance $v = 0.2$:

$$\begin{aligned}
 P(v = 0.2|c_1) &\propto f_{c_1}(v = 0.2) \\
 &= f(v = 0.2|\mu_1 = 1, \delta_1 = 0.25) \\
 &= \frac{1}{\sqrt{2 \times 0.25^2 \pi}} e^{-\frac{(0.2-1)^2}{2 \times 0.25^2}} \\
 &= 0.00954 \\
 P(v = 0.2|c_2) &\propto 2.5 \times 10^{-7} \\
 P(v = 0.2|c_3) &\propto 5.2 \times 10^{-24}
 \end{aligned}$$

As you can see from the Figure 11.12, $v = 0.2$ is on the far left side of the first (red) bell curve centered on 1, and has a very small probability density $P(v = 0.2|c_1) \propto 0.00954$. This is further remove from the centers of the second (blue) and third (green) normal curves, and probability densities corresponding to c_2 and c_3 are much smaller.

With these values, we can now compute $P(v)$, which is the weighted sum of probabilities that the 3 cluster models will generate data $v = 0.2$:

$$\begin{aligned}
 P(v = 0.2) &= \sum_{i=1}^3 P(c_i) f(v = 0.2|\mu_{c_i}, \delta_{c_i}^2) \\
 &= 1/3 \times 0.00954 + 1/3 \times 2.5 \times 10^{-7} + 1/3 \times 5.2 \times 10^{-24} \\
 &= 0.00318
 \end{aligned}$$

Plug these values into Equation [eq-org-pcv], we can compute the probability of data instance $v = 0.2$ belonging to each cluster for the *Expectation* step:

11. Clustering and Organization

$$\begin{aligned}
 P(c_1|v = 0.2) &= \frac{P(v = 0.2|c_1)P(c_1)}{P(v = 0.2)} \\
 &= \frac{0.00954 \times 1/3}{0.00318} \\
 &\approx 1 \\
 P(c_2|v = 0.2) &= 2.59 \times 10^{-5} \\
 P(c_3|v = 0.2) &= 5.46 \times 10^{-22}
 \end{aligned}$$

Apparently, data instance $v = 0.2$ is much more likely to belong to the cluster c_1 (the red bell curve on the left) than to the other clusters, which are far away as shown in Figure 11.12.

Now that we have identified the membership probabilities for the first instance, we shall continue to do the same for all other data instances ($N = 15$ for the IRIS data) and conclude the *Expectation* step with all probabilistic membership "assignment." For example, here are related probabilities for two more data instances in IRIS: $v = 0.4$ (2^{nd} instance) and $v = 2.3$ (15^{th} instance).

$$\begin{aligned}
 P(c_1|v = 0.4) &\approx 1 \\
 P(c_2|v = 0.4) &= 0.000177 \\
 P(c_3|v = 0.4) &= 1.7 \times 10^{-19} \\
 P(c_1|v = 1.3) &= 8.66 \times 10^{-6} \\
 P(c_2|v = 1.3) &= 0.128 \\
 P(c_3|v = 1.3) &= 0.872
 \end{aligned}$$

Figure 11.13 shows the color-coded result of the first *Expectation* step on the IRIS data. A data instance's color indicates the cluster with the highest probability – that is, the bell curve that best "covers" the data instance.

11.4. Probabilistic Clustering

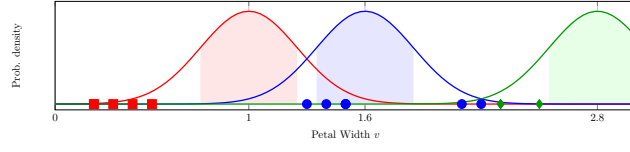


Figure 11.13.: First expectation result. Color of a data instance indicates the cluster membership with the highest probability.

💡 Put it into practice

Continue with the corresponding practice activity to turn **Expectation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

11.4.3. Maximization

The objective of clustering is to find clusters such that the overall probability of observed data instances is maximized. That is to maximize the joint probability for all data instances, which is the product of all $P(v|c)$ if we assume statistical independence:

$$\arg \max_{\theta_c} \prod_v P(v|c)$$

where θ are parameters of the distribution models and, with the normal distribution assumption, are μ and δ^2 of each cluster distribution. This is equivalent to maximizing the log-likelihood:

$$\begin{aligned} \log \prod_v P(v|c) &= \log \prod_v f(v|\mu_c, \delta_c^2) \\ &= \sum_v \log f(v|\mu_c, \delta_c^2) \end{aligned}$$

11. Clustering and Organization

The parameters can be estimated by:

$$\begin{aligned}\hat{\mu}_c &= \frac{\sum_v vP(c|v)}{\sum_v P(c|v)} \\ \hat{\delta}_c^2 &= \frac{\sum_v (v - \hat{\mu}_c)^2 P(c|v)}{\sum_v P(c|v)}\end{aligned}$$

where v is each one of N data instances ($N = 15$ in the small IRIS data subset here) and $P(c|v)$ is the probability that v is associated with cluster c .

It turns out that there is not a simple direct solution to maximizing the log-likelihood. However, if we look at the result from the *Expectation* step, there is a way in which we can maximize this by following the data and improving the *representative* of the cluster models, similar to how k-means iterates.

Now that data instances have been assigned, probabilistically, to each cluster, the original model for each cluster is no longer representative of data associated with them. In Figure 11.13, for example, the first (red) bell curve is not centered on the data instances most highly associated with it.

This is similar to k-means where the original centroid does no longer represent the center of member instances. And the models – with parameters mean μ_c and variance δ_c^2 for each cluster c – will have to be recomputed. This re-computation of parameters, in the context of model-based clustering, is the *Maximization* step.

Now back to the IRIS data. With the assigned posterior probabilities $P(c|v)$, we can now update the mean and variance for each cluster model. For cluster c_1 and $N = 15$ data instances, we have:

11.4. Probabilistic Clustering

$$\begin{aligned}
 \hat{\mu}_1 &= \frac{\sum_v v P(c_1|v)}{\sum_v P(c_1|v)} \\
 &= \frac{0.2 \times 1 + 0.4 \times 1 + \dots + 1.3 \times 8.66 \times 10^{-5}}{1 + 1 + \dots + 8.66 \times 10^{-5}} \\
 &= 0.522 \\
 \hat{\delta}_1 &= \sqrt{\frac{\sum_v (v - \hat{\mu}_1)^2 P(c_1|v)}{\sum_v P(c_1|v)}} \\
 &= \sqrt{\frac{(0.2 - 0.522)^2 \times 1 + (0.4 - 0.522)^2 \times 1 + \dots + (1.3 - 0.522)^2 \times 8.66 \times 10^{-5}}{1 + 1 + \dots + 8.66 \times 10^{-5}}} \\
 &= 0.433
 \end{aligned}$$

Likewise, we can update models for cluster c_2 with $\hat{\mu}_2 = 1.67$ and $\hat{\delta}_2 = 0.332$, and cluster c_3 with $\hat{\mu}_3 = 2.34$ and $\hat{\delta}_3 = 0.116$.

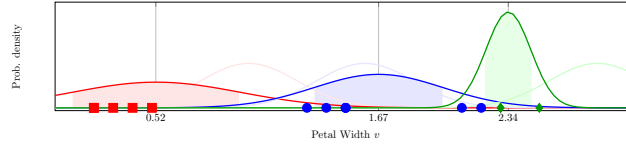


Figure 11.14.: First Maximization result. A bell curve with with a shaded area ($\pm delta$) is the new model with updated parameters for the cluster.

Figure 11.14 shows the result of this *Maximization* step. The models (normal curves) have moved closer to their more highly associated data instances. This is very similar to re-computation of centroids – thus causing them to move – in k-means clustering. With Maximization, more than one parameters are needed to specify a model. For normal distribution models, both mean μ and variance δ^2 are to be updated. If one assumes a

11. Clustering and Organization

different probability distribution, then a different set of parameters will be maximized.

Put it into practice

Continue with the corresponding practice activity to turn **Maximization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

11.4.4. Iterations of Expectation-Maximization

Now that the cluster models have changed, we need to go back to the *Expectation* step and re-compute the posterior probabilities $P(c|v)$. Once the probabilities are changed, the model parameters will be updated again in the next *Maximization* step. The process of Expectation-Maximization will repeat itself until the models no longer changed, where the overall probability of models generating observed data instances has been maximized.

Figure 11.15 shows further iterations of EM on the IRIS data. Visually, the cluster models (curves) move increasingly closer to the data instances they represent and, in the end, get well aligned with them.

Put it into practice

Continue with the corresponding practice activity to turn **Iterations of Expectation-Maximization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

11.4. Probabilistic Clustering

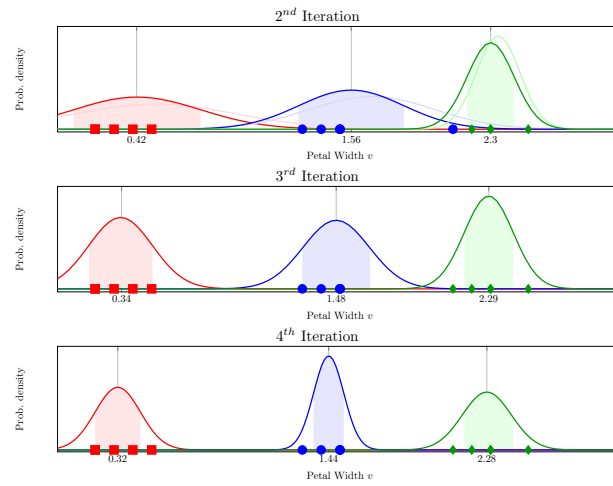


Figure 11.15.: Iterations of Expectation and Maximization. Note that there are 5 data instances in each iris species. But because their petal width values overlap on the only dimension, there appear to be 3 or 4 instances in each cluster, which, in fact, has 5.

11.4.5. EM with Higher Dimensionality

Our discussion has been focused on the it Expectation-Maximization method on one dimension with the univariate normal distribution. In real-world applications, however, there is hardly any clustering problem that can be resolved with this simplified version.

Normally EM has to take into account multiple, if not thousands or even millions of, variables. Instead of univariate distributions, one will have to rely on multivariate distributions. For example, rather than using univariate normal distributions with means and variances, one will instead rely on co-variances in a mixture Gaussian model.

In addition, the normal (Gaussian) distribution is only one of many distribution functions that can be used with EM clustering. In text clustering application, for example, it is very common to use poisson distributions. Regardless of the assumed distribution, the idea and process of expectation and maximization for the optimization of log-likelihood of observed data remain the same.

Put it into practice

Continue with the corresponding practice activity to turn **EM with Higher Dimensionality** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **Probabilistic Clustering** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Part III.

Language, Retrieval, and Structure

12. Text and Human Language

The big print giveth, and the fine print taketh away.

Fulton J. Sheen

Text has been the primary form of documentation in human history. Today large volumes of unstructured text data are being generated and disseminated at an unprecedented pace, via such media as tweets, emails, and scholarly publications. The magnitude of these data has far exceeded our human capacity to read and digest. How can we possibly sift through the world of written text and access what we need? In what ways can computers, which have helped create this ever-growing pile of data, be harnessed to also help us discover relevance and meaning?

12.1. Text Vectorization

Let's start with some basic issues and ideas for processing text.

Imagine having a long legal document, it can be stored digitally along with other documents and files. In its raw text format, you can hardly use the document to compute and compare – for example, to find other documents related to the same or similar cases – unless we can convert the text into some numbers to be used in the calculation.

Indeed, the first step of text processing involves a general process called *vectorization*, which identifies elements in the text and assigns numbers (weights) to them as its numerical representation. The result of this is a list of values, which can then be used for searching, ranking, and association.

12. Text and Human Language

12.1.1. Tokenization

Through *tokenization*, the text can be split into basic elements (tokens) such as paragraphs, sentences, phrases, and/or words. Taking single words as basic elements, the following text:

John is quicker than Mary.

can be split into individual tokens *John*, *is*, *quicker*, *than*, and *Mary*. In a simple classic approach, we only try to capture what tokens (keywords) occur in the text and pay no heed to the order in which they appear. This is referred to as the *bag of words* model, which will lead to some loss of information. Apparently, with this treatment, tokenization of the above text will result in the identical *bag of words* with the following:

Mary is quicker than John.

where the only difference is the order and the meaning is simply the opposite. Indeed, for analyses where the semantic interpretation of a text is critical, the model will suffer from the loss of order. In many practical applications, nonetheless, the *bag of words* works sufficiently well where the central problem is about finding associations through the keywords. For now, we stick with this basic model and revisit the issue of word order later.

Now that we identify the tokens (words), one may consider normalize them so that the same word types in various forms can be unified and treated as the same. In English, one can apply normalization techniques such as:

- *Case folding*, which changes all tokens into lower-case, e.g. *John* and *JOHN* $\rightarrow$ *john*;
- *Stemming*, which reduces tokens to their *roots*, e.g. *compression* and *compressed* $\rightarrow$ *compress*;
- *Lemmatization*, which converts word variants to their base form, e.g. *are* and *is* $\rightarrow$ *be*;

12.1. Text Vectorization

These techniques are language and domain-dependent. Aside from potential benefits of unifying tokens of the same type, token normalization may also lead to undesired consequences, e.g. to treat *CAT* (the company) and *cat* as the same. So careful examination of data and application is necessary to determine whether related normalization techniques may be instrumental or detrimental.

Also, note that tokens are not limited to single words. One can use a moving window to take two, three, or more consecutive words at a time to identify all possible phrases or n-grams. Likewise, you can use a moving window to identify any n consecutive characters as tokens. Character n-grams have shown its utility in applications such as spam detection, where junk emails may pad normal keywords with special characters to avoid being identified.



Put it into practice

Continue with the corresponding practice activity to turn **Tokenization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

12.1.2. Term Weighting

Once tokens – single words or phrases – have been identified, they can be sorted and merged into a set of unique tokens, which we refer to as the *dictionary*. Suppose we have a book collection of *The Chronicles of Narnia*, for which we only store their titles¹:

1. The Lion, the Witch and the Wardrobe
2. The Voyage of the Dawn Treader

¹Bear in mind that full-text is often considered and analyzed in real-world text data. We use the title only here for the simplicity of illustration.

12. Text and Human Language

3. The Horse and His Boy

Based on tokenization of single words with case-folding, the following dictionary (alphabetically ordered) can be identified from this small collection:

1	2	3	4	5	6	7	8	9	10	11
and	boy	dawn	his	horse	lion	the	treader	voyage	wardrobe	witch

Now we can use terms in this dictionary to represent each document based on their occurrences, 0 if one does not occur and 1 if it does. The following matrix shows such a *binary representation*:

	1	2	3	4	5	6	7	8	9	10	11
DOC	and	boy	dawn	his	horse	lion	the	treader	voyage	wardrobe	witch
1	1	0	0	0	0	1	1	0	0	1	1
2	0	0	1	0	0	1	1	1	1	0	0
3	1	1	0	1	1	0	1	0	0	0	0

The size of the dictionary will increase with the size of the text collection and the length of each document. However, one can imagine that, for each document, not all dictionary terms will occur and therefore some (many in fact) of vector values are 0s. A more efficient representation is a sparse vector, where only non-zero values (1 in the binary case here) are retained. For example, document 3 in the above matrix can be represented as:

(	1	2	4	5	7	)
	and	boy	his	horse	the	

which only includes terms that appear in the document and have 1 values.

12.1. Text Vectorization

The binary representation only whether or not a term occurs. This is a reasonable treatment for small text documents such as the above example and tweets. With longer text documents, when full-text is used for vectorization, it is useful to also consider *term frequency* (TF).

TF_{dt} is the number of times a term t appears in document d . It is often observed that a term with a higher term frequency in a document is more relevant to the document's topical theme and should, therefore, be given a higher weight. One can simply use the term frequency as term weight:

$$w_{dt} = TF_{dt}$$

One can also normalize the TF value with logarithm:

$$w_{dt} = \begin{cases} 1 + \log TF_{dt}, & \text{if } TF_{dt} > 0 \\ 0, & \text{otherwise} \end{cases}$$

A basic assumption of counting occurrences in the term frequency weight is that all occurrences are equal and should count equally. In light of the Zipf law, however, we know words are used unequally.

Some words appear more frequently simply because of the effect of *least effort*. Some are stop-words like *a* and *the* that contribute little in meaning. For many of these, there is one thing in common – they appear frequently not only in the current document but also in other documents as well.

In general, we observe that some terms are used so *broadly* in so many documents that they can hardly help differentiate one document from another. They are too broad to be *specific* and informative.

On the other extreme, some terms are used in only a very few documents and they very clearly distinguish these documents from others. These terms are rarer, more specific, and should be given more weight according to the observation here.

12. Text and Human Language

Imagine we randomly draw a document from the entire text collection, there is a probability that term t will appear, which can be estimated by:

$$\begin{aligned} p(t|C) &= \frac{n_t}{N} \\ p(t'|C) &= 1 - \frac{n_t}{N} \end{aligned}$$

where n_t is the number of documents containing term t and N is the total number of documents in the collection C . $p(t'|C)$ is the complimentary probability that t does NOT occur.

For a specific document d where term t appears, it is certain so the probability of observing the term is:

$$\begin{aligned} p(t|d) &= 1 \\ p(t'|d) &= 0 \end{aligned}$$

The amount of *information gain* or relative entropy, according to chapter [ch-info], by seeing the term in document d out of collection C , can be computed by:

$$\begin{aligned} Info(t) &= DL[P(t|d)||P(t|C)] \\ &= - \sum_{x \in (t, t')} p(x|d) \log \frac{p(x|C)}{p(x|d)} \\ &= -p(t|d) \log \frac{p(t|C)}{p(t|d)} - p(t'|d) \log \frac{p(t'|C)}{p(t'|d)} \\ &= -1 \cdot \log \frac{n_t}{N} - 0 \cdot \log \frac{p(t'|C)}{p(t'|d)} \\ &= -\log \frac{n_t}{N} \end{aligned}$$

This information gain of term t is in fact the exactly the classic *Inverse Document Frequency* (IDF) weighting scheme:

12.1. Text Vectorization

$$\begin{aligned} IDF_t &= Info(t) \\ &= \frac{N}{n_t} \end{aligned}$$

where n_t , the number of documents with term t , is commonly referred to as *document frequency* (DF).

IDF measures how informative a term is when it appears in a document. For a term that appears in all documents in the collection – the stop-words for example – n_t is the same as N and its IDF weight becomes:

$$\begin{aligned} IDF_t &= \log \frac{N}{N} \\ &= \log 1 \\ &= 0 \end{aligned}$$

For a term that appears in very few documents in a large collection, $n_t \ll N$ and the IDF weight is great. In this way, the IDF weight penalizes broader terms used in many documents and rewards more selective, rare terms.

Please note that IDF is a collection-wide statistic for each term whereas term frequency (TF) is term statistic specific to a document. They address two separate aspects of term weighting and are often combined for document representation. The combined weighting scheme, namely the classic $TF*IDF$, assigns a term's weight by:

$$w_{dt} = TF_{dt} \log \frac{N}{n_t}$$

or with log-transformed TF weight:

12. Text and Human Language

$$w_{dt} = \begin{cases} (1 + \log TF_{dt}) \log \frac{N}{n_t}, & \text{if } TF_{dt} > 0 \\ 0, & \text{otherwise} \end{cases}$$

where TF_{dt} is the number of times term t occurs in document d (TF), n_t is the number of documents having t (DF), and N is the total number of document in the collection.

In certain applications, TF may be normalized by document length to alleviate potential favoritism toward longer documents. This is one of the most widely used term weighting methods in information retrieval and text mining.

 Put it into practice

Continue with the corresponding practice activity to turn **Term Weighting** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Text Vectorization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

12.2. Similarity Measures

Once each document is represented as a vector of terms with weights, we can conduct various operations, such as clustering and ranking, on them. The basic unit of these computational tasks is to measure how similar or dissimilar one document is to another.

12.2. Similarity Measures

A simple method to compute the similarity of any two documents d_u and d_v is by the dot product of their vector values:

$$Sim(d_u, d_v) = \sum_{t \in T}^m w_{ut} w_{vt}$$

where for each term t in dictionary T , $w_{u,t}$ is the weight of t in document d_u , and $w_{v,t}$ is the weight of t in document d_v . For example, the similarity of doc 1 and 3 given the binary matrix of Table [tab-text-binary] can be computed by:

$$\begin{aligned} Sim(d_1, d_3) &= 1 \times 1 + 0 \times 1 + 0 \times 0 + \dots + 1 \times 0 \\ &= 2 \end{aligned}$$

The dot product coefficient in Equation [eq-text-dotp] can be used with any term weighting schemes including TF and TF*IDF. However, this similarity measure is not normalized by any size factor and its score can be inflated by document lengths and favor those longer documents. A remedy of this is the classic cosine similarity, which we will discuss in depth.

A binary sparse vector can also be regarded as a *set*, that is, a set including all unique terms that occur in the document. Several similarity coefficients can be computed based on sets. The *Dice Coefficient* computes the ratio between two sets' intersection size and the mean of their sizes:

$$\begin{aligned} Dice(d_u, d_v) &= \frac{|d_u \cap d_v|}{(|d_u| + |d_v|)/2} \\ &= \frac{2|d_u \cap d_v|}{|d_u| + |d_v|} \end{aligned}$$

where $|d_u|$ and $|d_v|$ are cardinalities (sizes) of the two sets, and $d_u \cap d_v$ is their intersection (the set of common terms).

12. Text and Human Language

A similar method, *Jaccard Coefficient*, measures the same intersection but as a fraction of the union of the two sets:

$$Jaccard(d_u, d_v) = \frac{|d_u \cap d_v|}{|d_u \cup d_v|}$$

Beyond binary representation, document vectors can be treated as data points in a dimensional space where each term is an independent dimension (coordinate). To simplify the discussion and visualize the dimensional space, suppose we have only two terms in the dictionary, *Data* and *Machine*.² Suppose we use TF term weighting and the following is a matrix representation of two documents where the terms appear:

	1	2
DOC	Machine	Data
1	1	5
2	4	1

These two documents can be plotted on the two-dimensional space, e.g. *machine* for the x dimension and *data* for the y dimension, shown in Figure 12.1:

With this dimensional representation, pair-wise similarity or difference can be measured by the geometric distance between two document data points. A classic metric of such is the *Euclidean Distance*, which measures the length of the straight line connecting the two data points in question, as shown in Figure 12.1. In an m -dimensional space, the general *Euclidean Distance* function is:

²In real world applications, the number of terms can be very large and we may be dealing with thousands, if not millions, of dimensions. While mathematically possible, we cannot visualize more than three dimension due to the limit of our visible, three-dimensional world.

12.2. Similarity Measures

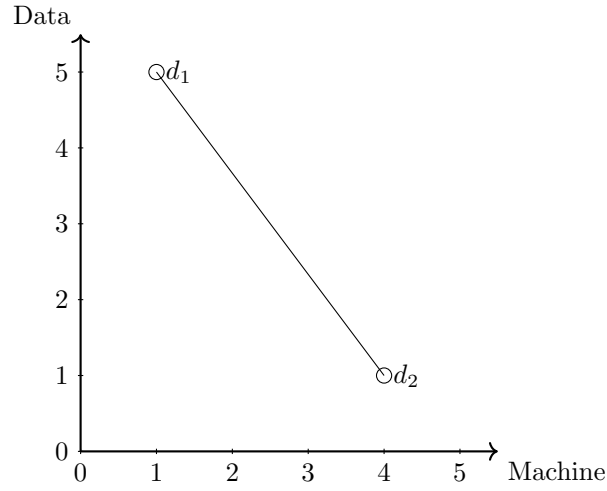


Figure 12.1.: Documents in a two-dimensional space

$$ED(u, v) = \sqrt{\sum_{i=1}^m (u_i - v_i)^2}$$

where u_i and v_i are the i^{th} dimensional value of data points u and v respectively. Given documents d_1 and d_2 in Figure 12.1, their euclidean distance is:

$$\begin{aligned} ED(d_1, d_2) &= \sqrt{(5-1)^2 + (1-4)^2} \\ &= 5 \end{aligned}$$

While *Euclidean Distance* is widely used in text mining, its performance may suffer in applications where there is a wide distribution of document sizes. A thought experiment will help illustrate the problem.

12. Text and Human Language

Imagine we have two documents $d_1 = (4, 5)$ and $d_2 = (5, 4)$ (with TF weights) that are highly similar and have a very small Euclidean distance between them, as show in Figure 12.2:

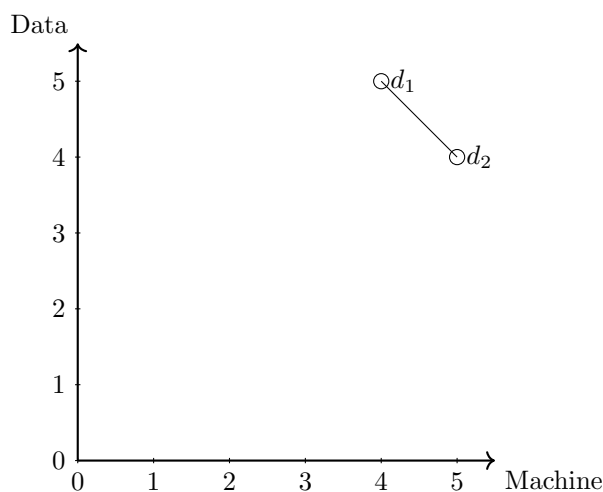


Figure 12.2.: Thought experiment with two documents

For now, one may think the line distance in Figure 12.2 is a reasonable estimate of their difference. But imagine make a double-copy of d_2 and treat the new copy as d_3 , resulting in Figure 12.2:

Because d_3 is copy of d_2 – the only difference is that it doubles everything in d_2 – its semantics should remain the same as d_2 . A fair distance measure should give a relatively small value, indicating d_3 is *close* to d_1 and especially d_2 . However, as shown in Figure 12.2, d_3 is much farther away from d_1 and consequently the euclidean distance is much greater.

In short, Euclidean Distance treats documents differently simply because they are of different sizes (magnitude). In this light, we should somehow disregard the sizes when measuring their similarities.

12.2. Similarity Measures

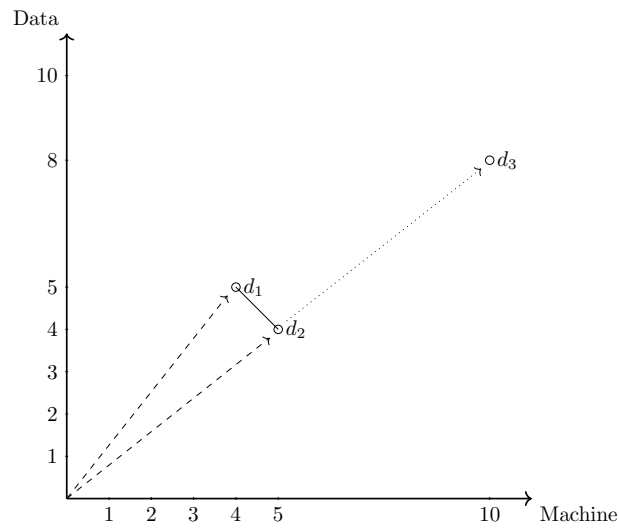


Figure 12.3.: Thought experiment a third document, where d_3 doubles the content of d_2

12. Text and Human Language

Another perspective on similarity vs. distance in the dimensional space is by measuring the *direction* of a vector. In physics, the notion of vector has been used extensively for quantities such as velocity, which has both a *magnitude* (speed) and direction. Likewise, as shown as dashed lines in Figure 12.3, documents such as d_1 and d_2 can be treated as vectors from the origin, with a magnitude (line length) and direction (arrow). The difference between the two documents can be measured by their angle distance.

Also in Figure 12.3, d_3 as copy of d_2 is pointing in the exact same direction of d_2 and their angle distance is 0. Consequently, the angle distance between d_1 and d_3 is the same as that of d_1 and d_2 .

All this makes sense with angle distance as d_2 and d_3 essentially have the same content and should be treated equivalent. In this *vector space* representation, a vector's direction is dependent on the *distribution of term weights* in the corresponding text document. While the magnitude is about the absolute values of term weights, the direction is a rather relative measure – that means, in the case of Figure 12.3, to what degree a document vector is more or less about *machine* (to the east) vs. *data* (to the north).

To measure an *angle distance*, one can calculate the degree of the angle between two vectors. Nonetheless, a classic approach is to measure the *cosine* of the angle as their *similarity*.

As shown in Figure 12.4, with the help line perpendicular to vector d_1 , the cosine of d_1 and d_2 is:

$$\text{Cosine}(d_1, d_2) = \frac{a}{c}$$

With the vector representations of any two documents u and v , the same cosine can be computed by:

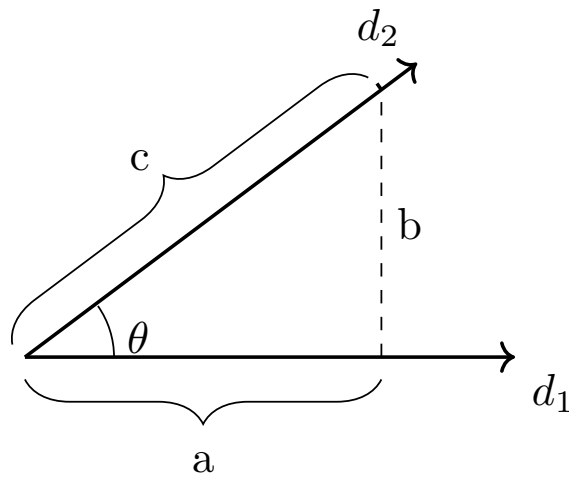


Figure 12.4.: Cosine of an angle

12. Text and Human Language

$$\text{Cosine}(d_u, d_v) = \frac{\sum_{i=1}^m u_i v_i}{\sqrt{\sum_{i=1}^m u_i^2} \sqrt{\sum_{i=1}^m v_i^2}}$$

where u_i is the weight of the i^{th} term in document u and v_i is the same term's weight in document v . For the three documents in the thought experiments, $d_1 = (4, 5)$, $d_2 = (5, 4)$, and $d_3 = (10, 8)$, we can compute their pairwise similarities:

$$\begin{aligned} \text{Cosine}(d_1, d_2) &= \frac{4 * 5 + 5 * 4}{\sqrt{4^2 + 5^2} \sqrt{5^2 + 4^2}} \\ &\approx 0.98 \\ \text{Cosine}(d_1, d_3) &= \frac{4 * 10 + 5 * 8}{\sqrt{4^2 + 5^2} \sqrt{10^2 + 8^2}} \\ &\approx 0.98 \\ \text{Cosine}(d_1, d_2) &= \frac{4 * 8 + 5 * 10}{\sqrt{4^2 + 5^2} \sqrt{10^2 + 8^2}} \\ &= 1 \end{aligned}$$

In this case, d_1 and d_2 are identical with cosine 1 whereas d_2 and d_3 have the same similarity compared to d_1 .

Cosine is inversely related to the angle θ (distance) and is a similarity measure. When two vectors are close in their direction, with a small angle shown in Figure 12.5, a is almost the same as c and cosine similarity is nearly 1. The two vectors are highly similar in their directions, with 1 indicating the same direction and perfect match.

On the other hand, when two vectors point to directions nearly perpendicular to one another, with a large angle shown in Figure 12.6, a becomes extremely small and cosine similarity is close to 0. For vectors based on text documents, their vector values are normally positive (in the first quadrant

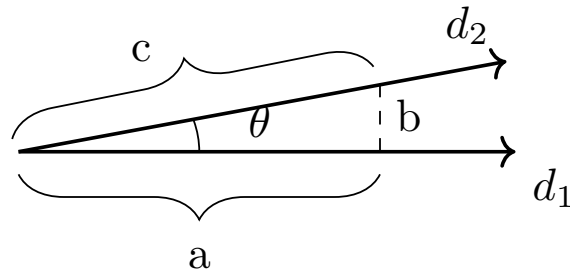


Figure 12.5.: Cosine of a small angle, i.e. vectors of close directions

in the two-dimensional space). Therefore, the smallest cosine similarity value one can obtain is 0 when two vectors are exactly perpendicular (orthogonal directions).

In short, cosine is a similarity measure in the vector space model, which ranges from 0 and 1. A greater cosine value indicates a higher similarity of two vectors.

 Put it into practice

Continue with the corresponding practice activity to turn **Simi-229** **larity Measures** into a calculation, implementation, visualization, diagnostic, or interpretation task.

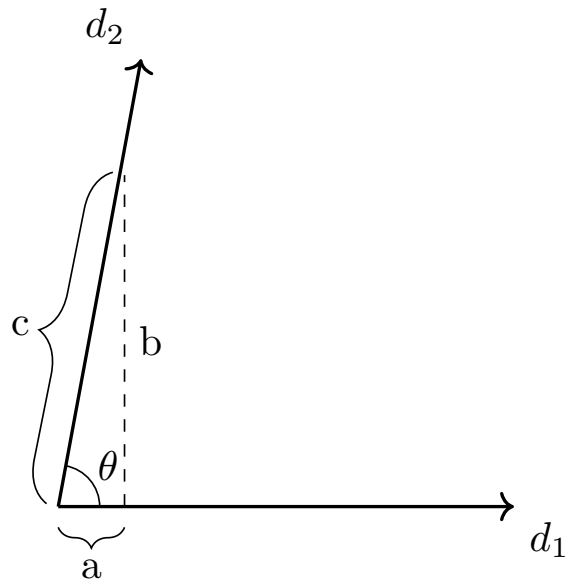


Figure 12.6.: Cosine of a large angle, i.e. vectors with nearly perpendicular directions

12.3. Probabilities and Implications

12.3.1. Zipf's Law

The notion of *least action* or *least effort* has long been observed in the natural world as well as human behavior. In spoken language, we tend to minimize our efforts by reusing old words and expressions, as if they are old tools meant for many applications. The results of this: words tend toward "a minimum in number and size" and "the older the tool (word) the more different jobs it can perform." (Zipf 1949)

Under the *Principle of Least Effort*, there appears to be a great divide between a small number of words so frequently used and those rarely mentioned. In particular, if we sort words by their frequencies, a term t 's frequency f_t is proportional to the inverse of its rank k_t :

$$f_t \propto \frac{1}{k_t}$$

where word t is ranked the k^{th} most frequent. If we consider probabilities rather than raw frequencies, the above can be written as:

$$p(t) = \frac{c}{k_t}$$

where c is a constant. In the English language, empirical data have shown $p(t) \approx 0.1/k_t$ within the top 1,000 rank. This relation can be visualized in Figure 12.7 (a).

With a long tail, Figure 12.7 (a) on normal coordinates is difficult to examine and it is often useful to apply the logarithmic transformation. By taking the log of both x (rank) and y (frequency) values, this can be transformed into Figure 12.7 (b). As shown, Zipf law is a linear function on log-log coordinates and is a special case of *power-law* distributions, which,

12. Text and Human Language

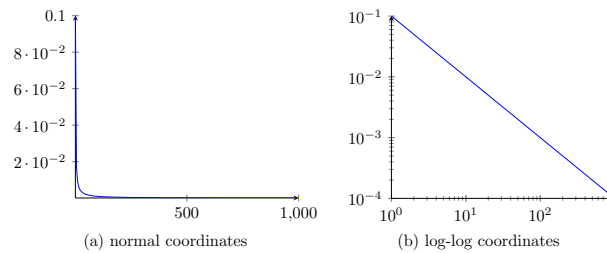


Figure 12.7.: Zipf law function, roughly for the top 1,000 in English

as we discussed in Chapter , are results of the *Matthew effect* (the rich get richer). In fact, there is an intrinsic connection between the *Principle of Least Effort* and the *Matthew effect* – as we tend to a minimum set of vocabulary, what we have frequently used (old ”rich” words) will be used even more frequently (richer).

💡 Put it into practice

Continue with the corresponding practice activity to turn **Zipf’s Law** into a calculation, implementation, visualization, diagnostic, or interpretation task.

12.3.2. Feature Selection

We have observed that any text processing may involve a very large number of features (terms) and it is a common practice to perform *feature selection* in such processes as clustering, classification, and retrieval (search).

In light of Zipf’s law, there are a large number of words that are rarely used, only in a few documents. While these terms might be informative when they occur, their rare occurrences make them less useful in such tasks as clustering and classification where documents are grouped by terms they

have in common. Given that there are many low frequent words – the long tail of the Zipf function – removing them will reduce the dictionary size (feature space) significantly.

In terms of document frequency (DF), on the other hand, we know highly frequent words such as stop words are not informative and can be safely removed in certain applications. In practice, simple feature selection techniques such as DF (document frequency) thresholding has performed very effectively with little computational cost.(Yang and Pedersen 1997) It has been shown that having the right number of features is key to the success of text clustering and classification – using all features not only slows down the processing but also contributes more noise than information for related tasks.(Ke 2015)

💡 Put it into practice

Continue with the corresponding practice activity to turn **Feature Selection** into a calculation, implementation, visualization, diagnostic, or interpretation task.

💡 Put it into practice

Continue with the corresponding practice activity to turn **Probabilities and Implications** into a calculation, implementation, visualization, diagnostic, or interpretation task.

12.4. Text Classification

With a distance or similarity measure in the vector representation, it is now possible to compare documents to one another and associate them with data groups or labels. For example, one may use an instance-based lazy learning method such as kNN (see Chapter [ch-choices]) with cosine

12. Text and Human Language

similarity to perform classification and prediction based on the nearest data vectors.

Another successful approach is to model the probability that a document belongs to each class (label) and assign the document to the class that maximizes the probability. That is, given a document representation d , we need to estimate the probability of each class $p(c_i|d)$, where c_i is one of the given classes to be predicted.

Say we are working on email *spam* filtering based on subject titles. This is a binary classification problem, for which we have two classes (labels) – an email is either a *ham* (normal) or a *spam* (junk). Suppose we have a small collection of email subjects which we have labeled *ham* or *spam* to train our model (learning):

DOC	Email Subject	Class
1	You have been selected!	spam
2	Have a prize for you!	spam
3	You won!	spam
4	Project meeting schedule	ham
5	Nice to meet you!	ham
6	Project update	ham
7	Update on research	ham

12.4.1. Naive Bayes with Binary Term Occurrences (Bernoulli)

Given the short subject titles, let us first look at how we can model probabilities in text if we only consider binary occurrences of terms. Assume we remove some of stop-words (e.g. have, been) and normalize some others (e.g. meeting $\rightarrow$ meet, won $\rightarrow$ win), we can represent the email using the following dictionary:

12.4. Text Classification

	1	2	3	4	5	6	7	8	9	10	
DOC	meet	nice	prize	project	research	schedule	select	update	win	you	Class
1							1			1	spam
2			1							1	spam
3									1	1	spam
4	1			1		1					ham
5	1	1								1	ham
6				1				1			ham
7					1			1			ham

Within a class c , there is a probability that a term t will occur in a (random) document $p(t|c)$. To estimate the probability, one can simply count the ratio between the number of emails t appears and the total number of emails in class c .

However, as we discussed in Chapter , this approach may lead to skewed estimates and potential zero probability values due to data variance in the sample. Now that we have a very small data set here, we will use the Laplace estimator with an added constant μ to any count, that is:

$$p(t|c) = \frac{n(t|c) + \mu}{N(c) + 2\mu}$$

where $n(t|c)$ is the number of documents in class c that contain term t (document frequency) and $N(c)$ is the total number of documents belonging to c . μ is a constant added to avoid zero probability estimates and 2μ in the denominator is to account for μ being added to the binary cases (for when t occurs and when it does not).

For example, with $\mu = 0.05$, some of the term probabilities in the spam class can be computed by:

12. Text and Human Language

$$\begin{aligned}
 p(\textit{meet}|\textit{spam}) &= (0 + 0.05)/(3 + 0.1) \\
 &= 0.016 \\
 p(\textit{prize}|\textit{spam}) &= (1 + 0.05)/(3 + 0.1) \\
 &= 0.339 \\
 p(\textit{you}|\textit{spam}) &= (3 + 0.05)/(3 + 0.1) \\
 &= 0.984
 \end{aligned}$$

The same probabilities in the ham class are:

$$\begin{aligned}
 p(\textit{meet}|\textit{ham}) &= (2 + 0.05)/(4 + 0.1) \\
 &= 0.5 \\
 p(\textit{prize}|\textit{ham}) &= (0 + 0.05)/(4 + 0.1) \\
 &= 0.012 \\
 p(\textit{you}|\textit{ham}) &= (1 + 0.05)/(4 + 0.1) \\
 &= 0.256
 \end{aligned}$$

In the binary representation, $w_t = 1$ when term t occurs in the document and $w_t = 0$ if it does not. Given $p(t|c)$ probability term t occurs in a (random) document in class c , $1 - p(t|c)$ is the probability that it does not occur. That is, the probability of observing a binary term weight w_t can be written as the *Bernoulli* probability mass function:

$$\begin{aligned}
 p(w_t|c) &= p(t|c)^{w_t}(1 - p(t|c))^{1-w_t} \\
 &= \begin{cases} p(t|c), & \text{if } w_t = 1 \\ 1 - p(t|c), & \text{otherwise } w_t = 0 \end{cases}
 \end{aligned}$$

For term *prize* in the spam class, the above can be written as:

12.4. Text Classification

$$\begin{aligned}
 p(w_{prize}|spam) &= p(prize|spam)^{w_{prize}}(1 - p(prize|spam))^{1-w_{prize}} \\
 &= \begin{cases} 0.339, & \text{if prize occurs } w_{prize} = 1 \\ 0.661, & \text{otherwise } w_{prize} = 0 \end{cases}
 \end{aligned}$$

Similarly, probability of a term weight for *you* in the ham class:

$$\begin{aligned}
 p(w_{you}|ham) &= p(you|ham)^{w_{you}}(1 - p(you|spam))^{1-w_{you}} \\
 &= \begin{cases} 0.256, & \text{if it occurs } w_{you} = 1 \\ 0.744, & \text{otherwise } w_{you} = 0 \end{cases}
 \end{aligned}$$

Following this, we can estimate the probability of *any* term weight w_t in the classes ham vs. spam. Now suppose we receive a new email and try to determine whether it is a *ham* or a *spam*:

Table 12.1.: A new email to be classified

DOC	Email Subject	Class
d	Your prize!	?

which can be represented as:

	1	2	3	4	5	6	7	8	9	10	
DOC	meet	nice	prize	project	research	schedule	select	update	win	you	Class
d			1							1	?

The question here is how we can estimate the probability of the email d being *ham* or *spam*, $p(ham|d)$ vs. $p(spam|d)$. Using the Bayes theorem, they can be computed by:

12. Text and Human Language

$$p(ham|d) = \frac{p(ham)p(d|ham)}{p(d)}$$

$$p(spam|d) = \frac{p(spam)p(d|spam)}{p(d)}$$

Both equations above have $p(d)$ as the denominator, which does not affect the ratio of the two values (classification outcome) and can be ignored in the calculation.

$p(ham)$ and $p(spam)$ can be estimated by counting the number of hams vs. spams. With a Laplace estimator, they are:

$$p(ham) = (4 + 0.05)/(7 + 0.1)$$

$$= 0.57$$

$$p(spam) = (3 + 0.05)/(7 + 0.1)$$

$$= 0.43$$

For $p(d|spam)$, email d can be viewed as a collection of its term weights w_t . Given the 10 terms in the dictionary, $p(d|spam)$ is their joint probabilities $p(w_t|spam)$ in the *spam* class. Assume the term probabilities are independent (naive assumption), the conditional probability can be computed by:

$$p(d|spam) = \prod_t p(w_{dt}|spam)$$

$$= p(w_{meet} = 0|spam)p(w_{nice} = 0|spam)p(w_{prize} = 1|spam)...p(w_{you} = 1|spam)$$

$$= 0.984 \times 0.984 \times 0.339... \times 0.984$$

$$= 0.132$$

Likewise, we can compute the probability conditioned on the *ham* class:

12.4. Text Classification

$$\begin{aligned}
 p(d|ham) &= \prod_t p(w_{dt}|ham) \\
 &= p(w_{meet} = 0|ham)p(w_{nice} = 0|ham)p(w_{prize} = 1|ham) \cdot p(w_{you} = 1|ham) \\
 &= 0.5 \times 0.744 \times 0.012 \dots \times 0.256 \\
 &= 0.013
 \end{aligned}$$

Plug the numbers back in Equations [eq-text-nb_ham] and [eq-text-nb_spam], we have:

$$\begin{aligned}
 p(spam|d) &= \frac{p(spam)p(d|spam)}{p(d)} \\
 &= \frac{0.43 \times 0.293}{p(d)} \\
 &= \frac{0.132}{p(d)} \\
 p(ham|d) &= \frac{p(ham)p(d|ham)}{p(d)} \\
 &= \frac{0.57 \times 0.013}{p(d)} \\
 &= \frac{0.007}{p(d)}
 \end{aligned}$$

Clearly $p(ham|d) < p(spam|d)$ and the new email d will be predicted as a spam.

Two important notes here. First, although the example only has two classes, the Naive Bayes classifier can be applied to more than two classes. Given a document d and a set of classes $C = [c_1, c_2, \dots, c_k]$, where $k \geq 2$, the general Naive Bayes model with Bernoulli distribution is to identify a class $\hat{c}$ such that the posterior probability $p(c|d)$ is maximized:

12. Text and Human Language

$$\begin{aligned}\hat{c} &= \arg \max_{c \in C} p(c|d) \\ &= \arg \max_{c \in C} \frac{p(c)p(d|c)}{p(d)} \\ &= \arg \max_{c \in C} \frac{p(c) \prod_t p(w_{dt}|c)}{p(d)}\end{aligned}$$

where w_{dt} is the binary weight of term t in document d (to be classified). $p(d)$ is the common denominator for all classes and can be ignored in the calculation.

Second, the product of joint probabilities (with $\prod_t$ in the above equation) in the model can become a very small value, especially when there are a great number of terms in the dictionary. To avoid this computational difficulty, it is a better practice to convert this into a log-likelihood (by taking the logarithm of the probabilities):

$$\begin{aligned}\hat{c}_{log} &= \arg \max_{c \in C} \log \frac{p(c) \prod_t p(w_{dt}|c)}{p(d)} \\ &= \arg \max_{c \in C} \log p(c) + \sum_t \log p(w_{dt}|c) - \log p(d)\end{aligned}$$

In this way, we perform the addition of log-likelihood and can maintain a better precision with float points. Again $\log p(d)$ is a constant across the classes and can be omitted in the computing.



Put it into practice

Continue with the corresponding practice activity to turn **Naïve Bayes with Binary Term Occurrences (Bernoulli)** into a calculation, implementation, visualization, diagnostic, or interpretation task.

12.4.2. Naive Bayes with Term Frequencies (Multinomial)

Now let us look beyond binary term occurrences for text classification. As we discussed earlier in the chapter, term frequency (TF) is a useful statistic, which we should also take advantage of in this task.

Suppose we extend the email representation from subject title to include email body (content) as well. Assume the dictionary (vocabulary) remains the same as in Table [tab-text-emails_bin] but obviously terms may occur multiple times in the email text. The following is a hypothetical TF representation of the same emails:

	1	2	3	4	5	6	7	8	9	10	
DOC	meet	nice	prize	project	research	schedule	select	update	win	you	Class
1							1			3	spam
2			3							2	spam
3									2	2	spam
4	2			1		3					ham
5	1	1								2	ham
6				2				1			ham
7					3			1			ham

The probability of term t in class c , $p(t|c)$ can be computed by its overall term frequency (in all documents) over the total term frequency of documents in that class. Given term frequencies in Table [tab-text-emails_tf], the following terms' probabilities in the spam class can be estimated by³:

³Again, we add a Laplace constant of 1, which is 1 added to the numerator and 2 added to the denominator.

12. Text and Human Language

$$\begin{aligned}
 p(\textit{nice}|\textit{spam}) &= (0 + 1)/[(1 + 3 + 3 + 2 + 2 + 2) + 2] \\
 &= 1/15 \\
 p(\textit{prize}|\textit{spam}) &= (3 + 1)/15 \\
 &= 4/15 \\
 p(\textit{you}|\textit{spam}) &= (3 + 2 + 2 + 1)/15 \\
 &= 8/15
 \end{aligned}$$

Likewise, these terms' probabilities in the ham class can be estimated by:

$$\begin{aligned}
 p(\textit{nice}|\textit{ham}) &= (1 + 1)/[(2 + 1 + 3 + 1 + 1 + 2 + 2 + 1 + 3 + 1) + 2] \\
 &= 2/19 \\
 p(\textit{prize}|\textit{ham}) &= (0 + 1)/19 \\
 &= 1/19 \\
 p(\textit{you}|\textit{ham}) &= (2 + 1)/19 \\
 &= 3/19
 \end{aligned}$$

Now given the following new document to be classified:

	1	2	3	4	5	6	7	8	9	10	
DOC	meet	nice	prize	project	research	schedule	select	update	win	you	Cl
<i>d</i>		1	3							2	?

Document *d* can be regarded as a sequence of terms that are drawn from their probability distributions. And the probability of generating the

12.4. Text Classification

document $p(d)$ is proportional to the joint probability of drawing the individual terms, following the Multinomial distribution⁴:

$$\begin{aligned} p(d) &\propto \prod_{k=1}^{n_d} p(t_k) \\ &= \prod_{t \in T} p(t)^{TF_{dt}} \end{aligned}$$

where n_d is the length of document d (the total number of terms including duplicates) and t_k is term t at position of k in the document. In the bag-of-words approach, positional information is irrelevant so $p(t_k) = p(t)$ for any position k . T is the dictionary of unique terms, and TF_{dt} is term frequency of term t in document d .

If document d belongs to be particular class c , the posterior probability can be computed in the same manner:

$$p(d|c) = \prod_{t \in T} p(t|c)^{TF_{dt}}$$

For document d in Table [tab-text-new_tf], IF it belongs to the spam class, then the log-likelihood of having document d is⁵:

⁴With the bag-of-words approach, the Multinomial probability mass function has to include as a factor $\frac{n_d!}{\prod_t TF_{dt}!}$ because of the permutations the terms can appear in. This Multinomial coefficient, however, is a constant given a fixed document d , is the same for all classes and can be omitted.

⁵Note that in the calculation we omit terms that do not occur in document because a 0 TF value will lead to 0 in the log-likelihood and does not affect the result.

12. Text and Human Language

$$\begin{aligned}
 \log p(d|spam) &= \log \prod_{t \in T} p(t|spam)^{TF_{dt}} \\
 &= \sum_{t \in T} TF_{dt} \log p(t|spam) \\
 &= 1 \log p(nice|spam) + 3 \log p(prize|spam) + 2 \log p(you|spam) \\
 &= \log \frac{1}{15} + 3 \log \frac{4}{15} + 2 \log \frac{8}{15} \\
 &= -3.444
 \end{aligned}$$

Likewise, we can compute the log-likelihood of d IF it belongs to the *ham* class instead:

$$\begin{aligned}
 \log p(d|ham) &= \sum_{t \in T} TF_{dt} \log p(t|ham) \\
 &= 1 \log p(nice|ham) + 3 \log p(prize|ham) + 2 \log p(you|ham) \\
 &= \log \frac{2}{19} + 3 \log \frac{1}{19} + 2 \log \frac{3}{19} \\
 &= -6.417
 \end{aligned}$$

Finally, we can estimate the posterior probability of document d belonging to either spam or ham by using the Bayes theorem. Again, the log-likelihood is used:

$$\begin{aligned}
\log p(\text{spam}|d) &= \log \frac{p(\text{spam})p(d|\text{spam})}{p(d)} \\
&= \log p(\text{spam}) + \log p(d|\text{spam}) - \log p(d) \\
&= \log 0.43 - 3.444 - \log p(d) \\
&= -3.811 - \log p(d) \\
\log p(\text{ham}|d) &= \log \frac{p(\text{ham})p(d|\text{ham})}{p(d)} \\
&= \log p(\text{ham}) + \log p(d|\text{ham}) - \log p(d) \\
&= \log 0.57 - 6.417 - \log p(d) \\
&= -6.66 - \log p(d)
\end{aligned}$$

In this case, $\log p(\text{spam}|d) > \log p(\text{ham}|d)$ so $p(\text{spam}|d) > p(\text{ham}|d)$ and document d should be labeled a spam.

Back to the general model of Naive Bayes with a Multinomial distribution, the idea is to consider term frequencies in the model and identify the class that maximizes the log-likelihood of the document's membership to that class. That is⁶:

$$\begin{aligned}
\hat{c} &= \arg \max_{c \in C} \log p(c) \prod_t p(t|c)^{TF_{dt}} \\
&= \arg \max_{c \in C} \log p(c) + \sum_{t \in T} TF_{dt} \log p(t|c)
\end{aligned}$$

where c is each class in the set of classes C , t is each term in dictionary T , $p(t|c)$ is the probability of term t in class c , and TF_{dt} is the term frequency of t in document d , the document to be classified.

⁶Here we omit $\log p(d)$ as it is constant for all classes.

12. Text and Human Language

Put it into practice

Continue with the corresponding practice activity to turn **Naive Bayes with Term Frequencies (Multinomial)** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **Text Classification** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Summary

The basic method for text processing is the *bag of words* approach, in which we disregard the order of *tokens*. In practice, for the English language, tokens are often single words which can be obtained using word boundaries such as space and punctuation. Nonetheless, a token can be a phrase or any number of characters in a sequence (character n-gram). For example, you may treat every four consecutive characters (4-gram) in a document as a token and use all unique 4-gram tokens as your vocabulary.

Some form of token normalization can be helpful. However, the extent of its use depends on the language, data domain, and use cases, e.g. how users express their queries vs. words used in written documents of the data collection.

Once tokens or terms are obtained and normalized, we can then assign certain weights to them for the representation of text documents. We have discussed term weighting schemes including the binary, term frequency (TF), and TF*IDF (Inverse Document Frequency). TF*IDF is a classic

treatment where a term's weight increases with the increase of term frequency in the current document and its rarity in other documents. IDF, or a term's rarity, indicates the degree of *specificity* and *informativeness* of the term.

Text documents can be treated as vectors based on the term weights. When comparing two vectors in the vector space model, we noticed that vector direction is more important than vector magnitude (length). Whereas the magnitude is a function of the absolute values of term weights, the vector direction is due to the distribution of these (relative) weights. Because of this nature, we often use an angle-based distance function such as cosine similarity to compare two vectors.

The probabilistic view provides an important approach to text processing. The Zipf's is an empirical generalization of the probabilistic occurrences of spoken or written words. The Bayes theorem lays the foundation for probabilistic modeling of problems such as text classification and retrieval.

13. Search and Retrieval

Ask, and it shall be given you; seek, and you shall find; knock, and it shall be opened to you.

The Gospel of Matthew

We are in constant pursuit of information, knowledge, and truth. We have questions and we ask. Today, we ask questions and seek information with the help of technologies. We search for information and web pages by Googling. We pose questions to digital assistants such as Amazon's Alexa. Regardless of differences in the specifics, the essence of these interactions is to find the right (relevant) piece of information that can satisfy us (the user) – be it a web page, a short answer, a prediction or some kind of recommendation.

This is about search and the general area of research is commonly referred to as *Information Retrieval*. In 1951 – when searches for information were primarily conducted in the libraries with the assistance of reference librarians without a computer – Calvin Mooers coined the term *Information Retrieval* as "the investigation of information description and specification for search and techniques for search operations." (Mooers 1951) A library catalog with index cards was the state of the art to help people seek and find quickly.

With the support of computers, we can perform the same type of search tasks even more efficiently, yet with ease on the user's side and sophistication on the system's side. Now that computers are essential to the various search systems we have in place, the notion of *Information Retrieval* (IR) may be redefined, in a narrower sense, as: (Manning et al. 2008)

13. Search and Retrieval

finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers)

With this narrower definition, a basic task of IR is, given a user query, to find *relevant* documents from a collection of text materials. Traditionally, the data collection such as a library of books is assumed to be static, and certain pre-computation and indexing can be conducted before querying processing. In reality, this assumption does rarely hold as data can grow to include new materials which in turn should be incorporated into the index.

Web search engines are *Information Retrieval* (IR) systems and the Web is a huge collection of data that constantly grow and change. The magnitude, dynamics, and decentralization of data on the Web pose great challenges, which have led to waves of innovation on Web IR¹. We will discuss some of these challenges and related ideas later in the chapter.

13.1. Basic Paradigm

In information retrieval, the basic problem is to search and find matched documents against a query representation based on a user's information need. Documents need to be stored in proper representation so that related matches can be computed (numerically). An index is also necessary to conduct the matching process timely.

¹First came Lycos and Yahoo, which borrowed traditional text indexing and reference catalog/directory techniques from libraries. AltaVista enhanced the technologies with a larger index and a responsive interface. Google and Baidu came after them and revolutionized search ranking by using *citation analysis*, another method that had been studied extensively in library science. Now very few people know AltaVista, not to mention Lycos. Looking forward, what will come in the next ten or twenty years? Will Google remain relevant then.

13.1. Basic Paradigm

As shown in Figure 13.1, the user with an information need issues a query to the retrieval system, which in turn match it against the document representation in the store to identify relevant documents in the search result. The result can also be ranked (sorted) in terms of predicted relevance of each document.

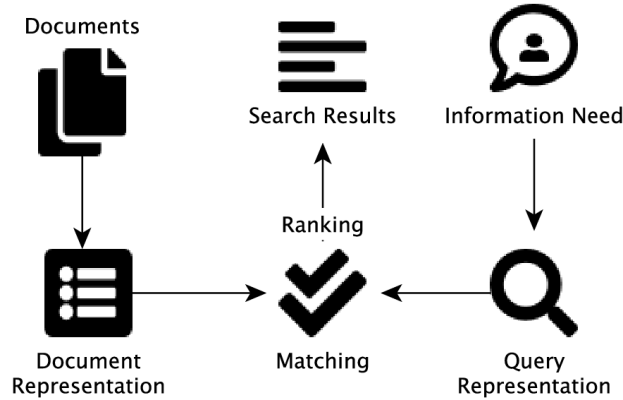


Figure 13.1.: Basic paradigm of an *Information Retrieval* system

The notion of *relevance* is key in information retrieval research. Yet it remains one of the least understood concepts in the field. Whether a piece of information is relevant depends on many factors such as the user's objectives and the context of the current task. Sometimes, these may be difficult for the user to articulate; and even if she does, the query expression thus constructed may hardly be precise.

Retrieval systems have to attack the issue of uncertainty often associated with understanding the user's need and what is truly relevant in context. We can factor this in when building a retrieval model. With probabilistic models, for example, the degree of uncertainty can be quantified for proper probabilistic ranking in light of probability and information theories.

However, uncertainty is often due to lack of information (data) and requires

13. Search and Retrieval

further data collection. Techniques that support relevance feedback and interactive, exploratory searching may help engage the user in order to understand more precisely what she is looking for.

Put it into practice

Continue with the corresponding practice activity to turn **Basic Paradigm** into a calculation, implementation, visualization, diagnostic, or interpretation task.

13.2. Indexing

For now, let's assume the basic model presented in Figure 13.1 and walk through some of the major ideas for finding matched (and potentially relevant) documents.

In Figure 13.1, the data collection can be very large with a great number of documents, e.g. a library of a million books. In order to conduct searches with matching and ranking responsively, some amount of data pre-processing is necessary. This is also practically sensible if the collection is relatively static (no dramatic changes frequently).

In chapter , we discuss basic processes for text vectorization. The result of vectorizing a text collection can be represented by a matrix with documents as rows and terms as columns. Each document here is a vector of values corresponding to terms in the dictionary. These values can be obtained using methods such as binary, term frequency (TF), or TF*IDF term weighting scheme.

Assume we start with the binary representation, which is very simple with 0s and 1s, for a collection of 1,000 documents each containing a dozen words. In terms of the Heaps' law, we probably obtain somewhere close to 1,000 unique terms, leading to a large matrix in the order of million

values. With an even larger collection, the entire matrix representation can become too expensive to be stored and computed.²

Now if we examine actual values of each document (row) in the matrix, we will quickly realize that most values are 0 (zero). This is due to the fact that even though we have a larger number of unique terms in the entire collection – thus many columns in the matrix – each document (row) only contains a small number of these terms. So instead of storing the entire matrix with all values, we can use a *Sparse Matrix* representation where only non-zero values are stored.

On the document level, this means each document is a *Sparse Vector* that only stores data about what terms actually appear in it. With the sparse representation, we can significantly reduce the amount of storage needed for document representation (i.e. better space efficiency).

However, the use of sparse vectors does not necessarily address the issue of query processing efficiency. That is, even with document sparse vectors, it still requires lots of computational effort on the system side to sift through all documents to find matches for a query. The amount of time to perform this matching task increases linearly with the increasing number of the documents. For searching a very large data collection such as the Web, this will become too slow to serve users on the fly.

Efficient query processing requires a better data structure that supports: 1) fast lookup of individual query terms with their associated documents, and 2) fast merging of all terms in the query expression. The 1st objective can be achieved if the terms are sorted where a binary search can be performed. Each of the terms on the sorted list can then point to the list of documents (using their IDs) containing it. We refer to such a sorted structure of terms with references to documents as an *Inverted Index*. Each term is now a

²On the benchmark Reuters RCV1 text collection, the observed Heaps' function indicates that the vocabulary size M , i.e. the number of unique terms, is proportional to $T^{0.5}$, where T is the total number of tokens. Given that T is about linearly associated with the number of documents N , the number of values in the matrix representation is $N \cdot M$, which is $\propto N^{1.5}$.

13. Search and Retrieval

vector of document IDs, inverted from the original document vectors of terms.

When presented with a single-term query, the system can conduct a binary search to quickly identify all documents matching (containing) the term from the inverted index. With a query expression of multiple terms, each term can be looked up in the same manner.

But how can we merge documents associated with the individual terms to find out which documents actually satisfy the entire query, where, either explicitly or implicitly, terms are used in a logical context with AND, OR, and/or NOT?

To achieve the 2nd objective of fast merging, documents for each term in the inverted index should also be sorted (by their IDs). So in the end when an inverted index is built, all the terms will be sorted, each of which in turn contains a list of sorted document IDs. In the next section, we will look at a couple of methods to see how we can take advantage of the sorted lists to merge query terms for document-query matching.



Put it into practice

Continue with the corresponding practice activity to turn **Indexing** into a calculation, implementation, visualization, diagnostic, or interpretation task.

13.3. Matching

One who is interested in reading about the U.S. revolution may issue a query "*The Declaration of Independence*" on a collection of historical artifacts to find that particular document. If we assume the system treats words *the* and *of* as stop-words, they will be disregarded in the query processing. In this case, the query has two terms, namely *Declaration*

13.3. Matching

and *Independence*, which can be further interpreted as *Declaration* AND *Independence* assuming the user is looking for both terms.

Given an inverted index with a sorted list of all unique terms in the collection, it takes binary search logarithmic time to find each term's entry. That is, given M terms in the index, the system makes $O(\log M)$ comparisons to find *Declaration* and *Independence* in the worse case. This is logarithmic time complexity is rather efficient and scalable. An example of the two term entries found in the inverted index may look like this:

Declaration	→	2	3	10	17	103	170	181	200	237	253	301	400	456	480	490	...
Independence	→	1	2	5	20	30	40	103	210	405	406	455	456	457	459	460	...

Figure 13.2.: Two postings from the inverted index. Highlighted document IDs with double border indicate matched documents that satisfy *Declaration* AND *Independence*.

We now refer to each term's list of documents – in fact only their IDs – as the *posting* of the term. An inverted index is thus a collection of *postings*. After examining the two postings, we can conclude that documents #2, #103, and #456 in Figure 13.2 contain both terms. But how can an algorithm merge the two postings to identify these matched documents?

A very simple approach is to go through the documents IDs in the posting for term *Declaration* and, for each of them, we compare it with every document ID in the second posting for term *Independence*. Suppose the first posting has n_1 documents and the second has n_2 , this will conduct $n_1 \cdot n_2$ comparisons. This approach is going to be very slow with large postings and it has yet to take advantage of the sorting (pre-computation) that has been performed on the document IDs.

Apparently, we can improve on the above brute force method is by conducting a binary search on the second posting for each document on the first posting. In this case, the time complexity of the matching process will become $O(n_1 \log n_2)$ as the binary search is of $O(\log n_2)$ complexity.

13. Search and Retrieval

Assume $n_2 > n_1$ is a large number, this will significantly reduce query processing time.

So far we have used one posting as the primary to iterate through and, in the process, compare each value there to those in the second posting. In this approach, the two postings play unequal roles in the matching process.

A different approach is to take the postings equally and iterate through them simultaneously. Assuming document IDs are sorted in the ascending order, we start at the beginning of each posting. In each step, we compare the current values of the two postings and the one with a small value will move to the next (greater) value. This continues until it has been found a value greater or equal to the current value of the other posting. In that case, we switch to the other posting to go next.

For the above example, we start at first document ID in each posting shown in Figure 13.3, where the *Declaration* posting has a value 2 and the *Independence* posting has 1 (see step 1 in the figure). As the one with a smaller document ID, the *Independence* posting will move to the next document ID, which is 2 and matches the the current value in *Declaration* (see step 2).

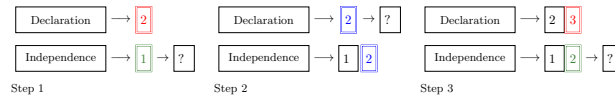


Figure 13.3.: Simultaneous walking to merge postings, starting at the first document ID. The posting with the smaller value will move to the next ID.

Now that the second posting (*Independence*) is already with an equal or greater document ID, the process will shift to the first posting (*Declaration*) to catch up, which, as shown in Figure 13.3 step 3, then goes to document

#3 and switches the turn back as it surpasses 2. This process will continue until no more matches can possibly be found.

In the worse case scenario, it iterates through the two postings from start to end, which is $O(n_1 + n_2)$ time complexity. Practically, this is better than the previous method with $O(n_1 \log n_2)$ complexity where $\log n_2$ may remain a large number ($\gg 2$).

These techniques are particularly useful to merge postings for logical AND, i.e. for document containing both or all query terms. They also work well with the OR query expressions, where documents containing any of the terms should be brought together after the removal of duplicate IDs.

The simultaneous iterative process can be further improved with skip pointers that the iterations faster by skipping a significant amount of document IDs in each posting. However, the skip pointers technique is of little use for OR queries as it skips documents that should be included with a logical OR.

Put it into practice

Continue with the corresponding practice activity to turn **Matching** into a calculation, implementation, visualization, diagnostic, or interpretation task.

13.4. Ranking

With techniques presented in the previous section, we know how to process a query with an inverted index and identify matched documents by merging postings for the query terms. This can be conducted rather efficiently if proper preprocessing is already in place, e.g. with sorting and skip pointers.

13. Search and Retrieval

If the user can articulate her need for information with relevant query terms in a precise boolean expression, then the system should be able to serve the user's need via matching. There is potentially a long list of documents that match the query. In the old days, this perhaps worked just fine for scholars who would like to find a list of research publications for a comprehensive literature review.

In the context of today's World Wide Web and online social media, it will not be helpful by retrieving an extremely long list of pages and presenting them all at once to the user. For one, the query expression of any ordinary user is most likely imprecise and a lot of matched documents will turn out to be non-relevant.

More important, many information seekers on the web are not looking for comprehensive literature. Often they are in need of the one page, if not a few, that can satisfy their need or at least something they can start with. In these cases, the challenge is not about the thoroughness of the result, but how precise and relevant the result is.

The common practice of today's web search engines is to present a ranked list of matched documents where, ideally, the most relevant documents will appear early on so the user can see them and follow with a clickthrough. So in addition to query matching, document ranking (sorting) in the retrieved result is essential for the success of a search engine.

13.4.1. Content-based Ranking

So in what order shall we rank documents or pages according to a query? Among the matched ones, what documents should be presented to the user early in the result?

As the key to *information retrieval* (IR) is relevance, the first reaction to the questions here is to rank documents according to how *relevant* they are to the query. But how do we quantify relevance? Without the user's subjective feedback on what is relevant vs. what is not, there is simply no

data to quantify relevance. The alternative is to assume that documents that match the user's query expression to a higher degree are more relevant to her information need.

Based on this assumption, there are several approaches we can take. For example, using the *Term Frequency* (TF) scheme discussed in chapter , we can count the total number of query term occurrences in each document. The matching score of a document d with regard to query q can be therefore computed by:

$$score(q, d) = \sum_{t \in q \cap d} TF_{t,d}$$

where t is each term appearing in both the query and the document, and $TF_{t,d}$ is the term frequency of t in d , i.e. the number of times it appears in the document.

Now one would argue that this scoring function will not work well as it give equal importance to each query term. With this method, one who issues a query like "the declaration" may retrieve a bunch of documents with lots of "the" in the writing (suppose "the" has not been removed as a stop-word from being indexed). We may therefore consider the relevance of individual terms by replacing the TF with TF-IDF weights (Term Frequency * Inverse Document Frequency):

$$score(q, d) = \sum_{t \in q \cap d} TF_{t,d} \cdot IDF_t$$

where IDF_t is IDF weight of term t .

Both TF and TF-IDF scoring functions above are dependent on the terms' occurrences. Apparently, longer documents (e.g. books) are more likely to contain more instances of the same terms than the shorter documents. To remedy for this bias, one can normalize term frequency by the document length:

13. Search and Retrieval

$$score(q, d) = \sum_{t \in q \cap d} \frac{TF_{t,d}}{L_d} \cdot IDF_t$$

where L_d is the length of document d , i.e. the number of terms in the documents including duplicates. Now that we factor in document length, short documents will have *equal* chances to be ranked and long documents will no longer dominate³. After all, for any document d , $\sum_t \frac{TF_{t,d}}{L_d} = 1$ regardless of its length. This is after the fact that the overall probability of *any term* drawn from the document is always 1.

The document-length-normalized TF is a probability estimate. Imagine if we put all the terms of document d in a black box and randomly draw a term from it, the likelihood of getting term t can be estimated by $\frac{TF_{t,d}}{L_d}$. The length-normalized scoring function here is essentially modeled on the probability of each query term's occurrence in the document.

Like document-length normalized TF, IDF is also based on a probability estimate. As discussed in chapter , a term's IDF weight is also a measure based on the term's likelihood to appear in any document drawn randomly from the entire data collection. The component $\frac{n_t}{N}$ in the IDF function, where N is the total number of documents and n_t the number of documents containing term t , is to estimate the term's collection-wide probability. The logarithmic transformation:

$$\begin{aligned} IDF_t &= -\log \frac{n_t}{N} \\ &= \log \frac{N}{n_t} \end{aligned}$$

does convert dramatically varied probability values into a more comparable scale and dampens their differences. However, this formulation is not

³An alternative to document length normalization is to use *cosine similarity* instead of a sum of term weights. Cosine is focused on the angle proximity (vector direction) in a vector space, which is essentially about terms' distributions rather than their absolute occurrences. See related discussions in chapter .

accidental but has in fact a concrete theoretical root in the information theory. IDF_t can be derived from measuring the amount of *relative entropy* (KL divergence) when term t is observed in a document from the collection. (Aizawa 2003)

In light of uncertainties involved in understanding user needs to quantify relevance, one may also think of ranking documents in terms of the *probability of relevance*. Indeed, early probabilistic IR research followed the *Probability Ranking Principle* (PRP), which states that documents should be ranked in order of the probability of relevance, or how likely documents will be useful to the user. (Robertson 1977) The principle can be operationalized by estimating those probability from data, e.g. data on term distributions and user relevance feedback.

Based on PRP and a logarithmic transformation of the Bayes rule, one can model the odds of a document's relevance given a query and derive a weighting function similar to IDF. This is known as the *Binary Independence Model* (BIM), where the notion of relevance is binary (either relevant or not relevant)⁴ and documents are assumed to be independent of one another (Robertson and Sparck-Jones 1976). The relevance weight w_t^{RSJ5} of a term t in BIM is computed by:

$$w_t^{RSJ} = \log \frac{(r_t + 0.5)(N - R - n_t + r_t + 0.5)}{(n_t - r_t + 0.5)(R - r_t + 0.5)}$$

where R is the number of relevant documents, n_t is document frequency of term t , and r_t is the number of *relevant* ones in the n_t documents. The weight approximates IDF when both the number of relevant documents r_t is much fewer n_t , which in turn is much smaller compared to the entire collection N . That is, $w_t^{RSJ} \approx IDF_t$ when $r_t \ll n_t \ll N$.

⁴This is not to confuse with the fact that the probability (of relevance) is still a continuous measure, not a binary one. In other words, even though the final answer is either yes (1) or no (0), the probability of yes vs. no remains a continuous value between 0 and 1.

⁵RSJ stands for Robertson and Spark-Jones, the authors of the weighting scheme.

13. Search and Retrieval

Further development of this probabilistic model led to another well-known method called *BM25* (Robertson and Zaragoza 2009), where a term's weight is computed by:

$$w_{t,d}^{BM25} = \frac{TF_{t,d}}{k_1(1-b) + b\frac{\bar{L}_d}{\bar{L}} + TF_{t,d}} \cdot w_t^{RSJ}$$

where $\bar{L}$ is the average document length in the collection, and k_1 and b are parameters to be optimized (tuned) through experiments. b adjusts the degree of document normalization, with full normalization at $b = 1$ and no normalization at $b = 0$. This highly resembles TF*IDF, where w_t^{RSJ} corresponds to IDF_t and the rest of the formula is $TF_{t,d}$ with document-length normalization. The scoring (ranking) function for BM25 is then the sum of term weights:

$$score(q, d) = \sum_{t \in q \cap d} w_{t,d}^{BM25}$$

Put it into practice

Continue with the corresponding practice activity to turn **Content-based Ranking** into a calculation, implementation, visualization, diagnostic, or interpretation task.

13.4.2. Popularity-based Ranking

With document ranking methods such as those based on TF*IDF and BM25, an *information retrieval* system will be able to sort documents according to their match scores, especially in terms of important keywords in the user query. This worked well in the traditional library context, where documents were in fact selected publications such as books and articles. Because of the editorial processes these publications went through, the

13.4. Ranking

contents of the documents were trustworthy in general. Ranking based on document content is a sensible solution in that setting.

When the World Wide Web started to grow in the 1990s, the first generation of web search engines adopted the same technologies developed in the libraries and continued to rely on text contents in their ranking. It soon became clear that this is easily susceptible to spamming.

The Web is a decentralized space for publication without a central editorial process. Everyone is a publisher who writes anything he wants, in his own way of expression. A ranking method based on $TF \cdot IDF$, for example, is impacted by TF and IDF. You probably can exert very little influence on the IDF part as it is a *global* statistic of the entire web collection. However, if you want to be ranked number one for queries like "The Declaration of Independence," you simply repeat those keywords in your page contents.

By simple repetition in the text, you increase the terms' frequencies (TFs) and therefore the ranking of your page to related queries. The point here is that page contents on the web are not necessarily trustworthy and we cannot simply rely on the *claims* of authors in this rather decentralized, autonomous environment.

Without a central authority that we can trust on individual pages, we may perhaps consider input from their *peers* – those who know these pages and what they have to say about them. On the web, pages reference one another via *hyperlinks*. Figure 13.4 shows an example of a web page, from which the user can click on a label (text), a button, or an image to visit another page.

With hyperlinks, we pages form a directed network on which structural analysis can be performed. And when a text label (i.e. anchor text) is provided as part of the hyperlink, it provides useful data about what the target page is about.

In Figure 13.4, when page A links to page B, descriptions or keywords page A uses on the hyperlink can be considered for the representation of page B. Assuming the pages are on different web sites with separate ownership,

13. Search and Retrieval



Figure 13.4.: Page references via hyperlinks. Page A links to several other pages, including B and C.

then this piece of anchor text represents an independent evaluation of page B's contents. By aggregating anchor text data from all other pages linking to page B, its inverted index can be adjusted with the crowd-sourced data, which are potentially more reliable than self-claimed page contents.⁶

Based on the classic BM25, BM25F is an example of related methods that go beyond the body text of a web page to include other structural elements (fields) and relevant data streams (particularly anchor text data). (Zaragoza et al. 2004) Among others, the use of anchor text data for indexing and matching is a common practice of web search engines.

When the user issues a search query, the question remains what is more *relevant* and should be ranked higher in the result. For web searches, *relevance* does not merely mean *on-topic* because a very large number of pages will be considered on topic in terms of their contents (anchor text included). The notion of *relevance* also entails certain degree of *importance* while on topic.

⁶Anchor text can also be hacked and abused to boost one's search engine ranking. There has been known practice of self- or mutual-linking from sites set up by the same group of people. On the other hand, websites can team up to *bomb* a target with off-topic anchor text in order to "promote" its ranking for those keywords, e.g. for political activism.

13.4. Ranking

Importance, however, can be subjective and hard to quantify. To borrow an idea from *scientometrics*, we can instead substitute *importance* with *impact* or *popularity*. While in scholarly communication scientific research is evaluated based on the amount of citations, web pages can be treated likewise in terms of hyperlinks.

If we regard each hyperlink as a citation to the target page, we can now measure a page's impact or popularity by counting the number of incoming links, i.e. *in-degree*. That is, the impact score R_d of a target document (page) d is computed by:

$$r_d = \sum_{p \in P_d} 1$$

where P_d is the collection of pages linking to d . For each linking page p , the contribution is considered equal with 1. In Figure 13.5, for example, page t has two in-links from pages p_1 and p_2 and its impact score is therefore 2.

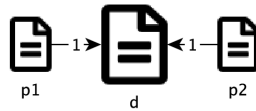


Figure 13.5.: Two links being equal. Without more data, the two links are assumed to contribute equally to the target page's impact.

But are all hyperlinks created equal? Let's consider an extreme case where we have more data for the two linking pages in Figure 13.5. Suppose p_1 is an ordinary page that is linked to by another page but links to many others (including page d). On the other hand, p_2 is a very popular page that is referenced by many pages but has chosen to link to only one page (i.e. page d). Figure 13.6 illustrates this scenario.

Based on the above hypothetical scenario, it is apparent that the links from p_1 and p_2 to the page t are not equal. For one, p_2 is much more

13. Search and Retrieval

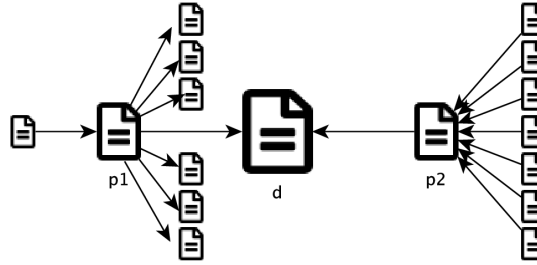


Figure 13.6.: Two links being different. When we know more about the pages linking to the target page, we can no longer assume their contributions are equal.

popular and has more impact than p_1 given the number of in-links it receives. Moreover, it gives out the only one link to page d , making its contribution (endorsement) even stronger. On the contrary, p_1 receives very little (from only one page) and gives out a lot, further diluting its already small share.

In reality, p_2 can be the page of an influential figure like the Pope who receives lots of media attention. If he chooses to post only one link and that goes to your page, you must be thrilled. We can also think of p_1 as the total opposite, a page of yours or mine with a long list of notes and references. The longer the list goes, the less significant each reference may become.

This discussion sheds lights on two important aspects for scoring a page's impact or popularity. First, it matters who gives the link and how impactful the *link giver* is. A link from an important figure should carry more weight in itself. Second, it also matters how many links the giver *gives out*. When one gives out to more recipients, the share of each link is diluted and the weight should be divided by the number of out-links. Mathematically, the second aspect can be easily quantified by counting the out-links for each page in P .

On the first point, however, the solution does not appear straightforward. Again in Figure 13.6, the impact of document d depends on the impact of linking pages p_1 and p_2 , which in turn depend on the ones that link to them. And this will go on. How can we proceed to solve this problem of *chicken and egg* without going into a loop? But the solution is indeed a loop, or a recursive process.

 Put it into practice

Continue with the corresponding practice activity to turn **Popularity-based Ranking** into a calculation, implementation, visualization, diagnostic, or interpretation task.

13.4.3. PageRank

The *directed graph* of web pages pointing to one another can be thought of as a city of one-way streets (hyperlinks) you can follow and locations (pages) that you can visit via these streets. If you want to simply explore the city, you can randomly pick a street to get where it leads to. Then from there, you can pick a random street again and continue on.

Imagine if you continue this over and over again, ultimately you may be able to reach every corner of the city (if every location is connected). Over time, you will be able to visit some locations for many times. And chances are, you will visit some locations more often than others – in other words, some locations are more *popular* than others on your tireless, random tour of the city.

So which ones will be visited more frequently by chance? Apparently, those with more incoming streets, which in turn have more incoming streets, and so on. And this appears to match the problem we had earlier, where we wanted to calculate the impact or popularity score of a page, which depends on the impact of linking pages, which in turn depends on their linking pages, and so on.

13. Search and Retrieval

In this thinking, the impact or popularity score of a web document (similar to a location in the above imaginary city) is equivalent to its *chance* (frequency) of being visited on a random walk over time – that is, its *long-term visit rate*.

This is essentially the *Random Surfer* (random "walker" on the web) model proposed for the Google's original *PageRank*⁷ function. (Page et al. 1999) In this model, the computer simulates a user following the link structure of the web after pages have been collected (crawled) and parsed. It starts at any page and, at each step, goes out along one (randomly chosen) of the links on page to reach another. Ultimately, each page will have a long-term visit rate if this simulation is conducted with sufficient time (to reach the steady state).

There is one problem, however. Some pages do not have out-links – they are dead ends where the surfer will get stuck. To avoid this issue, the surfer should be allowed to jump (*teleport*) out of the current page to visit another random page. Also for pages with few or no in-links to be visited a bit more frequently – so that they are not totally disregarded in the ranking model – the surfer will *teleport* periodically regardless of out-links. The probability associated with this periodic teleporting is referred to as the *damping factor*. With this model, the impact score r_d of a page d is equivalent to the long-term visit rate, which is computed by:

$$r_d = \alpha \sum_{p \in P_d} \frac{r_p}{n_p} + \beta$$

where n_p is the out-degree (number of out-links) of page p , which links to d and is having a current score of r_p . On the right side of the equation, the first part $\alpha \sum r_p/n_p$ is the score one receives from in-links and the term β represents the amount each page is given "for free." Both α and β can be regarded as parameters related to the damping factor.

⁷While PageRank is indeed about ranking *pages* on the web, the name is in fact after its primary author and Google co-founder, Larry Page.

13.4. Ranking

For example, with a periodic teleporting probability of 0.15, we can set $\alpha = 1 - 0.15 = 0.85$ for the earned portion (from links) and $\beta = 0.15 \frac{1}{N}$ for the "free" portion, where N is the total number of documents (pages) in the collection.⁸ In this way, β represents the chance each page will be visited with teleporting.

With equation [eq-pagerank], one needs to follow the recursive approach of a random surfer to calculate the ultimate PageRank scores for all pages. Because we do not know these values (i.e. the r_p values), which are needed to start the computation (i.e. to calculate r_d), we will kick off with arbitrary initial values, for example, by setting every page's score to $1/N$ – that is, they are treated equally likely to be visited before we proceed to find out the actual values. In each iteration, we update the all pages' scores using equation [eq-pagerank] until there is very little or no change at all to the scores.

In the end, the PageRank score of a page corresponds to the probability that the page will be visited by the random surfer. The more prominent a page is in the web graph, the more likely (frequently) it will be visited and therefore will have a higher PageRank score. PageRank scores are query independent as they are only determined by the network structure. When a search engine ranks pages with regard to a specific user query, results should first limited to those meeting query criteria before PageRanks are combined with query-related weights such as those based on TF*IDF.

Although the basic version of PageRank computes its scores regardless of a query, one can easily modify equation [eq-pagerank] to make it query-dependent. For example, the β does not have to be a universal constant for all the pages. If we assume each document (page) can have its individual β_d value, then the PageRank function becomes:

$$r_d = \alpha \sum_{p \in P_d} \frac{r_p}{n_p} + \beta_d$$

⁸The original PageRank paper used 0.15 for the damping factor based on earlier experimental results.

13. Search and Retrieval

where we may assign different β_d to pages based on, for example, other indicators about their relevance to a search query.

PageRank, which is essentially a *centrality* measure in network analysis, is an extension of computing eigenvector and Katz centrality.(Newman 2010) The Hyperlink-Induced Topic Search algorithm, or HITS, can be regarded as a further extension of PageRank.(Kleinberg 1999) While PageRank only considers contributions from in-links, HITS gives credit to pages that point to others (out-links) as well. With *hub* (centrality based on out-links) and *authority* (centrality based on in-links), the algorithm propagates the scores in a similar iterative process until stabilized.

With HITS, the analysis is performed on a sub-collection including pages relevant to a query (text-wise) as well as other pages with in-links from and/or out-links to the relevant ones. This setup makes HITS query-dependent and the relevance to a query is reinforced through the propagation of *hubs* and *authorities*.

Put it into practice

Continue with the corresponding practice activity to turn **PageRank** into a calculation, implementation, visualization, diagnostic, or interpretation task.

Put it into practice

Continue with the corresponding practice activity to turn **Ranking** into a calculation, implementation, visualization, diagnostic, or interpretation task.

13.5. Information Filtering

Information Filtering (IF) looks at the same information seeking problem from the opposite angle of IR – instead of reaching out to find information, IF is to eliminate the non-relevant so that the user only receives the relevant. IR and IF are considered to be “two sides of the same coin.” (Belkin and Croft 1992)

Information filtering usually operates on an ongoing information stream such as emails. Look at how many junk emails or spams⁹ you receive daily and you see the importance of having a good spam filter. Filtering for recommendation, e.g. on news and movies, is another popular application of information filtering.

One major challenge in information filtering is the information stream that constantly feeds and changes in its content. Patterns that you have discovered from past spam emails, for example, may turn out to be useless for future spam filtering as spammers learn and change their behavior. News that you used to follow closely may become irrelevant in the next news cycle or as your interests move on. It requires constant learning of the user and the environment to adapt to the ongoing changes.



Put it into practice

Continue with the corresponding practice activity to turn **Information Filtering** into a calculation, implementation, visualization, diagnostic, or interpretation task.

⁹We refer to relevant emails as *hams* and the irrelevant as *spams*.

14. Graphs and Structural Analysis

14.1. Graphs as Data

 Put it into practice

Continue with the corresponding practice activity to turn **Graphs as Data** into a calculation, implementation, visualization, diagnostic, or interpretation task.

14.2. Paths, Connectivity, and Traversal

 Put it into practice

Continue with the corresponding practice activity to turn **Paths, Connectivity, and Traversal** into a calculation, implementation, visualization, diagnostic, or interpretation task.

14.3. Centrality and Influence

 Put it into practice

Continue with the corresponding practice activity to turn **Centrality and Influence** into a calculation, implementation, visualization, diagnostic, or interpretation task.

14.4. Communities and Structural Roles

 Put it into practice

Continue with the corresponding practice activity to turn **Communities and Structural Roles** into a calculation, implementation, visualization, diagnostic, or interpretation task.

14.5. Link Prediction

 Put it into practice

Continue with the corresponding practice activity to turn **Link Prediction** into a calculation, implementation, visualization, diagnostic, or interpretation task.

14.6. From Graph Features to Graph Learning

 Put it into practice

Continue with the corresponding practice activity to turn **From Graph Features to Graph Learning** into a calculation, implementation, visualization, diagnostic, or interpretation task.

14.7. Summary, Exercises, and Lab

Part IV.

Trustworthy and Practical Machine Learning

15. Evaluation and Experimentation

Imagine you are asked to design an algorithm to help detect credit card fraud. Suppose you are told that your algorithm will be tested for its accuracy and, as a way to motivate you, your boss promises a bonus based on its accuracy. Accuracy, as it turns out, can be roughly defined as the fraction of answers that are correct. In the context of credit card fraud detection, one can produce a confusion matrix by cross-examining predictions (model) vs. real outcomes (ground truth):

Table 15.1.: A confusion matrix

	Predicted Positive $\hat{\oplus}$	Predicted Negative $\hat{\ominus}$
Actual Positive $\oplus$	True Positive (TP)	False Negative (FN)
Actual Negative $\ominus$	False Positive (FP)	True Negative (TN)

Based on the counts in Table 1.1, accuracy can be computed by:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

which is again the total number of correct answers (true positive and true negative) divided by the total number of answers.

Given the evaluation based on accuracy, how would you do to maximize your reward? Surely you can develop a very sophisticated algorithm with massive data and effort. There is, nonetheless, a simple response to this

15. Evaluation and Experimentation

challenge. If you look at statistics on credit cards, fraud transactions represent only a very small fraction. Suppose the likelihood of a fraud is 10 out of every 1000. One may simply classify all 1000 transactions as non-fraud and the confusion matrix will become:

Table 15.2.: Confusion matrix based on the zero-effort solution, by classifying all transactions to non-fraud.

	Predicted Fraud	Predicted Non-Fraud
Actual Fraud	0	10
Actual Non-Fraud	0	990

And the accuracy is:

$$\begin{aligned} \text{Accuracy} &= \frac{0 + 990}{1000} \\ &= 0.99 \end{aligned}$$

This solution requires virtually zero effort but its accuracy appears to be pretty good: 99%. There are several reasons why an overall accuracy can be rather misleading, if not completely useless here. For one, the evaluation metric ignores the data distribution among class labels (e.g. fraud vs. non-fraud) and does not account for potential *chance* accuracy. In the above example, it is very easy to cheat and get rewarded just *by chance*. Here you simply favor the dominant class (non-fraud) to boost accuracy.

Second, an evaluation metric should align properly with the purpose of the assigned task. For credit card fraud detection, one can argue that missing a *fraud* transaction is much more costly and undesirable than misclassifying a normal transaction as fraud. In other words, one should be motivated to identify most if not *all* fraud transactions; and the result should be heavily penalized if real frauds are identified as non-fraud (false negative). We shall look into these issues to see what alternatives may offer.

15.1. Precision

Given a skewed distribution of data in the classes, it may be useful to evaluate accuracy within each predicted class. This is equivalent to a metric commonly known as *precision*, which is the fraction of those predicted positive to a class that actually belong to that class. In terms of the confusion matrix Table 1.1, precision is computed by:

$$Precision = \frac{TP}{TP + FP}$$

When both TP and FP are zero, precision is 1 (100%) by definition. Based on the numbers in Table 1.2, we can compute precision for those predicted to be fraud (in the first column of numbers):

$$\begin{aligned} Precision(fraud) &= \frac{0}{0 + 0} \\ &= 1 \end{aligned}$$

This does not seem to resolve the issue with accuracy, to which precision is very similar. This is due to the second reason we discussed, that there should be a greater emphasis and penalty on the *false negative*. Now instead of looking at the columns to compute precision, we can examine the rows and evaluate the same result from a very different perspective.

Put it into practice

Continue with the corresponding practice activity to turn **Precision** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15. Evaluation and Experimentation

15.2. Recall

Recall, a classic evaluation metric, refers to the fraction of actual positive that have been classified positive. In the example here, it is the fraction of real fraud transactions that have been identified as such. Recall can be computed in terms of the confusion matrix Table 1.1:

$$Recall = \frac{TP}{TP + FN}$$

where false negative (FN) is part of the denominator. Applying this to data in Table 1.2, we get a recall of:

$$\begin{aligned} Recall(fraud) &= \frac{0}{0 + 10} \\ &= 0 \end{aligned}$$

By focusing on true positive vs. false negative, recall gives a better indication than precision on the poor performance in fraud detection. Precision and recall are often used together in evaluation. It is attempting to compute an average score of the two so there is only one score to represent the overall performance. The arithmetic mean of precision and recall here is $(1 + 0)/2 = 0.5$, which still appears too good for a zero-effort solution.

Arithmetic mean gives equal weights to both metrics and is not very indicative of the overall performance. We suggest that such an "averaging" measure show an alarming score if one of the scores is extremely low – that is, we will regard a method as bad if it performs miserably in one metric. One function that leans more heavily on the lower score is the F measure of precision and recall:

$$F_{\beta} = (1 + \beta^2) \frac{Precision \cdot Recall}{\beta^2 \cdot Precision + Recall}$$

15.2. Recall

where β value can be adjusted in favor of precision or recall. When $\beta = 1$, this becomes the F_1 measure, i.e. the *harmonic mean*:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

With $\text{Precision} = 1$ and $\text{Recall} = 0$ in the above example, the harmonic mean is $F_1 = 0$ – here, the one zero score (recall in this case) completely dominates and the average suffers.

In applications where a model can be adjusted or tuned, we can evaluate the model based on more than one pairs of precision and recall scores. A common practice is to plot the precision-recall curve. Figure 15.1 shows precision-recall curves of two methods, where method 1 appears to perform better as its curve is closer to the top-right (higher precision and recall).

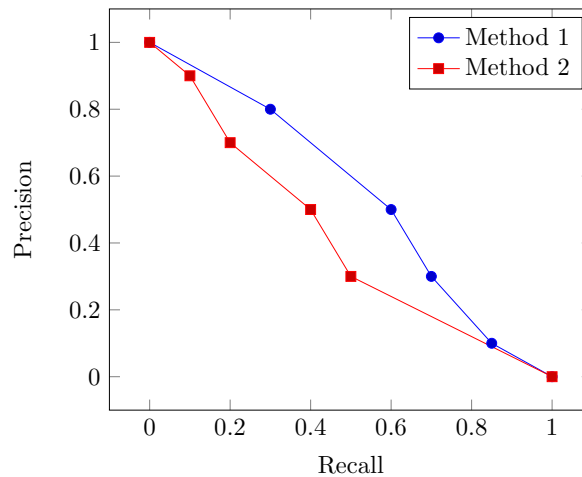


Figure 15.1.: Precision-recall curve

15. Evaluation and Experimentation

Put it into practice

Continue with the corresponding practice activity to turn **Recall** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15.3. Sensitivity and Specificity

Precision and recall are both focused on the fraction of true positive. Recall is also referred to as the *true positive rate* (TPR), sensitivity, or the probability of detection. It measures how *sensitive* a method is in identifying the actual positives.

Conversely, one can also measure the *true negative rate* (TNR) or the fraction of actual negatives that have been correctly identified as such. This is often referred to as *specificity*, which is computed by:

$$Specificity = \frac{TN}{TN + FP}$$

Sensitivity and specificity are concepts relative to the class in question. Sensitivity for the *fraud* class is specificity for the *non-fraud* class; and vice versa.

The false negative rate (FNR) is complementary to specificity or the true negative rate (TNR), i.e. $FNR = 1 - TNR$, and is often referred to as fall-out or *probability of false alarm*. Similar to precision vs. recall, sensitivity and fall-out (i.e. 1 - specificity) are often plotted to produce an *ROC*¹ curve.

¹ROC stands for *receiver operating characteristic* and was initially used in the military to evaluate the performance of a radar detector.

15.3. Sensitivity and Specificity

ROC analysis has been widely used and well studied for model performance assessment in many fields including defense and medicine. It not only enables the evaluation of model effectiveness in terms of true positive and true negative rates, but can also help identify best practice with cost constraints.

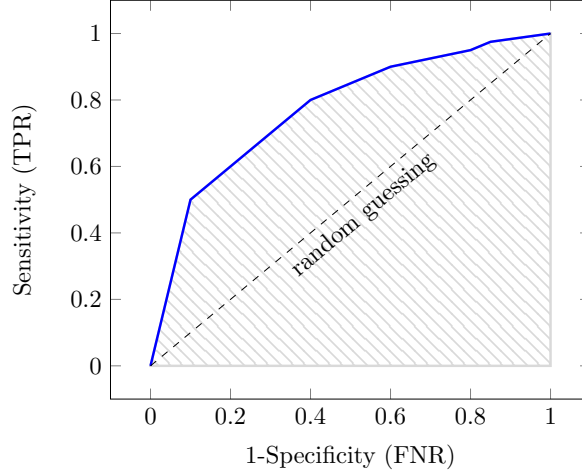


Figure 15.2.: ROC curve and area under curve (AUC)

Figure 15.2 illustrates a plot of an ROC curve, where the diagonal line (dashed) defines the acceptable performance. To understand what the diagonal line means, imagine we create a random model that takes a 50-50 guess during classification. As a result of this random guessing, there will be an equal split between the predicted positives $\hat{\oplus}$ and predicted negatives $\hat{\ominus}$. Hence, column values of the same row in Table 1.1 will be identical:

$$\begin{aligned} N_{\hat{\oplus}} &= N_{\hat{\ominus}} \\ TP &= FN \\ FP &= TN \end{aligned}$$

15. Evaluation and Experimentation

where $N_{\oplus}$ and $N_{\ominus}$ are the numbers of predicted positives and negatives respectively. It follows that $TPR = FNR$, i.e. $Sensitivity = 1 - Specificity$ or $Sensitivity + Specificity = 1$. This equation defines the diagonal line in Figure 15.2 and is the result of random guessing.

Below the diagonal line $Sensitivity + Specificity = 1$, the model performs worse than random and is undesirable. The farther the ROC curve is above the diagonal line, the better the performance with a greater area under curve. Therefore, the *area under curve* (AUC), i.e. the shaded area in the plot, is a single score that can be computed to measure the overall performance in terms of ROC.

Put it into practice

Continue with the corresponding practice activity to turn **Sensitivity and Specificity** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15.4. Kappa and Chance Agreement

So far the metrics are focused on different aspects of the confusion matrix in evaluation. However, we are yet to address one important problem in assessing the agreement between the predicted and the ground truth – chance agreement. A method may hit the correct answer simply by chance. The evaluation cannot be fairly conducted unless we find a way to discount the chance agreement.

Let's first look at some of the chances or probabilities. In binary classification, for example, the outcome is either positive (e.g. fraud) or negative (e.g. non-fraud). Based on the confusion matrix Table 1.1, the likelihood of a positive prediction vs. a negative prediction can be estimated:

15.4. Kappa and Chance Agreement

$$\begin{aligned}
 P(\hat{\oplus}) &= \frac{N_{\hat{\oplus}}}{N} \\
 &= \frac{TP + FP}{TP + FN + FP + TN} \\
 P(\hat{\ominus}) &= \frac{N_{\hat{\ominus}}}{N} \\
 &= \frac{TN + FN}{TP + FN + FP + TN}
 \end{aligned}$$

where $N_{\hat{\oplus}}$ and $N_{\hat{\ominus}}$ are the number of predicted positive and negative answers respectively. N is the total number of instances. $TP + FP$ is the sum of the *Predicted Positive* $\hat{\oplus}$ column where as $TN + FN$ is the sum of the *Predicted Negative* $\hat{\ominus}$ column in Table 1.1. These are probabilities of model predictions.

Likewise, we can estimate probabilities of having a positive vs. negative outcome in the ground truth:

$$\begin{aligned}
 P(\oplus) &= \frac{N_{\oplus}}{N} \\
 &= \frac{TP + FN}{TP + FN + FP + TN} \\
 P(\ominus) &= \frac{N_{\ominus}}{N} \\
 &= \frac{TN + FP}{TP + FN + FP + TN}
 \end{aligned}$$

where $N_{\oplus}$ and $N_{\ominus}$ are the numbers of actual positive and negative results respectively. $TP + FN$ is the sum of the *Actual Positive* row and $TN + FP$ is the sum of the *Actual Negative* row in Table 1.1. These are probabilities based on the ground truth.

With these probability, it is now possible to estimate chance agreement. If we assume the model and ground truth are independent, then the

15. Evaluation and Experimentation

(hypothetical) probabilities that they will reach the same conclusion, namely the joint probability of both saying "yes" (positive) and the joint probability of both saying "no" (negative) can be computed by:

$$\begin{aligned} P_e(\oplus \cap \hat{\oplus}) &= P(\oplus)P(\hat{\oplus}) \\ P_e(\ominus \cap \hat{\ominus}) &= P(\ominus)P(\hat{\ominus}) \end{aligned}$$

The overall chance agreement, the probability of either both "yes" or both "no" is:

$$\begin{aligned} P_e &= P_e[(\oplus \cap \hat{\oplus}) \cup (\ominus \cap \hat{\ominus})] \\ &= P_e(\oplus \cap \hat{\oplus}) + P_e(\ominus \cap \hat{\ominus}) \\ &= P(\oplus)P(\hat{\oplus}) + P(\ominus)P(\hat{\ominus}) \end{aligned}$$

Note that we add the two probabilities to compute the OR ($\cup$) probability because *positive* and *negative* are mutually exclusive.

Now that we know the chance agreement P_e , we can use it to discount the overall agreement P_o :

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o is the relative agreement of two, e.g. two raters such as a model and an expert (the ground truth), and is identical to the accuracy metric in our discussion $P_o = Accuracy$. This is commonly known as the *Kappa* coefficient, a statistic that measures the agreement of two sets of results.

As Equation [eq-eval-kappa] shows, Kappa takes into account potential agreement by chance and discounts the overall coefficient by the chance agreement. This makes it harder for a model to "cheat" on the result by simply favoring the dominant classes (e.g. classifying all transactions into non-fraud).

15.4. Kappa and Chance Agreement

With the result in Table 1.2, the chance agreement P_e is:

$$\begin{aligned} P_e &= P(\oplus)P(\hat{\oplus}) + P(\ominus)P(\hat{\ominus}) \\ &= \frac{10}{1000} \times 0 + \frac{990}{1000} \times \frac{1000}{1000} \\ &= 0.99 \end{aligned}$$

By discounting this chance agreement, we get a *Kappa* coefficient of zero:

$$\begin{aligned} \kappa &= \frac{P_o - P_e}{1 - P_e} \\ &= \frac{0.99 - 0.99}{1 - 0.99} \\ &= 0 \end{aligned}$$

A kappa score of 0 does not mean there is no agreement. There is in fact an agreement between the model prediction and the ground truth. However, this agreement is purely due to chance and is reduced to 0 after discounted for the chance agreement.

Indeed, $\kappa = 0$ means completely chance agreement. Its minimum value, $\kappa = -1$, indicates total disagreement whereas a maximum value $\kappa = 1$ shows the two are in full agreement even after accounting for chances.

 Put it into practice

Continue with the corresponding practice activity to turn **Kappa and Chance Agreement** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15. Evaluation and Experimentation

15.5. Averaging

When evaluating model performance on multiple classes – even binary classification entails two classes – there are two major strategies to compute an overall score (average).

Suppose precision is the evaluation metric, one can construct a confusion matrix for each class c and compute its precision by:

$$P_c = \frac{TP_c}{TP_c + FP_c}$$

And the overall (average) precision can be computed by:

$$\bar{P}_{macro} = \frac{\sum_{c \in C} P_c}{|C|}$$

where C is the set of unique classes and $|C|$ is the size of the set or the number of classes, e.g. 2 for binary classification. This is *macro averaging*, which computes the average based on class-level scores. It gives equal treatment to each class and disregard their relative sizes (significance).

The alternative is micro averaging which takes into account the composition of each classes and produces a final average weighted by class sizes. With micro averaging, all related counts are summed up before a ratio is taken. For example, average precision can be computed by:

$$\bar{P}_{micro} = \frac{\sum_{c \in C} TP_c}{\sum_{c \in C} TP_c + FP_c}$$

This turns out to be the overall fraction of predictions that are correct – in fact, a micro averaged precision is identical to *accuracy* in binary classification.

Whether one should use macro or micro averaging depends on the relative importance of classes in the final assessment. If all classes are equally important regardless of their sizes, macro-averaging should be preferred. If one aims to emphasize the absolute number of correct predictions, then micro-averaging is appropriate.

 Put it into practice

Continue with the corresponding practice activity to turn **Averaging** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15.6. Rank Evaluation

There are applications where the focus is not about definitive correct vs. incorrect answer. In information retrieval, for example, a model may return a sorted list of search results. Such a rank list requires a different approach to evaluation.

Surely one can regard a rank list as the set of correct answers and use the discussed metrics such as precision and recall. More precisely, we can create a cut-off at a certain position k in the rank and use the top k as the set to be evaluated. For example, precision at $k = 10$ is useful to evaluate early results in web searches, where the emphasis is often on the first result page (top 10).

If one knows the total number of relevant document r (recall) for a query, the cut-off can be set at position r to measure it precision. This is referred to as *R-Precision* because at this particular position precision is equal to recall. It is also possible to measure average precision by starting with the top position $k = 1$ and increasing it until recall reaches 100%.

15. Evaluation and Experimentation

Several other metrics can be derived from variations of these strategies. However, they only attack the problem of rank list evaluation by indirectly addressing positions in the list.

A direct attack on this issue is to assign a *different score* (amount of reward) for a correct (relevant) answer in a *different position*. A relevant answer at the top should be highly rewarded, which should be discounted down the rank.

This is the idea behind the *Normalized Discounted Cumulative Gain*, or nDCG, which has been widely used in the information retrieval community since its proposal. (Järvelin and Kekäläinen 2002) Given the relevance rel_i ² of an answer at position i , the Discounted Cumulative Gain at position k can be computed by:

$$\begin{aligned} DCG_k &= \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \\ &= rel_1 + \sum_{i=2}^k \frac{rel_i}{\log_2(i+1)} \end{aligned}$$

where a relevant (correct) answer at position 1 will receive its full credit and others will be discounted by their positions down the rank, with $\log_2(i+1)$.

Among all possible arrangements of the rank list for a query, there exists at least one ideal (perfect) list in which answers are sorted by their relative relevance scores in the descending order. This produces the best possible DCG score, which we call the ideal DCG or IDCG. Comparing a DCG score against its corresponding ideal score, we can obtain the *normalized* DCG or nDCG:

²Relevance scores can be on any scale dictated by the evaluation protocol. For example, they can be as simple as 1 for a relevant (correct) answer and 0 if otherwise.

15.7. Evaluation of Numeric Predictions

$$nDCG_p = \frac{DCG_p}{IDCG_p}$$

Normalized by the ideal score, an nDCG score is always between $[0, 1]$, with 1 indicating a perfect ranking and 0 no relevant answer at all in the list³. nDCG is a popular metric for web search evaluation and one can adjust the position k to emphasize relevant results at the very top, e.g. with $nDCG_3$.

 Put it into practice

Continue with the corresponding practice activity to turn **Rank Evaluation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15.7. Evaluation of Numeric Predictions

When the outcome of a model is on a continuous scale rather than discrete values, the evaluation is no longer about *correct* answers because it is rarely possible to make predictions that are *exactly* the actual outcome. Instead, the focus should be on how closely predictions are to the actual values or how much apart they are (errors).

15.7.1. Error Metrics

Many evaluation metrics for numeric predictions are indeed based on the error terms. The mean absolute error (MAE), mean squared error (MSE),

³Here assume relevance scores are non-negative, e.g. 0 no reward for non-relevance. In certain applications, it might be reasonable to assign negative scores for non-relevant answers to penalize a ranking with any non-relevant results.

15. Evaluation and Experimentation

and root mean-squared error (RMSE), for example, compute mean errors in their absolute, squared, and root squared forms:

$$\begin{aligned}MAE &= \frac{\sum_{i=1}^N |\hat{v}_i - v_i|}{N} \\MSE &= \frac{\sum_{i=1}^N (\hat{v}_i - v_i)^2}{N} \\RMSE &= \sqrt{\frac{\sum_{i=1}^N (\hat{v}_i - v_i)^2}{N}}\end{aligned}$$

where N is the total number of instances to test the numeric predictions, $\hat{v}_i$ is the predicted value on the i^{th} instance, and v_i is the actual value of the i^{th} instance. Relatively speaking, metrics based on squared errors such as MSE and RMSE are amplify the impact of large errors and are more sensitive to outliers.

Each of these error metrics can be normalized by the corresponding error of a pseudo-model predicting the mean value of training data, to obtain its *relative* error. For example, relative squared error (RSE) can be computed by:

$$RSE = \frac{\sum_{i=1}^N (\hat{v}_i - v_i)^2}{\sum_{i=1}^N (\bar{\mu} - v_i)^2}$$

where $\bar{\mu}$ is the estimated mean based on training data. Root relative squared error (RRSE) and relative absolute error (RAE) can be established in the like manner.



Put it into practice

Continue with the corresponding practice activity to turn **Error Metrics** into a calculation, implementation, visualization, diagnostic,

or interpretation task.

15.7.2. Correlation

While RSE measures the deviation of predicted values $\hat{v}_i$ from actual values v_i , standard deviations of predicted and actual values can be computed by:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (v_i - \mu)^2}{N - 1}}$$

$$\hat{\sigma} = \sqrt{\frac{\sum_{i=1}^N (\hat{v}_i - \hat{\mu})^2}{N - 1}}$$

where μ is the mean of actual values and $\hat{\mu}$ the mean of predicted values. The covariance between the predicted and actual is:

$$\text{cov}(\hat{v}, v) = \frac{\sum_{i=1}^N (\hat{v}_i - \hat{\mu})(v_i - \mu)}{N - 1}$$

By normalizing the covariance by the product of both standard deviations (also a value of variance), we can obtain the Pearson correlation between the predicted and actual values:

$$\begin{aligned} \text{cor}(\hat{v}, v) &= \frac{\text{cov}(\hat{v}, v)}{\hat{\sigma}\sigma} \\ &= \frac{\sum_{i=1}^N (\hat{v}_i - \hat{\mu})(v_i - \mu)}{\sqrt{\sum_{i=1}^N (v_i - \mu)^2} \sqrt{\sum_{i=1}^N (\hat{v}_i - \hat{\mu})^2}} \end{aligned}$$

15. Evaluation and Experimentation

Pearson correlation measures the similarity in their variances from means and is equivalent to cosine of v and $\hat{v}$ as vectors from their means. Figure 15.3 illustrates the connection using a 2-dimensional space. To simplify the discussion, suppose there are only $N = 2$ instances on which the actual outcomes are (v_1, v_2) and predicted values are $(\hat{v}_1, \hat{v}_2)$, with means μ and $\hat{\mu}$ respectively.

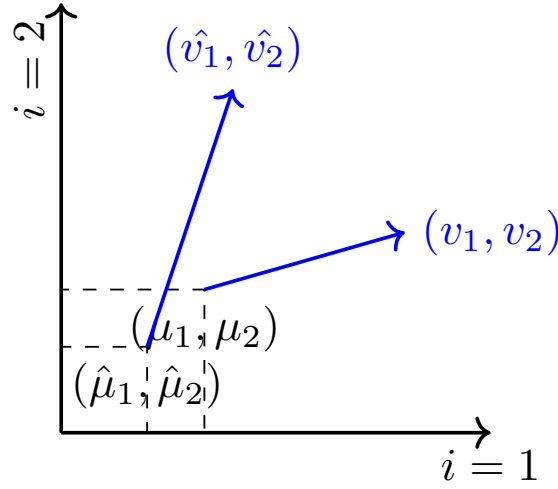


Figure 15.3.: Pearson correlation as cosine of vectors originated from means, with $N = 2$ for a 2-dimensional visualization

As shown in Figure 15.3, the actual values can be regarded as a vector pointing in the $\vec{v} = (v_1 - \mu, v_2 - \mu)$ direction, whereas the predicted is represented by the vector $\vec{\hat{v}} = (\hat{v}_1 - \hat{\mu}, \hat{v}_2 - \hat{\mu})$. In this representation, Pearson correlation is identical to the cosine of the two vectors, i.e. $cov(\hat{v}, v) = \cosine(\vec{\hat{v}}, \vec{v})$. Chapter has more on the cosine similarity.

Because of this connection, Pearson correlation shares important characteristics with cosine. It reaches its maximum value of 1 when errors from

their mean, i.e. $\hat{v}_i - \hat{\mu}$ vs. $v_i - \mu$ for all instances, are distributed identically (in the same vector direction) – they are perfectly correlated in this case. When the errors are distributed in absolutely *orthogonal* directions, the two are perfectly independent and has no correlation. When their errors distribute in the opposite direction, they are negatively correlated, with the minimum value of -1 indicating perfectly opposite.

 Put it into practice

Continue with the corresponding practice activity to turn **Correlation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

 Put it into practice

Continue with the corresponding practice activity to turn **Evaluation of Numeric Predictions** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15.8. Experimentation

Now that we know some of the metrics for model evaluation, how do we actually conduct the assessment with data? In the ideal world, one builds a model using training data with known outcomes and rolls out the model to be tested in the operating environment. In the real world, however, we have to make sure a model works reliably well for its purpose before it is used in production for important predictions and decision making.

We often have to use existing data to test models with experiments. It is a common practice to split the data into training and testing subsets – one to build and train the model, and the other to evaluate its performance.

15. Evaluation and Experimentation

Random sampling can be used to create the training and testing subsets. Once a model is trained, it can be experimented with the test data and evaluated using metrics suitable for related objectives. This process can be repeated to test different models or the same model with different parameters or configurations. It can be done with different random samples.

Along the process, one may be able to find out what leads to best performances in terms of certain metrics. Given various model configurations, it is also possible to create plots such as precision-recall or ROC curves, where one can examine related parameters and tradeoffs.

The ratio to split training vs. testing depends the amount of available data and the amount required to sufficiently train the specific model. One may be tempted to conduct a 50-50 split. In this case, you use half of the data (sample) for training and the other half for testing. Afterwards, it also makes sense to switch the two subsets, testing data now for training and training for testing.

This is cross validation. And you can do it on 50-50, a 2-fold validation, you can certainly do it on more folds. Practically, it is popular to conduct a 10-fold cross validation, where the data is split into 10 random subsets. Each time, 1 subset is held out for testing and the other 9 are used for training. This is repeated 10 times so the model is tested on every fold of the data.



Put it into practice

Continue with the corresponding practice activity to turn **Experimentation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

15.9. On Efficiency

In this chapter, we have focused on the evaluation of model *effectiveness* or, loosely speaking, how satisfactory predictions are compared to the ground truth. This is about the *quality* of a model.

The other side of evaluation is on the *efficiency* of operations. This is about the time and cost necessary to train a model and to perform assigned tasks. It depends on the space and time complexity of the model, and the structure and magnitude of data the model has to work on in the production settings.

A thorough evaluation should include an assessment of both effectiveness and efficiency. We will elaborate on important aspects of efficiency as well as *scalability* in Chapter .



Put it into practice

Continue with the corresponding practice activity to turn **On Efficiency** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16. Generalization, Model Selection, and Fit

Torture the data, and it will confess to anything.

Ronald Coase

16.1. Training Performance and Generalization

 Put it into practice

Continue with the corresponding practice activity to turn **Training Performance and Generalization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16.2. Bias, Variance, and Model Capacity

 Put it into practice

Continue with the corresponding practice activity to turn **Bias, Variance, and Model Capacity** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16.3. Validation and Cross-Validation

 Put it into practice

Continue with the corresponding practice activity to turn **Validation and Cross-Validation** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16.4. Regularization

 Put it into practice

Continue with the corresponding practice activity to turn **Regularization** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16.5. Learning Curves and Diagnostics

 Put it into practice

Continue with the corresponding practice activity to turn **Learning Curves and Diagnostics** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16.6. Hyperparameter Selection Without Leakage

 Put it into practice

Continue with the corresponding practice activity to turn **Hyperparameter Selection Without Leakage** into a calculation, implementation, visualization, diagnostic, or interpretation task.

16.7. Summary, Exercises, and Lab

Under- and over-fitting...

Bias, variance, and error...

Combined, ensemble learning...

17. Scaling, Deployment, and Responsible Use

17.1. Time, Space, Data, and Compute

 Put it into practice

Continue with the corresponding practice activity to turn **Time, Space, Data, and Compute** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.2. Batch, Streaming, and Distributed Workflows

 Put it into practice

Continue with the corresponding practice activity to turn **Batch, Streaming, and Distributed Workflows** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.3. From Experiment to Reproducible Pipeline

 Put it into practice

Continue with the corresponding practice activity to turn **From Experiment to Reproducible Pipeline** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.4. Deployment, Monitoring, and Drift

 Put it into practice

Continue with the corresponding practice activity to turn **Deployment, Monitoring, and Drift** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.5. Privacy, Security, and Misuse

 Put it into practice

Continue with the corresponding practice activity to turn **Privacy, Security, and Misuse** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.6. Energy, Sustainability, and Proportionality

 Put it into practice

Continue with the corresponding practice activity to turn **Energy, Sustainability, and Proportionality** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.7. Responsible Decisions and Human Oversight

 Put it into practice

Continue with the corresponding practice activity to turn **Responsible Decisions and Human Oversight** into a calculation, implementation, visualization, diagnostic, or interpretation task.

17.8. Summary and Exercises

Part V.

**Practice and Projects: A
Companion Volume**

Practice and Projects

This volume turns the conceptual development in Volume I into observable, testable work. It is intentionally placed after the theory chapters so readers can follow the conceptual narrative continuously, while every substantive theory section provides a direct route here.

How to use this volume

- **Read-first path:** finish a theory section, then follow its *Put it into practice* link to the aligned activity or development scaffold.
- **Build-first path:** start from an activity, use its *Theory connection* to revisit the underlying model, and return with a concrete question.
- **Course path:** combine selected activities into short labs, assignments, and a final project without changing the theory sequence.

Recovered activities preserve their original wording and code wherever possible. They are rendered without execution until dependencies, datasets, and expected outputs have been modernized and tested. Sections marked as development scaffolds make the missing practical coverage explicit rather than hiding it.

Computational Toolkit

The book's practice volume uses Python and Jupyter-compatible notebooks. These recovered primers are optional for readers who already have a working Python environment.

Environment setup

Continue with Python and Jupyter Setup.

Python programming primer

Continue with Python Programming Primer.

Control flow, randomness, and input/output

Continue with Python Input and Output, Python Selection, Python Randomness, and Python Repetition.

File processing

Continue with Python File Processing.

Core data types and structures

Continue with Python Data Types, Python Data Structures, and Python Data Structures Assignment.

Python and Jupyter Setup

Recovered activity

i Historical source and execution note

This activity was recovered from `setup.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Purpose

- Organize your code and files
- Organize your data
- Organize your analysis (output)

Now you are to write Python code to process and analyze data, you need to have good tools to help you manage your code as well your data. Once you write a piece of code, it is a good idea to store the program on the computer so we can run it over and over again without retyping everything.

Traditionally, here are some options to organize your Python programs:

1. A text editor to create and modify your programs, e.g.:
 - Notepad on Windows

Python and Jupyter Setup

- TextEdit on Mac
 - vi on Linux
2. An Integrated Development Environment (IDE), e.g.:
- PyCharm, specifically for Python
 - Atom, general purpose for many languages

Today, there is a growing need to organize and manage data and program output as well. A data science notebook, e.g. **Jupyter Notebook**, is a popular choice.

Your Options

We will be using **Python3**, the language in its 3rd generation.

It works on different platforms including *Windows*, *MacOS*, and *Linux*.

Use CCI's JupyterHub (recommended)

The IT team at CCI has setup a JupyterHub server where: + Current students can login with their **preconfigured** accounts + Use Jupyter Notebook and Python right away, **in the browser**.

In this case, you don't need to do any setup regarding Python on your computer.

You do, however, need to have **Drexel VPN** so you can be connected to "campus" (virtually) before accessing the JupyterHub.

Here are steps and related instructions: 1. Get connected to Drexel **VPN**: <https://docs.cci.drexel.edu/display/CD/VPN> 2. Once connected, visit **JupyterHub** at: <https://jupyterhub.cci.drexel.edu/> 3. Enter your username and **TUX password**.

Note the TUX password is the one you used for `tux.cs.drexel.edu`, which you received in an encrypted message when you started at CCI. Send an email to `ihelp@drexel.edu` if you don't know what it is.

Setup on Your Computer (optional)

Python and Jupyter Notebook

You can install the following on your computer: * Python: <https://www.python.org/>
* Jupyter Notebook: <https://jupyter.org/>

You may proceed with your own setup of Python environment only if are confident to do so. This option does offer you the flexibility of having everything on your computer and does not require VPN. But depending on the platform of your computer and version, this may require additional effort.

You may install Python and Jupyter separately based on instructions from the above sites.

Conda

You may consider Conda to help you install and manage your Python platform:

<https://www.anaconda.com/distribution/>

For example, once Conda is installed, you can run:

```
conda install -c conda-forge notebook
```

to install the Jupyter Notebook.

Python and Jupyter Setup

Run Jupyter Notebook

From command line, you can launch your jupyter notebook:

```
jupyter notebook
```

This will start a server and open a local website at <http://localhost:8888/> by default.

Result

In the end, whether you are using:

- A JupyterHub at: <https://jupyterhub.cci.drexel.edu/>
- Or a local Jupyter Notebook installation at: <http://localhost:8888/>

The interface should be similar. Here are a couple of screenshots:

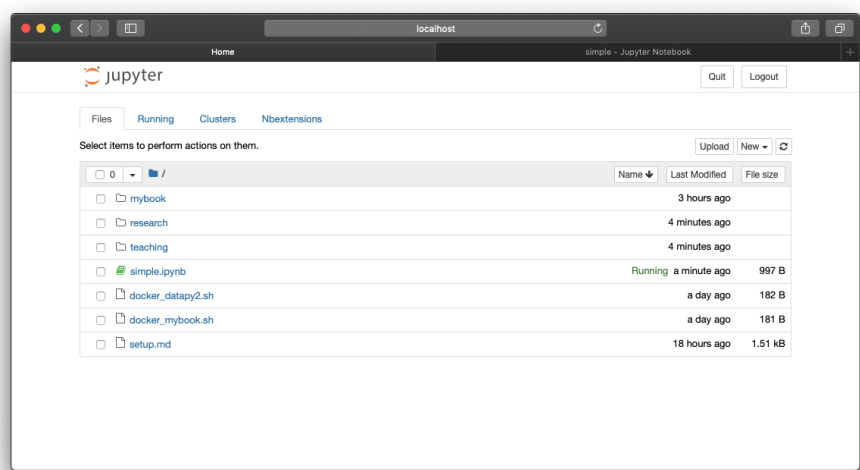


Figure 17.1.: First Screen

Python and Jupyter Setup

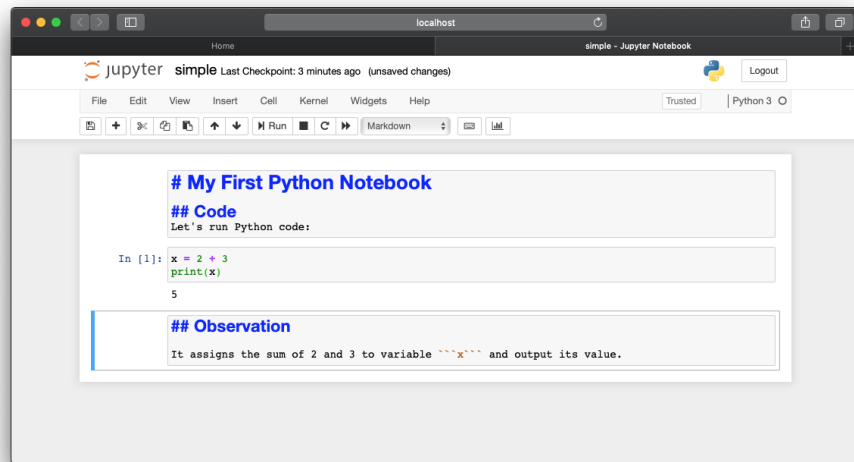


Figure 17.2.: Second Screen

Python Programming Primer

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_intro.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Programming

What is computer programming? Programming is to create a step-by-step list of instructions to solve a problem. Today's computers are very capable of computing, that is, to add, to subtract, to multiple, and to divide, etc. They are good at crunching numbers.

To program a solution to a problem with computers, that problem has to be computable. We also refer to the program thus created as an **algorithm**.

Let's consider a daily example of a program or algorithm: to bake a brownie:
1. Preheat the oven to 325 °F; 2. Gather the ingredients; 3. Mix the ingredients in a bowl; 4. Pour the mixture into a baking pan; 5. Put the pan in the oven for 30 minutes; 6. Check if baking is done; * If it is NOT done: wait 3 more minutes and go back to step 6; * Otherwise – if it is done – remove the pan; 7. Let the brownie cool before serving it. 8. Enjoy!

From the example, we observe the following structures: + **Sequential**: one step follows another in a sequence; + **Selection**: one does things differently depending on the condition; + **repetition**: one repeats a task for some times.

Each step above is a statement or instruction about what to do. Most of the steps follow one after another (**sequential**). For step 6, however, it has a different structure. It has IF...OTHERWISE (else)..., which does some things for a specific condition (e.g. wait if the brownie is NOT done), or something else for a different condition (e.g. remove it if it IS done). This is a **selection** structure because it does things selectively based on conditions.

Programming

Finally, step 6 also has another unique programming structure, **repetition**. You will be waiting for 3 minutes, and over and over again, until it is all done. We also call this repeating structure a loop.

So with the simple example of baking a brownie, we have seen three different programming structures in play: sequential, selection, and repetition. In fact, every computable algorithm or program can be written by the three structures and these three **ONLY**.

Your creativity is about how to mix them together, just like making your own brownie! :)

Introduction to Python

Now that you know about the general logic of programming, how do you actually do it? In what language can you give instructions to the computer so it can follow and do the job?

Python Language

- A high-level language that resembles English
- Popular choice for introductory programming
- No. 1 for data sciences, e.g. for data processing and analytics

There are many so-called high-level languages you can use, programming languages that resemble the style of English and can be translated into computer languages to give instructions.

Python is one of these high-level languages. It is, in fact, one of the most popular today, especially for the very hot topic of **data sciences**. Python was first developed by a computer programmer named Guido van Rossum, who wanted “computer programming for everybody.” As of today (2019), Python is the de facto official language for introductory programming and data analytics at the college level.

With this brief introduction, let’s meet Python “in person” and interact with it.

Working Environment

Jupyter Notebook (or Hub)

If you have Jupyter Notebook installed or have access to a JupyterHub server (website), you can create a New Python3 Notebook and test your Python code there.

Command Line

For a computer with Python installed, you can go to the command line, i.e.

- `cmd` on Windows, OR
- `Terminal` on Mac or Linux

and type the following command:

```
python3  
>>>
```

You will be prompted with `>>>` and that is where you give the computer instructions in the language of Python.

First Interactions

Now type a statement (instruction) in Python, in the notebook and Python command line:

```
print(2+3)
```

and the computer will spit out 5 on the screen. What it does here is to compute the result of $2+3$, which is 5, and then execute the `print` statement to show the result.

Likewise, you can do any of the following:

```
print(4*5)
print((2+3)*7)
print(12-8)
print(36/9)
print(100%3)
print(3**4)
```

It should be easy to tell what `+` and `-` do? Now have you figured out what `*`, `/`, `%`, and `**` are about? All these are operators you can use to tell a computer how to compute.

The `print` here is a function, a.k.a. a process or a method, that is already part of python. You can call a function by its name, i.e. `print` here, followed by a pair of parentheses `()`. The `print` function requires a piece of data (information) about what to print, and you put this piece of data (called an **argument**), e.g. $2+3$, in the parentheses.

Variables

Often, your program can be more complex and it may take many steps before you want to print out the result.

For example, if you want to calculate the average height of four people in inches: 1. Joe 81'' 2. Carol 76'' 3. Larry 77'' 4. Brenna 48''

you may want to save these numbers somewhere before adding them up and divide the total by 4.

Do this in Python:

Introduction to Python

```
j = 81 # joe's height
c = 76 # carol's height
l = 77 # larry's height
b = 48 # brian's height
total = j + c + l + b
avg = total / 4
print(avg)
```

Note that anything after **#** is a **comment** (to help programmers remember/understand the code) and is **NOT executed**.

The **j**, **c**, **l**, and **b** are variables, which are in the computer's **memory** holding the heights of Joe, Carol, Larry, and Brian respectively.

Be cautious, however, that:

... the equal sign =

... *does NOT mean* “equal to.”

It is an **assignment** operator that

... takes the value (result) from its right side

... and assign it to the variable on its left.

For example:

```
j = 81
```

means $j \leftarrow 81$.

That is, to take the value of 81 and put it in a computer memory location marked as **j**. This way you can use **j** later to get the value (e.g. Joe's height) on that location.

Python Input and Output

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_io.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Output

```
print('Hello!')
```

We know we can use the `print()` function to show information on the screen.

We can also show the value (data) stored in a variable such as:

```
name = 'John Paul'  
print(name)
```

The `print()` function can also display multiple things at one time:

Python Input and Output

```
print(name, "how are you?")
```

Notice that with multiple values, they are separated by a *space* ' ', which is called the **separator**. We can actually change the separator with the **sep=** parameter in the **print()** function.

An example of **print** with a **sep** (separator) specified:

```
print(name, "how are you?", sep=', ')
```

This is basic python output – it is the *output* from the computer to the *user* (person) who can see it by using the output device (e.g. looking at the screen).

Input

How does the *user* – the person who uses your program – enter data into the program? The **input()** function in Python does just that.

For example:

```
name = input('What is your name? ')
```

The **input** function first displays a message (prompt) **What is your name?** and waits for the user to type. Once the user enters something and presses the **Enter** key, the input data (e.g. letters of a name) will be stored in the **name** variable.

Remember, with the assignment operator **=**, the statement here is to take the value on the right (**input**) and store it in the variable on the left (**name**).

Now if you use:

Example Program: Knock-knock

```
print("You entered: ", name)
```

It shows what the user just typed in and has been stored in the variable `name`.

Example Program: Knock-knock

Python Program

The Game (Algorithm)

Consider a knock-knock joke, it consists of the following steps between two persons, say John and Mary:

1. John pretends to knock on the door, saying, “Knock knock.”
2. Mary responds with “Who is there?”
3. John answers with a **name**, e.g. “Boo”
4. Mary is confused by the name and continue to ask in a pattern like “**name** who?”, e.g. “Boo who?”
5. The final response depends on the joke and can be difficult to program. For now, let’s simply repeats it by saying “Don’t cry!”

An example knock-knock:

1. John: Knock knock!
2. Mary: Who is there?
3. John: Boo.
4. Mary: Boo who? (sounds like Boohoo...)
5. John: Please don’t cry...

The Program Code

Suppose **John** is the *user* who will play with the program, here is the simple Python code:

```
print("John: Knock knock.")
print("Mary: Who is there? ")
name = input("Enter your response: ")
print("Mary:", name, "who?") # sounds like name hoo...
print("John: Please don't cry...")
```

Python Selection

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_selection.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

In the programming introduction, we discussed three different programming structures: sequential, selection, and repetition.

- **Sequential** is to follow program instructions one by one (in a sequence);
- **Selection** allows the program to do something depending on the situation;
- **Repetition** enables the program to repeat something many times.

Today we discuss the **selection** programming structure in Python. Let's start with an example:

```
name = input("What is your name? ")
```

Here the program displays a message and asks the user to enter his or her name. To make this more personal, we can change the program to display

Python Selection

What's your name, sir? for a male, and What's your name, madam? for a female.

So instead of asking for the name, we ask for the user's gender first:

```
gender = input("Are you a lady (L) or a gentleman (M)? Enter L or M: ")
```

IF... Structure

Now if one enters L and hits Enter, the variable **gender** will contain this value and we shall display a message concerning a lady.

In Python, we can use an **if... elif... else** structure to find out the condition and act accordingly.

For the problem here, we can have:

```
if gender=="L":  
    name = input("What is your name, madam? ")
```

You probably notice that we have **two spaces** (indentation) before **name=input("What is your name, madam?")**. This is because this **madam** message is part of the **if** structure where the user enters L. In Python, we can use 2 (or 4) spaces as indentation to structure a program.

Also, pay attention that we have double equal signs **==** in the **if** condition. Double equals **==** is an equality operator, which determines whether the values on its two sides (left and right) are equal (true) or not (false).

Compare this to a single equal sign **=**, which is the **assignment** operator. For example, if you put **gender="L"**, it will change the value of variable **gender** to "L", which is **NOT** what we wanted here.

Now, do you understand the difference between **=** and **==**?

IF... ELIF... ELSE... Structure

Back to the program, L is not the only situation because the user may be a man and enters M, or, if she or he does not want to tell, the value of **gender** can be **anything else**. Let's expand on the above to include the other two situations:

```
gender = input("Are you a lady (L) or a gentleman (M)? Enter L or M: ")
if gender=="L":
    name = input("What is your name, madam? ")
elif gender=="M":
    name = input("What is your name, sir? ")
else:
    name = input("What is your name? ")
```

Note that **elif...** stands for **else if**, which means, if you are NOT “L” (in the **if** condition), **then** are you ...?

When entering the gender information, the user may enter in lowercase or capital. Run your program and try entering lowercase l or m and see what happens.

We should further revise the code a bit to accommodate situations of different cases, for example:

```
if gender=="L" or gender=="l":
```

Here **or** is a logical **OR** operator – the entire condition is true if any one of the conditions (“L” or “l”) is true. In other programs, you may need the **and** operator, which turns true when both conditions are true.

Likewise, can you revise the code for a boy (“M” or “m”)?

Python Selection

```
# Enter your code and try it here...  
# gender = input("Are you a lady (L) or a gentleman (M)? Enter L or M: ")  
# ...
```

Python Randomness

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_random.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

When you flip a coin or roll a die, you can guess what the outcome will be but cannot be certain about it. The result is random.

In computer programming, generating a random value (number) is very useful in many applications. Take the educational app *ExtraMath*, for example, you can generate random numbers combined with arithmetic operators to test a student on basic math skills such as addition and multiplication.

In Python, we can generate random numbers using the `random` library by importing it first:

```
import random
```

To generate a random integer, we can call the `random.randint()` function:

Python Randomness

```
import random
x = random.randint(1,9)
print(x)
```

Do it one more time:

```
x = random.randint(1,9)
print(x)
```

and chances are you will get different numbers.

In the above code, the `randint()` function takes two parameters 1 and 9, which define the range for the random number. The generated random number is limited to between 1 and 9. That is, the above code will generate any integer among [1,2,3,4,5,6,7,8,9].

Example: Addition Game

Now that we know how to generate a random number, let's attempt to create a math game about **addition**.

Here is the idea –

1. generate two random integers from 1 to 9;
2. store them in two variables `x` and `y`;
3. calculate `x+y` and store the result in `z` (correct answer);
4. ask the user for the answer with `input` and store it in the `answer` variable;
5. finally, compare the correct answer in `z` vs. that from the user in `answer`.

Example: Addition Game

Try the following code:

```
import random

# Generate two numbers x and y
x = random.randint(1, 9)
y = random.randint(1, 9)
# Save the result of addition to z
z = x + y

# Show the numbers to the user
print(x, "+", y)

# And ask the user for an answer
answer = input("= ")

# Feedback to the user about correctness
if int(answer) == z:
    print("You are right!")
else:
    print("Wrong! The correct answer is", z)
```

Run the code and test the program.

Run it again, and see what happens – what is different now? What is still the same?

Python Repetition

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_loop.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Sometimes you may want to repeat something in a program.

Consider the following robotic questioning in python –

```
print("1 What is your name, sir?")
print("2 What is your name, sir?")
print("3 What is your name, sir?")
print("4 What is your name, sir?")
print("5 What is your name, sir?")
```

It repeats the same question 5 times. However, if the program has to do this for hundreds – if not millions – of times, can you manage to type hundreds of python statements here? Perhaps you can, but it is going to be extremely tedious, to say the least.

Loop Basics

As we discussed, there is a programming structure called `loop` that can help you repeat doing something without typing the code over and over again. In particular, you can use a `for` loop to repeat something for a certain number of times.

The above Python code can be rewritten using a `for` loop –

```
for i in [1,2,3,4,5]:  
    print(i, "What is your name, sir?")
```

Here `[1,2,3,4,5]` is a list (array) of five numbers from 1 to 5, and `i` is a variable that repeats itself, each time taking a value from the list: the first time 1, the second time 2, ..., and finally the fifth time 5.

The above code can also be simplified as:

```
for i in range(1,6):  
    print(i, "What is your name, sir?")
```

Here, `range(1,6)` produces a list of numbers between 1 and 6 (6 not included), which is the same as `[1,2,3,4,5]`.

With the `range()` function, it is easy to change the range and repeat for even more times. For example:

```
for i in range(1,100):  
    print(i, "What is your name, sir?")
```

How many times does it repeat?

Loop for a Pattern

Now, consider this –

```
print("0 "*1)
print("0 "*2)
print("0 "*3)
print("0 "*4)
print("0 "*5)
```

When you use the expression "Some String" * n, where n is an integer, it repeats the string n times.

You can use a loop to do the same:

```
for i in range(1,6):
    print("0 "*i)
```

How about the following code?

```
for i in range(1,6):
    print("0 "*(6-i))
```

Explain what you see and why.

Nested Loop

Earlier, the code such as `print("0 "*3)` repeats a string a number of times. This itself can be implemented with a loop such as:

Python Repetition

```
for j in range(1,4):  
    print("O ", end="")
```

Here, to output 3 Os in the result, you use a loop with `range(1,4)`, which is `range(1, 3+1)`.

With this in mind, the above code can be rewritten as:

```
i = 3  
for j in range(1,i+1):  
    print("O ", end="")  
print("")
```

If you change `i` to 5:

```
i = 5  
for j in range(1,i+1):  
    print("O ", end="")  
print("")
```

You see the repetition (for `j` loop) code here is equivalent to `print("O "*i)`, which repeats “O” several times depending on the value of `i`.

Remember this code:

```
for i in range(1,6):  
    print("O "*i)
```

where the above pattern is created with `"O "*i` expression in the loop?

We can replace the expression with our `j` loop, within the `i` loop:

```
for i in range(1,6):  
    for j in range(1,i+1):  
        print("0 ", end="")  
    print("")
```

Compare the code. Does it make sense?

Now you see the *j* loop is a sub-structure embedded in the *i* loop. This is a **nested loop**.

A nested loop is useful to process data and patterns in more than one dimensions. Here the two-level nested loop (*i* and *j*) creates a 2-dimensional pattern and can be used for two-dimensional data such as a **matrix**.

Python File Processing

Recovered activity

Historical source and execution note

This activity was recovered from `python/python_data_files.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

In this lecture, we discuss some of the basic techniques in Python for file processing, especially when accessing data in text files.

Outline

- Python file data processing
 - Text file processing
 - Processing strings and statistics
 - CSV file processing
- Alternatives

Text Files

What text files?

A text file is any file containing only readable characters: - Characters being letters, numbers... - As well as other symbols and exotic symbols - English and other languages

A text file may letters, numbers, and other symbols, in English or other languages.

In a text file:

- **Symbols** can have special meanings, e.g. math $A = r^2$
- **Code** in special syntax, e.g. HTML code
- Writing/data in various **structures**, e.g. text in paragraphs or tables:

Column 1	Column 2	Column 3
...	...	...
...	...	...

A scientific article is an example of a text file, where you may encounter math equations and symbols with special meanings. It may be a web page with HTML code, which follows a special syntax defined by the W3 consortium. It may contain descriptions in paragraphs and data in tabulated structures.

What is not a text file?

- Binary files
 - JPEG or GIF pictures
 - MP3 audio files
 - Binary document formats, e.g. PPT
- Binary files require special programs

Text files can be viewed and edited by any text editor, e.g. Notepad on PC, TextEdit on Mac, and vi on Linux/Unix, etc. But binary files require special software to access them.

In this lecture, we focus on processing text files.

Reading a file

- To read and print the content of a text file:

```
f = open("months.txt")
print(f.read())
```

The example reads the entire content of a file. The `open` function creates a file object (a way of getting at the contents of the file), which is then stored in the variable `f`. `f.read()` tells the file object to read the full contents of the file, and return it as a string.

Reading a file by smaller pieces

- To read and print character by character:

```
f = open("months.txt")
next = f.read(1)
while next != "":
    print(next)
    next = f.read(1)
```

The example here reads a file a character at a time. This calls the `read()` function with parameter value 1, which specifies the maximum number of characters to read from the file. When it reaches the end of the file, the `f.read()` returns an empty “” and ends the while loop.

Reading a file by line

- To read and print line by line:

```
f = open("months.txt")
next = f.readline()
while next != "":
    print(next)
    next = f.readline
```

It is very common to read a file one text line at a time. Here the code calls the `readline()` function to read a line at a time. When it reaches the end of file, `readline()` returns an empty “” and terminates the loop.

Reading all lines in a file

- To read all lines:

```
f = open("months.txt")
for month in f.readlines():
    print("Month " + month.strip())
```

Another alternative, shown here, is to read a file and load all text lines:

- The `readlines()` method reads all the lines in a file and returns them as a Python list.
- You can create a loop to go through the list of lines:
- The `strip()` method removes leading and trailing characters, i.e. spaces by default.

Writing to files

- Default mode of `open()` is to read
- To open a file to write:

```
f = open("awesomenewfile.txt", "w")
```

- Parameter:
 - `r` to read
 - `w` to write (overwrite)
 - `w+` to write (and create if file does not exist)
 - `a` to append

With the option “w”, you can open a file to write. All existing content in the files will be overwritten. If you only want to add new lines to the existing content, use option `a` to append. `w+` create the file if it does not exist yet. `r` is the default option to read from a file.

Writing to files

- Writing text to file:

```
f = open("names.txt", "w+")
f.print("John")
f.write("Luke\n")
f.write("Matt\n")
f.close()
```

It is a good practice to close the file in the end with the `close()` method. The `print` method writes the text and adds a new line character at the end by default. The `write()` does not do that by default so if you need a new line character and you have to include the `\n` (newline character) manually.

More examples

Here are a few more examples to recap what we have discussed.

To open a text file:

```
fh = open("hello.txt", "r")
```

To read the entire content of a text file:

```
fh = open("hello.txt","r")
print fh.read()
```

To read one line at a time:

More examples

```
fh = open("hello.txt", "r")
print fh.readline()
```

To read a list of lines:

```
fh = open("hello.txt.", "r")
print fh.readlines()
```

To write a text line to a file, and replace its existing content:

```
fh = open("hello.txt", "w")
write("Hello World")
fh.close()
```

To write mutiple text lines to a file:

```
fh = open("hello.txt", "w")
lines_of_text = ["a line of text", "another line of text", "a third line"]
fh.writelines(lines_of_text)
fh.close()
```

To append a text file to a file:

```
fh = open("hello.txt", "a")
write("Hello World again")
fh.close
```

And finally, to close a file after you are done with it:

```
fh = open("hello.txt", "r")
print fh.read()
fh.close()
```

Strings Processing and Statistics

Now that we can read/write text files, there are a lot we can do to process data in the text.

Example problem

- Voting for favoured radish:
 - What is the most popular?
 - What are the least popular?
 - Did anyone vote twice?

In `radishsurvey.txt`:

```
Evie Pulsford - April Cross
Matilda Condon - April Cross
Samantha Mansell - Champion
geronima trevisani - cherry belle
Alexandra Shoebridge - Snow Belle
...
```

The example here is a survey about the types of radish people like. In this survey data of radish votes, each vote is recorded in the format of:

Name of Person followed by the Name of Radish

with a - to separate the two.

Reading survey

- Read and split:

```
for line in open("radishsurvey.txt"):
    line = line.strip()
    parts = line.split(" - ")
    name = parts[0]
    vote = parts[1]
    print(name + " voted for " + vote)
```

First, we read the file one line at a time with a `for` loop.

The `strip()` method removes leading and trailing spaces in a text line, and then we `split()` the text line into a list of two parts, name and radish, using the delimiter `-`.

On list and split

In Python, you can assign a list of values to multiple variables:

```
a, b, c = [1, 2, 3]
print(b)
print(a)
```

When you split something into a list in Python, you can assign the result directly to multiple variables:

```
name, cheese, cracker = "Fred,Jarlsberg,Rye".split(",")
print(cheese)
```

Counting votes

- Counting votes for a specific radish:

```
print("Counting votes for White Icicle...")
count = 0
for line in open("radishsurvey.txt"):
    line = line.strip()
    name, vote = line.split(" - ")
    if vote == "White Icicle":
        count = count + 1
print(count)
```

Back to the radish survey.

The code here counts the number of votes for a specific radish such as `White Icicle`. It initializes the `count` as 0, goes through every vote line in the data, and, whenever there is a match, adds 1 to the count.

This does the job well. However, if you want to count different kinds of radish, the code is not very reusable.

- General function for counting votes:

```
def count_votes(radish):
    print("Counting votes for " + radish + "...")
    count = 0
    for line in open("radishsurvey.txt"):
        line = line.strip()
        name, vote = line.split(" - ")
        if vote == radish:
            count = count + 1
    return count
```

```
print(count_votes("White Icicle"))
print(count_votes("Daikon"))
print(count_votes("Sicily Giant"))
```

We can put the code into a function called `count_votes()`, with a `radish` parameter to tell the function what to count on when we call it. Now the function can be called to count different radish types.

Counting ALL Votes

- What if you don't know the radish names
- Count all of them in the survey?
- Use a dictionary to keep track to every radish

Still, calling the function requires your knowledge of what radish might have been voted for in the survey. To make the vote-counting task easier, perhaps it is better to count all the votes in the survey and whatever radish appears in the survey will be included in the final counts.

One specific data structure we will be using is **dictionary**, which will help us keep track of radish names (as keys) and their counts (as values).

```
# create an empty dictionary
counts = {}

# go through the survey and add counts for each radish
for line in open("radishsurvey.txt"):
    line = line.strip()
    name, vote = line.split(" - ")
    if vote not in counts:
        # Initial value for a radish, if it is in the count yet
        counts[vote] = 1
    else:
```

```
# Increment the vote count
counts[vote] = counts[vote] + 1
print(counts)
```

First, in line 2, we create an empty dictionary to keep track of votes.

Then, starting in line 5, it goes through the survey and adds 1 to a count associated with each type of radish.

If the radish has not been voted before the current vote, it has not been counted and the key (radish name) is not in the dictionary yet. In this case, as shown in code line 10, we need to add the key and set its initial value to 1.

If the radish has been voted already, simply add 1 to the existing count (line 13).

In the end, we use the `print()` function to output data in the dictionary structure.

Output

- Use Python module pprint (pretty) print:

```
from pprint import pprint
pprint(counts)
```

If the format of the `print()` output does not look good, you may consider the `pprint`, or pretty print, module in Python.

- Print the votes in a custome format:

Example problem

```
for name in counts:  
    count = counts[name]  
    print(name + ": " + str(count))
```

Or you may consider rendering the output using the loop.

Data Cleaning

Problem in the final counts:

```
Red King: 1  
red king: 3  
White Icicle: 1  
Cherry Belle: 2  
daikon: 4  
Cherry  Belle: 1  
...
```

Shown here is an example output of the program we have created.

In the final result of counts, did you notice that capital **Red King** and lowercase **red king**, **Cherry Belle** and **Cherry Belle** (with two spaces in the middle) are treated as different radishes?

Apparently, when people vote for their favorite radish, they enter the names in slightly format formats. Now that we know about the dirty data, we should unify (normalize) them by removing the extra spaces and converting the strings into the same format – that is, the same capitalization and one space between words.

Insert the code in the loop:

```
vote = vote.replace("  ", " ").capitalize()
```

Results will look like:

```
Red king: 4
White icicle: 1
Cherry belle: 3
Daikon: 4
...
```

The code here, inserted after `name, vote = line.split(" - ")`, will replace two spaces with one and capitalize the names. In the end, with this improvement, the different variations of Red King and Cherry Belle will be merged.

String methods

- `s.strip()` returns a string with whitespace removed from the start and end
- `s.startswith('other')` or `s.endswith('other')` tests if the string starts or ends with the given other string
- `s.replace('old', 'new')` returns a string where all occurrences of 'old' have been replaced by 'new'
- `s.split('delim')` returns a list of substrings separated by the given delimiter.
- `s.join(list)` opposite of `split()`, joins the elements in the given list together using the string as the delimiter.

CSV Data Files

In the radish survey example, data are delimited by -. Today, it is very common to encounter data in the CSV format. So let's take a close look at this format and how to process CSV files with Python.

CSV

- CSV, or comma-separated values
 - A common way to express structured data in text files
 - Supported by spreadsheet software, e.g. Excel
 - Can be used to import to / export from relational databases

In `coffee.csv`:

```
"Coffee","Water","Milk","Icecream"  
"Espresso","No","No","No"  
"Long Black","Yes","No","No"  
"Flat White","No","Yes","No"  
"Cappuccino","No","Yes - Frothy","No"  
"Affogato","No","No","Yes"  
...
```

In the example here, the first line is the header with the names of data fields. Each text line after that is a data instance, where values are separated by a comma.

Python CSV

Use the Python CSV module to process:

```
import csv
f=open("coffee.csv")
for row in csv.reader(f):
    print(row)
```

The `csv.reader()` reads a file in the CSV format and returns each row as a list of values. The module takes care of the split of column values so you don't need to do that again.

- Read CSV and print columns:

```
import csv
f = open("airports.dat")
for row in csv.reader(f):
    print(row[1])
```

Now that the `row` variable is a list of values, you can access each value using the index. `row[1]`, for example, returns the value of the second column or field.

- Read and print filtered data:

```
import csv
f = open("airports.dat")
for row in csv.reader(f):
    if row[3] == "Australia" or row[3] == "Russia":
        print(row[1])
```

- DictReader and DictWriter use dictionary objects
 - With keys and values
 - Read data using field names (keys)


```
import csv
with open('name.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        print(row['first_name'], row['last_name'])
```

We noticed earlier that a CSV usually provides the header with the names of the fields.

When there are many fields in the data, it is easier to use names instead of a numeric index to access the corresponding value.

With DictReader and DictWriter, you can access data in the CSV using column names in the header.

- Write to CSV files:

```
import csv

infile = open('test.csv', "rb")
reader = csv.reader(infile)
outfile = open('ttest.csv', "wb")
writer = csv.writer(outfile, delimiter=',', quotechar='"', quoting=csv.QUOTE_ALL)

for row in reader:
    writer.writerow(row)

infile.close()
outfile.close()
```

Because CSV format is a plain text format that uses comma as the delimiter, you can simply open a file and write to it using the comma-delimited style.

To make sure data are written consistently according to the format, especially if you have special characters such as comma in the data, you may consider using the `csv.writer()`.

Other Formats and Tools

- JSON, JavaScript Object Notation
- XML, Extensible Markup Language
- YAML, YAML Ain't Markup Language, a human-friendly data serialization format
- Relational database and SQL

There are other commonly used formats such as JSON and XML, and we will discuss them in another lecture.

Pandas

- Data loading and processing, ...
- Data preprocessing, selection, ...

```
import pandas as pd
iris_filename = 'datasets-uci-iris.csv'
iris = pd.read_csv(iris_filename, sep=',', decimal='.',
                   header=None, names= ['sepal_length', 'sepal_width',
                                       'petal_length', 'petal_width',
                                       'target'])
```

Besides basic file read/write, Python has a wide range of tools and packages for loading, cleaning, and preprocessing data. **Pandas**, for example, is a very powerful library for data analysis and manipulation; along with **NumPy**,

which makes it easier to compute data in vectors and multi-dimensional arrays, and SciPy for scientific computing.

NumPy

- NumPy for data processing and operations on vectors, arrays, etc.

SciPy

- SciPy for scientific computing

References

- Chapter 2 ## Working with Data, of Hector Cuesta (2013). Practical Data Analysis. <https://ebookcentral-proquest-com.ezproxy2.library.drexel.edu/lib/drexel-ebooks/detail.action?docID=1507840>
- Working with Text Files:
 -
 - <http://opentechschoool.github.io/python-data-intro/core/text-files.html>
 - https://www.tutorialspoint.com/python/python_files_io.htm
 - <https://www.guru99.com/reading-and-writing-files-in-python.html>
- Working with strings: —
- <http://opentechschoool.github.io/python-data-intro/core/strings.html>
- CSV files:
 -

Python File Processing

- <https://code.tutsplus.com/tutorials/how-to-read-and-write-csv-files-in-python--cms-29907>
- <http://opentechschoool.github.io/python-data-intro/core/strings.html>

Practice 1: From Data to Information and Meaning

Theory connection: Volume I, Chapter 1

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

From Data to Meaning

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Concept Generalizability

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 2: Data Types and Representations

Theory connection: Volume I, Chapter 2

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Datum

Recovered starting point: Python Data Types.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Numerical

Recovered starting point: Python Data Types.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 2: Data Types and Representations

Categorical

Recovered starting point: Python Data Types.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Data

Recovered starting point: Python Data Structures and Python Data Structures Assignment and Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Sets

Recovered starting point: Python Data Structures and Python Data Structures Assignment and Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Vectors

Recovered starting point: Python Data Structures and Python Data Structures Assignment and Data, Vectors, and Cosine Assignment.

Data

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Python Data Types

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_data_types.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Programming – Data Types

- Basic data representation: binary digits
- We need data types
 - With respect to the problem at hand
 - Based on primitive data types as building blocks
- Computation is associated with data types:
 - Characters and strings can be sorted, concatenated
 - Integers and numbers can added, subtracted, and multiplied

On the lower level, all data and variables in a computer are represented with binary digits, 0 and 1s. On the application level, however, we need different data types and ways in which data can be operated with respect

Python Data Types

to the problems we have. We need the basic building blocks, or primitive data types, to construct more complex data objects.

Ultimately, whatever computation we can perform is associated with data types. For example, characters and strings can be sorted and concatenated. Integers and numbers in general, on the other hand, can be added, subtracted, and multiplied.

Data abstraction

Abstract data type (ADT)

- Logical description of data and related operations on the data
- Abstract from actual implementation
- Actual implementation (data structure) using programming constructs and primitive data types

The notion of Abstract Data Type (ADT) represents a logical description of data and related operations on the data. It is an abstraction or a blueprint of the actual implementation, which will be based on programming constructs and primitive data types.

Basic Python: Atomic Data Types

Numeric types:

- `int` class for integer
- `Float` class for floating point data types

Operations allowed:

```
+ - * /  
**      exponentiation  
%        modulo  
//       integer division
```

There are built-in basic atomic data types, or classes, in Python. Numeric types such as `int` and `Float` can be computed with operators such as division, exponentiation, modulo, among others.

Basic Python – Atomic Data Types

Relational and Logical Operators:

Operation Name	Operator	Meaning
less than	<	
greater than	>	
less than or equal to	<=	
greater than or equal to	>=	
equal	==	
not equal	!=	
logical and	and	Both true for result to be true
logical or	or	Either true for result to be true
logical not	not	Negates true to false, false to true

Numeric values are ordinal data and you can use relational operators such as less than or greater than to evaluate and compare different values. The result of an relational expression is either true or false, a boolean type.

There are ways in which boolean values or boolean expressions can be combined. Using `and`, `or`, and `not`, you will be able to construct any complex conditions.

Basic Python – Atomic Data Types

Try these numeric operator examples:

```
print("5+3*4 =", 5+3*4)
print("20/5 =", 20/5)
print("11/4 =", 11/4)
print("11//4 =", 11//4)
print("11%4 =", 11%4)
print("5/20 =", 5/20)
print("5//20 =", 5//20)
print("2**10 =", 2**10)
print("100**2 =", 100**2)
```

You can pause here and test some the operators. Change some of the expression and run them again. Do they meet your expectation?

Basic Python – Boolean

Boolean data type: + bool class + True or False value

Examples:

```
x = True
print("x is", x)
y = False
print("y is", y)
print("x or y is", x or y)
print("x and y is", x and y)
print("not (x and y) is", not (x and y))
```

You can see a boolean type is either `True` or `False`. And you can combine them using operators such as `and`, `or`, and `not`.

Another set of examples:

```
print("Is 5 equal to 10?", 5 == 10)
print("Is 10 greater than 5?", 10 > 5)
x = 15
print("Is x between 10 and 20?", (x>=10) and (x<=20))
```

Here you see the result of an equality and inequality comparison is also a bool value. For example, `5==10` is `False`, and it is `True` that `10>5`. Again, you can combine multiple conditions using the logical operators.

Again, change the expressions and test them for yourself. Do logical operators such as `and`, `or`, and `not` make sense?

Basic Python - Collections

Collection Data Types + Ordered collections: 1. Lists 2. Strings 3. Tuples
+ Unordered collections: * Sets * Dictionaries

So far the data types are associated with single data values, such as a number, an boolean expression.

In data processing, we often need to deal with a collection of data values. There are multiple types of collection data. You may have an ordered collection such as a list, a sequence of characters as a string, and data tuples. In other situations, an unordered collection such as a set or a dictionary might have been what you need.

Let's discuss some of these collection data types.

Basic Python - List

List: > an ordered collection of zero or more references to data objects

- Empty list: []
- Items are delimited by comma
- Can be heterogeneous (data objects from same or different classes / types)

If `x` variable is a list: + `x[index]` references the item in the `index+1` position of the list + `x[0]` the first item, `x[1]` the second, and so on + You can retrieve the value, or assign a new value using `x[index]=...`

A list is an ordered collection of zero or more (references to) data objects. An empty list without data is constructed by a pair of square brackets. By adding values to the brackets, we can add new data to the list. Multiple values are separated by comma. In Python, a list can be either homogeneous (i.e. consisting of data of the same type) or heterogeneous (i.e. data of different types or classes). This makes programming flexible as well as dangerous.

To access an individual element in the list, you can use an index to refer to a specific position in the list. The index is an integer starting from 0, which references the first element; 1 refers to the second, and so on. You can get the value of an element, or assign a new value to it using the assignment operator.

Examples of list:

```
students = ["John", "Mary", "Peter", "Paul"]
print("Students are: ", students)
x = True
mix = ["rock", "paper", x, 5]
print("What's in there? ", mix)
```


In the examples, we can have a list of names and they are homogeneous – every element is a string here in the students list. You can also have data of different types in the list such as the `mix` variable, which has strings as well as a bool value and a number.

```
m = [30]*12
print("Does each month have 30 days? ", m)
print("How many days in February? ", m[1])
m[1] = 29
print("Days in February, now that it is a leap year: ", m[1])
```

To quickly create a list of many data values, you can simply multiply a list by a number. Here for example, you have a one-element list `[30]` and, when multiplied by 12, it becomes a list of 12 items with the same value.

You can then change the value of a specific item by using the indexing we discussed early. For example, `m[1]` references the second item in the list, which can be used for the month of February.

Basic Python - List Methods

Additional operations of a list: + Using predefined functions (methods) of the list class) + For a variable that is of the list type

For a list variable `x`, here are some of these functions:

Method	Use.	Meaning
<code>append</code>	<code>x.append(item)</code>	Adds a new item to the end
<code>insert</code>	<code>x.insert(i, item)</code>	Inserts a new item at position <code>i</code>
<code>pop</code>	<code>x.pop()</code>	Removes and returns last item
<code>pop</code>	<code>x.pop(i)</code>	Removes and returns an item at position <code>i</code>

Python Data Types

Method	Use.	Meaning
sort	x.sort()	Sort the list in the ascending order by default
reverse.	x.reverse()	Reverse the current order of the list

More can be found at: https://www.w3schools.com/python/python_ref_list.asp

List method examples:

```
week = ["Sunday", "Monday", "Wednesday", "Thursday", "Friday", "Sabbath?"]
week.pop()          # removes the last item
week.append("Saturday") # adds Saturday
week.insert(2, "Tuesday") # inserts Tuesday
print("Weekdays", week)
```

In the example here, we first have a list of 6 weekdays in the variable `week`. The `week.pop()` is called, the last item is called and `Sabbath?` is removed; and with `week.append()`, we add Saturday to the end of the list, and insert Tuesday at position 2, which will be placed before the existing `Wednesday`.

Basic Python - List functions

List-related functions:

- `range(..)` defines a sequence of values in a range (iterable)
- `list(...)` constructs a range of values based on an iterable
- `len(x)` returns the number of items in `x`

Examples:

```
it = range(10)
print("The iterable is in", it)
sequence = list(it)
print("A list thus generated is", sequence)
print("which has", len(sequence), "values")
```

`range(10)` create a range between 0 and 10, but does not include 10. A list can then be created based on the range, which is an iterable, and the results are 10 integers from 0 to 9.

Basic Python - String

String, a list of characters: > a sequential collection of zero or more letters, numbers, and other symbols

```
season = "Spring"
print("The string has", len(season), "characters")
print("The third character is", season[2])
```

Because a string is essentially a list, you can continue to use functions such as `len()` for a list and the same indexing mechanism to access individual items or characters in the string.

String methods, with a string variable `x`:

Method	Use.	Meaning
find	<code>x.find(item)</code>	Returns the index of first occurrence of item
lower	<code>x.lower()</code>	Returns the string in lowercase
upper	<code>x.upper()</code>	Returns the string in uppercase

Python Data Types

Method	Use.	Meaning
split	x.split(sep)	Split the string into substrings with a separator

More can be found at: https://www.w3schools.com/python/python_strings.asp

```
name = "John P. Smith"
p = name.find("P")
print("First P appears at", p)
(f,m,l) = name.split(" ")
print("First name is", f.upper())
```

In the example, you can use the `find()` method to search the string and find out where a substring such as “P” appears first. You can use the `split()` method to divide the full name into first, middle, and last name, with the space as delimiter.

Basic Python - Tuple

Tuple > immutable sequence of data objects

- Similar to list
- Cannot be changed (immutable) once constructed
- Less overhead and more memory efficient than list
- Values comma-delimited in parentheses ()

```
slogan = ("I", "am", "here", "to", "stay")
# works like a list
print(len(slogan))
print("What did you say?", slogan[4])
# slogan[4] = "leave"      # Sorry, but you cannot make me leave
```

A Tuple behaves like a list. But try if you can uncomment the last statement and see if you can change the value in a tuple. What happened? Why?

Basic Python – Set

A set is > an unordered collection of zero or more immutable data objects.

- Every item is unique (with no duplicates)
- Empty set with `set()` or `{}`
- Values comma-delimited in curly braces `{}`

```
chess = {"King", "Queen", "Knight", "Knight", "Bishop", "Bishop"}  
print(chess)
```

So you see only the unique elements are preserved even though we enter multiple `knights` and `bishops` in the set.

Set methods, with a variable `x` of the type:

Method	Use.	Meaning
union	<code>x.union(y)</code>	Returns a new set with unique elements from both
intersection	<code>x.intersection(y)</code>	Returns a new set with unique elements in common
difference	<code>x.difference(y)</code>	Returns a new set with elements from 1st set ONLY
issubset	<code>x.issubset(y)</code>	Returns whether x is a subset of y

There are also methods to add and remove elements.

More can be found at: https://www.w3schools.com/python/python_ref_set.asp

Python Data Types

```
john = {"Python", "Java", "R", "C++"}
mary = {"Python", "R", "Javascript"}
team = john.union(mary)
print("Team skills", team)
print("Common skills", john.intersection(mary))
print("Mary's unique skills", mary.difference(john))
```

With built-in methods of the `set` class, you can easily find out – between John and Mary – what’s in common, what’s the joint skill set, or what unique skills Mary has.

Basic Python – Dictionary

A dictionary is > an unordered structure of associated pairs of items:

- Each pair is a key and a value
- Pairs are comma delimited in curly braces
- Each value can be referenced using its key in square bracket `[key]`

```
# state population data
pop = {"PA":12813969, "NY":19491339, "FL":21646155, "TX":29087070, "CA":3974}
print("Pennsylvania:", pop["PA"])
ca_to_pa = pop["CA"] // pop["PA"]
print("CA population is about", ca_to_pa, "times that of PA.")
```

A dictionary is very useful and highly efficient to keep track of data associated with unique identifiers. The example here keeps state population data and can quickly look it up using a two-letter, unique state name.

Dictionary methods, with a variable `x`:

Method	Use.	Meaning
keys	x.keys()	Returns the keys in the dictionary
values	x.values()	Returns the values in the dictionary
items	x.items()	Returns the key-value pairs in the dictionary
get	x.get(k, alt)	Returns the value of key k, or alt if no such key

More can be found at: https://www.w3schools.com/python/python_ref_dictionary.asp

```
votes = {"John":10, "Mary":3}
for candidate in votes:
    print(candidate, "received", votes[candidate], "votes, so far. ")

# count last three votes
votes["John"] += 1
votes["Mary"] = votes["Mary"] + 1
votes["Paul"] = votes.get("Paul",0) + 1      # if no such key, treat current value as 0
print("Final votes:", votes)
```

In this example of voting counting. Every time we see a vote for one candidate, we can add 1 to the candidate's key in the directory. However, a potential problem may arise when a key does not exist yet. The `get` method is useful when certain keys may not have existed in the middle of a counting process.

Paul, for example, has not received a vote so his name is not in the dictionary yet. So when you use `votes.get("Paul",0)`, it will return 0 if the key is not there yet so we can add 1 to it. Without the default value 0, the program will trigger an error when you try to add 1 to a null value.

References

1. Chapter 1 Introduction, of Miller and Ranum (2013). Problem Solving with Algorithms and Data Structures using Python. <http://interactivepython.org/runestone/static/pythonds/index.html>
2. Data Structures in Python: <http://opentechschool.github.io/python-data-intro/core/data.html>

Python Data Structures

Recovered activity

Historical source and execution note

This activity was recovered from `python/python_data_structures.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Outline

- Stacks
- Queues
- Deques

Basic data structures

Linear structures + Stacks, queues, deques, and lists + Data collections where * Items are ordered depending on how they are added/removed * Position relative to the other elements that came before and after + Two ends of linear structures * Where to add and where to remove?

In this lecture, I'd like to focus on linear data structures. Stacks, queues and deques behave like a list, but they differ in the order of adding new items and the positions where items can be accessed and removed from the list. The design to allow the access of an item from one end but not the other seems restrictive but makes it easier, and sometimes more efficient, to translate a real world problem into a computer solution.

Stacks

Here is a stack of books, when you put one on top of the other. There is a natural order in which the books are placed in a stack. You start with the one in the bottom, and add another on top, and continue that way. When you remove it, the most natural way is to remove one book at a time from the top.

So regardless of whether you are to add or to remove, you access the stack from the top.

A stack, or “push-down stack”: + An ordered collection of items where * the addition of new items and the removal of existing items * always takes place at the same end. + Last-in first-out (LIFO)

Because of this mechanism of placing one on top of the last and pushing it down, a stack can be thought of as a push-down stack. It is an ordered collection of items where the addition and removal happens at the same end, that is, the top.

Think about this. When you try to remove a book from a stack, you first remove the last one you put in. So a stack is last-in first-out.

Stack Design

Abstract data type (class): + **Stack()** creates a new stack that is empty. It needs no parameters and returns an empty stack. + **push(item)** adds a

new item to the top of the stack. It needs the item and returns nothing. + `pop()` removes the top item from the stack. It needs no parameters and returns the item. The stack is modified. + `peek()` returns the top item from the stack but does not remove it. It needs no parameters. The stack is not modified. + `is_empty()` tests to see whether the stack is empty. It needs no parameters and returns a boolean value. + `size()` returns the number of items on the stack. It needs no parameters and returns an integer.

Imagine if you are to design Stack data structure, perhaps here are some of the operations or methods you need to have: to push and add a new item, to pop or remove the last item, and to check the size of the stack, that is, the number of items in it.

Stack Implementation

An example Stack class, based on a Python list:

```
class Stack:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[len(self.items)-1]
```

```
def size(self):  
    return len(self.items)
```

Now if you run this code, it will work fine but you get nothing from it. For now, this only defines the class or the blueprint of what a Stack is. Until you construct an actual Stack (instance) based on the blueprint, it is not real yet.

Let's look at the Stack blueprint (class) first. The `__init__` is the constructor function that will be called when a new Stack instance is created and, at the very moment of creation, whatever code you have here will be used for initialization. Here, `self` refers to the new instance just created, within which you have an `items` variable holding an empty list with the `[]`.

The other functions implement related methods to add, remove, and access the last (top) item in the stack, as well as those to evaluate the size of the stack.

Stacks in Action

Now that we have the class (blueprint), let's create Stack instances (objects) and put them into action.

```
cards = Stack()  
print("Is the stack empty for now?", cards.is_empty())  
cards.push("J")  
cards.push("Q")  
cards.push("K")  
print("The last card is", cards.peek())  
print("There are", cards.size(), "cards in the stack.")  
cards.pop()  
print("We removed the top.")
```

```
print("Now the last card become", cards.peek())
print("There remain", cards.size(), "cards in the stack.")
```

In the code, we first create an empty Stack `cards` by calling the `Stack()` constructor, which executes whatever you have in the `__init__` function. Next, we push or add new cards to the `cards` stack, and when we peek, we can only access the last one, “King” which is on the top. And now if we call `pop()`, we remove the “King”.

Stack Applications

Why do we need Stacks? + Isn't a list more flexible? + Why FILO? Isn't that restrictive and making programming more difficult?

There are very natural questions from students. But there are applications where the restrictive design is a blessing. When the problem you try to solve also exhibits the same restrictive characteristics, the design will make sure related assumptions and conditions are not violated. In fact, the proper mapping of the problem space and the design makes coding easier.

Example Application

Math formula checker

Correct:

$7 - (5 + (2+3)*10)$

Incorrect:

$7 - (5 + (2+3)*10))))$

Let's take a look at real world problems. Imagine you write a math equation, whenever you open with (, you will have close it to properly with) at some point. The whole equation should be balanced, e.g. two opening (matched with two closing) just like the first equation above. The second equation is incorrect because there are more closing) than the number of opened ones.

Related problems in HTML markup:

```
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    Page Content
  </body>
</html>
```

Checking math equations is similar to checking code in programs and markup languages. In HTML – XML and XHTML in particular – tags appear in pairs. A regular HTML tag consists of an opening `<tag>` and a closing one `</tag>`. They should be likewise balanced. So how can we check them, using the idea of a stack?

So let's try to create a piece of code to check the balance of a match equation.

```
# Math checker
m = "7 - (5 + (2+3)*10)"
s = Stack()
balanced = True
for symbol in m:
    if symbol == "(":
        s.push(symbol)
```

```

elif symbol == ")":
    if s.is_empty():
        balanced = False
    else:
        s.pop()

if not s.is_empty():
    balanced = False

print("Is it balanced? ", balanced)

```

For now, we hard code the math equation to show the basic idea. You can certainly make this a function or even a method of a class so you can reuse it.

We create an empty Stack `s` first and assume the result to be correct with `balanced = True`, unless we find something wrong.

Now we take each `symbol` in the equation `m` from left to right, which is what the loop does. For each symbol:

1. if it opens with a (, we push it to the stack.
2. if it closes with a):
 - if stack is empty, the equation is problematic as no opening (has preceeded it;
 - if stack not empty, we remove (pop) the last opening (on top as it has been **balanced out** (good).

So going through the math here $7 - (5 + (2 + 3) * 10)$, from left to right:

															Bal-
Step7	-	(	5	+	(	2	+	3	)	*	1	0	)	Stack	anced
1.	7														True

Step	7	-	(	5	+	(	2	+	3	)	*	1	0	)	Stack	Bal- anced	
2.		-														True	
3.			(												+	(	True
4.				5													True
5.					+												True
6.						(									+	((	True
7.							2									((	True
8.								+								((	True
9.									3							((	True
10.									)						-	(	True
11.											*					(	True
13.												1				(	True
14.													0			(	True
15.														)	-		True

In the end, **balanced** remains true and the stack is empty (balanced out). So everything is good.

Now for the second equation we had earlier, ****7 - (5 + (2+3)*10))****:

Step	7	-	(	5	+	(	2	+	3	)	*	1	0	)	)	)	Stack	Bal- anced
15.														)				True
16.														)				False
17.														)				False

The process will continue with a few more steps with the extra **)))**. Now that the stack is already empty at step 15, it triggers an **if** condition and **balanced** will be set to **False**.

Step	7	-	(	5	+	(	(	(	2	+	3	)	*	1	0	)	Stack	Balance			
..																					
6.						(											+	((	True		
7.							(											+	((((	True	
8.								(											+	(((((	True
..																					
16.															)			-	((	True	
17.															)			-	((	True	

```
if not s.is_empty():
    balanced = False
```

```
if not s.is_empty():
    balanced = False
```

395

sound like a good idea for the problem here? Think about HTML code, do you think it is similar to develop a program to check whether HTML tags are opened and closed properly?

Queue

I suppose you don't like waiting, especially if you are late and there is already a long line. It might be tempting to push in, but hey, you are not supposed to do that.

A queue: + An ordered collection of items where * the addition of new items happens at one end (rear) * the removal of existing items occurs at the other (front) + Queues are **first-in first-out** (FIFO), or first come first served

A waiting line is a queue, and the rule is first come first served.

A queue is defined as an ordered collection of items where the addition of new items and the removal of existing items occur at the different ends. This is quite the opposite of a Stack, where you access from the same end.

So queues are first-in first-out, or first come first served.

Queue Design

Queue abstract data type:

- `Queue()` creates a new queue that is empty. It needs no parameters and returns an empty queue.
- `enqueue(item)` adds a new item to the rear of the queue. It needs the item and returns nothing.
- `dequeue()` removes the front item from the queue. It needs no parameters and returns the item. The queue is modified.

- `is_empty()` tests to see whether the queue is empty. It needs no parameters and returns a boolean value.
- `size()` returns the number of items in the queue. It needs no parameters and returns an integer.

Again, imagine you are designing a data structure to solve a problem related to queues. Here are potential operations you need to include to support the construction of related algorithms, for example: enqueue to add a new item to the end, dequeue to remove one from the front, and methods to check the size of the queue.

Queue Implementation

```
class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0, item)

    def dequeue(self):
        return self.items.pop()

    def size(self):
        return len(self.items)
```

The Queue class is similar to the Stack class we implemented earlier, both using a list to store items. The `dequeue()` method is similar to the `pop()` in the Stack class, which removes an item at the end of a list.

Here, conceptually, the end of the items list is in fact the **front** of the queue. Conversely, the `enqueue()` method adds an item to position 0, which is, in this case, the **rear** of the queue.

```
orders = Queue()          # An empty queue first
orders.enqueue("Coke")
orders.enqueue("Burger")
orders.enqueue("Fries")

item = orders.dequeue()    # Coke is served and dequeued
print("Served", item)
item = orders.dequeue()    # Next, Burger is served
print("Served", item)
print("How many orders remain?", orders.size())
item = orders.dequeue()
print("Served", item)
print("Now, has everyone been served?", orders.is_empty())
```

Queue has a wide range of applications, from business transactions, to printer jobs, to any processes where things get done on a first-come first-served basis. The example here shows a sequence of actions to place (add) orders and to get them processed and served (removed).

Dequeues

A deque, or double-ended queue + An ordered collection of items similar to the queue * Two ends, front and rear * Items can be added and removed from either front and rear + A deque has all capabilities of a stack and a queue + No longer restricted to LIFO and FIFO, with access from both ends + Consistent use of addition and removal in the context of application

Deque is an extension of the queue structure.

Deque, or a double-ended queue, is an ordered collection of items with two ends, the front and the rear, and items can be added and removed from both ends. In this design, a deque has all capabilities of a stack as well as those of a queue. It is no longer last-in first-out or first-in first-out.

If deque sounds a like weird idea, think about a train. You can add or remove cars to the end of a train; likewise, you attach them to the front. An engine can be attached to either side depending on which direction you want to go.

Deque Design

Deque abstract data type: + **Deque()** creates a new deque that is empty. It needs no parameters and returns an empty deque. + **add_front(item)** adds a new item to the front of the deque. It needs the item and returns nothing. + **add_rear(item)** adds a new item to the rear of the deque. It needs the item and returns nothing. + **remove_front()** removes the front item from the deque. It needs no parameters and returns the item. The deque is modified. + **remove_rear()** removes the rear item from the deque. It needs no parameters and returns the item. The deque is modified. + **is_empty()** tests to see whether the deque is empty. It needs no parameters and returns a boolean value. + **size()** returns the number of items in the deque. It needs no parameters and returns an integer.

When designing a deque type, you need to have methods to add and remove from both ends: **add_front()**, **add_rear()**, **remove_front()**, and **remove_rear()**.

Deque class implementation

```
class Deque:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return self.items == []

    def add_front(self, item):
        self.items.append(item)

    def add_rear(self, item):
        self.items.insert(0, item)

    def remove_front(self):
        return self.items.pop()

    def remove_rear(self):
        return self.items.pop(0)

    def size(self):
        return len(self.items)
```

Compare this Deque class code to the Queue and Stack we had earlier. It is a combination of methods from the two classes, in different names though.

Deque in Action

```
train_cars = Deque()

# attach the cars in the end
train_cars.add_rear("Passengers")
train_cars.add_rear("Dining")
train_cars.remove_rear()
train_cars.add_rear("Parlor")
train_cars.add_rear("Dining")

# attach caboose
train_cars.add_rear("Caboose")

# put engine in front
train_cars.add_front("Engine")

print("How long is the train?", train_cars.size(), "cars")
```

Imagine you are to arrange train engine with different cars. You may play with the code to see the different orders in which the same sequence of cars can be constructed.

References

- Chapter 1 Introduction, of Miller and Ranum (2013). Problem Solving with Algorithms and Data Structures using Python. <http://interactivepython.org/runestone/static/pythonds/index.html>
- Chapter 4 Data Structures, of Miller and Ranum (2013). Problem Solving with Algorithms and Data Structures using Python. <http://interactivepython.org/runestone/static/pythonds/index.html>

Python Data Structures Assignment

Recovered activity

Historical source and execution note

This activity was recovered from `problems/python_data_structure.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Objectives

Please solve the following problem with Python programming using proper data structures.

1. Create a **Jupyter Notebook** with code and markdown descriptions;
2. **Test your code** and make sure it works properly;
3. Submit your Jupyter Notebook (.ipynb) to **blackboard**.

A similar application to the parentheses matching problem is HTML (hypertext markup language) validation. In HTML, tags exist in both opening and closing forms and must be balanced to properly describe a web document.

Python Data Structures Assignment

This simple HTML code (string) here:

```
<html>
<head>
<title>
Example>
</title>
</head>
<body>
<h1>
Hello, world
</h1>
</body>
</html>
```

shows the matching and nesting structure for tags in the language.

Please create a Jupyter notebook and write a Python program that can **validate HTML code** with proper opening and closing tags. To simplify the problem, we suppose the opening and closing tags are provided in **separate lines without indentation** (as shown in the above example).

Consult the example on the math checker (parentheses matching) in lecture slides/notes:

python-data-structures.qmd#sec-lab-python-data-structures

0. Academic Honesty Statement

Please put the Academic Honesty Statement with your full name printed on your notebook.

Your submission **will not be graded without the statement**.

1. Create a Stack abstract data type (2 points)

1. Create a Stack abstract data type (2 points)

The Stack class should include the following methods:

- `push(item)` function to add a new item to the stack
- `pop()` function to remove and **return** the top item from the stack
- `peek()` to return the top item in the stack
- `is_empty()` to test whether the stack is empty
- `size()` to tell the size (number of items) in the stack

```
class Stack:
    """
    The Stack class to be implemented.
    Please define all required methods
    within the class structure.
    """
```

2. Create a check_html function (5 points)

```
def check_html(html_string):
    """Validates an HTML string (text)
    Arguments:
        html_string (string): the HTML string with tags in each line (separated by \n)

    Returns:
        bool: True if the HTML tags are balanced; False if otherwise.
    """
```

Here is what the function should do:

- Create a **balanced** variable and set its initial value to **True**

Python Data Structures Assignment

- Create a new Stack object;
- Use the `split()` method of a string to split it into text lines;
- Create a loop to read each text line:

– You can use a for loop such as –

```
```python
for line in text_lines:
```
```

- If the text line matches an opening tag (e.g. `<h1>`), add the tag name (e.g. “h1”) to the stack;
 - * You can use expressions such as `"<" in line` to check if the symbol “<” appears in the string variable
 - * You should make sure “<” is in the text line AND “/” is NOT in it for an opening tag;
 - * Use the logical **and** operator to connect multiple logical conditions;
 - * Use statements such as `tag_name = line.replace("<", "")` to remove special symbols, so you can reduce it to its tag name only;
- If the text line is a closing tag (e.g. `</h1>`), check whether the stack is empty:
 - * The HTML code is unbalanced if stack is currently empty:
 - Please use the `is_empty()` method of the stack object.
 - * If not empty, use the stack object’s `pop()` to read and remove the top (last) item from stack:
 - Again, use the `replace()` method of the text line string variable to remove symbols `</>`, to reduce it to its tag name only;

3. Function Calls (0.5 point)

- If the top (last) tag matches (==) the current closing tag, then the HTML code is balanced (True);
 - Otherwise, the HTML code is unbalanced (False).
- If the text line is ordinary content without a HTML tag, skip the line;
 - * You do not actually need to do anything in this case.
- After the loop, check:
 - If the Stack is empty (balanced out), return the value of the `balanced` variable
 - Otherwise, `return False`

3. Function Calls (0.5 point)

Create two string variables, one with *balanced HTML* code and the other with *unbalanced HTML code*.

Call the `html_checker()` function **twice** in your code, each time with each of the two variables as argument to test the program. For example:

```
str1 = "(Valid/balanced HTML code here)"
str2 = "(Invalid/unbalanced HTML code here)"
print("Is the first string valid?", check_html(str1))
print("Is the second string valid?", check_html(str2))
```

You can use the syntax in the box here to create a string with multiple lines.

4. Markdown and Comments (2 point)

Besides your Python code, you should also have: + Markdown headers and text to provide basic descriptions of your work + Comments in the Python code for each function and major step in the code

Bonus (+1 point)

Create a loop to read user input for multiple lines of HTML code (until the user enters an empty line to signal the end) and run the `check_html()` function to validate the entered HTML.

Important Notes

- Do **NOT copy and paste** other people's code without understanding it and proper acknowledgment.
- Make sure your **code runs** properly before submitting it.
- The grade will be significantly penalized if code does NOT work.
- Get in touch with the instructor if you need **guidance** / assistance.

Practice 3: Vectors, Matrices, and Geometric Thinking

Theory connection: Volume I, Chapter 3

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Basic Matrix Operation

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Sum of Squares

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Vector Magnitude (Length)

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Vector Distance

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Vector Angle

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Vector Cross Product

i Practice development scaffold

Create an activity that makes **Vector Cross Product** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Optimization

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Kernel Functions

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Graph Analysis

Recovered starting point: Reinforcement Learning Extension.

Practice 3: Vectors, Matrices, and Geometric Thinking

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Data, Vectors, and Cosine Assignment

Recovered activity

i Historical source and execution note

This activity was recovered from problems/python_data_init.ipynb. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Preparation

Please import all necessary packages and setup `%matplotlib` for inline plots in the report.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from numpy import linalg
```

```
from scipy import stats
from scipy.stats import norm, probplot
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

%matplotlib inline
```

1. Basic Statistics (2 points)

Suppose that the data for analysis includes the attribute age. The age values for the data tuples are (in increasing order):

```
age = [13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 33, 33]
```

(a) What is the mean of the data? What is the median?

```
# Use Numpy's np.mean(age)
# Use Numpy's np.median(age)
```

(b) What is the mode of the data?

```
# Use Scipy's stats.mode(age)
```

(c) Plot the distribution and comment on the data's modality (i.e., bimodal, trimodal, etc.)

```
# Use Matplotlib's plt.hist(age)
```

(d) Can you find (roughly) the first quartile (Q1) and the third quartile (Q3) of the data?

2. IRIS Data Statistics

```
# Try: np.percentile(age, [25, 75])
```

(e) Give the five-number summary of the data.

(f) Show a boxplot of the data.

```
# Try: plt.boxplot(age)
```

(g) What does the boxplot tell? Be thorough.

2. IRIS Data Statistics

Consult IRIS data documentation online about the dataset:

<http://archive.ics.uci.edu/ml/datasets/Iris>

which has been included in related packages such as Sklearn and Seaborn.

Here is an example to load data into a data frame.

```
iris = load_iris()
iris_df = pd.DataFrame(data= np.c_[iris['data'], iris['target']], columns= iris['feature_names'], index= iris['target'])
iris_df.head()
```

2.1. Attribute Types (1 point)

Examine data values in the data frame and create a table to list all attribute data types:

Data, Vectors, and Cosine Assignment

| At-tribute | # Values | Order? | Interval? | Zero Point? | Data Type |
|--------------|----------|----------|-----------|-------------|--------------------------------------|
| Sepal Length | # | Yes / No | Yes / No | Yes / No | Nominal / Ordinal / Interval / Ratio |
| ... | | | | | |

2.2. Data Distributions (1.5 points)

Examine data distributions for each **numeric** attribute, using a histogram with a density plot.

For example, you can use the Seaborn's `distplot()` function to create a histogram:

```
sns.distplot(iris_df["sepal length (cm)"])
```

Questions

1. Does it look like a normal distribution?
2. What modality is the distribution? (# modes)
3. Is the distribution skewed? If so, positively or negatively?

Repeat for Each Attribute

- Repeat the above for each numeric attribute, and answer the questions.
- Pick a distribution that is unlikely a normal distribution
- Generate a QQ plot on the actual quantiles vs. normal distribution quantiles
- Is it a normal distribution according to QQ? Why or why not?

2. IRIS Data Statistics

Example code to generate a QQ plot against normal:

```
probplot(iris_df["sepal length (cm)"], dist="norm", plot=plt)
```

2.3. Subset Data Distributions (2 points)

Now take a data subset for a specific **target** (species). For example:

```
iris_df[iris_df["target"]==0.0]
```

will limit the data to rows with target value 0.0.

Produce and examine each numeric attribute's data distribution for each target.

Example code to plot subset distribution:

```
sns.distplot(iris_df[iris_df["target"]==0.0]["sepal length (cm)"])
```

Questions

1. Does it look like a normal distribution?
2. What modality is the distribution? (# modes)
3. Is the distribution symmetric or skewed? If so, positively or negatively?

Repeat for Each Target

- Repeat the above for **each numeric attribute** and **each target** level (0, 1, 2).
- Pick a distribution that looks like a normal distribution.

Data, Vectors, and Cosine Assignment

- Generate a QQ plot on the actual quantiles vs. normal distribution quantiles
- Is it a normal distribution according to QQ? Why or why not?

Example code to generate a QQ plot against the normal distribution:

```
probplot(iris_df[iris_df["target"]==1.0]["petal length (cm)"], dist="norm", )
```

2.4. Boxplots vs. Target Levels (1.5 points)

For each numeric attribute, generate boxplots across the target levels.

For example:

```
sns.boxplot(x="target", y="sepal length (cm)", data=iris_df)
```

For Each Attribute

Examine the boxplots and discuss the following questions:

1. Do the distributions show a **constant variance** on the different target levels?
2. On each level, does it show a **symmetric** distribution? Or is it positively or negatively skewed?
3. Are the means constant or different? What does it mean (about potential correlation)?

Please do the boxplots and answer the question for each numeric attribute.

3. Correlation, Similarity and Distance

3.1. Scatter Plots (0.25 point)

Reload the IRIS data from Seaborn and create pairwise scatter plots for IRIS variables:

```
iris = sns.load_dataset("iris")    # reload data from seaborn
sns.pairplot(iris)
```

Optional: You may also consider using the hue parameter to color code the target (species) on the plots:

```
sns.pairplot(iris, hue="species");
```

3.2. Visual Examination (1 point)

Examine the scatterplots and identify **two pairs** of numeric attributes:

1. One pair of **strongly correlated** attributes; Briefly explain **why or whether** they are positively or negatively correlated? 2. One pair of weakly correlated or **uncorrelated** attributes; Briefly explain **why** they appear to be uncorrelated.

3.3. Pearson Correlation (1 point)

Now compute Pearson correlation on the data attributes.

```
pcorr = iris.corr(method='pearson')
pcorr
```

For the two pairs of attributes you identified earlier, single out their Pearson correlation scores:

Data, Vectors, and Cosine Assignment

1. Strongly correlated pair: Pearson score
2. Uncorrelated pair: Pearson score

Note: please keep **3 digits** after the decimal point, e.g. *0.123*.

What do the scores tell you? Are they consistent with your visual observation?

3.4. Euclidean Distance (0.25 point)

Compute Euclidean distances for the two pairs of attributes. For example:

```
a = iris['sepal_length']
b = iris['sepal_width']
d = linalg.norm(a-b)
d
```

3.5. Cosine 1 with Original Values (0.25 point)

Compute Cosine similarities for the two pairs of attributes. For example:

```
a = iris['petal_length']
b = iris['petal_width']
c1 = np.dot(a,b)/(linalg.norm(a)*linalg.norm(b))
c1
```

3.6. Cosine 2 with Vectors from Mean (0.25 point)

Subtract the mean from each attribute value and make each attribute (column) a vector from its mean (instead of from the origin), and compute Cosine similarities. For example:

3. Correlation, Similarity and Distance

```
a = iris['petal_length']
a2 = a - np.mean(a)
b = iris['petal_width']
b2 = b - np.mean(b)
c2 = np.dot(a2,b2)/(linalg.norm(a2)*linalg.norm(b2))
c2
```

3.7. Summary and Observation (1 point)

Compile all scores (pearson, euclidean distance, cosine 1 and cosine 2) together as one table below:

| Pair | Pearson | Euclidean | | |
|---------------------------------------|---------|-----------|----------|----------|
| | | Dist | Cosine 1 | Cosine 2 |
| Strong correlation: attr 1 vs. attr 2 | | | | |
| Weak correlation: attr 3 vs. attr 4 | | | | |

Examine the scores for each pair in the table, and answer the questions:

1. Does a stronger Pearson correlation necessarily lead to a smaller or greater Euclidean distance?
2. Does a strong Pearson correlation lead to a greater Cosine 1 (vectors from the origin)?
3. What do you observe is the relation between Pearson correlation and Cosine 2 (vectors from the mean)?

Practice 4: Probability and Statistical Thinking

Theory connection: Volume I, Chapter 4

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Probability

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probability Basics

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 4: Probability and Statistical Thinking

Joint probability

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Independent events

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Dependent Events

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

The Bayesian Theorem

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Naive Bayes' Model

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probability Estimators

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probability Distributions and Models

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Continuous Probability Distribution

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probability Estimators

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Discrete Probability Estimation

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

MLE for a Continuous Variable

Recovered starting point: Probability and Linearity in Data.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probability and Linearity in Data

Recovered activity

i Historical source and execution note

This activity was recovered from `demo/data_prob_linear.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Import necessary libraries in Python:

```
%matplotlib inline
import math
import matplotlib.pyplot as plt
from matplotlib import animation, rc
import numpy as np
import pandas as pd
import seaborn as sns
from numpy import linalg
from scipy import stats
from IPython.display import HTML, display

plt.rcParamsdefaults()
# plt.xkcd(scale=1, length=300, randomness=5)
```

A. Probabilities in Language

Let's look at a collection of emails, with spams and hams: <https://www.kaggle.com/uciml/sms-spam-collection-dataset>

Here is a local copy of the data: spam.csv.

A.1. Load Data

```
emails = pd.read_csv("../data/spam.csv", encoding="iso8859-1")
emails[['v1', 'v2']].head()
```

A.2. Zipf's Law

Let's tokenize the emails and count word frequencies.

```
from sklearn.feature_extraction.text import CountVectorizer

# instantiate a count vectorizer
vect = CountVectorizer(analyzer="word")
# obtain vocabulary dictionary and return doc-term matrix
X = vect.fit_transform(emails.v2)
# get the identified terms
terms = vect.get_feature_names()
# sum terms' frequencies in the entire collection
freqs = X.toarray().sum(axis=0)
terms = np.array(terms)
freqs = np.array(freqs)

# organize the terms and frequencies in t and f columns
tfs = pd.DataFrame({'t':terms, 'f':freqs}, columns=['t','f'])
```

A. Probabilities in Language

Rank the terms by their frequencies:

```
tfs = tfs.sort_values(by=['f'], ascending=False)
tfs = tfs.reset_index(drop=True)
tfs.head()
```

Add the rank r column to start with 1:

```
tfs['k'] = tfs.index + 1
tfs[['t', 'k', 'f']].head()
```

Plot the frequency f_t vs. rank k_t :

```
plt.plot(tfs['k'], tfs['f'])
plt.xlabel('Rank $k_t$')
plt.ylabel('Frequency $f_t$')
plt.show()
```

Transform X and Y to log-log coordinates:

```
plt.loglog(tfs['k'], tfs['f'])
plt.xlabel('Rank $k_t$')
plt.ylabel('Frequency $f_t$')
plt.show()
```

Now each term's probability to occur can be estimated by:

$$p_t = \frac{f_t}{\sum_t f_t}$$

Probability and Linearity in Data

```
ttf = tfs['f'].sum()
tfs['p'] = tfs['f'] / ttf
tfs[['t','k','p']].head()
```

Plot probability p_t vs. rank k_t :

```
plt.loglog(tfs['k'], tfs['p'])
plt.xlabel('Rank  $k_t$ ')
plt.ylabel('Probability  $p_t$ ')
plt.show()
```

Does it look like a straight line on log-log coordinates? Does it follow the Zipf's law?

Let's perform a linear regression on the log-transformed p_t and k_t :

```
from sklearn.linear_model import LinearRegression

# log transformation of k and p values
x = tfs['k'].values.reshape(-1,1)
xlog = np.log(x)
y = tfs['p'].values.reshape(-1,1)
ylog = np.log(y)

# perform linear regression on log values
lm = LinearRegression()
lm.fit(xlog,ylog)
yplog = lm.predict(xlog)

# plot the fitted (predicted) line
# along with actual k_t and p_t data
plt.loglog(tfs['k'], tfs['p'], label="Observed Data")
plt.loglog(x, np.exp(yplog),
```

A. Probabilities in Language

```
label="$p_t = k_t^{" + "{:.2f}".format(lm.coef_[0][0]) + "}$")
plt.xlabel('Rank $k_t$')
plt.ylabel('Probability $p_t$')
plt.legend()
plt.show()
```

So the fitted line is roughly:

$$p_t = k_t^{-1.16} \quad (17.1)$$

$$= \frac{1}{k_t^{1.16}} \quad (17.2)$$

Not exactly $p_t = \frac{1}{k_t}$ as the Zipf's law suggests.

A.3. Probabilities in Training Data

Given the above dataset, let's split it into two random subsets: 80% training and 20% testing.

```
# generate random true (80% chance) or false values
train_index = np.random.rand(len(emails)) < 0.8
# use the above list to take the training (true values)
train = emails[train_index]
# the opposite (rest) is for testing
test = emails[~train_index]
```

The probability of a ham $p(ham)$ in the training data:

Probability and Linearity in Data

```
# get the total number of instances
n = train.shape[0]
# get the total number of hams
counts = train.groupby("v1")["v1"].value_counts()
ham = counts[0]
# compute probability
p_ham = ham / n
print("p(ham) = {:.3f}".format(p_ham))
```

The probability of a spam $p(\text{spam})$:

```
# get the total number of spams
spam = counts[1]
# compute probability
p_spam = spam / n
print("p(spam) = {:.3f}".format(p_spam))
```

Now we vectorize the training data:

```
vect = CountVectorizer(analyzer="word")
X_train = vect.fit_transform(train.v2)
```

And use the trained dictionary to vectorize test data:

```
X_test = vect.transform(test.v2)
```

We split the training data into spam and ham subsets:

```
ham_index = train['v1']=='ham'
hams = train[ham_index]
spams = train[~ham_index]
```


A. Probabilities in Language

```
X_hams = vect.transform(hams.v2)
X_spams = vect.transform(spams.v2)
X_hams.shape
```

Let's locate a term in the list of terms (dictionary):

```
terms = vect.get_feature_names()
t = "prize"
i = terms.index(t)
print("The {:d}th term is: {:s}".format(i, t))
```

We can compute conditional probability of a term in the ham $p(t|ham)$:

```
hamf = X_hams.toarray().sum(axis=0)
hamttf = hamf.sum()
f = hamf[i]
p = f / hamttf
ps = (f+1)/(hamttf+2)
print("p({:s}|ham) = {:.7f}, smoothed to {:.7f}".format(t, p, ps))
```

You see it is very unlikely for term **prize** to appear in a ham (normal email).

Now let's look at the conditional probability of the same term in the spam $p(t|spam)$:

```
terms = vect.get_feature_names()
spamf = X_spams.toarray().sum(axis=0)
spamtff = spamf.sum()
f = spamf[i]
p = f / spamtff
ps = (f+1)/(spamtff+2)
print("p({:s}|spam) = {:.7f}, smoothed to {:.7f}".format(t, p, ps))
```

What do $p(\text{prize}|\text{ham})$ and $p(\text{prize}|\text{spam})$ mean respectively?

Apparently, it is much more likely to see **prize** in a spam than in a ham, i.e. $p(\text{prize}|\text{ham}) \gg p(\text{prize}|\text{spam})$.

Take a look at a few other terms (words) in spam vs. ham to see if their conditional probabilities make sense.

```
t1 = terms.index("prize")
t2 = terms.index("meeting")
t1, t2
```

```
# Example to plot two term (column vectors) on x and y
plt.plot(X_hams.toarray()[:,t1], X_hams.toarray()[:,t2], '+', color="blue", )
plt.plot(X_spams.toarray()[:,t1], X_spams.toarray()[:,t2], 'o', color="red", )
plt.xlabel("prize")
plt.ylabel("meeting")
plt.legend()
```

A.4. Probabilistic Binary Model (Bernoulli Naive Bayes)

The above probabilities form the basis of the Naive Bayes model (probabilistic classification).

Now let's train a Bernoulli Naive Bayes classifier, with binary term occurrences:

```
from sklearn.naive_bayes import BernoulliNB

# build the Bernoulli Naive Bayes classifiers
# with parameters, such as:
# 1. alpha value for the Laplace estimator (smoothing)
# 2. binarize, the threshold/cutoff for 0 or 1 values
bNB = BernoulliNB(alpha=.01, binarize=0.0)
```

```
bNB.fit(X_train, train.v1)
v1p = bNB.predict(X_test)
```

```
print(v1p)
```

Now let's compare and compute predicted values $v1p$ and actual class labels $v1$ in the testing data. We first look at the confusion matrix:

```
from sklearn.metrics import confusion_matrix, plot_confusion_matrix
confusion_matrix(v1p, test.v1)
display(plot_confusion_matrix(bNB, X_test, test.v1, values_format='d'))
```

Look at the confusion matrix here and think about these questions: + Among the four values, which one would you like to maximize for spam filtering? + Which ones would you minimize?

```
from sklearn.metrics import accuracy_score, cohen_kappa_score
acc = accuracy_score(v1p, test.v1)
kappa = cohen_kappa_score(v1p, test.v1)

print("Accuracy: {:.4f}".format(acc))
print("Kappa : {:.4f}".format(kappa))
```

Adjust the parameters, train and test the data again. Does it make any difference?

A.5. Probabilistic Term Frequency Model (Multinomial Naive Bayes)

Likewise, we can train and test the Naive Bayes model with the Multinomial distribution (term frequencies).

```
from sklearn.naive_bayes import MultinomialNB

# build the Bernoulli Naive Bayes classifiers
# with parameters, such as:
# 1. alpha value for the Laplace estimator (smoothing)
mNB = MultinomialNB(alpha=1)
mNB.fit(X_train, train.v1)
```

Take a look at some of the probabilities:

```
import math

pt_ham = mNB.feature_log_prob_[0][i]
pt_spam = mNB.feature_log_prob_[1][i]
print('p({:s}|ham) = {:.7f}'.format(t, math.exp(pt_ham)))
print('p({:s}|spam) = {:.7f}'.format(t, math.exp(pt_spam)))
```

Compare the estimated values to the same conditional probabilities we obtained earlier. Are they similar, relatively?

Now let's use the trained model (estimated probabilities) to predict on the test data:

```
v1p = mNB.predict(X_test)
print()
display(plot_confusion_matrix(mNB, X_test, test.v1, values_format='d'))
```

Evaluation with related metrics:

```
acc = accuracy_score(v1p, test.v1)
kappa = cohen_kappa_score(v1p, test.v1)

print("Accuracy: {:.4f}".format(acc))
print("Kappa : {:.4f}".format(kappa))
```

B. Linear Classification Model

Again, you may play with the alpha parameter (Laplace smoothing) for the model. What model performs better?

Are accuracy and kappa good metrics for the evaluation here? Think about what your objective is for email spam filtering and what your objectives might be? Are there any other evaluation metrics – precision, recall, specificity, etc. – that you should consider here?

B. Linear Classification Model

Here is a small dataset about car evaluation: cars.txt. It is originally from UCI's Car Evaluation Data Set (<https://archive.ics.uci.edu/ml/datasets/Car+Evaluation>) and has been modified (**idealized** for now) for demonstration here.

We focus on each car's: 1. **Buying price** b 2. **Maintenance cost** m 3. **Car evaluation**, i.e. class a with values of acceptable (1) or unacceptable 0.

```
cars = pd.read_csv("../data/cars.txt", sep=" ")
display(cars[['b', 'm', 'a']].sample(frac=0.5))
```

Now take a look at the car's evaluation a (acceptable vs. unacceptable) on the two attribute dimensions, buying price b and maintenance cost m :

```
positive = cars['a']>0
cars0 = cars[~positive]
cars1 = cars[positive]
# for name, group in groups:
plt.plot(cars0.b, cars0.m, linestyle='', marker='o',
         color="red", ms=6, label="Unacceptable")
plt.plot(cars1.b, cars1.m, linestyle='', marker='+',
         color="blue", ms=10, label="Acceptable")
```

```
plt.xlabel("Buying Price $b$")
plt.ylabel("Maintenance Cost $m$")
plt.legend(loc="center", shadow=True)
plt.show()
```

B.1. Linear Classification (Perceptron) with Idealized Data

Let's use the cars data to train a perceptron:

```
from sklearn.linear_model import Perceptron

# build a perceptron (single layer neural network)
# with parameters:
# 1. 20 iterations
# 2. learning rate of 0.15
net1 = Perceptron(max_iter=20, eta0=0.15, random_state=0)
net1.fit(cars[['b', 'm']], cars['a'])
w0 = net1.intercept_[0]
w1, w2 = net1.coef_[0]
print("The linear model is: {:.2f}{:+.2f}b{:+.2f}m = 0".format(w0, w1, w2))
```

Now we can visualize the linear function, a line on the 2-dimensional space:

```
# function to plot data with linear model
def plot_linear(ax, title, cars0, cars1):
    global w0, w1, w2
    ax.plot(cars0.b, cars0.m, linestyle='', marker='o',
            color="red", alpha=0.2)
    ax.plot(cars1.b, cars1.m, linestyle='', marker='+',
            color="blue", alpha=0.2)
    x = np.array([1, 2.5])
```

B. Linear Classification Model

```
y = -(w0 + w1 * x)/w2
ax.plot(x, y, color="black", linewidth=2, label=title)
ax.legend(loc="center")

positive = cars['a']>0
cars0 = cars[~positive]
cars1 = cars[positive]
plot_linear(plt, "20 Epochs", cars0, cars1)
plt.show()
```

Now that the Perceptron takes 20 iterations to find the weight, let's go through the iterations to see how the values of $[w_0, w_1, w_2]$ are updated and converge.

```
import warnings

it, w0s, w1s, w2s = [], [], [], []
for i in range(1,16):
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        net1 = Perceptron(max_iter=i, eta0=0.15, random_state=0) #, warm_start=True)
        net1.fit(cars[['b','m']], cars['a'])
    it.append(i)
    w0s.append(net1.intercept_[0])
    w1s.append(net1.coef_[0][0]) # / (net1.intercept_[0] + 0.000000001)
    w2s.append(net1.coef_[0][1]) # / (net1.intercept_[0] + 0.000000001)
```

Now we can plot how the w values change over the iterations (epochs):

```
fig, (ax1, ax2) = plt.subplots(2,1)
ax1.plot(it, w0s, label="w_0")
ax1.legend(loc="lower right")
```

```
ax2.plot(it, w1s, label="w_1")
ax2.plot(it, w2s, label="w_2")
ax2.set(xlabel="Epochs")
# ax1.ylabel("Values")
ax2.legend()
# fig.show()
plt.show()
```

Here is the visualized evolution of the linear function (straight line) over the iterations (epochs):

```
fig, axs2 = plt.subplots(3,3)
i=0
for axs in axs2:
    for ax in axs:
        ep = it[i]
        w0 = w0s[i]
        w1 = w1s[i]
        w2 = w2s[i]
        plot_linear(ax, "#{0}".format(ep), cars0, cars1)
        i += 1
```

B.2. Linear Classification with Not-So-Ideal Data

Now if we simply add a **new data** point (2,2,1) to the idealized data:

```
new = pd.DataFrame([[2,2,1]], columns=['b','m','a'])
newcars = cars.append(new)

positive = newcars['a']>0
newcars0 = newcars[~positive]
newcars1 = newcars[positive]
```


B. Linear Classification Model

```
# for name, group in groups:
plt.plot(newcars0.b, newcars0.m, linestyle='', marker='o',
         color="red", ms=6, label="Unacceptable")
plt.plot(newcars1.b, newcars1.m, linestyle='', marker='+',
         color="blue", ms=10, label="Acceptable")
plt.xlabel("Buying Price")
plt.ylabel("Maintenance Cost")
plt.legend(loc="center", shadow=True)
plt.show()
```

Now our training data in `newcars` have a **new situation** not encountered before.

Let's repeat the process: 1. Train and visualize the Perceptron model on the `newcars` data (as we did on the `cars` data earlier). 2. Do you think we will obtain a good linear function (perceptron)?

```
net1 = Perceptron(max_iter=100, eta0=0.15, random_state=0)
net1.fit(newcars[['b','m']], newcars['a'])
w0 = net1.intercept_[0]
w1, w2 = net1.coef_[0]
print("The linear model is: {:.2f}{:+.2f}b{:+.2f}m = 0".format(w0, w1, w2))
```

Here is the model after 100 iterations (epochs):

```
plot_linear(plt, "100 Epochs", newcars0, newcars1)
```

Even after many iterations (epochs) of training, the linear function cannot seem to separate the two classes.

The w coefficients did converged but the results here are not as satisfactory.

```
it, w0s, w1s, w2s = [], [], [], []
for i in range(1,51):
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        net1 = Perceptron(max_iter=i, eta0=0.15, random_state=0) #, warm_start=True
        net1.fit(newcars[['b','m']], newcars['a'])
    it.append(i)
    w0s.append(net1.intercept_[0])
    w1s.append(net1.coef_[0][0])
    w2s.append(net1.coef_[0][1])

fig, (ax1, ax2) = plt.subplots(2,1)
ax1.plot(it, w0s, label="w_0")
ax1.legend(loc="lower right")
ax2.plot(it, w1s, label="w_1")
ax2.plot(it, w2s, label="w_2")
ax2.set(xlabel="Epochs")
# ax1.ylabel("Values")
ax2.legend()
# fig.show()
plt.show()
```

C. Non-Linear Model with Multi-layer Neural Network

Now that the data cannot be linear separated, let's try a Multi-layer Neural Network (non-linear classifier) on the data:

```
from sklearn.neural_network import MLPClassifier
net2 = MLPClassifier(hidden_layer_sizes=(2), activation='logistic',
                    solver='lbfgs', max_iter=100)
net2.fit(newcars[['b','m']], newcars['a'])
net2.predict(newcars[['b','m']])
```

C. Non-Linear Model with Multi-layer Neural Network

To better understand the model, let's use the notation here according to the Non-Linear Classification Model lecture:

Note: A difference in `MLPClassifier` we built here is the **additional links** from $v_1 \rightarrow h_2$ and $v_2 \rightarrow h_1$ – the above lecture/figure skips those links to simplify the calculation and it still works.

v variables for the input: + v_1 : the buying price variable **b** + v_2 : the maintenance cost variable **m**

h variables for the hidden layer (only one layer): + $h_0 = f(1)$: is the hidden layer for overall bias input. + $h_1 = f(s_1)$: is activation based on the weighted sum s_1 to hidden node 1. + $h_2 = f(s_2)$: is activation based on the weighted sum s_2 to hidden node 2.

where f is the sigmoid (logistic) function for activation:

$$f(s) = \frac{1}{1 + e^{-s}}$$

Based on the coefficients obtained from the above model, we have: + From bias v_0 to hidden layer:

```
print(net2.intercepts_[0])
```

- From v_1 to hidden layer, i.e. first layer (index 0) and first variable (index 0):

```
print(net2.coefs_[0][0])
```

- From v_2 to hidden layer, i.e. first layer (index 0) and second variable (index 1):

```
print(net2.coefs_[0][1])
```

Following the model, we define a general function for the sum:

```
import math
# weight sum of input layer to a hidden node
# v1 and v2 are input values
# i is the index of the layer (0 from input to hidden, 1 from hidden to output)
# j is the index of the node to receive the sum
def s(v1, v2, i, j):
    w0 = net2.intercepts_[i][j]
    w1 = net2.coefs_[i][0][j]
    w2 = net2.coefs_[i][1][j]
    return w0 + w1*v1 + w2*v2

def sigmoid(s):
    return 1 / (1 + math.exp(-s))
```

Hence, given input values such as ($v_1 = 2, v_2 = 2$), the hidden nodes receive the following weighted sums:

```
s1 = s(2,2,0,0)
s2 = s(2,2,0,1)
print("Weighted sum to hidden node 1: s1 = {:.2f}".format(s1))
print("Weighted sum to hidden node 2: s2 = {:.2f}".format(s2))
```

Using the sigmoid function, the hidden layer activated values are:

```
h1 = sigmoid(s1)
h2 = sigmoid(s2)
print("Sigmoid activation of hidden node 1: h1 = {:.2f}".format(h1))
print("Sigmoid activation of hidden node 2: h2 = {:.2f}".format(h2))
```

C. Non-Linear Model with Multi-layer Neural Network

Now if we apply this process to all instances in the `newcars` data:

```
# compute the hidden layer output
# weighted sum
newcars['s1'] = s(newcars['b'], newcars['m'], 0, 0)
newcars['s2'] = s(newcars['b'], newcars['m'], 0, 1)
# sigmoid transformation
newcars['h1'] = 1/(1 + np.exp(- newcars['s1']))
newcars['h2'] = 1/(1 + np.exp(- newcars['s2']))

# compute the final output
# weighted sum and then sigmoid transformation
newcars['so'] = s(newcars['h1'], newcars['h2'], 1, 0)
newcars['o'] = 1/(1 + np.exp(- newcars['so']))
```

And we can plot the hidden layer result h_1 and h_2 , which is a non-linear transformation of v_1 and v_2 using a sigmoid (logistic) function. You can see here that the data are now linearly separable:

```
positive = newcars['a']>0
newcars0 = newcars[~positive]
newcars1 = newcars[positive]
# for name, group in groups:
plt.plot(newcars0.h1, newcars0.h2, linestyle='', marker='o',
         color="red", alpha=0.5)
plt.plot(newcars1.h1, newcars1.h2, linestyle='', marker='+',
         color="blue", alpha=0.5)
plt.xlabel("Hidden Node $h_1$")
plt.ylabel("Hidden Node $h_2$")
x = np.array([0, 1])

w0 = net2.intercepts_[1][0]
w1 = net2.coefs_[1][0][0]
```

```
w2 = net2.coefs_[1][1][0]
y = -w0/w2 - w1 * x / w2

equation = "${: .0f}{:+.0f}h_1{:+.0f} h_2 = 0$".format(w0, w1, w2)

plt.plot(x, y, color="black", linewidth=2, label=equation)
plt.legend()
plt.show()
```

The solid line here is in fact the linear function obtained by the Multi-layer Perceptron at the output layer where:

$$\sum_i w_i h_i = 0$$

With the three w weights estimated here:

```
print("Weights (w0, w1, w2) at the output: ({: .0f}, {: .0f}, {: .0f})".format(w0, w1, w2))
```

The final linear equation at the output can be written as:

$$\sum_i w_i h_i = 9 - 18h_1 - 17h_2 \quad (17.3)$$

$$= 0 \quad (17.4)$$

which is the solid line in the above plot.

You see the multi-layer perceptron is essentially transforming data with non-linear functions (e.g. a sigmoid) – sometimes multiple transformations through multiple hidden layers – and, in the end, use a linear equation (weighted sum) to separate the transformed data.

D. Further Research and Exercise

In this exercise, we discuss multiple classification models and datasets.

Here are models that we have trained and/or tested:

| Model | Spam Data
(Text) | Idealized Cars
(linear) | Cars
(non-linear) |
|--------------------------|---------------------|----------------------------|----------------------|
| Bernoulli NB
(prob) | [x] | | |
| Multinomial NB
(prob) | [x] | | |
| Single Perceptron | | [x] | [x] Not fit |
| ML Perceptron
(NN) | | | [x] |

Please consider some additional tasks: 1. Apply the probabilistic models to the cars data. 2. Apply the Perceptron – single or multi-layer – to the spam (text) data. 3. Evaluate and compare their performances.

You may also consider some other classification models available in **scikit learn**: <https://scikit-learn.org/stable/modules/classes.html#module-sklearn.svm>

For example: + Support Vector Machines (SVM): `sklearn.svm.LinearSVC`, `sklearn.svm.SVC` + Decision Tree: `sklearn.tree.DecisionTreeClassifier`
+ Lazy Learning: `sklearn.neighbors.KNeighborsClassifier`

Practice 5: Information, Entropy, and Divergence

Theory connection: Volume I, Chapter 5

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Shannon Entropy

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Properties of Shannon Entropy

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 5: Information, Entropy, and Divergence

Entropy in Thermodynamics?

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Applications

i Practice development scaffold

Create an activity that makes **Applications** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Limits and Other Measures

i Practice development scaffold

Create an activity that makes **Limits and Other Measures** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Relative Entropy

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Jensen-Shannon Divergence

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

IDF and KL Divergence

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Least Information Theory (LIT)

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 6: Data Preparation and Feature Engineering

Theory connection: Volume I, Chapter 6

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Data Quality and Provenance

Recovered starting point: Python Data Preprocessing and Outliers versus the Normal.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Missing, Noisy, and Inconsistent Data

Recovered starting point: Python Data Preprocessing and Outliers versus the Normal.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Scaling, Normalization, and Transformation

Recovered starting point: Python Data Preprocessing.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Encoding Categorical and Structured Variables

Recovered starting point: Python Data Preprocessing.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Feature Selection and Dimensionality Reduction

Recovered starting point: Python Data Preprocessing.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Leakage and Reproducible Pipelines

Recovered starting point: Python Data Preprocessing.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Python Data Preprocessing

Recovered activity

i Historical source and execution note

This activity was recovered from `python/python_data_preprocessing.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Data Quality

- Today's real-world data likely origins from multiple, heterogeneous sources
- Therefore, we are very likely to handle low-quality data
- It is important to properly address data quality issues
- Even simple descriptive analysis can be applied such that:
 - Sort all values in ascending or descending order
 1. If all values are identical: no information. Ignore this variable!
 2. If some values occur abnormally with very high frequency (e.g., “Alabama” in 20% cases for state entry: need to

examine whether it is caused by any interesting pattern (e.g., football fan survey) or error (e.g., default value)

- All techniques (e.g., data cleaning, integration, reduction, transformation, and discretization) related to data quality need to be properly addressed

In this exercise, we are going to utilize a dataset distributed as a part of the Yelp Dataset Challenge (<https://www.yelp.com/dataset>). The ultimate goal is to create a classifier to predict the usefulness of each review by using various attributes. For instance, the count of words can be a good indicator of the usefulness of reviews because longer reviews can deliver more information than shorter reviews can do. The sentiment of reviews might be important because too positive/negative reviews might not be objective. Metrics related to social networks such as degree, betweenness, and eigenvector might be important by representing the social status of reviewers. To achieve the ultimate goal, preprocessing data is the first step. Let's open the data. There are 1,000 tuples and 26 attributes. The names of attributes are self-explanatory.

```
## Imports all necessary modules
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from numpy import linalg
from scipy import stats

## This loads the csv file from disk
yelp_data = pd.read_csv(
    filepath_or_buffer = "../data/Yelp_Usefulness_Practice.csv", sep = ",",

print(yelp_data.head(20))
## print the dimension of the data
print(yelp_data.shape)
```


Applying descriptive analysis

As explained earlier, we can apply a descriptive analysis to understand the distribution of data. We can count the number of unique values as well.

```
## Set the maximum number of columns as "None" to display all columns
pd.set_option('display.max_columns', None)

## Pandas' describe method can provide basic statistics for numeric variables at default
print(yelp_data.describe())

## In the above, only 24 attributes got results.
## We need to select "only" categorical attributes to get descriptive statistics for categorical attributes
print(yelp_data[["review_id", "class"]].describe())
```

For the categorical attributes, we found that there are no cases such that all values are identical or some values occur abnormally with very high frequency.

However, in the case of numeric attributes, we found that there are three missing values in the “liking” attribute. The total count of tuples is 997, although all other attributes’ count is 1,000. We can also check missing values by using Pandas’ `isna` method. Also, “dislike” attribute has all zero values. That said, we need to drop this attribute by using Pandas’ `drop` method. However, we will keep this for now.

```
## Show the number of data per attributes that has missing values
print(yelp_data.isna().sum())
```

Python Data Preprocessing

```
## check the missing values across tuples of the "liking" attribute
print(yelp_data["liking"].isna())
```

Now, let's create a box plot to see the distribution of the "liking" attribute. This will help us to determine how to handle the missing data.

```
## Initialize a 6 x 6in figure
fig = plt.figure(figsize = (6, 6))

## Make the box plot
plt.boxplot(
    ## We need to drop tuples that include missing values
    yelp_data["liking"][~yelp_data["liking"].isna()], labels = ["Liking"]
)

## Adjust the tick and label font size
plt.tick_params(labelsize = 15)

## Set the title
plt.title("The normalized number of words related to 'liking' sentiment", font
```

It seems that most of the values are zeros except for one tuple. It is skewed. Therefore, it might be better to use the median central tendency.

```
## We should exclude missing tuples to get median
median = np.median(yelp_data["liking"][~yelp_data["liking"].isna()])
print ("The median is: ", median)

## Replace missing values with median
yelp_data["liking"].fillna(median, inplace=True)

## Check whether the missing value were filled
print(yelp_data.isna().sum())
```

Outlier Detection

We can use Z-score and IQR score to detect outliers. There are other methods, too.

```
## The first attribute (review_id) and the last attribute (class) are categorical
## Therefore, we cannot calculate z-score
## The "dislike" attribute has 0 mean and sd. Therefore, we cannot calculate z score.
## We create indices that exclude review_id, dislike, and class
positions = list(range(1,13))
positions.extend(list(range(14,25)))

z = np.abs(stats.zscore(yelp_data.iloc[:, positions]))
print(z)
```

The above numbers represent z scores of our data. It is difficult to say which data point is an outlier. Let's try and define a threshold to identify an outlier. To be conservative, we will use 3 as a threshold. We can exclude 0.3% of data.

```
threshold = 3
print(np.where(z > 3))
```

The first array contains the list of row numbers, and the second array corresponding column numbers that have a Z-score higher than 3, which mean they are: `z[7][1]`, `z[18][17]`, `z[18][23]`, and so on.

We may want to remove tuples in which the most attributes are outliers. We can remove all tuples regardless of the number of attributes that are outliers. For this exercise, we adopt a very conservative approach. Therefore, we count the frequency of tuples that includes outliers.

```
unique_elements, counts_elements = np.unique(np.where(z > 3)[0], return_counts=True)
print(np.asarray((unique_elements, counts_elements)))
```

The first list represents the row number when the z score is larger than 3. The second list represents the frequency of the corresponding row. The row 69 and 262 have the largest number of outliers. Therefore, we want to remove these tuples from the dataset.

```
yelp_data = yelp_data.drop([ 69, 262])

print(yelp_data.shape)
```

As you can see, two tuples were dropped. I will leave outlier detection exercise using IQR to you.

Data Integration

One potential problem in data integration is redundancies in attributes and/or tuples. As for attributes, we can check redundancies by applying correlation analysis.

```
pcorr = yelp_data.corr(method='pearson')

## Set the maximum number of columns as "None" to display all columns
pd.set_option('display.max_columns', None)
pcorr
```

It is hard to see. We can use a heatmap to visualize the correlation coefficients.

```
plt.figure(figsize=(12,10))
sns.heatmap(pcorr, annot=True, cmap=plt.cm.Reds)
plt.show()
```

We found the strongest correlation (0.94) between degree and eigenvector. The degree and betweenness are also strongly correlated (0.83). These results represent redundancies in attributes. We can drop one or two of them. However, we will leave it to the feature subset selection method in this exercise.

Data Integration 2

One common challenge in data integration is attribute construction. New attributes need to be constructed from the given ones if existing attributes do not adequately reflect the target label/value. For instance, the “area” is a better indicator of housing prices than the “length” and “width” of a house. We observed that attributes related to sentiment are sparse. Most of them are zeros. We will create new attributes from the given ones. The attributes in the left of the arrow mark are existing ones, and the attributes in the right are new ones. + joy, love, affection, liking, enthusiasm -> positive_emotion + sadness, dislike, despair, horror, distress -> negative_emotion

```
yelp_data["positive_emotion"] = yelp_data["joy"] + yelp_data["love"] + yelp_data["affection"]
yelp_data["negative_emotion"] = yelp_data["sadness"] + yelp_data["dislike"] + yelp_data["despair"]

## Drop existing attributes
yelp_data = yelp_data.drop(["joy", "love", "affection", "liking", "enthusiasm", "sadness", "dislike", "despair"])

print(yelp_data.head())
```

Dimensionality Reduction + Normalization

As a method of data reduction, we can apply dimensionality reduction. We covered PCA and feature subset selection in the lecture. In this exercise, we will cover feature subset selection. We are also going to apply normalization before selecting a feature subset. We will implement the forward selection and bi-directional stepwise selection.

```
## Create feature matrix by dropping the review_id and label attribute
## Review_id is not going to be helpful to predict the usefulness of reviews
X = yelp_data.drop(["review_id", "class"], 1)

## Pre-processing. Sklearn takes integer as label
# ## Create target attribute
yelp_data[yelp_data['class'] == 'useful'] = 1
yelp_data[yelp_data['class'] == 'not_useful'] = 0

# ## Specify the data type. Before specifying, the type was unknown
y = yelp_data["class"].astype('int')

print(yelp_data["class"])
```

```
print(X.shape[1])

## import the necessary libraries
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from sklearn.naive_bayes import MultinomialNB
from sklearn import preprocessing

## Apply Min_Max Scaler
min_max_scaler = preprocessing.MinMaxScaler()
X_minmax = min_max_scaler.fit_transform(X)
X_minmax = pd.DataFrame(X_minmax)
```

```
## Sequential Forward Selection(sfs)
## Please refer to http://rasbt.github.io/mlxtend/user\_guide/feature\_selection/SequentialFea
## for the detailed arguments options
sfs = SFS(MultinomialNB(),
          k_features=(1,X_minmax.shape[1]),
          forward=True,
          floating=False,
          scoring = 'r2',
          cv = 10)

sfs.fit(X, y)
## Get the final set of features
sfs.k_feature_names_
```

```
from mlxtend.plotting import plot_sequential_feature_selection as plot_sfs
import matplotlib.pyplot as plt

## performance graph (y-axis: r-squared values)
fig = plot_sfs(sfs.get_metric_dict(), kind='std_dev')
plt.title('Sequential Forward Selection (w. StdErr)')
plt.grid()
plt.show()
```

Thirteen attributes were selected. Let's try bi-directional stepwise selection (combination of forward and backward selection) and compare results.

```
## Bi-directional stepwise selection
## Please note that floating option was changed to "True"
sfs1 = SFS(MultinomialNB(),
            k_features=(1,X_minmax.shape[1]),
            forward=True,
            floating=True,
```

```
        scoring = 'r2',
        cv = 10)

sfs1.fit(X, y)
# Get the final set of features
sfs1.k_feature_names_

## performance graph (y-axis: r-squared values)
fig1 = plot_sfs(sfs1.get_metric_dict(), kind='std_dev')
plt.title('Sequential Bi-directional Stepwise Selection (w. StdErr)')
plt.grid()
plt.show()
```

Now, we have seven attributes ('review_stars', 'word_count', 'correct_spell_ratio', 'price_included', 'eigenvector', 'positive_emotion', 'negative_emotion'). We have a reduced set of attributes. The performance from both selection methods seems to be not very different. Potentially, using a small set will be better if the performance is not very different? However, it is still early to conclude. We did not separate data into training and testing sets. Also, what is the goal of your classifier? Is it effectiveness or efficiency? Usually, we need to sacrifice one to achieve another. In other words, effectiveness increases when efficiency decreases, and vice versa. We will cover these topics later.

Outliers versus the Normal

Recovered activity

Historical source and execution note

This activity was recovered from `demo/demo_outliers.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

```
%matplotlib inline
import math
import matplotlib.pyplot as plt
from matplotlib import animation, rc
import numpy as np
import pandas as pd
import seaborn as sns
from numpy import linalg
from scipy import stats
from IPython.display import HTML

plt.rcParamsdefaults()
# plt.xkcd(scale=1, length=300, randomness=5)
```

A. A NORMAL Spender Model

Mean and deviation

Suppose, on average you purchase $\mu = 20$ dollars with deviation $\sigma = 10$:

```
mu, sigma = 20, 10
```

Here we set the mean μ and standard deviation σ to 20 and 10 respectively.

Normal Distribution (Model)

Let's simulate 100 purchases you *may* make following a **normal** (Gaussian) distribution:

```
purchases = np.random.normal(mu, sigma, 100)
```

We use the NumPy's normal distribution model to generate the 100 purchase amounts, just like what you could have done.

Now we show the distribution of your data using a histogram:

```
count, bins, ignored = plt.hist(purchases, 20, density=True, color=(0.1, 0.1
```

Normal Density Distribution

Compare to the normal distribution density function:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/2\sigma^2} \quad (17.5)$$

B. Outliers in Belgium International Phone Calls

Now that we assume the data follow a normal distribution – and it does – the data should follow the normal distribution curve. So let's compare the histogram to the NORMAL model that generates the data.

```
count, bins, ignored = plt.hist(purchases, 20, density=True, color=(0.1, 0.1, 0.1, 0.1), edgecolor='b')
plt.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) *
         np.exp( - (bins - mu)**2 / (2 * sigma**2)),
         linewidth=2, color='b')
plt.show()
```

The blue curve depicts the MODEL, i.e. the normal distribution where the data come from. But it is easy to tell that the actual data, shown as the histogram, do not form the exact bell shape of the model.

The actual spending, for example, may tend to have more purchases with a greater amount, than what the model suggests. This is because the model is probabilistic and comes with a degree of randomness. That leads to the variance in the data and unpredictability of individual data instances.

And this is the **noise**, which may appear to be outliers but, in fact, comes from the one same model that generates all data here.

In terms of the distribution here, it is very unlikely that you will spend more than 50 dollar on an order:

```
from scipy.stats import norm
normal_spender = norm(mu, sigma)
prob = 1-normal_spender.cdf(50)
print("How likely is an order of over 50 dollars? {:.2f}".format(prob))
```

B. Outliers in Belgium International Phone Calls

The number of international phone calls (millions) from Belgium during 1950 - 1973:

Outliers versus the Normal

Data

```
phones = pd.read_csv("../data/phones.csv")
phones[['year', 'calls']].head(10)
```

Statistics

```
phones[['year', 'calls']].describe()
```

Distribution

```
sns.distplot(phones["calls"], color="gray")
```

Potential Correlation

```
phones[['year', 'calls']].corr(method="pearson")
```

Linear Regression

LSM, least Squares from the mean: + Least **mean** squares to be minimized
+ Tries to fit all values

If we conduct a linear regression based on the classic method of least squares, which is to minimize the sum of squared errors from the **mean**, we obtain:

B. Outliers in Belgium International Phone Calls

```
from sklearn.linear_model import LinearRegression

x = phones['year'].values.reshape(-1,1)
y = phones['calls'].values.reshape(-1,1)

def lsm_mean():
    lsm = LinearRegression()
    lsm.fit(x, y)
    # print(lsm.intercept_, lsm.coef_)
    global yp
    yp = lsm.predict(x)

    # plt.scatter(x, y, color='gray')
    plt.plot(x, yp, color='red', linewidth=2)
    plt.xlabel("Year")
    plt.ylabel("# Calls (tens of millions)")
    plt.show()
```

```
lsm_mean()
```

The linear regression model shows the positive relation between year and the number of phone calls. Over time, there is an increasing number of calls from Belgium, which makes sense.

Again, the regression model here is to minimize the sum of least squared errors, from the mean. That is, the model try to fit the regression line along the mean or average over the years.

Robust Linear Regression

LAD, least absolute deviation from the median: + A quantile regression, with $q=0.5$ (median) + Robust regression models + **Median**, instead of mean, deviation + **Least affected by extreme values**, e.g. outliers

Outliers versus the Normal

An alternative regression technique is referred to as the Quantile Regression, which is part of a family of techniques for robust regression. We know that the quantile at 0.5 or 50% is the middle of all values and is the median. So a quantile regression with $q=0.5$ is “median regression,” based on least absolute deviation.

In this case, when fitting the model, values and errors in the middle have a greater impact on the regression model. Extreme values, which are at the two ends of a distribution, will hardly have an impact on the median error estimate. So outliers will be less likely to impact the results.

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

def lad_median():
    # rlm = sm.RLM(x, y, M=sm.robust.norms.HuberT())
    rlm = smf.quantreg('calls ~ year', phones)
    rlm_res = rlm.fit(q=0.5)
    # print(rlm_res.summary())
    global yp2
    yp2 = rlm_res.params[0] + x * rlm_res.params[1]

    # plt.scatter(x, y, color='gray')
    plt.xlabel("Year")
    plt.ylabel("# Calls (millions)")
    plt.plot(x, yp, color='red', linewidth=2, label="Least Square (Mean)")
    plt.plot(x, yp2, color='blue', linewidth=2, label="Least Deviation (Median)")
    plt.legend()
    plt.show()
```

```
lad_median()
```

Now after we perform the “median regression”, we can compare its regression line (the blue one) to the regression based on least mean squares (the

B. Outliers in Belgium International Phone Calls

red line). The two lines are very different. In particular, the least mean square model has a much steeper regression line. What happened?

```
def show_data():
    plt.scatter(x, y, color='gray', label="Data")
    plt.xlabel("Year")
    plt.ylabel("# Calls (millions)")
    plt.plot(x, yp, color='red', linewidth=2, label="Least Square (Mean)")
    plt.plot(x, yp2, color='blue', linewidth=2, label="Least Deviation (Median)")
    plt.legend()
    plt.show()
```

```
show_data()
```

Now if we plot the actual data, the figure gives us some clues about what makes the difference.

As shown in the data, there are anomalies in the data, specifically during years 1964 - 1969, where the numbers of calls appear to be unusually greater.

The model based on least mean squares, the red line, is fitting the regression line in terms of mean squared errors. The mean, as we know, is affected by all values. So essentially, the least mean square model is trying to “please” every single data point and the group of outliers will have a major impact on the final regression line, which goes in between the normal data points and outliers.

The “median regression” (so to speak), the blue line, is immune to the outliers because it relies on the median deviation. As you can see, it fits the normal data points very well.

But what are the outliers? What do they represent? Where do they come from?

Outliers versus the Normal

```
def show_data_groups():
    phones1 = phones[phones["calls"]<100]
    plt.scatter(phones1["year"], phones1["calls"], color='black', label="Normal")
    phones2 = phones[phones["calls"]>=100]
    plt.scatter(phones2["year"], phones2["calls"], color='red', label="Outliers")
    plt.xlabel("Year")
    plt.ylabel("# Calls (millions)")
    plt.legend()
    plt.show()
```

```
show_data_groups()
```

In our definition of outliers, we declare that they must have come from a process or model different from the one that generates the normal data. So if the normal data points come from the number of phone calls Belgium people make? What the outliers?

It turns out that the outliers are still about phone calls from people in Belgium. However, they were mistakenly recorded as the number of minutes, instead of counts of individual calls. So while the black normal data points come from a “counting” process, and the red outliers come from the time log. They are indeed two separate models.

C. Parametric Detection: What is a NORMAL temperature?

As we discussed, outliers come from a different process or model that generates normal data. These are generative models or stochastic (random) processes hidden behind the data. The idea here is that if we can find out the model that produce normal data, we will then be able to estimate the likelihood or probability of potential outliers.

C. Parametric Detection: What is a NORMAL temperature?

1. **Parametric** methods: data are generated by a distribution **model** with parameters.
2. **Nonparametric** methods: do not assume a model but try to determine it **from data**.

Example, temperature data in °C:

```
x = [24.0, 28.9, 28.9, 29.0, 29.1, 29.1, 29.2, 29.2, 29.3, 29.4]
```

```

<div style="position:absolute; top:50%; left:50%; transform:translate(-
50%, -50%); font-size:48pt;">
  <font style="color:blue; background-color: white;">0&deg;C</font>
</div>
```

Given the above sample data and the NORMAL assumption, how likely are the following? 1. A temperature **below 0 °C**? 2. A temperature **above 30 °C**?

Are the values normal or outliers?

1. Estimate Mean and Deviation from Sample

Given normal distribution, two parameters to be estimated: 1. Mean: $\hat{\mu} = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ 2. Variance: $\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$

```
import math
import numpy as np
mu = np.mean(x)
std = np.std(x)
```

Outliers versus the Normal

```
print("Mean = {:.2f}".format(mu))  
print("Deviation = {:.2f}".format(std))
```

2. Build the Normal Model with Estimated Parameters

```
from scipy.stats import norm  
model = norm(mu, std)
```

3. Compute the Likelihood of Temperature Ranges

```
t1,t2 = 0, 30  
print("How likely is a temperature below 0 C?", model.cdf(t1))  
print("How likely is a temperature above 30 C?", 1-model.cdf(t2))
```

Given the observed temperature data, $t_1 = 0$ is extremely unlikely to be generated by the normal distribution model here and is most likely an outlier.

D. NORMAL Age in Context

Some data do not appear to be outlier, until they are viewed in the context. Let's look at the normal **age** of a working person in the context of demographical groups.

1. Census Income Data

The data are based on the 1994 Census of the U.S.

<https://archive.ics.uci.edu/ml/datasets/Adult>

The dataset comes with attributes including age, education, sex, and income levels.

```
people = pd.read_csv("../data/adult.data", header=None, skipinitialspace=True)
people.columns = ["age", "workclass", "fnlwgt", "education",
                  "education_num", "marital_status", "occupation",
                  "relationship", "race", "sex", "capital_gain",
                  "capital_loss", "hours_per_week", "native_country",
                  "income"]
people.head()
```

```
people.shape
```

```
sns.distplot(people['age'], bins=20)
```

What is the chance of a working 18-year old or younger? Assume a normal distribution.

```
mu, std = norm.fit(people['age'])
normal_age = norm(mu, std)
p = normal_age.cdf(18)
print("{:.2f}".format(p))
```

The probability is about 7%. It is less common but not necessarily abnormal.

2. Working Age of Male vs. Female

Age distributions for **male vs. female**:

```
counts = people.groupby('sex').size()
print(counts)
```

```
male = people[(people['sex'] == 'Male')]
female = people[(people['sex'] == 'Female')]
print(male.shape, female.shape)
```

```
# male_age = male['age']
# female_age = female['age']
# male_age.hist(histtype='stepfilled', color='blue', alpha=0.5, bins=20)
# female_age.hist(histtype='stepfilled', color="red", alpha=0.5, bins=20)
sns.distplot(male['age'], color='blue', bins=20)
sns.distplot(female['age'], color='red', bins=20)
```

```
# female_age.hist(density=True, histtype='stepfilled', color="red", alpha=0.5)
# male_age.hist(density=True, histtype='stepfilled', color=sns.desaturate("blue", 0.5))
```

3. NORMAL Age with a Graduate Degree

The distributions of income data in the **education** groups.

```
counts = people.groupby('education').size()
print(counts)
```

Consider people with a Masters or Doctorate degree:

```
graduate = people[(people['education'] == 'Masters') | (people['education'] == 'Doctorate')]  
sns.distplot(graduate['age'], bins=20)
```

Again, we can estimate the normal distribution based on the sample with a graduate degree:

```
mu, std = norm.fit(graduate['age'])  
print(mu, std)  
normal_graduate_age = norm(mu, std)
```

How likely is `age=18` or younger in the population with a graduate degree?

```
p = normal_graduate_age.cdf(18)  
print("{:.3f}".format(p))
```

The chance is $< 1\%$. It is quite abnormal and suggests likely outliers in the context of having a `graduate` degree.

References

- Han, Jiawei and Kamber, Micheline (2011). Data Mining: Concepts and Techniques (3rd Edition). Morgan Kaufmann Publishers, San Francisco.
- Ian H. Witten, Eibe Frank, Mark A. Hall, Christopher J. Pal (2017). Data Mining: Practical Machine Learning Tools and Techniques (Morgan Kaufmann Series in Data Management Systems) 4th Edition.
- Laura Igual and Santi Seguí (2017). Introduction to Data Science: A Python Approach to Concepts, Techniques, and Applications. Springer.

Practice 7: Classification

Theory connection: Volume I, Chapter 7

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Lazy Learning

i Practice development scaffold

Create an activity that makes **Lazy Learning** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

K Nearest Neighbors (kNN)

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 7: Classification

Other Distance Metrics

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Linear Classifiers

Recovered starting point: Probability and Linearity in Data and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Between Centroids

Recovered starting point: Data, Vectors, and Cosine Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Support Vector Machines

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Perceptron: Toward a Neural Network

Recovered starting point: Probability and Linearity in Data and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Non-Linear Models

Recovered starting point: Probability and Linearity in Data and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Kernel SVM

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Multi-Layer Neural Networks

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 8: Multiclass and Structured Decisions

Theory connection: Volume I, Chapter 8

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

From Binary to Multiple Classes

Recovered starting point: Classification Patterns and Evaluation Assignment and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

One-vs-Rest and One-vs-One Strategies

Recovered starting point: Classification Patterns and Evaluation Assignment and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Scores, Probabilities, and Softmax

Recovered starting point: Classification Patterns and Evaluation Assignment and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Decision Trees and Rule-Based Decisions

Recovered starting point: Classification Patterns and Evaluation Assignment and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Multilabel and Ordinal Outcomes

Recovered starting point: Classification Patterns and Evaluation Assignment and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Error Analysis and Calibration

Recovered starting point: Classification Patterns and Evaluation Assignment and Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 9: Numeric Prediction and Regression

Theory connection: Volume I, Chapter 9

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Prediction with Continuous Outcomes

Recovered starting point: Outliers versus the Normal and Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Linear Regression

Recovered starting point: Outliers versus the Normal and Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Loss Functions and Fitting

Recovered starting point: Outliers versus the Normal and Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Regularization

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Nonlinear Regression

Recovered starting point: Outliers versus the Normal and Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Uncertainty and Diagnostics

Recovered starting point: Outliers versus the Normal and Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Classification Patterns and Evaluation Assignment

Recovered activity

i Historical source and execution note

This activity was recovered from `problems/assignment_2_final.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Preparation

Please import all necessary packages and setup `%matplotlib` for inline plots in the report. Also, load the dataset, which you can download from here: `Yelp_Usefulness_Assignment2_1.csv`. There are several versions of the dataset, so please make sure that you load the correct version. Please refer to the Python Data Proprocessing for details about the dataset.

```
## Imports all necessary modules
import pandas as pd
import matplotlib.pyplot as plt
```

```
import numpy as np
import seaborn as sns
from numpy import linalg
from scipy import stats
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from mlxtend.plotting import plot SequentialFeatureSelector as plot_sfs
from sklearn.naive_bayes import MultinomialNB
from sklearn import preprocessing

## This loads the csv file from disk
yelp_data = pd.read_csv(
    filepath_or_buffer = "../data/Yelp_Usefulness_Assignment2_1.csv", sep = ','

print(yelp_data.head(20))
## print the dimension of the data
print(yelp_data.shape)
```

1. Data Cleaning

The data from the real world is not necessarily clean. Therefore, you need to clean your data. If it is not cleaned properly, it can have a significant impact on the analysis. You need to find three problems in the loaded data. It should be noted that there might be more than three problems. As far as you can find three major problems, you will meet the expectation. Examine the data by applying descriptive analysis and other measures. The data preprocessing exercise should include the basic code and procedures for this. Once you found those problems, please describe those problems and explain how you will handle the problems. You also need to provide rationale and pros/cons for your approach in detail.

1.1. The first problem (1 point)

1.1.1 What is the first problem you found? How did you find it?

Provide codes to find the problem

Provide your open-ended answer

1.1.2 How will you handle this problem? Please code your approach.

Provide your open-ended answer

Provide codes to handle the problem

1.1.3 What are pros/cons of your approach?

Provide your open-ended answer

1.2. The second problem (1 point)

1.2.1 What is the second problem you found? How did you find it?

Provide codes to find the problem

Provide your open-ended answer

1.2.2 How will you handle this problem?. Please code your approach.

Provide your open-ended answer

Classification Patterns and Evaluation Assignment

```
## Provide codes to handle the problem
```

1.2.3 What are pros/cons of your approach?

```
## Provide your open-ended answer
```

1.3. The third problem (1 point)

1.3.1 What is the third problem your found? How did you find it?

```
## Provide codes to find the problem
```

```
## Provide your open-ended answer
```

1.3.2 How will you handle the problems. Please code your approach.

```
## Provide your open-ended answer
```

```
## Provide codes to handle the problem
```

1.3.3 What are pros/cons of your approach?

```
## Provide your open-ended answer
```

2. Data Normalization + Reduction

In data mining, you need to scale data to the same range to avoid dependence on the choice of measurement units (e.g., lb vs. kg). You also want

2. Data Normalization + Reduction

to obtain a reduced representation of the dataset that is much smaller in volume but yet produces the same (or almost the same) analytical results.

In this assignment, you will explore different options for feature subset selection (e.g., metrics and methods) and normalization (e.g., z-transformation and max-min). After exploring all options, the goal is to create the best classifier. The performance will be measured by accuracy, the ratio of the number of correct predictions to the total number of input samples, considering the even distribution of the class. You are not allowed to explore any options other than feature subset selection and normalization. For instance, you can only use one machine learning algorithm (Logistic Regression). You cannot create a new attribute, and so on. In this way, you can know how feature subset selection and normalization would contribute to the modeling. It should be noted that we are not splitting the dataset into the training set and testing set, although the generalizability of the classifiers' performance can be tested only with the unseen data (i.e., testing data). We will learn about this topic later.

Please load the dataset (Yelp_Usefulness_Assignment2_2.csv). There are several versions of the dataset, so please make sure that you load the correct version.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

## This loads the csv file from disk
yelp_data = pd.read_csv(
    filepath_or_buffer = "../data/Yelp_Usefulness_Assignment2_2.csv", sep = ",", header=0 )

print(yelp_data.head(20))
```

Then, we are going to build a baseline model by using the Logistic Regression classifier. At the end of the following code, you can see the accuracy of this baseline classifier (0.752). After exploring various options for the

Classification Patterns and Evaluation Assignment

normalization and feature subset selection, you would want to beat this baseline model.

```
## Create feature matrix by dropping the review_id and label attribute
## Review_id is not going to be helpful to predict the usefulness of reviews
X = yelp_data.drop(["review_id", "class"], 1)

## Pre-processing. Sklearn takes integer as label
## Create target attribute
yelp_data[yelp_data['class'] == 'useful'] = 1
yelp_data[yelp_data['class'] == 'not_useful'] = 0

## Specify the data type. Before specifying, the type was unknown
y = yelp_data["class"].astype('int')

## Create a model
clf = LogisticRegression()
clf.fit(X, y)

## predict target class based on the trained model
predictions = clf.predict(X)

## Calculate the performance of the classifier
accuracy = accuracy_score(predictions, y)

print(accuracy)
```

Here is an example to scale data using z-scale.

```
## Apply z-transformation
z_scaler = preprocessing.StandardScaler()
X_scaled = z_scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns = X.columns)
```


2. Data Normalization + Reduction

Here is an example to scale data using max-min transformation.

```
## Apply Min_Max Scaler
min_max_scaler = preprocessing.MinMaxScaler()
X_minmax = min_max_scaler.fit_transform(X)
X_minmax = pd.DataFrame(X_minmax, columns = X.columns)
```

Here is an example of using forward feature selection with max_min transformation.

```
## import the necessary libraries
from mlxtend.feature_selection import SequentialFeatureSelector as SFS

min_max_scaler = preprocessing.MinMaxScaler()
X_minmax = min_max_scaler.fit_transform(X)
X_minmax = pd.DataFrame(X_minmax, columns = X.columns)

## Sequential Forward Selection(sfs)
sfs = SFS(LogisticRegression(),
          k_features=(1,X_minmax.shape[1]),
          forward=True,
          floating=False,
          scoring = 'accuracy',
          cv = 10)

sfs = sfs.fit(X_minmax, y)
## Get the final set of features
print(sfs.k_feature_names_)

X_selected = sfs.transform(X_minmax)

# Fit the estimator using the new feature subset
# and make a prediction on the test data
```

Classification Patterns and Evaluation Assignment

```
clf.fit(X_selected, y)
predictions = clf.predict(X_selected)

## Calculate the performance of the classifier
accuracy = accuracy_score(predictions, y)

print(accuracy)
```

Actually, the performance would be lower than the baseline classifier, if you use the forward feature selection with `max_min` transformation. Now, explore all possible options. For instance, you can set the “forward” argument of the `SequentialFeatureSelector` as “False” to use backward feature selection. If you set the “floating” argument as “True”, then you will use bi-directional stepwise feature selection. You can also change “scoring” and “cv” options.

Please refer to the following link for the detailed arguments options: http://rasbt.github.io/mlxtend/user_guide/feature_selection/SequentialFeatureSelector/#sequential-feature-selector

Enjoy all options, find the best classifier, and answer the following questions.

2.1. Performance (1 point)

What was the best performance you got? Please provide the code for your best classifier. The student that achieves the best accuracy will be given ONE bonus point towards his/her final grade and, of course, fame and glory.

```
## Provide the best accuracy
```

3. Association Rule Mining: Theory

```
## Provide codes to find the best classifier
```

2.2. Insight and Explanation (2 points)

Provide a couple of paragraphs description of what you tried, what worked, and what did not work. Describe the lesson you got by this exercise. Please be comprehensive to deliver what you have done. Perhaps, using graphs or tables will be helpful to find a meaningful pattern from your experiments.

```
## Provide your open-ended answer
```

3. Association Rule Mining: Theory

A typical example of association rule mining is market basket analysis. This process analyzes transaction data to find associations between the different items that customers place in their “shopping baskets”. The discovery of these associations can help retailers develop marketing strategies by understanding which items are frequently purchased together by customers. Also, online retailers can develop a recommendation system based on association rule mining.

You are going to find association rules from the following virtual transactional data.

Transaction ID	Items bought
1	Computer, Mouse
2	Computer, Tablet, Smart Phone, Smart Watch
3	Computer, Smart Watch, Game Console
4	Mouse, Game Console
5	Tablet, Smart Watch, Smart Phone
6	Smart Phone, Smart Watch

3.1. Rule Calculation

3.1.1 Calculation of support (1 point)

First of all, you need to calculate the first rule, support, which is the frequency of an itemset. You are going to use relative support, which is the fraction of transactions that contain X itemset (i.e., the probability that a transaction contains X itemset). If you are not familiar with this, please watch the lecture video or read the textbook (section 6.1). Please note that both minimum support and minimum confidence are 50% (i.e., 0.5).

First, you need to calculate support for frequent 1-itemsets that consist of 1 item. If each item does not pass the minimum support threshold, it does not need to be included for 2-itemsets candidates that consist of 2 items (e.g., computer and mouse) according to the Apriori property. Examine data values in the previous table and include the calculation process. If it takes too much to use the Jupyter Notebook to write the calculation process, you can write them on paper and turn them in as attached. Please create tables to list corresponding values:

Frequent 1-itemsets

Itemset	Count	Total # of Transactions	Support	Passing Minimum Support ?
Com-puter	3	6	0.5	Yes
Mouse	#	#	.xx	Yes / No
Smart Watch	#	#	.xx	Yes / No
Tablet	#	#	.xx	Yes / No

3. Association Rule Mining: Theory

Itemset	Count	Total # of Transactions	Support	Passing Minimum Support ?
Smart Phone	#	#	.xx	Yes / No
Game Console	#	#	.xx	Yes / No

Frequent 2-itemsets

Itemset	Count	Total # of Transactions	Support	Passing Minimum Support ?
Com-puter, Mouse	#	#	.xx	Yes / No
Com-puter, Smart Watch	#	#	.xx	Yes / No
...				

Frequent 3-itemsets

Classification Patterns and Evaluation Assignment

Itemset	Count	Total # of Transactions	Support	Passing Minimum Support ?
Com-puter, Tablet, Smart Phone	#	#	.xx	Yes / No
...				

3.1.2 Association Rules (1 point)

Itemsets that pass the minimum support can be used to find the association rule $X \Rightarrow Y$. Find all the rules $X \Rightarrow Y$ with minimum support and confidence. Please see the following equations:

$$support(A \Rightarrow B) = P(A \cup B)$$

$$confidence(A \Rightarrow B) = P(B|A) = \frac{support(A \cup B)}{support(A)}$$

Please calculate the support and confidence for each rule you found and include the calculation process. If it takes too much to use the Jupiter Notebook to write the calculation process, you can write them on paper and turn them in as attached. Please create a table to list corresponding values:

3. Association Rule Mining: Theory

Rule	support (A B)	confidence (A B)	Passing Minimum Support ?	Passing Minimum Confidence ?
Com- puter Smart Watch	.xx	.xx	Yes / No	Yes / No
...				

3.2. Pattern Evaluation Method

The support-Confidence framework is often limited because not all rules found strong by this framework are interesting. Therefore, we need to utilize other interestingness measures such as lift and chi-squared (χ^2). You are going to calculate those other interestingness measures from the table and compare their pros/cons.

	Smart Watch	Not Smart Watch
Smart Phone	500	350
Not Smart Phone	100	50

$$Lift(A, B) = \frac{Confidence(A \Rightarrow B)}{Support(C)} = \frac{Support(A \cup B)}{Support(A) \times Support(B)}$$

3.2.1 Calculation of interestingness measures (0.5 points)

Please calculate the lift and chi-squared (χ^2) for each rule and include the calculation process. If it takes too much to use the Jupyter Notebook to write the calculation process, you can write them on paper and turn them in as attached. Please create a table to list corresponding values:

Classification Patterns and Evaluation Assignment

Rule	support (A B)	confidence (A B)	Lift (A B)	chi-squared (χ^2)
Smart Watch Smart Phone	.xx	.xx	.xx	xx.xx
Not Smart Watch Smart Phone	.xx	.xx	.xx	xx.xx

3.2.2 Insight and Explanation (0.5 points)

If you calculated correctly, each measure might tell you different stories. Let's assume that both minimum support and minimum confidence are 35% (i.e., .35). What do these interestingness measures tell you? Please explain what each measure means. In other words, can you tell whether there is a strong association or not? What are the pros/cons of each measure, particularly related to this example? How are you going to use these measures in the future according to what you learned?

Provide your open-ended answer

4. Association Rule Mining: Practice

You will practice with one of the advanced pattern mining topics, quantitative association rules. So far, you have practiced with categorical variables and calculate rules based on the contingency table. However, relational and data warehouse data often involve quantitative attributes and measures. For instance, in the Yelp dataset, most attributes are not categorical

4. Association Rule Mining: Practice

variables but numeric variables. To apply association rule mining, you can convert those attributes to binary attributes by using either median or mean split. This is not an ideal approach because it will lose information, but good enough for illustration. In this assignment, you are going to use the mean split. Based on this discretization, you will find association rules and analyze those rules. Perhaps, these association rules can help you find discriminative attributes to predict the usefulness of reviews. In other words, you can use the association rule mining for feature selection. You will need to use evaluation metrics and your own (subjective) judgments to find meaningful associations.

4.1. Discretization (1 point)

Now you need to create a binary representation of attributes. We will need to include “class” attribute, but drop “review_id” attribute.

Here is an example of converting textual representation to numeric representation. The library (MLXtend) you will use 1/0 representation for binary attributes.

```
yelp_data['class'][yelp_data['class'] == 'useful'] = 1
yelp_data['class'][yelp_data['class'] == 'not_useful'] = 0
yelp_data["class"].astype('int')
```

Here is an example for discretization.

```
yelp_data["degree"] = np.where(yelp_data["degree"] >= np.mean(yelp_data["degree"]),1,0)
yelp_data["degree"].astype('int')
```

You need to iteratively apply the above transformation for all attributes.

Please load the dataset (Yelp_Usefulness_Assignment2_2.csv). As noted, there are several versions of the dataset, so please make sure that you load

Classification Patterns and Evaluation Assignment

the correct version. Convert the data by using your own code and show the first 20 lines.

```
## This loads the csv file from disk
yelp_data = pd.read_csv(
    filepath_or_buffer = "../data/Yelp_Usefulness_Assignment2_2.csv", sep = ','

print(yelp_data.head(20))
print(yelp_data.shape)
```

```
## Provide codes
```

4.2. Implementation (1 point)

Now you need to create association rules between attributes. Please refer to the Association Rule Mining exercise for more details. You need to change minimum support and/or other parameters to filter out less interesting rules.

Please provide the code for your association rules. If you were not able to transform the data in the previous step, you can load the preprocessed data (Yelp_Usefulness_Assignment2_3.csv). Although you use this preprocessed data, any points will not be deducted.

```
## Provide codes
```

4.3. Insight and Explanation (1 point)

Provide a couple of paragraphs' description of what rules you have found. Did you find any interesting rules? Pick one interesting rule and explain how it is interesting and how you used evaluation measures. Pick one not-interesting rule and explain how it is not interesting and how evaluation

4. Association Rule Mining: Practice

measures overate the rule. Describe the lesson you got by this exercise. Please be comprehensive to deliver what you learned. Perhaps, using graphs or tables will be helpful in finding a meaningful pattern from your experiments. Also, you can compare association rules with the attributes selected from Section 2.2.

Provide your open-ended answer

Practicum: From Pixels to CNN Features

Theory connection: Convolutional Neural Networks: Learning Spatial Structure

This practicum tells one continuous story. You will first solve MNIST with a multilayer perceptron (MLP), stop to make convolution visible, solve the same task with a convolutional neural network (CNN), and finally inspect how kernels activate on handwritten 1s and 7s.

Download the complete Jupyter notebook. It contains the runnable code and representative executed output. The web and print book retain a concise set of results so that the explanation remains readable without requiring TensorFlow during rendering.

To reproduce the authoring environment from the repository root:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r practice/requirements-cnn.txt
jupyter lab practice/notebooks/cnn-mnist-practicum.ipynb
```

The `.venv` directory is intentionally ignored by Git. A standard Google Colab runtime is an easier alternative when local TensorFlow installation is inconvenient.

Learning Question and Evidence Rules

What changes when a classifier preserves spatial neighborhoods and reuses local detectors before making a class prediction?

The comparison controls the data split, scaling, optimizer, batch size, early-stopping rule, and approximate parameter budget. The architecture is the main planned difference. The validation set guides stopping; the test set is evaluated only after each model is fixed.

By the end, you should be able to:

- explain what flattening preserves and what it no longer makes explicit;
- calculate and visualize a kernel response;
- distinguish a kernel, a feature map, and a class prediction;
- read training and validation loss as evidence about fit; and
- make a cautious, evidence-based MLP-versus-CNN comparison.

i Representative results, not universal constants

The displayed results come from one seeded full-data run using TensorFlow 2.20 and Keras 3.15. Small numerical differences are normal across software and hardware. Your explanations should survive those differences.

Setup: One Dataset, Three Roles

MNIST contains 60,000 training images and 10,000 test images. We shuffle the original training data once, use 54,000 examples for fitting, and reserve 6,000 for validation. Each image is a 28×28 grayscale array.

Setup: One Dataset, Three Roles

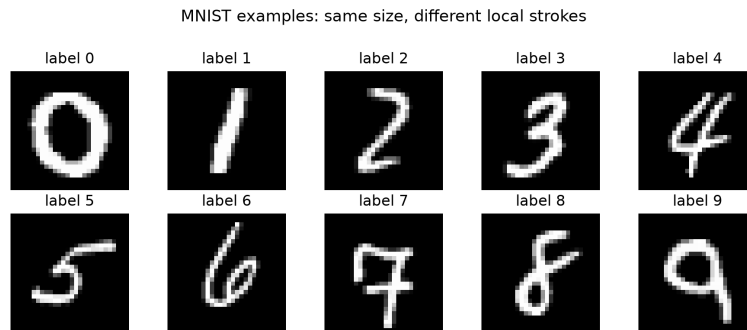


Figure 17.3.: One example from each MNIST class. The task is simple enough to train quickly, but the handwriting contains varying strokes, slants, and loops.

```
import time
import numpy as np
import matplotlib.pyplot as plt
import keras
import tensorflow as tf
from keras import layers

SEED = 42
BATCH_SIZE = 128
MAX_EPOCHS = 12
keras.utils.set_random_seed(SEED)
tf.config.experimental.enable_op_determinism()

(x_train_raw, y_train_raw), (x_test_raw, y_test) = (
    keras.datasets.mnist.load_data()
)

# Reproduce the same 54,000/6,000 fit/validation split.
```

```
order = np.random.default_rng(SEED).permutation(len(x_train_raw))
x_train_raw, y_train_raw = x_train_raw[order], y_train_raw[order]
x_fit_raw, x_val_raw = x_train_raw[:54000], x_train_raw[54000:]
y_fit, y_val = y_train_raw[:54000], y_train_raw[54000:]

def scale(images):
    return images.astype("float32") / 255.0

fig, axes = plt.subplots(2, 5, figsize=(8, 3.5))
for digit, axis in enumerate(axes.flat):
    index = np.flatnonzero(y_fit == digit)[0]
    axis.imshow(x_fit_raw[index], cmap="gray")
    axis.set_title(f"label {digit}")
    axis.axis("off")
plt.tight_layout()
```

Check before continuing. Which set may influence the stopping epoch? Why should neither the validation set nor test set be passed to `model.fit` as training examples?

Part I: Establish an MLP Baseline

Flatten the Grid

The MLP receives a vector, not an image grid:

```
x_fit_mlp = scale(x_fit_raw).reshape(-1, 784)
x_val_mlp = scale(x_val_raw).reshape(-1, 784)
x_test_mlp = scale(x_test_raw).reshape(-1, 784)

assert x_fit_mlp.shape == (54000, 784)
```



```
assert x_val_mlp.shape == (6000, 784)
assert x_test_mlp.shape == (10000, 784)
```

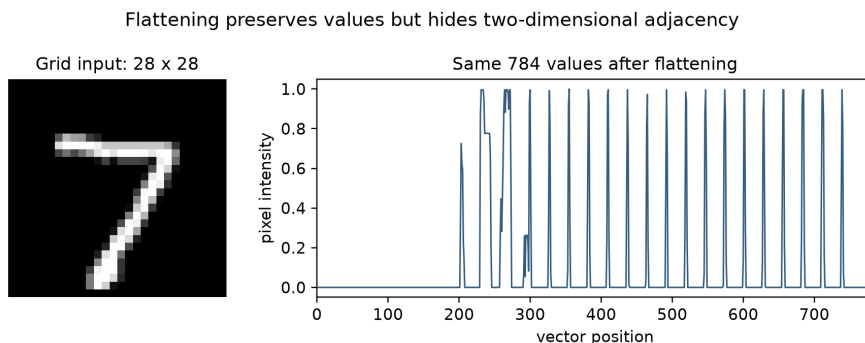


Figure 17.4.: A digit 7 represented as a two-dimensional image and as the same 784 values in row-major vector order.

Flattening neither deletes nor randomizes a pixel. It changes how the model is allowed to use the values: two neighboring pixels no longer share an explicit two-dimensional neighborhood, and the same stroke in a new location reaches different dense-layer weights.

Explain. If pixel (10, 10) and pixel (10, 11) are adjacent in the grid, where do they appear in the vector? Why can the MLP still learn from them, even though it does not reuse one local detector across locations?

Build and Train the MLP

The model is intentionally ordinary and compact:

```
def build_mlp():
    keras.utils.set_random_seed(SEED)
    model = keras.Sequential([
```

```
        keras.Input(shape=(784,)),
        layers.Dense(128, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ], name="mnist_mlp")
    model.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

mlp_model = build_mlp()
assert mlp_model.count_params() == 109386
mlp_model.summary()
```

Use validation loss, not training accuracy, to decide when to stop:

```
def fit_model(model, x_fit, x_val):
    stop = keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=2,
        restore_best_weights=True,
    )
    start = time.perf_counter()
    history = model.fit(
        x_fit,
        y_fit,
        validation_data=(x_val, y_val),
        batch_size=BATCH_SIZE,
        epochs=MAX_EPOCHS,
        callbacks=[stop],
        verbose=2,
```

```
)
    return history.history, time.perf_counter() - start

mlp_history, mlp_seconds = fit_model(
    mlp_model, x_fit_mlp, x_val_mlp
)
mlp_loss, mlp_accuracy = mlp_model.evaluate(
    x_test_mlp, y_test, verbose=0
)
mlp_pred = mlp_model.predict(x_test_mlp, verbose=0).argmax(axis=1)
```

Record the parameter count, stopping epoch, lowest validation loss, test loss, test accuracy, number of errors, and training time. Do not tune the MLP after looking at the CNN's test results; that would make the comparison asymmetric.

Part II: Make Convolution Visible

One Transparent Operation

A kernel produces one number per local window: multiply matching entries, add the products, and then move the same weights to the next location. The function below implements valid two-dimensional cross-correlation for one channel:

```
def cross_correlate2d(image, kernel, stride=1, padding=0):
    image = np.pad(np.asarray(image, dtype=float), padding)
    kernel = np.asarray(kernel, dtype=float)
    kh, kw = kernel.shape
    out_h = (image.shape[0] - kh) // stride + 1
    out_w = (image.shape[1] - kw) // stride + 1
```

```
output = np.empty((out_h, out_w))

for row in range(out_h):
    for col in range(out_w):
        r, c = row * stride, col * stride
        window = image[r:r + kh, c:c + kw]
        output[row, col] = np.sum(window * kernel)
return output
```

Test the worked example from the theory chapter:

```
small_image = np.array([
    [1, 1, 0, 0],
    [1, 1, 0, 0],
    [1, 1, 0, 0],
    [1, 1, 0, 0],
], dtype=float)

boundary_kernel = np.array([[1, -1], [1, -1]], dtype=float)
feature_map = cross_correlate2d(small_image, boundary_kernel)

expected = np.array([[0, 2, 0], [0, 2, 0], [0, 2, 0]])
np.testing.assert_array_equal(feature_map, expected)
feature_map
```

Predict before running. What changes when `stride=2`? What padding keeps a 28×28 image at the same spatial size with a 3×3 kernel and stride one? Verify both answers with array shapes.

Kernels Produce Evidence, Not Labels

The next comparison uses three hand-designed 3×3 kernels. After cross-correlation, ReLU sets negative responses to zero. Bright locations show

where a local window agrees with a kernel.

```
kernels = {
    "vertical": np.array([
        [-1, 2, -1], [-1, 2, -1], [-1, 2, -1]
    ]),
    "horizontal": np.array([
        [-1, -1, -1], [2, 2, 2], [-1, -1, -1]
    ]),
    "diagonal": np.array([
        [-1, -1, 2], [-1, 2, -1], [2, -1, -1]
    ]),
}

def relu(response):
    return np.maximum(response, 0)

indices = [np.flatnonzero(y_test == digit)[0] for digit in (1, 7)]
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
for row, (digit, index) in enumerate(zip((1, 7), indices)):
    image = scale(x_test_raw[index])
    axes[row, 0].imshow(image, cmap="gray")
    axes[row, 0].set_ylabel(f"digit {digit}")
    axes[row, 0].axis("off")
    for col, (name, kernel) in enumerate(kernels.items(), start=1):
        response = relu(cross_correlate2d(image, kernel))
        axes[row, col].imshow(response, cmap="magma")
        axes[row, col].set_title(name if row == 0 else "")
        axes[row, col].axis("off")
plt.tight_layout()
```

The vertical and diagonal kernels respond to parts of *both* digits. The horizontal kernel responds strongly to the top bar of this 7, but it also finds

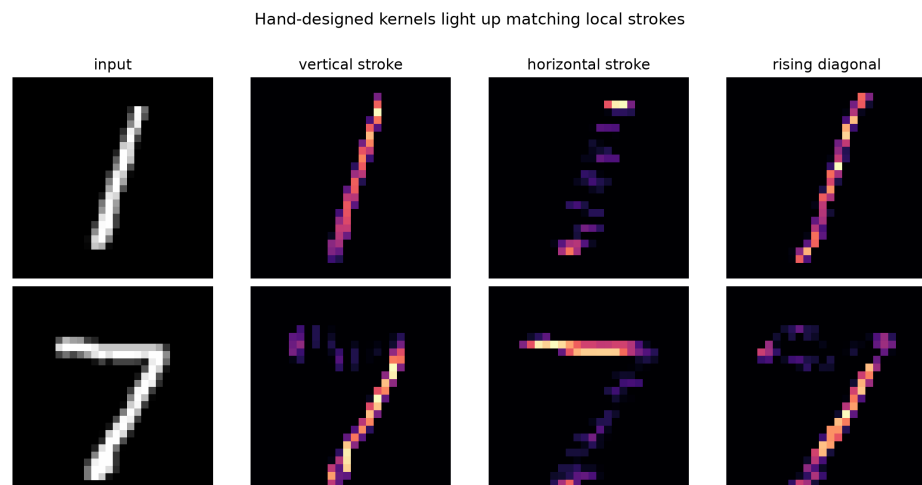


Figure 17.5.: A digit 1 and a digit 7 passed through vertical, horizontal, and diagonal hand-designed kernels. Bright regions mark strong positive responses.

short horizontal transitions in the 1. A single feature map is therefore not a class decision. A CNN learns several filters and combines their spatial patterns across layers.

Experiment. Change one kernel entry at a time. For each change, predict which region will brighten or darken, then test your prediction. Finally design a kernel for a falling diagonal. State why none of these kernels alone is a reliable digit classifier.

Pooling and Parameter Counting

Max pooling retains the largest activation in each window and learns no weights:

```
def max_pool2d(feature_map, size=2, stride=2):
    out_h = (feature_map.shape[0] - size) // stride + 1
    out_w = (feature_map.shape[1] - size) // stride + 1
    output = np.empty((out_h, out_w))
    for row in range(out_h):
        for col in range(out_w):
            r, c = row * stride, col * stride
            output[row, col] = feature_map[
                r:r + size, c:c + size
            ].max()
    return output

def conv_parameters(kh, kw, channels_in, channels_out):
    weights = kh * kw * channels_in * channels_out
    biases = channels_out
    return weights + biases

assert conv_parameters(3, 3, 1, 16) == 160
assert conv_parameters(3, 3, 16, 32) == 4640
```

Why does image height not appear in the parameter formula? Why does it still affect the number of operations and the size of the feature map?

Part III: Replace the MLP with a CNN

Preserve the Grid and Add a Channel Axis

The CNN sees the same scaled values but retains their two-dimensional arrangement:

```
x_fit_cnn = scale(x_fit_raw)[..., np.newaxis]
x_val_cnn = scale(x_val_raw)[..., np.newaxis]
x_test_cnn = scale(x_test_raw)[..., np.newaxis]

assert x_fit_cnn.shape == (54000, 28, 28, 1)
assert x_val_cnn.shape == (6000, 28, 28, 1)
assert x_test_cnn.shape == (10000, 28, 28, 1)
```

Build a Nearly Parameter-Matched CNN

```
def build_cnn():
    keras.utils.set_random_seed(SEED)
    model = keras.Sequential([
        keras.Input(shape=(28, 28, 1)),
        layers.Conv2D(
            16, 3, padding="same", activation="relu", name="conv_1"
        ),
        layers.MaxPooling2D(2),
        layers.Conv2D(
            32, 3, padding="same", activation="relu", name="conv_2"
```



```
    ),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
], name="mnist_cnn")
model.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
return model

cnn_model = build_cnn()
assert cnn_model.count_params() == 105866
cnn_model.summary()
```

Before viewing the summary, calculate every output shape and parameter count. Where are most CNN parameters located? What useful transformation occurs before that large dense layer?

Train and evaluate under the same protocol:

```
cnn_history, cnn_seconds = fit_model(
    cnn_model, x_fit_cnn, x_val_cnn
)
cnn_loss, cnn_accuracy = cnn_model.evaluate(
    x_test_cnn, y_test, verbose=0
)
cnn_pred = cnn_model.predict(x_test_cnn, verbose=0).argmax(axis=1)

results = {
    "MLP": {
```

```
        "accuracy": mlp_accuracy,
        "errors": int(np.sum(mlp_pred != y_test)),
        "parameters": mlp_model.count_params(),
    },
    "CNN": {
        "accuracy": cnn_accuracy,
        "errors": int(np.sum(cnn_pred != y_test)),
        "parameters": cnn_model.count_params(),
    },
}
results
```

Compare Outcomes, Then Explain

This representative run produced:

Evidence	MLP	CNN
Trainable parameters	109,386	105,866
Epochs run	10	7
Lowest-validation-loss epoch	8	5
Test loss	0.0874	0.0370
Test accuracy	97.56%	98.75%
Misclassified test images	244	125
Training time on the authoring machine	5.4 s	17.1 s

The CNN reduced the test error count by 119, a relative reduction of about 49%, despite having slightly fewer parameters. This observation is consistent with the value of locality and weight sharing for images. One run does not prove that CNNs always win: the result depends on the data, protocol, implementation, and task.

Part III: Replace the MLP with a CNN

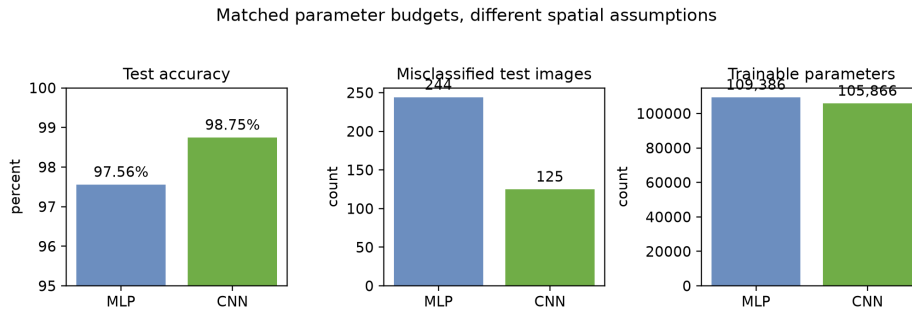


Figure 17.6.: Test accuracy, error count, and parameter count for the matched MLP and CNN run.

```
fig, axes = plt.subplots(1, 3, figsize=(9, 3))
names = ["MLP", "CNN"]
colors = ["#6C8EBF", "#70AD47"]
axes[0].bar(names, [100 * results[n]["accuracy"] for n in names], color=colors)
axes[0].set(title="test accuracy", ylabel="percent", ylim=(95, 100))
axes[1].bar(names, [results[n]["errors"] for n in names], color=colors)
axes[1].set(title="test errors", ylabel="count")
axes[2].bar(names, [results[n]["parameters"] for n in names], color=colors)
axes[2].set(title="parameters", ylabel="count")
plt.tight_layout()
```

Inspecting individual errors keeps the aggregate comparison connected to the images. The following code shows five mistakes from each model:

```
fig, axes = plt.subplots(2, 5, figsize=(9, 3.8))
for row, (name, predictions) in enumerate(
    [("MLP", mlp_pred), ("CNN", cnn_pred)]
):
    wrong = np.flatnonzero(predictions != y_test)
    for axis, index in zip(axes[row], wrong[:5]):
```

```
axis.imshow(x_test_raw[index], cmap="gray")
axis.set_title(
    f"true {y_test[index]} / pred {predictions[index]}"
)
axis.axis("off")
axes[row, 0].set_ylabel(name)
plt.tight_layout()
```

Use Loss to Diagnose Overfitting

```
def plot_history(history, title):
    epochs = np.arange(1, len(history["loss"]) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
    axes[0].plot(epochs, history["loss"], marker="o", label="training")
    axes[0].plot(epochs, history["val_loss"], marker="o", label="validation")
    axes[1].plot(epochs, history["accuracy"], marker="o", label="training")
    axes[1].plot(epochs, history["val_accuracy"], marker="o", label="validation")
    axes[0].set(title=f"{title}: loss", xlabel="epoch")
    axes[1].set(title=f"{title}: accuracy", xlabel="epoch")
    for axis in axes:
        axis.legend()
    plt.tight_layout()

plot_history(mlp_history, "MLP")
plot_history(cnn_history, "CNN")
```

For both models, training loss keeps falling after validation loss has stopped improving. That widening gap is evidence of overfitting: the model continues to fit the training examples without improving performance on held-out data. Accuracy alone makes the turn harder to see; cross-entropy loss

Part III: Replace the MLP with a CNN

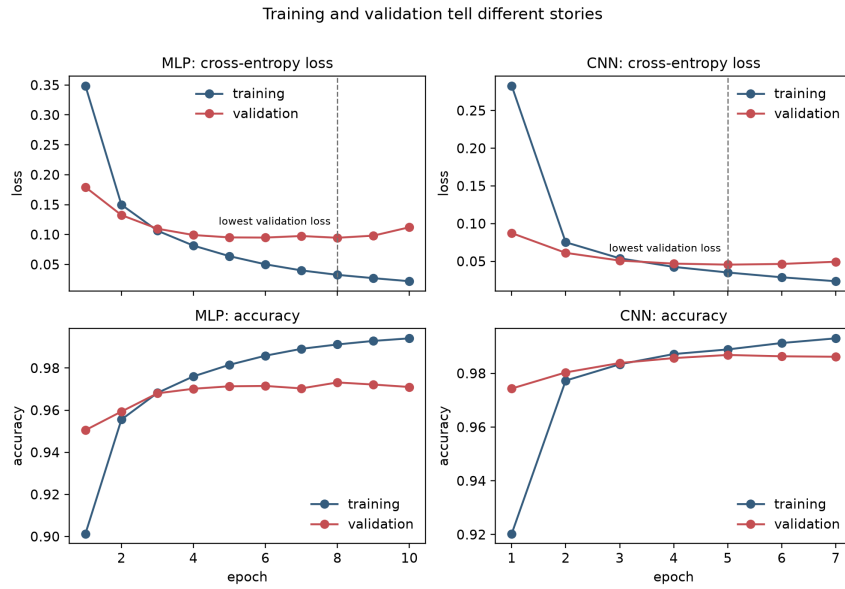


Figure 17.7.: Training and validation loss and accuracy for the MLP and CNN. Dashed lines mark the minimum validation loss restored by early stopping.

reacts to changes in confidence even when the predicted class stays the same.

Interpret. Identify the first epoch after the minimum validation loss for each model. Explain why early stopping restores the earlier weights rather than keeping the final epoch. Then distinguish the evidence (the curve) from the diagnosis (overfitting).

Part IV: Return to Kernels: Learned Activations for 1 and 7

The hand-designed kernels made the mechanism transparent. We now ask what the trained CNN learned. A small **probe model** returns the first convolutional layer's output instead of the final softmax probabilities:

```
probe = keras.Model(
    inputs=cnn_model.inputs,
    outputs=cnn_model.get_layer("conv_1").output,
)
feature_maps = probe.predict(x_test_cnn, verbose=0)

# Compare each channel's share of total first-layer activation.
channel_means = feature_maps.mean(axis=(1, 2))
channel_share = channel_means / (
    channel_means.sum(axis=1, keepdims=True) + 1e-12
)
mean_for_1 = channel_share[y_test == 1].mean(axis=0)
mean_for_7 = channel_share[y_test == 7].mean(axis=0)
difference = mean_for_1 - mean_for_7

channel_1_relative = int(np.argmax(difference))
channel_7_relative = int(np.argmin(difference))
```

```
print(channel_1_relative, channel_7_relative)

first_kernels, _ = cnn_model.get_layer("conv_1").get_weights()
channels = [channel_1_relative, channel_7_relative]
indices = [np.flatnonzero(y_test == digit)[0] for digit in (1, 7)]

fig, axes = plt.subplots(3, 3, figsize=(7, 7))
axes[0, 0].text(
    0.5, 0.5, "weights learned\nfrom labels",
    ha="center", va="center"
)
axes[0, 0].axis("off")
for col, channel in enumerate(channels, start=1):
    kernel = first_kernels[:, :, 0, channel]
    limit = np.abs(kernel).max()
    axes[0, col].imshow(
        kernel, cmap="coolwarm", vmin=-limit, vmax=limit
    )
    axes[0, col].set_title(f"kernel {channel}")
    axes[0, col].axis("off")
for row, (digit, index) in enumerate(zip((1, 7), indices), start=1):
    axes[row, 0].imshow(x_test_raw[index], cmap="gray")
    axes[row, 0].set_ylabel(f"digit {digit}")
    axes[row, 0].axis("off")
    for col, channel in enumerate(channels, start=1):
        axes[row, col].imshow(
            feature_maps[index, :, :, channel], cmap="magma"
        )
        axes[row, col].axis("off")
plt.tight_layout()
```

Normalization matters here because the chosen 7 contains more illuminated pixels than the chosen 1. Without it, a class could have greater mean

activation in nearly every channel simply because it contains more ink.

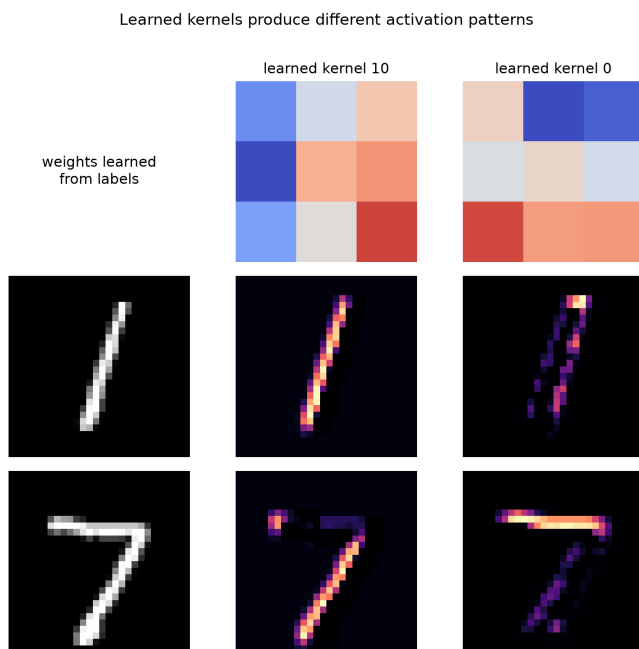


Figure 17.8.: Two learned first-layer kernels and their feature maps for a representative 1 and 7. The channels were selected by their average relative activation difference across all test examples of the two classes.

The bright regions show *where* each learned kernel produces a positive response. Kernel 10 has a larger relative share of activation for 1s across the test set; kernel 0 has a larger relative share for 7s. Neither is a literal digit detector. The complete prediction combines 16 first-layer maps, 32 second-layer maps, their spatial arrangement, and the dense classification head.

Final Investigation

1. Reproduce Figure 17.8 with a different 1 and 7. Which bright regions persist, and which change with handwriting style?
2. Select channels using only one example rather than class averages. Explain why the resulting interpretation is less reliable.
3. Compare a correctly classified 7 with a 7 misclassified by either model. Describe the pixels and feature maps separately from any causal claim.
4. Shift both digits two pixels to the right without wrapping. Does the feature map shift as expected? Does the final predicted probability remain constant?
5. Design a new box-based image like Jason’s proposed letter task. Predict which local edges or corners an early CNN layer would need before building the model.

What to Submit

Submit the completed notebook plus a short report containing:

- your MLP/CNN comparison table;
- one hand-worked kernel response and its matching code result;
- annotated training/validation loss curves;
- the 1-versus-7 hand-designed and learned activation comparisons;
- three representative errors from each model; and
- a 300-500 word conclusion answering the learning question.

Your conclusion must separate **observation** from **explanation**. “The CNN made 119 fewer errors in this run” is an observation. “Locality and weight sharing helped it exploit image structure” is a plausible explanation supported by the controlled design and activation evidence, not a universal theorem about every image problem.

Practice 10: Clustering and Organization

Theory connection: Clustering and Organization

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Example Data

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Hierarchical Clustering

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 10: Clustering and Organization

Issues of HAC

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

K-means Partitioning

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Issues of K-means

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probabilistic Clustering

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Initialization

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Expectation

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Maximization

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Iterations of Expectation-Maximization

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 10: Clustering and Organization

EM with Higher Dimensionality

Recovered starting point: Text and Cluster Analysis.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Text and Cluster Analysis

Recovered activity

i Historical source and execution note

This activity was recovered from `demo/python_clustering.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Introduction

- Clustering is a process of grouping a set of data objects into multiple groups or clusters so that objects within a cluster have high similarity, but are very dissimilar to objects in other clusters.
- There are a lot of areas to apply cluster analysis, such as collaborative filtering, outlier detection, and dynamic trend detection.
- In this exercise, we will practice with topic clustering. In other words, we will group documents by topic to support navigation and exploration.
- If you have not done yet, please review the text vectorization exercise: `text-vectorization.qmd#sec-lab-text-vectorization`.

In this exercise, we are going to utilize a dataset distributed as a part of the Yelp Dataset Challenge (<https://www.yelp.com/dataset>). This is

Text and Cluster Analysis

the same dataset you used for the data preprocessing and assignment 2. The previous dataset had attributes processed from text, while it did not include text. In this exercise, you would directly process text. Let's open the data. There are 1,000 tuples and 4 attributes. The names of attributes are self-explanatory: review_id, text, categories, and class. You are going to work with the text attribute.

```
## Imports all necessary modules
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import nltk, re

# Sklearn
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn import metrics
from pprint import pprint

## This loads the csv file from disk
yelp_data = pd.read_csv(
    filepath_or_buffer = "../data/yelp_reviews.csv", sep = ",", header=0 )

print(yelp_data.head(20))
## print the dimension of the data
print(yelp_data.shape)
```

Text vectorization

You loaded the data. Now, you need to vectorize text so that they can be used for clustering. Before vectorization, you have to preprocess text, including tokenization, stopwords removal, and stemming.


```
# Convert text to a list of data
data = yelp_data.text.values.tolist()

# Created a list of ids
ids = yelp_data.review_id.values.tolist()

# Remove new line characters
data = [re.sub('\s+', ' ', sent) for sent in data]

pprint(data[:5])
```

```
## Conduct text preprocessing such as tokenization, stopwords removal, and stemming.

from nltk.tokenize import word_tokenize

## tokenization
tokens = [word_tokenize(review) for review in data]

print(tokens[:5])
```

```
## import NLTK stopwords
from nltk.corpus import stopwords
stop_words = stopwords.words('english')

## following punctuations and contractions do not deliever any special meaning in text processing
stop_words.extend(["!", ".", ",", "?", "'", "-", "(", ")", "'s", "'ve", "'m" ])

## remove stopwords
tokens_no_stopwords = [[word.replace("'", "") for word in review if word not in stop_words] for review in tokens]

print(tokens_no_stopwords[:5])
```

Text and Cluster Analysis

```
## load nltk's SnowballStemmer
from nltk.stem.snowball import SnowballStemmer
stemmer = SnowballStemmer("english")

## stemming and normalization by converting all characters to lower cases
tokens_stemmed = [[stemmer.stem(word) for word in review] for review in tokens]

print(tokens_stemmed[:5])

## concatenate tokens
## the input for vectorizer should be in the form of a list of string
tokens_concatenated = [" ".join(token) for token in tokens_stemmed]
print(tokens_concatenated[:5])

##define vectorizer parameters
tfidf_vectorizer = TfidfVectorizer(
    analyzer = 'word',
    max_df=0.8,
    max_features=2000,
    min_df=0.05,
    stop_words='english',
    use_idf=True,
    ngram_range=(1,3),
    token_pattern='[a-zA-Z0-9]{3,}', # num of characters in token
)

tfidf_matrix = tfidf_vectorizer.fit_transform(tokens_concatenated)

print(tfidf_matrix.shape)

terms = tfidf_vectorizer.get_feature_names()

## print features you selected.
```

```
print(terms)
```

```
## it is useful to know what terms are included. But knowing the real tf-idf values will be
review_index = [n for n in ids]
import pandas as pd
df = pd.DataFrame(
    tfidf_matrix.T.todense(), index=terms, columns=review_index)
print(df.transpose())
```

Great! You have applied text preprocessing and created a tf-idf term matrix so far. As you can see, there are a few parameters you can modify in the `TfidfVectorizer`, such as `max_df` and `min_df`. If a term occurs too much, then the term is not unique enough to deliver any information. These are terms such as stopwords. Also, if a term rarely occurs, then the term does not have enough evidence to prove to be discriminative. Therefore, changing these parameters would give you different tf-idf matrices. Especially, the number of terms will change. For details, please look at the documentation: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

K-means clustering

Now, we are ready to move onto the next fun step, which is the actual clustering using K-means algorithm. Please note that you should specify the number of clusters (k) to the algorithm in advance. Also, this algorithm finds a local optimum instead of a global optimum. Therefore it is wise to run the k-means algorithm multiple times with different k . We will practice this in the later part.

```
from sklearn.cluster import KMeans

## specify K
```

Text and Cluster Analysis

```
num_clusters = 5

km_model = KMeans(n_clusters=num_clusters)

km_model.fit(tfidf_matrix)

## retrieve cluster ids
clusters = km_model.labels_.tolist()

print(clusters)
```

Now you can see the cluster id of each review. However, this simple number does not deliver much information useful for you to understand the results. Therefore it might be useful if you could get representative words for each topic (i.e., cluster id) and the distribution of topics.

```
## Create Pandas dataframe by using the following dictionary
review = { "review_id" : ids, 'cluster': clusters }

result = pd.DataFrame(review, index = [ids] , columns = ['cluster'])

print(result)

# Review topics distribution across reviews

df_topic_distribution = result['cluster'].value_counts().reset_index(name="Num Reviews")
df_topic_distribution.columns = ['Topic Num', 'Num Reviews']
print(df_topic_distribution)
```

```
from __future__ import print_function

print("Top terms per cluster:")
print()
```

```
# Sort cluster centers by proximity to centroid
order_centroids = km_model.cluster_centers_.argsort()[:, :-1]

# Set the number of top keywords for each cluster
num_top_keywords = 15

for i in range(num_clusters):
    print("Cluster %d top keywords:" % i, end='')
    print() #add whitespace

    for ind in order_centroids[i, :num_top_keywords]:
        print(terms[ind])
    print() #add whitespace
```

After reviewing top keywords for each cluster, it seems that cluster 0 is about hotels, cluster 1 is about clubs or bars, and cluster 2 to 4 is about restaurants. This result makes sense because most reviews in Yelp are for restaurants. Then, now it is going to be important to find the best k for clustering because we simply assume that 5 might be a good number of clusters. We will do it by using Silhouette measure, which is an intrinsic measure you learned in the lecture.

```
from sklearn.metrics import silhouette_score

sil = []
kmax = 20 ## change this value as necessary

## dissimilarity would not be defined for a single cluster, thus, minimum number of clusters
## increase the number of K from 2 to 20
for k in range(2, kmax+1):
    kmeans = KMeans(n_clusters = k).fit(tfidf_matrix)
    labels = kmeans.labels_
```

```
sil.append(silhouette_score(tfidf_matrix, labels, metric = 'euclidean'))

K = range(2, kmax+1)
plt.plot(K, sil, 'bx-')
plt.xlabel('k')
plt.ylabel('Silhouette score')
plt.title('Silhouette Method For Optimal k')
plt.show()
```

So, it seems that $k = 17$ or 20 seems to create the best cluster. However, you will have different results for every run, because K-means algorithm depends on the random initial seeds. Also, there are a lot of other measures, and they might make a different prediction. To see other measures, please refer to the following documentation: https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_digits.html#sphx-glr-auto-examples-cluster-plot-kmeans-digits-py

Bottom-up method

Now it is time to practice another clustering algorithm. I will leave this to you. As mentioned in the lecture, K-means clustering is sensitive to noisy data. According to Zipf's law, text distribution is usually very skewed, which means a lot of outliers. Perhaps, K-means algorithm might not be the best algorithm for topic clustering. Agglomerative hierarchical clustering aims at finding the best step at each cluster fusion, and our data might have an intrinsic hierarchy in it. Think about the category of restaurants. It might start from *Western* vs. *Eastern*. *Eastern* might have sub-categories such as *Korean*, *Indian*, *Chineses*, *Japanese*, and so on. If the data has a hierarchy like this, probably hierarchical clustering might be a better choice. Go for it and see whether it really makes any difference. Please see the following link for implementation: <https://scikit-learn.org/stable/modules/clustering.html#hierarchical-clustering>

References:

<https://scikit-learn.org/stable/modules/clustering.html#k-means>
<http://brandonrose.org/clustering> <https://www.machinelearning-plus.com/nlp/topic-modeling-gensim-python/>

Practice 11: Text and Human Language

Theory connection: Text and Human Language

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Text Vectorization

Recovered starting point: Text Vectorization and Text Vectorization Programming Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Tokenization

Recovered starting point: Text Vectorization and Text Vectorization Programming Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 11: Text and Human Language

Term Weighting

Recovered starting point: Text Vectorization and Text Vectorization Programming Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Similarity Measures

Recovered starting point: Data, Vectors, and Cosine Assignment and Text Vectorization.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Probabilities and Implications

i Practice development scaffold

Create an activity that makes **Probabilities and Implications** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Zipf's Law

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Feature Selection

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Text Classification

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Naive Bayes with Binary Term Occurrences (Bernoulli)

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 11: Text and Human Language

Naive Bayes with Term Frequencies (Multinomial)

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Text Vectorization

Recovered activity

Historical source and execution note

This activity was recovered from `demo/python_text_vector.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Objectives

This exercise is to process a small collection of course titles (as text documents) and compute the following statistics:

1. Term Frequencies (TF) of terms in each document.
2. Inverse Document Frequencies (IDF) of terms in the entire collection.

1. Data

We have a small collection of 3 course titles:

- 1 Information and System
- 2 Data and Information
- 3 System and System Programming

2. Create a *Course* class

```
class Course:
    def __init__(self, course_id):
        # create a new course with an ID
        self.id = course_id
        # create an empty dictionary
        # to hold term frequency (TF) counts
        self.tfs = {}

    def count_words(self, text):
        # split a title into words,
        # using space " " as delimiter
        words = text.lower().split(" ")
        for word in words:
            # for each word in the list
            if word in self.tfs:
                # if it has been counted in the TF dictionary
                # add 1 to the count
                self.tfs[word] = self.tfs[word] + 1
            else:
                # if it has not been counted,
                # initialize its TF with 1
                self.tfs[word] = 1
```

NOW OUTSIDE THE COURSE CLASS

3. Create a `print_dictionary` function

```
def print_dictionary(dict_data):  
    # print all key-value pairs in a dictionary  
    for key in dict_data:  
        print(key, dict_data[key])
```

4. Steps to Vectorize the Course Titles

4.1 Create an empty list of courses

```
courses = []
```

4.2 Create a list of the 3 course titles

```
titles = ["Information and System", "Data and Information", "System and System Programming"]
```

4.3 Use a loop to process data

1. Create a Course object for each title;
2. Process the title and run the TF counts;
3. Add the Course object (with TF statistics) to the list `courses`.

Text Vectorization

```
for i in range(0, len(titles)):
    title = titles[i]

    # create a new course with an ID
    course = Course(i+1)

    # process title and compute term frequencies (TF)
    course.count_words(title)

    # add the course to the list
    courses.append(course)
```

4.4 Create an empty dictionary to keep track of DF values

```
dfs = {}
```

4.5 Go through the Courses' TF dictionaries to get DF values

```
for course in courses:
    for word in course.tfs:
        # add 1 to DF count if the word appears in a doc (TF)
        dfs[word] = dfs.get(word,0) + 1
```

4.6 Print DF (Document Frequency) Statistics

```
print("DF statistics: ")
print("=====")
print_dictionary(dfs)
```


4.7 Print TF (Term Frequency) Statistics

```
print("TF statistics: ")
print("=====")
for course in courses:
    print("COURSE #{0}".format(course.id))
    print("-----")
    print_dictionary(course.tfs)
    print("")
```


Text Vectorization Programming Assignment

Recovered activity

i Historical source and execution note

This activity was recovered from `problems/python_text_vector.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Objectives

This assignment is to process a collection of text files (documents) and compute the following statistics:

1. Term Frequencies (TF) of terms in each document.
2. Inverse Document Frequencies (IDF) of terms in the entire collection.

A. Data Files (1 point)

Please collect about **20 text data instances** (e.g. brief news reports or research abstracts) and save them as individual .txt files. Files names should be named in a sequential order such as 1.txt, 2.txt, 3.txt, ... and 20.txt.

B. Create a *Document* abstract data type/class (2 points)

```
class Document:
    """
    The Document class to be implemented.
    Please define all required methods
    within the class structure.
    """
    def __init__(self, doc_id):
        """to be implemented"""

    def tokenize(text):
        """to be implemented"""
```

The Document class should have:

- A `__init__(self, doc_id)` method as the constructor method
 1. Assign `doc_id` to `self.id`
 2. Create an empty **dictionary variable** with

`self.variable_name`

C. NOW OUTSIDE THE DOCUMENT CLASS

Later you will keep track of all unique words and their frequencies in the document.

- A `tokenize(text)` method that:
 1. Splits text into single words using `space` and `punctuation` as delimiter;
 2. Use a loop to go through all the words, and for each word:
 - Convert the word to lower case using the `lower()` method of a string;
 - If it does not appear in the dictionary, add it to the dictionary and set its count/frequency to 1;
 - If it is already in the dictionary, increment its count/frequency by adding 1 to it;

Hint: See the lecture on data structures and follow the *radish vote counting* example.

C. NOW OUTSIDE THE DOCUMENT CLASS

C1. Create a `save_dictionary` function (1 points)

```
def save_dictionary(dict_data, file_path_name):
    """Saves dictionary data to a text file.
    Arguments:
        dict_data: a dictionary with key-value pairs
        file_path_name: the full path name of a text file (output)
    """
```

The function should accept **two arguments**:

1. One argument for the dictionary with data to be saved;

Text Vectorization Programming Assignment

2. Second argument about the file pathname to save the data;

The function saves all data/statistics in the dictionary to text files, with each key-value pair in one text line separated by a tab (`"\t"`).

The output file should look like:

```
Key1 value1
Key2 value2
Key3 value3
... ..
```

C2. Create a vectorize function (5 points)

```
def vectorize(data_path):
    """Vectorizes text files in the path.
    Arguments:
        data_path: the path to the directory (folder) where text files are
    """
```

The function should:

- Take a **string argument** as the path (folder) to where the text data files are;
- Process all data files in the path and produces TF and DF statistics;

Here are steps in the function:

1. Create a **dictionary variable** (this is a **global dictionary variable** outside the class) to keep track all unique words and their DF (document frequency) values;
2. Create a loop to go through every **.txt** files in the path argument. File names should be: **1.txt**, **2.txt**, ..., **20.txt**.

C. NOW OUTSIDE THE DOCUMENT CLASS

3. **For each** .txt file (within the loop):

- Create a Document object (based on the Document class) using the filename as `doc_id` parameter. For example, `doc_id` should be 2 for `2.txt`.
- Read the content (text lines) from the text file.
- Call the document object's `.tokenize()` function to process the text content.
- Call the `save_dictionary()` function to save the document's dictionary with TF (term frequencies) to a file, where the filename should be `tf_DOCID.txt` in the same path.
 - The TF dictionary can be accessed via the **document object's** `.dictionary_variable_name`.
 - For example, after processing `1.txt` file, the data should be saved to `tf_1.txt` file in the same directory.
- Create a **nested loop** (a smaller/indented loop inside the above loop), and **for each word in the document's dictionary**:
 - If it does not appear in the **dictionary for DF (the dictionary variable outside class)**, then add the word to the DF dictionary;
 - If it is already in the DF dictionary, increment its DF value by adding 1 to itself;

4. After all files are processed (**OUTSIDE the BIG LOOP above**), call the `save_dictionary()` function **again** to save the DF dictionary to a file named `df.txt` in the same path with the input text files.

Text Vectorization Programming Assignment

C3. Finally, call the vectorize function (1 point)

For example, suppose all text files are in the same directory as your code (notebook):

```
vectorize("./") # to process text files in the current directory/folder
```

Bonus (optional, +1 point)

Compute pair-wise cosine similarities among the documents using Term Frequency (TF) weights, and save the results to a file.

Your output may look like (id1 id2 score):

```
1 2 0.37
1 3 0.12
1 4 0.98
. . . . .
18 19 0.33
18 20 0.57
19 20 0.49
```

Submission

1. Test and debug to make sure your **code works** properly;
2. Submit a **ZIP file** containing your Jupyter Notebook (**code**) and text files (**data**) to blackboard.

Integrated Data Learning Assignment

Recovered activity

i Historical source and execution note

This activity was recovered from `problems/data_learning.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Preparation

Before completing the assignment, please get yourself familiar with the models and techniques discussed in Week 6. In particular, you will be **well prepared** for the assignment if you have done **the following exercise**:

- ☐ Probability and Linearity in Data

A. Data Preparation (1 points)

Please identify or prepare a text classification dataset. You can either: + Find and download an **existing dataset**, OR + **Create a dataset** on your own

Integrated Data Learning Assignment

Some of the websites to search for a dataset: + <https://www.kaggle.com/datasets> + <https://archive.ics.uci.edu/ml/datasets.php> + <https://vincentarelbundock.github.io/Rdatasets/datasets.html>

Specific **requirements** of your dataset: 1. [] It has to have **text** data, e.g. abstracts, news reports. 2. [] It has to have a **class** attribute, e.g. with at least **two** categories or labels. 3. [] It has to have at least: + If it is an **existing dataset**, **300+** data instances total and **100+** in each class, **OR** + If you **create** the dataset, **30+** data instances total and **10+** in each class. 4. [] Please **avoid** any dataset already used in **existing** exercises or assignments in this class.

Please do: + [] Include a **link** to your data or **submit** your data. + [] Include a **brief description** of your data, attributes, and instances. + [] Discuss the classification **task** on the data and **objectives**.

B. Probabilities and Zipf (3 points)

B.1. Class Distributions and Probabilities (0.5 point)

Find out the number of instances in each class c and compute its probability $p(c)$. Compile a table like this (example):

Class c	Instances n_c	Probability $p(c)$
Fake	100	$\frac{100}{300} = 1/3$
True	200	$\frac{200}{300} = 2/3$

B.2. Term Probabilities and Zipf's Law (2.5 points)

Conduct analysis in the following steps to obtain the term probability pattern in your text data. You can follow the example in

B. Probabilities and Zipf (3 points)

Probability and Linearity in Data to use `CountVectorizer` from `sklearn.feature_extraction.text`.

B.2.1. Rank, Frequency, and Probability

Rank terms by frequency and show the top five with probabilities (example):

	t	k_t	f_t	p_t
0	to	1	2242	
1	you	2	2240	
2	the	3	1328	
3	and	4	979	
4	in	5	898	

B.2.2. Probability vs. Rank Plot

Produce a probability p_t vs. k_t plot on **log-log** coordinates.

B.2.3. Regression line

Use linear regression to fit the $\log(p_t) \sim \log(k_t)$ relation: 1. Identify the **coefficient** value from the regression, e.g. a value like -1.2 means $p_t \propto \frac{1}{k_t^{1.2}}$. 2. Plot the regression line on the p_t vs. k_t plot. 3. Do your data follow Zipf's law? Discuss the visual pattern, the fitted coefficient, and the regression line.

C. Text Vectorization (2 points)

C.1. Training and Test Data

Split your data into 80% training and 20% test data.

C.2. Text Vectorization

Use the `CountVectorizer` to: 1. [] Vectorize (fit) your **training** data, based on the `text` field. 2. [] Your vectorizer should now have a set of **features** (words) from the training data 3. [] Transform your **test** data (text field), with the same vectorizer.

C.3. Terms and Conditional Probabilities

Think about two terms (words): 1. [] One term (word) t_1 that is relevant to one class c_1 , e.g. **prize** for a **spam** class for spam classification. 2. [] A second term (word) t_2 that is relevant to another class c_2 .

Identify their conditional probabilities:

$$p(t_1|c_1) = \dots \quad (17.6)$$

$$p(t_1|c_2) = \dots \quad (17.7)$$

$$p(t_2|c_1) = \dots \quad (17.8)$$

$$p(t_2|c_2) = \dots \quad (17.9)$$

$$(17.10)$$

Discuss and explain: 1. [] What do these probabilities mean? 2. [] Are the above probability values reasonable (sensible)? Why or why not? 3. [] Do you think they will be helpful in the classification task?

D. Classification (5 points)

D.1. Probabilistic Naive Bayes Model (2 points)

Pick one of the Naive Bayes models we discussed: `BernoulliNB` **OR** `MultinomialNB`, and conduct the following experiments:

1. Pick an **alpha** parameter, build and train (fit) the model using the 80% training data.
2. Test (predict) the model on the 20% test data.
3. Evaluate, show the **confusion matrix**.
4. Discuss, for your task and objectives, **which number(s)** in the confusion matrix is most important, that you wish to minimize or maximize? Why?
5. Compute **accuracy**, **kappa**, and a **3rd metric** (do some research in `sklearn.metrics`) that best evaluates evaluate your objective in the above #4 bullet point.
6. Change **alpha**, train, test, and evaluate again.

Model	Accuracy	Kappa	3rd Metric
Naive Bayes α_1			
Naive Bayes α_2			

D.2. Linear Model (2 points)

D.2.1. Linearity

Remember the two terms t_1 and t_2 for two classes c_1 and c_2 you picked earlier?

Scatter plot the training data with:

1. t_1 and t_2 as the X and Y axes.

Integrated Data Learning Assignment

2. **Color code** data points based on their classes c_1 and c_2 .
 - You only need to plot data in the two classes.
 - Make sure the two colors are distinguishable.
3. Discuss whether:
 - The two classes are (roughly) **separable** on the plot?
 - The two classes are **linearly** separable on the plot?
 - (Optional) It is possible they are linearly separable in a higher dimensional space with **all term features**?

D.2.2. Linear Classification

1. Pick a linear classification model: **Perceptron OR** `sklearn.svm.LinearSVC`.
2. Conduct the same experiments outlined in section D.1 (for the Naive Bayes model).
3. Make sure you research the model, pick a **parameter**, and use **two** different values for the parameter to train, test, and evaluate the model.

Model	Accuracy	Kappa	3rd Metric
Linear model, param1			
Linear model, param2			

D.3 Non-linear Classification or Alternative (1 points)

Pick **one classification model** from the following: + Non-linear Support Vector Machines (SVM) `sklearn.svm.SVC` + Non-linear Multi-layer Perceptron: `sklearn.neural_network.MLPClassifier` + Decision Tree: `sklearn.tree.DecisionTreeClassifier` + Lazy Learning: `sklearn.neighbors.KNeighborsClassifier` + Or any other classification model in `sklearn`

E. Conclusion (1 point)

Conduct the **same analysis as in D.2.2.**

Model	Accuracy	Kappa	3rd Metric
Third model, param1			
Third model, param2			

E. Conclusion (1 point)

- ☐ Compare the results in section D.
- ☐ Review your classification task and objectives.
- ☐ Which model gives you the best result so far? Why?
- ☐ Thoughts on future work on the data?

Practice 12: Search and Retrieval

Theory connection: Search and Information Retrieval

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Basic Paradigm

Recovered starting point: SQLite Practice and SQLite with Python and SQLite and Data Retrieval Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Indexing

Recovered starting point: SQLite Practice and SQLite with Python and SQLite and Data Retrieval Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Matching

Recovered starting point: SQLite Practice and SQLite with Python and SQLite and Data Retrieval Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Ranking

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Content-based Ranking

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Popularity-based Ranking

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

PageRank

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Information Filtering

Recovered starting point: Integrated Data Learning Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Recovered retrieval questions and exercise

Recovered questions

I do want to stress the key advantage AND disadvantage of normalization, which is to treat terms of different forms as the same.

1) There are situations in which the different variations are indeed about the same and should be treated as one term. This way, users who uses one variation in the query may be able to find documents containing terms in another.

2) But there are also situations where normalization is an over-treatment and can lead to results that are in fact not relevant (false positives). There are plenty of examples in both 1) and 2). In specific domains, there might be more in 1) vs. 2), or vice versa. By examining your domain data and

Practice 12: Search and Retrieval

use cases, you may find whether it is better to do certain normalization or not (e.g. casefolding, but not stemming).

Now, we have increasingly more storage and computing power at a cheaper price. So often we can do more with these technologies. For example, it is common to have multiple indices even for the same data, e.g. one with certain normalization and one without. Then at search time, you combine evidence from all indices to final matching and ranking of results.

We know terms have different discriminative powers – that some terms are more useful than the others. We know stopwords (highly frequent terms) are generally less useful. What about rare terms, i.e., terms that are used in a very tiny portion of documents? What about terms in between (neither rare or too frequent)? If you have to choose a limited number of terms to index your documents, what kind of terms would you pick?

In the vector space model, cosine similarity is often used to measure how close (or distant) two vectors are? For example, you can compare a query (vector) with each document (vector), and rank them according to their cosine similarities. Given that cosine is about the angle (vector directions) and disregards vector magnitude (length), why is vector direction more important than magnitude here (for term-based text representation)?

Your analogue is good in that both "generators" are in a dynamic process of generating something, which can trap a crawler in their space (in both worlds).

The generator here in the context of the web is a general reference to dynamic web pages that generate page contents based on different given parameters.

An example of this is the "next page" link to help users navigate to additional lists of information. An ordinary user may click on "next", and "next", ... for a few iterations.

But a spider, without sufficient intelligence and design, may go on and on by simply "clicking" on (following) the "next page" link nonstop... The

notion of "trap" goes both ways – it traps the crawler on the site gathering data on and on; but it also leads to intensive load on the generator (as there appear to be lots of clicks and page loads to the web server). Years ago when I was a graduate student, I developed a crawler with an aim to collect all web pages of the entire university (where I was studying) and, the first night I ran it, it was able to collect 2 million+ pages with a cost. The crawler was trapped by another college/school's site with a page "generator" and, in the end, brought the entire server down... You can imagine what happened the next morning when the IT manager found out... And it did take me some time to settle the issue as a student. ;)

Good question.

For website or pages that no one knows and links to, crawlers won't be able to collect them. A crawler starts with seed websites (those already known) and, ultimately, will get to those where there is a path. But there are "islands" to which there is no path. There are also pages that can be reached from certain parts of the web and cannot be reached either unless you have seeds to crawl from specific locations...

There is always a part of the web where crawlers cannot penetrate. We generally refer to this as the deep web. There are private sites, sites with access control, and sites that can be interacted with but too difficult for crawler to collect... they represent a space much much greater than the regular web covered by search engines.

Recovered retrieval calculation

Assume we have a tiny collection of two documents with the following terms: Doc 1: apple, pear, dessert, cream, cook, work, berry Doc 2: work, computer, disk, apple, network, system Given a free text query "apple computer," rank the documents using Dice's coefficient. Please provide the formula and list calculation steps.

Practice 12: Search and Retrieval

Which ones (more than one) of the following statements about Euclidean distance are true? It measures that distance between the end points of two vectors. It measures that angle between two vectors. Euclidean distance is affected by vector magnitudes. Euclidean distance is zero when two vectors point to the same direction.

Which ones (more than one) of the following statements about Cosine similarity are true? It measures that distance between the end points of two vectors. It measures that angle between two vectors. Cosine is affected by vector magnitudes. Cosine is 1 (max value) when two vectors point to the same direction.

Does vector length normalization (dividing a vector by its length/magnitude) have any impact on Cosine ranking?

SQLite and Data Retrieval Assignment

Recovered activity

Historical source and execution note

This activity was recovered from `problems/python_sqlite.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

This assignment is to load data from CSV, populate them in an SQLite database, and run in-database analytics with SQL.

Please create a Jupyter Notebook for this assignment.

A. Data

The data is provided as `CCSubset.csv`, which is a subset of the UCI Credit Card data set. You only need to use this provided subset for the assignment.

Please open and review the data file `CCSubset.csv` to determine the following when creating your table structure: 1. The name of the table: please

SQLite and Data Retrieval Assignment

use a name with **NO Space** in it. 2. How many fields (columns) do you have and what are they? 3. What are their field types?

The full dataset is available at: <https://www.kaggle.com/uciml/default-of-credit-card-clients-dataset>.

B. SQLite Database Schema (2 points)

Make sure you import sqlite3 module by:

```
import sqlite3
```

Create a new database, connect to it, and create a table structure corresponding to data fields in the CSV data source.

- Use `sqlite3.connect("DatabaseFileName")` to connect the database and once you establish the connection, use its `.cursor()` method to maintain the cursor. For example:

```
my_connection = sqlite3.connect("../data/mydatabase.db")
my_cursor = my_connection.cursor()
```

- Use the cursor's `.execute()` method to run SQL statements;
- Use `CREATE TABLE` statement as a parameter for the `execute()` method to create a new table.

Syntax of the `create table` statement in SQL:

```
CREATE TABLE table1 (
    field1 int primary key,
    field2 varchar(100),
    field3 numeric
)
```


C. Load Data into SQLite (2 points)

C. Load Data into SQLite (2 points)

Read data from the CSV file, use a loop to read each CSV line (data instance), and insert it into the SQLite table:

- Include `import csv` so you can use CSV-related functions;
- Please use the following program structure to read CSV data:

```
with open('students.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
```

Inside the loop:

- You can use `row['CSVColumnName']` to read each column value;
- Execute a SQL statement using `INSERT INTO`, combined with data from CSV, for example, within the for loop:

Example:

```
sql = "INSERT INTO table1 (field1, field2, field3)" \
      " VALUES({0}, '{1}', {2})".format(row["column1"], row["column2"], row["column3"])
my_cursor.execute(sql)
```

Change (and add) names such as `field1/2/3/...` and `column1/2/3/...` to the correct ones in database and CSV respectively.

D. In-database Query and Analytics with SQLite Data (6 points)

Outside the previous structure, create code in Python with related SQL statements to do the following:

1. **Update** the data so that current marriage=2 (single) and marriage=3 values (others) are merged/updated to 2 (single).
2. Remove all data records with **negative** BILL_AMT values using **DELETE**.
3. Select and show the first 10 records in the database table, using **SELECT ... LIMIT...**
4. Select and show all records with a BILL_AMT amount > 500,000;
5. Compute the **total number of records** count(*), average (avg) AGE, min LIMIT_BAL, max LIMIT_BAL in the data.
6. Count the # records, average AGE, min LIMIT_BAL, max LIMIT_BAL, using **GROUP BY**, for each group of DefaultNext (0 and 1 levels).
7. Count the # records, average AGE, min LIMIT_BAL, max LIMIT_BAL for each Marriage group (1, 2), again using **GROUP BY**.
8. Count the # records in each combination of Marriage groups and DefaultNext levels, using **GROUP BY Marriage, DefaultNext**.

For Items 1 and 2 above, you only need to call the cursor's **.execute()** method with proper SQL statements to execute.

For Items 3 and after, when you execute the SQL statement, you need to print the retrieved results using a loop such as:

Bonus (+1 point)

```
print("Say a few words about what you are printing here")
rows = my_cursor.fetchall()
for row in rows:
    print(row[0], row[1], ...)
```

Bonus (+1 point)

Organize the above result printing code as a **function** so that:

- 1) You can call the function to print all rows and columns each time you query the database;
- 2) You reduce redundant code on printing results from item 3 through 8.

Submission

1. Complete all tasks of this assignment in a Jupyter Notebook.
2. Test the code and make sure it works and produces proper results in the notebook.
3. Submit the notebook to blackboard for A3.

SQLite Practice

Recovered activity

i Historical source and execution note

This activity was recovered from `sql/sqlite.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

SQLite

SQL: structured query language for relational databases

- SQLite implements a simple SQL dialect
- SQLite uses data files
 - No need to setup a database server
- Widely used and supported, e.g. on mobile phones

SQLite Interactive

- SQLite has a SQL interactive tool

SQLite Practice

- Please download from: <http://www.sqlite.org/download.html>
- Guide to SQLite interpreter: <http://sqlite.org/cli.html>

SQLite Interactive

1. Download and install SQLite
2. Go to your Terminal or Command Line, and enter:

```
sqlite3
```

3. Start using SQLite interactive with
 - dot commands
 - SQL statements

Here is the interface on the Mac terminal / command line:

Enter `.help` (a dot command) on SQLite prompt will list all supported commands.

```
sqlite> .help
```

Load SQL Magic

```
%load_ext sql
```

Database college.db

Suppose we are to manage the following data in a college: 1. Degree programs with ID, program name, and description 2. Students in the programs, with: ID, name, program information, and class year.

Load SQLite Database

```
%sql sqlite:///college.db
```

Let's open – create if it does not exist – a database file called `college.db`:

```
sqlite> .open college.db
```

Screenshot on Mac terminal:

We can now create the two tables using SQL statements with the SQLite3 interactive.

Please note that `%%sql` symbols, for example:

```
%%sql  
select * from my_table;
```

in this tutorial is for SQL magic in the Jupyter Notebook and should **be excluded** from your actual SQL statement.

programs table

Create the `programs` table structure:

```
%%sql  
drop table if exists programs;  
create table programs (  
    program_id varchar(10) not null primary key,  
    program_name varchar(20),  
    program_desc varchar(500)  
);
```

SQLite Practice

Now add (insert) program data:

```
%%sql
insert into programs (program_id, program_name, program_desc)
      values ('IS', "Information Systems", "Information systems with
insert into programs values ('DS', "Data Science", "New data science program
insert into programs values
      ('CS', "Computer Science", "Traditional computer science"),
      ('CST', "Computing & Security Technology", "CST with a new focus on s
```

Now let's take a look at data in the table:

```
%%sql
select * from programs;
```

students table

Create the students table:

```
%%sql
drop table if exists students;
create table students (
    student_id integer primary key,
    student_name varchar(100),
    program_id varchar(10),
    class_year int
);
```

For the students tables, we have prepared a csv file with students records, which can be downloaded from: [students.csv](#).

Suppose data file `students.csv` is in the **same directory as the college.db**:


```
1  John Smith,      IS,  2020
2  Albert Einstein, DS   2021
3  George Washington, CS, 2019
4  Donald Trump,    CST, 2020
5  Barack Obama,    IS,  2021
.    ...           ..   ....
```

You can use the following **dot commands** to import the CSV data:

```
sqlite> .mode csv
sqlite> .import students.csv students
```

Now check out the data:

```
%%sql
select * from students limit 5;
```

```
%%sql
select count(*) from students;
```

Select and Filter

Now that we have the data, we can go ahead to search the data and run statistics.

For example, if you are interested in students in the DS program:

```
%%sql
select * from students where program_id='DS';
```

Or, you would like to know who have graduated before 2019:

SQLite Practice

```
%%sql
select * from students where class_year<2019;
```

Group Aggregates

Perhaps, you only want to know statistics in each program:

```
%%sql
select program_id,
       count(*) as student_count,
       min(class_year) as first_class,
       max(class_year) as last_class
from students
group by program_id
order by student_count DESC;
```

Update Student Data

Now that you see from the statistics some data values are unrealistic such as a class year > 2030. Perhaps we can reset all these values to 2030 if they are greater than that:

```
%%sql
update students set class_year = 2030 where class_year > 2030;
```

We can run the statistics again to see if data have been updated:

```
%%sql
select program_id,
       count(*) as student_count,
```

```
        min(class_year) as first_class,  
        max(class_year) as last_class  
from students  
group by program_id  
order by student_count DESC;
```

Join Tables

Suppose you want to retrieve student data with their corresponding program information (e.g. program name):

```
%%sql  
select student_name, program_name, class_year  
from students s join programs p on s.program_id = p.program_id  
limit 10;
```

We can rerun the previous statistics based on the program names, instead of program IDs:

```
%%sql  
select program_name,  
       count(*) as student_count,  
       min(class_year) as first_class,  
       max(class_year) as last_class  
from students s join programs p on s.program_id = p.program_id  
group by program_name  
order by student_count DESC;
```


References

- Chapter 3 Managing Data, section Managing Data using SQL, of LauraCassell and AlanGauld (2014). Python Projects.<https://ebookcentral-proquest-com.ezproxy2.library.drexel.edu/lib/drexel-ebooks/detail.action?docID=1875898>
- SQLite operations in Python:http://sebastianraschka.com/Articles/2014_sqlite_in_python_tutorial.html

This is not the most efficient method to join and run the **group by** clause. But for now, it is good for the small dataset here.

SQLite with Python

Recovered activity

Historical source and execution note

This activity was recovered from `sql/sqlite_python.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

SQLite with Python

- SQLite: File-based relational SQL database
- Python: A popular language in data science, etc.
- SQLite is well supported in Python

This tutorial will follow the same `college.db` example in the SQLite Practice presentation.

Python Code for SQLite

Import SQLite

```
import sqlite3
```

Load SQLite Database

To open and connect to the database file `college.db`:

```
db = 'college.db'
conn = sqlite3.connect(db)
c = conn.cursor()
```

We can now create the two tables:

1. Programs with ID, program name, and description;
2. Students with ID, name, program info, and class year.

using SQL statements with the SQLite3 interactive.

programs table

Create the `programs` table structure:

```
c.execute("drop table if exists programs")
c.execute("""create table programs (
            program_id varchar(10) not null primary key,
            program_name varchar(20),
            program_desc varchar(500)
        )""")
conn.commit()
```


Now we can start adding students data:

```
insert_sql = "insert into programs (program_id, program_name, program_desc) values (?, ?, ?)"

c.execute(insert_sql, ('IS', 'Information Systems', 'Information systems with a business angle'))
c.execute(insert_sql, ('DS', 'Data Science', 'New data science program'))
c.execute(insert_sql, ('CS', 'Computer Science', 'Traditional computer science'))
c.execute(insert_sql, ('CST', 'Computing & Security Technology', 'CST with a new focus on security'))

conn.commit()
```

Now let's take a look at data in the table:

students table

Create the students table:

```
c.execute("drop table if exists students")
c.execute("""
    create table students (
        student_id integer primary key,
        student_name varchar(100),
        program_id varchar(10),
        class_year int
    )
""")
conn.commit()
```

For the students tables, we have prepared a csv file with students records, which can be downloaded from: [students.csv](#).

Suppose data file **students.csv** is in the **data** sub-directory (folder):

SQLite with Python

1	John Smith,	IS,	2020
2	Albert Einstein,	DS	2021
3	George Washington,	CS,	2019
4	Donald Trump,	CST,	2020
5	Barack Obama,	IS,	2021
.	...	..	

Let read the CSV file and load data into the `students` table:

```
import csv

# remove all student data first
delete_sql = "delete from students"
c.execute(delete_sql)

# load CSV data
f=open("data/students.csv")
insert_sql = "insert into students values (?, ?, ?, ?)"
for row in csv.reader(f):
    c.execute(insert_sql, row)

conn.commit()
```

Now check out the data:

```
c.execute("select * from students limit 10")
rows = c.fetchall()
for row in rows:
    print(row)
```

Now let's count the number of records in the table:

```
c.execute("select count(*) from students")
rows = c.fetchall()
for row in rows:
    print("The total number of students:", row[0])
```

Group Aggregates

Perhaps, you only want to know statistics in each program:

```
query = """select program_id,
        count(*) as student_count,
        min(class_year) as first_class,
        max(class_year) as last_class
from students
group by program_id
order by student_count DESC
"""
c.execute(query)
rows = c.fetchall()
for row in rows:
    print(row[0], row[1], row[2], row[3])
```

Update Student Data

Now that you see from the statistics some data values are unrealistic such as a class year > 2030. Perhaps we can reset all these values to 2030 if they are greater than that:

```
update_sql = "update students set class_year = 2030 where class_year > 2030"
c.execute(update_sql)
conn.commit()
```

SQLite with Python

We can run the statistics again to see if data have been updated:

```
from pprint import pprint

query = """select program_id,
              count(*) as student_count,
              min(class_year) as first_class,
              max(class_year) as last_class
from students
group by program_id
order by student_count DESC
"""

c.execute(query)
rows = c.fetchall()
pprint(rows)
```

Join Tables

Suppose you want to retrieve student data with their corresponding program information (e.g. program name):

```
query = """
    select student_name, program_name, class_year
    from students s join programs p
        on s.program_id = p.program_id
    limit 10
"""

c.execute(query)
rows = c.fetchall()
for row in rows:
    print("{:s} {:s} {:d}".format(row[0].ljust(20), row[1].ljust(40), row[2]))
```

References

- Chapter 3 Managing Data, section Managing Data using SQL, of Laura Cassell and Alan Gauld (2014). Python Projects. <https://ebookcentral-proquest-com.ezproxy2.library.drexel.edu/lib/drexel-ebooks/detail.action?docID=1875898>
- SQLite operations in Python: http://sebastianraschka.com/Articles/2014_sqlite_in_python_tutorial.html

Practice 13: Graphs and Structural Analysis

Theory connection: Graphs and Structural Analysis

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Graphs as Data

i Practice development scaffold

Create an activity that makes **Graphs as Data** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Paths, Connectivity, and Traversal

i Practice development scaffold

Create an activity that makes **Paths, Connectivity, and Traversal** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Centrality and Influence

i Practice development scaffold

Create an activity that makes **Centrality and Influence** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Communities and Structural Roles

i Practice development scaffold

Create an activity that makes **Communities and Structural Roles** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Link Prediction

i Practice development scaffold

Create an activity that makes **Link Prediction** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

From Graph Features to Graph Learning

i Practice development scaffold

Create an activity that makes **From Graph Features to Graph Learning** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Practice 14: Evaluation and Experimentation

Theory connection: Evaluation and Experimentation

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Precision

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Recall

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Sensitivity and Specificity

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Kappa and Chance Agreement

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Averaging

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Rank Evaluation

Recovered starting point: Integrated Data Learning Assignment and Association Rule Mining.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Evaluation of Numeric Predictions

i Practice development scaffold

Create an activity that makes **Evaluation of Numeric Predictions** observable through a calculation, small implementation, visualization, diagnostic, or written interpretation. Include a reproducible input, a checkable result, and one reflection question.

Error Metrics

Recovered starting point: Classification Patterns and Evaluation Assignment.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Correlation

Recovered starting point: Classification Patterns and Evaluation Assignment.

Practice 14: Evaluation and Experimentation

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Experimentation

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

On Efficiency

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Association Rule Mining

Recovered activity

i Historical source and execution note

This activity was recovered from `demo/python_association_rule_mining.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Introduction

- A typical example of association rule mining is market basket analysis.
- This process analyzes transaction data to find associations between the different items that customers place in their “shopping baskets”.
- The discovery of these associations can help retailers develop marketing strategies by understanding which items are frequently purchased together by customers.
- Also, online retailers can develop a recommendation system based on association rule mining.

In this exercise, we are going to utilize a dataset downloaded from Kaggle (<https://www.kaggle.com/irfanasrullah/groceries>). Please see the data description on the site, if you are interested in details. Please note that the

Association Rule Mining

file format is somewhat different from what we used to be familiar with. Tuples contain itemsets purchased together in a transaction. Therefore, each tuple contains a different number of attributes, i.e., items. The ultimate goal is to find strong associations among items. I left up to six items per transaction on purpose. If there were more than six items, I deleted them. Let's open the data. There are 9,835 tuples.

```
import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, association_rules
import matplotlib.pyplot as plt

## This loads the csv file from disk
basket_data = pd.read_csv("../data/groceries/groceries_modified.csv", sep = '\t')

print(basket_data.head(20))
## print the dimension of the data
print(basket_data.shape)
```

You can see a lot of NaN from the Pandas dataframe. It's because they were empty in the original data. You may be interested in how many unique items are actually in the data.

```
items = list()
for attr in range(basket_data.shape[1]):
    items.extend(list(basket_data[attr].unique()))
print(set(items))
print(len(set(items)))
```

Preprocessing

There are 168 items in total that made up the entire data. Let's find some interesting associations between them!

Before starting the fun part, we need to preprocess the dataset. The library (MLXtend) requires 1/0 representation for attributes. With that being said, the shape of the dataframe should be changed. As there are 168 unique items, the number of columns should be 168. In tuples, the values should represent whether each item occurs or not in the transaction. If an item is in the basket, it should be represented as 1. If not, it should be represented as 0.

```
# Create empty list to contain the converted data
converted_vals = []
for index, row in basket_data.iterrows():
    labels = {}
    # Find items that do not occur in the transaction
    not_occurred = list(set(items) - set(row))
    # Find items that occur in the transaction
    occurred = list(set(items).intersection(row))
    for nc in not_occurred:
        labels[nc] = 0
    for occ in occurred:
        labels[occ] = 1
    converted_vals.append(labels)

converted_basket = pd.DataFrame(converted_vals)
print(converted_basket.head())
```

If you look at the table carefully, you will find one unwelcomed guest, “NaN”. It was included to process empty values, now plays like a real value. We should drop this. You see the importance of data preprocessing.

```
converted_basket = converted_basket.drop([np.nan], 1)
print(converted_basket.head())
```

Implementation

Now It's time for fun. You will create association rules between attributes. You need to change minimum support and/or other parameters to filter out less interesting rules.

```
frequent_itemsets = apriori(converted_basket, min_support=0.3, use_colnames=True)
print(frequent_itemsets.head())
```

You got no frequent itemsets. You have relatively large transactions, so having high support is challenging. You have to tune this parameter to get a good number of candidates.

```
frequent_itemsets = apriori(converted_basket, min_support=0.04, use_colnames=True)
print(frequent_itemsets)
```

```
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
print(rules)
```

We can filter the dataframe. Let's look for cases whose lift is larger than 2 and confidence is larger than 0.3.

```
rules[ (rules['lift'] >= 2) & (rules['confidence'] >= 0.3) ]
```

Visualization

Now It might be helpful to visualize the result. The visualization will help you understand how each rule performs.

```
plt.scatter(rules['support'], rules['confidence'], alpha=0.7)
for i in range(rules.shape[0]):
    plt.text(rules.loc[i,"support"], rules.loc[i,"confidence"], str(i))
plt.xlabel('support')
plt.ylabel('confidence')
plt.title('Support vs Confidence')
plt.show()
```

Now, you are ready to apply association rule mining to find interesting patterns. Please find more details about the libraries you used at: http://rasbt.github.io/mlxtend/user_guide/frequent_patterns/apriori/ and http://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/.

Practice 15: Generalization, Model Selection, and Fit

Theory connection: Generalization, Model Selection, and Fit

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Training Performance and Generalization

Recovered starting point: Classification Patterns and Evaluation Assignment and Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Bias, Variance, and Model Capacity

Recovered starting point: Classification Patterns and Evaluation Assignment and Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Validation and Cross-Validation

Recovered starting point: Classification Patterns and Evaluation Assignment and Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Regularization

Recovered starting point: Classification Patterns and Evaluation Assignment and Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Learning Curves and Diagnostics

Recovered starting point: Classification Patterns and Evaluation Assignment and Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Hyperparameter Selection Without Leakage

Recovered starting point: Classification Patterns and Evaluation Assignment and Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Practice 16: Scaling, Deployment, and Responsible Use

Theory connection: Scaling, Deployment, and Responsible Use

Use this chapter as the practical companion to the corresponding theory chapter. Recovered activities are linked where a strong match exists; other sections are explicit development scaffolds for the next writing cycle.

Time, Space, Data, and Compute

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Batch, Streaming, and Distributed Workflows

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

From Experiment to Reproducible Pipeline

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Deployment, Monitoring, and Drift

Recovered starting point: Data Mining Project.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Privacy, Security, and Misuse

Recovered starting point: Data Mining Project and Reinforcement Learning Extension.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Energy, Sustainability, and Proportionality

Recovered starting point: Data Mining Project and Reinforcement Learning Extension.

Responsible Decisions and Human Oversight

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Responsible Decisions and Human Oversight

Recovered starting point: Data Mining Project and Reinforcement Learning Extension.

Use the recovered material as evidence and raw material; revise package calls, expected outputs, and assessment criteria before assigning it in a current course.

Data Mining Project

Recovered activity

i Historical source and execution note

This activity was recovered from `problems/data_mining_project.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Please think about tentative ideas for your final project, which will account for 24% of your final grade.

This is a group project so please find other students to work together (**up to four** students).

Please **choose one** of the following project types:

I. Data Mining and Report

Find an **application domain** where you can use apply the methods of data mining for knowledge discovery (see topics above). Use real (not hypothetical) **data** and problems related to your interests. While there are various types of data you can use, make sure there are sufficient data for your problems.

Data Mining Project

General **topics of data mining** you can think about: + Generalization: integration, aggregation, summarization, etc. + Association and correlation analysis: frequent patterns, rules + Classification: training and testing (predictions) + Cluster analysis: major patterns, themes, discovery of new categories + Outlier analysis: outliers, noise or exception + Sequence, trend and evolution analysis + Graph mining, network analysis, web mining, etc.

The focus is on your mining *processes, analysis, and report*.

II. Algorithm Implementation

Pick a specific **data mining method** and implement it as a piece of software that can be tested with data. You can develop the method in Python or any other languages. The method (algorithm) can be any process in:

1. Data cleaning
2. Data integration
3. Data selection
4. Data transformation
5. Data mining
6. Pattern evaluation
7. Knowledge representation

The focus is your *design, code, test, and brief documentation*.

III. Literature Review

If you are interested in a specific **research** area in data mining, you may choose to do a literature review about it. For this type of project, your survey of the area, critical review, and writing will be the focus of your

Group Submission

work. There are a wide range of topics that you can review (see above). You can consult the textbook and related revenues such as KDD, CIKM, etc for additional references.

We do not cover everything in this class so feel free to explore.

Group Submission

To submit the project abstract, you need to:

1. Form a group of up to 4 students.
2. Pick a category and identify your topic/problem.
3. Write and submit an abstract about your project (about **150 words**).

Reinforcement Learning Extension

Recovered activity

Historical source and execution note

This activity was recovered from `research/rl_taxi.ipynb`. Its code is preserved but is not executed during the public book build, so readers can inspect it without requiring legacy package versions. Download the source notebook to run and modernize it interactively.

Motivation

- Classification of individual data instances
- More complex problems:
 1. No clearly defined class labels
 2. Better or worse, instead of true or false
 3. Learning from a sequence of trials

In supervised learning or classification, we focus on an individual data instance at a time and try to make a prediction or decision based on features of that single instance. While we may have built a model based on a large number of training samples, testing is conducted more or less on an isolated data instance, where the result of other test data has no impact.

Reinforcement Learning Extension

In reality, however, we have many situations and problems that are much more complex than what can be treated as classification for a number of reasons. For one, there might not be a clearly defined set of labels or training data for supervised learning. Perhaps, it is not possible to tell whether an action or decision is right (true) or wrong (false) under the current circumstance. Rather, it should be focused on what is better (improving) or worse. So the objective has more to do with optimization than classification.

More important, it normally takes more than one trials to figure out what is working vs. what is not. So whatever learning model we employ for tasks will have to analyze a sequence of actions and returns to identify an effective model.

Example, a robot learning to play soccer: + When to dribble? When to pass a ball? + Timing is key, e.g. a sequence of (teamwork) actions leading to a goal

Imagine if you are a striker, the success of your **actions** (e.g. passing, dribbling, and scoring) depends on your interactions with the **environment**, for example: + Who has the ball and where the ball is going + Where your teammates are or running toward + Where players on the other team are and/or moving to ...

It is a very dynamic environment that is constantly changing and you need to be alert about what may happen next so you can be there in the right place at the right moment, e.g. to receive the ball and score.

Reinforcement Learning

The soccer game illustrates several important concepts in Reinforcement Learning:

```
from graphviz import Digraph
dot = Digraph(comment="Reinforcement Learning", graph_attr={'rankdir':'LR'}, engine='dot')
dot.node('A', 'Agent', shape='circle')
# with dot.subgraph(name='child', node_attr={'shape': 'box'}) as c:
#     c.node('c', 'Action', shape='box')
#     c.node('s', 'State', shape='egg')
#     c.node('r', 'Reward', shape='egg')
dot.node('E', 'Environment', shape='circle')
# dot.edge('A', 'c')
# dot.edges(['Ac', 'cE', 'sA', 'rA', 'Es', 'Er'])
dot.edge('A', 'E', label='Action')
dot.edge('E', 'A', label='State')
dot.edge('E', 'A', label='Reward')
```

dot

1. **Agent** is a software program, just like a soccer player, who learns and adapts to perform **Actions**, e.g. to play in a soccer game.
2. **Environment** refers to the entire dynamically changing setting where agents operate, e.g. the soccer field, players, referees, and their interactions under the rules.
3. **State** of the environment such as the location of the ball and moving speeds of players.
4. **Reward** or **Penalty** as a result of an agent's action. For example, there are big rewards such as scoring a goal or penalties such as receiving a red card; but there are also minor rewards such as passing the ball accurately and being able to have longer procession, which may lead to opportunities to score.

Q-Learning

Through actions and interactions (state and reward) with the environment, it is possible for an agent to gain knowledge and improve its actions. We refer to as

The **Policy**: the method (strategy) an agent uses to act on its knowledge to maximize its long-term rewards.

Q-Learning is a reinforcement learning policy, in which the agent keeps track of **rewards** associated with each **state** and **action**.

Q-Table

Specifically, the agent stores rewards in a so-called **Q-table** like this:

	A_1	A_2	A_3	..	A_m
S_1	0	0	0		0
S_2	0	0	0		0
..	0	0	0		0
S_n	0	0	0		0

where: + $S_1, S_2, \dots, S_n$ are all possible states (rows). + $A_1, A_2, \dots, A_m$ are all possible actions (columns) for each state.

The Q-table is initiated with 0 values and gets updated whenever the agent performs an action and receives a reward (or penalty) from the environment.

Suppose at time t , the agent is at the state of s_t , performs an action a_t and receives a reward of r_t .

Here is one strategy to update the Q-table:

$$Q'(s_t, a_t) \leftarrow (1 - \beta)Q(s_t, a_t) + \beta(r_t + \gamma \max_a Q(s_{t+1}, a)) \quad (17.11)$$

where: $\beta \in [0, 1]$ is the learning rate. A greater β leads to more aggressive updating. $\gamma \in [0, 1]$ is the discount factor for future rewards. A greater γ tends to put more emphasis on long-term rewards.

Exploration vs. Exploitation

An agent learns from trials and errors. So learning requires (random) **exploration** in the environment. At some points, the agent may gain “sufficient” knowledge to make wiser decisions based on a trained Q-table. However, in order to allow opportunities to continued learning, one should avoid complete **exploitation** of Q-table for action selection.

For the tradeoff, we introduce a third parameter ϵ (probability), at time t :

1. With ϵ probability, the agent selects a random action.
2. With $1 - \epsilon$ probability, the agent selects the action with the maximum reward for state s_t in the Q-table: $a_t = \underset{a}{\operatorname{argmax}} r(s_t, a)$

Taxi Cab

```
#####
# Some common functions to show simulations
#####

import re
from IPython.display import clear_output
```

```
from time import sleep
from IPython.display import HTML

# Convert ASCII output to HTML (more graphical)
def str2html(out):
    out2 = out.replace(":", "~")
    # 4 locations
    # out2 = out2.replace("|R", "|<span style='color:red;vertical-align:middle'>R</span>")
    # out2 = out2.replace("|Y", "|<span style='color:yellow;vertical-align:middle'>Y</span>")
    out2 = out2.replace("\x1b[34;1m", "<div class='pickup'>")
    out2 = out2.replace("\x1b[35m", "<div class='target'>")
    # taxi cab
    out2 = out2.replace("\x1b[42m", "<div class='cab_loaded'>")
    out2 = out2.replace("\x1b[43m", "<div class='cab'>")
    # out2 = out2.replace("\x1b[43mR", "<div class='cab'>")
    # out2 = out2.replace("\x1b[43mG", "<div class='cab'>")
    # out2 = out2.replace("\x1b[43mG", "<div class='cab'>")
    # out2 = out2.replace("\x1b[43mY", "<div class='cab'>")
    out2 = out2.replace("\x1b[0m", "</div>")

    # table cells
    out2 = out2.replace("\n+-----+", "\n</table>")
    out2 = out2.replace("+-----+\n", "<table style='border:solid 1px black'>")
    out2 = out2.replace("\n|", "\n<tr><td class='street' style='text-align:center'>")
    out2 = out2.replace("| \n", "</td></tr>\n")
    out2 = out2.replace("|", "</td><td class='stwall' style='border-left: solid 1px black'>")
    out2 = out2.replace("~", "</td><td class='street' style='border-left: dashed 1px black'>")
    css = """
    <style>
        .street {
            width: 60px;
            height: 60px;
        }
    """
```

```
        text-align: center;
        border-left: dashed 2px white;
    }
    .stown {
        width: 60px;
        height: 60px;
        text-align: center;
        border-left: solid 3px black;
    }
    .pickup {
        width: 50px;
        height: 50px;
        border-radius: 50%;
        font-size: 20px;
        color: #fff;
        line-height: 50px;
        text-align: center;
        background: blue
    }
    .target {
        width: 50px;
        height: 50px;
        border-radius: 50%;
        font-size: 20px;
        color: #fff;
        line-height: 50px;
        text-align: center;
        background: red
    }
    .cab {
        background-image: url(figures/cab2.png);
        background-size: 20px;
        background-repeat: no-repeat;
```

```
        background-position: center;
        height: 45px;
        width: 45px;
        border: dashed 2px blue;
    }
    .cab_loaded {
        background-image: url(figures/cab2.png);
        background-size: 20px;
        background-repeat: no-repeat;
        background-position: center;
        height: 45px;
        width: 45px;
        border: solid 2px red;
    }

    </style>"""
    return css + "\n" + out2

# print simulation frames
def print_frames(frames, offset, limit, sleep_time):
    rewards = 0
    for i, frame in enumerate(frames):
        if i >= offset:
            rewards += frame['reward']
            clear_output(wait=True)
            out2 = str2html(frame['frame'])

            #         print(f"Timestep: {i+1}")
            #         print(f"State: {frame['state']}")
            #         print(f"Action: {frame['action']}")
            #         print(f"Reward: {frame['reward']}")
            out2 = """
            {0}
```



```

        <table>
            <tr><td>Timestep: </td><td>{1}</td></tr>
            <tr><td>State: </td><td>{2}</td></tr>
            <tr><td>Action: </td><td>{3}</td></tr>
            <tr><td>Reward: </td><td>{4}</td></tr>
            <tr><td>Total Reward: </td><td>{5}</td></tr>
        </table>""".format(out2, i+1, frame['state'], frame['action'], frame['reward'],
display(HTML(out2))
sleep(sleep_time)
if limit>0 and i>(offset+limit):
    # print(frame['frame'])
    break

# visualize the frames side by side
def compare_frames(t1, frames1, t2, frames2, offset, limit, sleep_time):
    rewards1, rewards2 = 0, 0
    for i, frame1 in enumerate(frames1):
        if i>=offset:
            frame2 = frames2[i]
            rewards1 += frame1['reward']
            rewards2 += frame2['reward']

            clear_output(wait=True)
            # first frame
            out1 = str2html(frame1['frame'])
            out1 = """
            <td>
            <h1>{0}</h1>
            {1}
            <table>
                <tr><td>Timestep: </td><td>{2}</td></tr>
                <tr><td>State: </td><td>{3}</td></tr>
                <tr><td>Action: </td><td>{4}</td></tr>

```

Reinforcement Learning Extension

```
        <tr><td>Reward: </td><td>{5}</td></tr>
        <tr><td>Total Reward: </td><td>{6}</td></tr>
    </table>
</td>""".format(t1, out1, i+1, frame1['state'], frame1['action'])

# second frame
out2 = str2html(frame2['frame'])
out2 = ""
<td>
<h1>{0}</h1>
{1}
<table>
    <tr><td>Timestep: </td><td>{2}</td></tr>
    <tr><td>State: </td><td>{3}</td></tr>
    <tr><td>Action: </td><td>{4}</td></tr>
    <tr><td>Reward: </td><td>{5}</td></tr>
    <tr><td>Total Reward: </td><td>{6}</td></tr>
</table>
</td>""".format(t2, out2, i+1, frame2['state'], frame2['action'])

display(HTML("<table><tr>" + out1 + "\n" + out2 + "</tr></table>"))
sleep(sleep_time)
if limit>0 and i>(offset+limit):
    # print(frame['frame'])
    break
```

The Taxi task in OpenAI's gym is a simplified problem of driving a taxi cab in city blocks.

Here is an visualization of city blocks, more like a parking lot:

```
import gym
env = gym.make("Taxi-v3")
env.reset()
out_text = env.render(mode="ansi")
out_html = str2html(out_text)

display(HTML(out_html))
```

Problem

Major tasks of the taxi cab (agent) include:

1. Pick up a passenger from a location (in blue color), at R, G, B, or Y.
2. Drop off the passenger at the target location (in red), at R, G, B, or Y.
3. Travel as little as possible to complete the tasks.

In each state, the cab can take any of the following actions:

0. Move south
1. Move north
2. Move east
3. Most west
4. Pickup passenger
5. Dropoff passenger

There are related rewards and penalties for each action:

- −1 point penalty for each action/move (just like gasoline consumption).
- −10 points penalty for illegal pickup or dropoff.
- 20 points reward for a successful delivery of the passenger.

Reinforcement Learning Extension

For example, given the following `state` where the cab is to: 1. Pick up passenger at pickup location (blue); 2. Drive to the destination at destination (red); 3. Drop off the passenger.

```
env.reset()
env.s = 328
```

```
display(HTML(str2html(env.render(mode="ansi"))))
```

Action without Learning

How can the agent (cab) learn to perform the tasks?

We will first start with a dummy version where the cab: + simply selects a random action + and does not learn

```
action = env.action_space.sample()
state, reward, done, info = env.step(action)
```

```
epochs = 0
penalties, reward = 0, 0
frames = []
done = False

while not done:
    action = env.action_space.sample()
    state, reward, done, info = env.step(action)
    if reward == -10:
        penalties += 1

    frames.append({
        'frame': env.render(mode='ansi'),
```

```

        'state': state,
        'action': action,
        'reward': reward
    })

    epochs += 1

print("Timesteps taken: {}".format(epochs))
print("Penalties incurred: {}".format(penalties))

print_frames(frames, 0, 0, 0.2)

```

You can tell that the cab is fooling around and its moves and actions are random and meaningless. By looking at the behavior of the cab, we can hardly have confidence in it to develop any intelligence and to perform a better job.

The strategy of random actions does not work!

Explore and Learn

Now let's follow the Q-Learning idea so the agent (cab) can learn from its failures and successes. In particular, we would like the agent to remember what works vs. what does not, that is:

What are past **rewards** (or **penalties**) for each **action** taken in each **state**.

We know there are 6 different actions the cab can select from. But how many states are there?

Here are factors to be considered in the combinations of states:

1. The cab can be at one of the 5×5 blocks.

Reinforcement Learning Extension

2. The passenger can be at one of the 4 pickup locations (RGBY) or in the cab: $\times 5$.
3. The cab is to drop off the passenger at one of the 4 locations: $\times 4$.

The total number of states: $5 \times 5 \times 5 \times 4 = 500$.

So we need to create a Q-table of 500 rows (states) $\times$ 6 columns (actions):

```
import numpy as np
q_table = np.zeros([env.observation_space.n, env.action_space.n])
```

We start with a Q-table like this (first 10 states):

```
q_table[0:10]
```

Each row has 6 columns because there is one value for each of the 6 actions. We only show first 10 states here but remember there are 500 rows (states) in total.

Now we will let the agent (cab) explore a little bit, with $\epsilon = 0.1$.

Given the current **state**, the agent has: + 10% probability to explore and select random actions.

```
action = env.action_space.sample()
```

- 90% probability to select the best action based on the Q-table (memory)

```
action = np.argmax(q_table[state])
```

After the action, the agent (cab) receives a **reward** and updates its Q-table:

$$Q'(s_t, a_t) \leftarrow (1 - \beta)Q(s_t, a_t) + \beta(r_t + \gamma \max_a Q(s_{t+1}, a)) \quad (17.12)$$

where: $\beta = 0.1$ is the learning rate + $\gamma = 0.6$ is the discount for future rewards + $Q(s_t, a_t)$ is the value in Q-table for state s_t and action a_t + r_t is the the received reward (or penalty) + $Q'(s_t, a_t)$ denotes the updated value in Q-table

```
%%time
"""Agent under training"""

import random
from IPython.display import clear_output

# Hyperparameters
a = 0.1 # alpha
g = 0.6 # gamma
e = 0.1 # epsilon

# keeping track of
all_epochs = []
all_penalties = []

# Train the cab for 100,000 tasks
for i in range(1, 10**5+1):
    state = env.reset()
    epochs, penalties, reward = 0, 0, 0
    done = False

    while not done:
        if random.uniform(0, 1) < e:
            action = env.action_space.sample() # random exploration
```

Reinforcement Learning Extension

```
        else:
            action = np.argmax(q_table[state]) # act on knowledge

            next_state, reward, done, info = env.step(action)
            next_max = np.max(q_table[next_state])

            old_value = q_table[state, action]
            new_value = (1 - a) * old_value + a * (reward + g * next_max)
            q_table[state, action] = new_value

            if reward == -10:
                penalties += 1

            state = next_state
            epochs += 1
    if i % 100 == 0:
        clear_output(wait=True)
        print(f"Episode: {i}")

print("Agent is trained!")
```

After training the agent (cab) for: + 100,000 tasks + about 1 minute on a Mac

we obtain the following Q-table:

```
np.set_printoptions(precision=1)
print(q_table[0:10])
```

The values are different from the initial values but how useful are they?

Act on Knowledge

Let's test the trained model (with an updated Q-table) for 100 tasks (episodes):

```
total_epochs, total_penalties = 0, 0
episodes = 100
trained_frames = []

for _ in range(episodes):
    state = env.reset()
    epochs, penalties, reward = 0, 0, 1

    done = False

    while not done:
        action = np.argmax(q_table[state])
        state, reward, done, info = env.step(action)

        trained_frames.append({
            'frame': env.render(mode='ansi'),
            'state': state,
            'action': action,
            'reward': reward
        })

        if reward == -10:
            penalties += 1

        epochs += 1

    total_penalties += penalties
    total_epochs += epochs
```

```
print(f"After {episodes} episodes:")
print(f"Average timesteps per episode: {total_epochs / episodes}")
print(f"Average penalties per episode: {total_penalties / episodes}")
```

Summary of the test, After 100 episodes: + Average timesteps per episode: 12.85 + Average penalties per episode: 0.0

```
print_frames(trained_frames, 0, 100, 0.5)
```

How did the agent learn?

Example, State 0:

- Cab location: (2,3)
- Pickup location B: 3
- Dropoff location G: 1

How does the cab learn to go **south** first to maximize its long-term reward?

```
env.encode(2,3,3,1)
```

Branches

At each state, there are 6 actions to perform, which can be visualized as:

```
import matplotlib.pyplot as plt
import networkx as nx

G = nx.balanced_tree(6, 1)
```

How did the agent learn?

```
names = {
    0: "State",
    1: "South",
    2: "North",
    3: "East",
    4: "West",
    5: "Pickup",
    6: "Dropoff"
}
G = nx.relabel_nodes(G, names)
```

```
nx.draw(G, with_labels=True, node_size=2000, node_color='0.85', alpha=1)
plt.show()
```

By taking an action, the agent (possibly) moves to another state and, from there, can select another action from the six.

This can be visualized as new branches from each one of the actions taken:

```
G = nx.balanced_tree(6, 2)
names = {
    0: "State",
    1: "South",
    2: "North",
    3: "East",
    4: "West",
    5: "Pickup",
    6: "Dropoff"
}
G = nx.relabel_nodes(G, names)
```

```
nx.draw(G, with_labels=False, node_size=300, node_color='0.85', alpha=1)
plt.show()
```

As the agent goes down the path, the number of potential choices and actions increase exponentially. How does the agent get rewarded for one of the most efficient path?

In particular, how does the agent maximize its **long-term award** by moving **south first** in the following step?

The path is equivalent to:

```
def show_graph():
    G = nx.path_graph(10)
    names = {
        0: "0",
        1: "1.South",
        2: "2.South",
        3: "3.Pickup",
        4: "4.East",
        5: "5.North",
        6: "6.North",
        7: "7.North",
        8: "8.North",
        9: "9.Dropoff"
    }

    positions = {
        "0": (30,60),
        "1.South": (30,40),
        "2.South": (30,20),
        "3.Pickup": (40,20),
        "4.East": (50,20),
        "5.North": (55,30),
```

How did the agent learn?

```

    "6.North": (60,40),
    "7.North": (65,50),
    "8.North": (70,60),
    "9.Dropoff": (75,70)
}

colors = ['0.75','0.75','0.75','lightblue','0.75','0.75','0.75','0.75','0.75','orange']

G = nx.relabel_nodes(G, names)
plt.figure(figsize=(12, 6))
plt.xlim(25,80)
plt.ylim(10,90)
nx.draw(G, with_labels=True, pos=positions, node_size=2200, node_color=colors, alpha=1)
plt.show()

```

```
show_graph()
```

For the sake of discussion, suppose values in Q-table are all the same 0.

By taking the action **south** at state 0: 1. The agent receive a penalty of -1 . 2. It will update its Q-value for **state** 0 with action **south**.

$$Q'(0, south) \leftarrow (1 - \beta)Q(0, south) + \beta(r_t + \gamma \max_a Q(s_{t+1}, a)) \quad (17.13)$$

$$= 0.9 \times 0 + 0.1 \times (-1 + 0.6 \times 0) \quad (17.14)$$

$$= -0.1 \quad (17.15)$$

- 0.1 \end{eqnarray}

Given: + Learning rate: $\beta = 0.1$ + Future award discount: $\gamma = 0.6$

So every *state* $\rightarrow$ *action* on the shown path will receive the same penalty of -1 and updated with a new value of -0.1 in the Q-table. One may

Reinforcement Learning Extension

think with this penalty along the path, the agent may not favor similar actions in the future.

```
show_graph()
```

However, when the cab reaches to the destination and selects an **dropoff** action at **state 8**:

1. It receives a reward of 20
2. It updates its Q-values for action **dropoff** at state 8.

$$\begin{aligned} Q'(8, dropoff) &\leftarrow (1 - \beta)Q(8, dropoff) + \beta(r_t + \gamma \max_a Q(s_{t+1}, a)) \\ &= 0.9 \times 0 + 0.1 \times (20 + 0.6 \times 0) \quad (17.17) \\ &= 2 \quad (17.18) \end{aligned}$$

2 \end{eqnarray}

With the new updated value of 2 for a **dropoff** at state 8, the agent will favor this specific action at this specific state in the future. But how about the previous actions at previous states leading to the dropoff? How do they get rewarded as well?

Remember in the Q-table updating policy, we have:

$$Q'(s_t, a_t) \leftarrow (1 - \beta)Q(s_t, a_t) + \beta(r_t + \gamma \max_a Q(s_{t+1}, a)) \quad (17.19)$$

For example, next time, when an action at state 7 is being updated:

$$Q'(7, North) \leftarrow \dots + \beta \gamma \max_a Q(s_{t+1}, a) \quad (17.20)$$

$$= \dots + 0.1 \times 0.6 \max_a Q(s_{t+1}, a) \quad (17.21)$$

How did the agent learn?

Because: 1. Action **north** at state 7 leads to state 8 2. The most rewarding action at state 8 is to **dropoff**, with Q-value 2

To update $Q'(7, north)$, the future reward becomes:

$$0.06 \max_a Q(s_{t+1}, a) = 0.06 \max_a Q(8, a) \quad (17.22)$$

$$= 0.06 \times Q(8, dropoff) \quad (17.23)$$

$$= 0.06 \times 2 \quad (17.24)$$

```
show_graph()
```

The main takeaway here is that a reward at one state will be “backpropagated” to an earlier *state, action* leading to that favorable state. With a sufficient number of iterations to train the agent, the rewards or penalties will be carried all the way back to even earlier state-actions.

In the end, the reward received for **dropoff** at state 8 will have an impact on going **south** at state 0, even though they are only remotely connected. Overall, the Q-learning policy will help improve the taxi cab’s behavior toward greater long-term rewards.

Comparison

Let’s compare the dummy cab (random model) and the trained model side by side:

```
compare_frames("Random Model", frames, "Trained Model", trained_frames, 0, 200, 0.5)
```

Conclusion

It is easy to tell that while the random model continues to lose and accumulate penalties, the trained model is much smarter in knowing how to move directly to pickup and dropoff locations to maximize its rewards.

Applications of Q-learning certainly go beyond the simplified taxi cab example here. A recent example is the AlphaGo that defeated the best human player. One can also imagine a similar reinforcement learning policy to be used by a smart floor vacuum to maximize the amount of dirt it can pick up to make your rooms clean.

References

- Wolfgang Ertel (2017). Introduction to Artificial Intelligence. Chapter 10 Reinforcement Learning. Springer.
- Miroslav Kubat (2017). An Introduction to Machine Learning. Chapter 17. Reinforcement learning. Springer.

References

- Aizawa, Akiko. 2003. “An Information-Theoretic Perspective of TF-Idf Measures.” *Information Processing and Management* (Tarrytown, NY, USA) 39 (1): 45–65. [https://doi.org/10.1016/S0306-4573\(02\)00021-3](https://doi.org/10.1016/S0306-4573(02)00021-3).
- Barabási, Albert-László, and Réka Albert. 1999. “Emergence of Scaling in Random Networks.” *Science* 286 (5439): 509–12. <https://doi.org/10.1126/science.286.5439.509>.
- Bekenstein, Jacob D. 2003. “Information in the HOLOGRAPHIC UNIVERSE.” *Scientific American* 289 (2): 58–65. <http://www.jstor.org/stable/26060403>.
- Belkin, Nicholas J., and W. Bruce Croft. 1992. “Information Filtering and Information Retrieval: Two Sides of the Same Coin?” *Commun. ACM* (New York, NY, USA) 35 (12): 29–38. <https://doi.org/10.1145/138859.138861>.
- Cover, T., and P. Hart. 1967. “Nearest Neighbor Pattern Classification.” *IEEE Transactions on Information Theory* 13 (1): 21–27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Endres, D. M., and J. E. Schindelin. 2006. “A New Metric for Probability Distributions.” *IEEE Transactions on Information Theory* (Piscataway, NJ, USA) 49 (7): 1858–60. <https://doi.org/10.1109/TIT.2003.813506>.
- Järvelin, Kalervo, and Jaana Kekäläinen. 2002. “Cumulated Gain-Based

References

- Evaluation of IR Techniques.” *ACM Trans. Inf. Syst.* (New York, NY, USA) 20 (4): 422–46. <https://doi.org/10.1145/582415.582418>.
- Kaggle. n.d. *The State of Data Science & Machine Learning 2017*. <https://www.kaggle.com/surveys/2017>.
- Ke, Weimao. 2015. “Information-Theoretic Term Weighting Schemes for Document Clustering and Classification.” *International Journal on Digital Libraries* 16 (2): 145–59. <https://doi.org/10.1007/s00799-014-0121-3>.
- Ke, Weimao. 2017. “Text Retrieval Based on Least Information Measurement.” *Proceedings of the ACM SIGIR International Conference on Theory of Information Retrieval* (New York, NY, USA), ICTIR ’17, 125–32. <https://doi.org/10.1145/3121050.3121075>.
- Kleinberg, Jon M. 1999. “Authoritative Sources in a Hyperlinked Environment.” *J. ACM* (New York, NY, USA) 46 (5): 604–32. <https://doi.org/10.1145/324133.324140>.
- Kullback, S., and R. A. Leibler. 1951. “On Information and Sufficiency.” *Annals of Mathematical Statistics* 22: 79–86. <https://doi.org/10.1214/aoms/1177729694>.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. “Gradient-Based Learning Applied to Document Recognition.” *Proceedings of the IEEE* 86 (11): 2278–324. <https://doi.org/10.1109/5.726791>.
- Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.
- Mooers, Calvin N. 1951. “Zatocoding Applied to Mechanical Organization of Knowledge.” *American Documentation* 2 (1): 20–32. <https://doi.org/10.1002/asi.5090020107>.

References

- Newman, Mark. 2010. *Networks: An Introduction*. Oxford University Press, Inc.
- Page, Lawrence, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report Nos. 1999-66. Stanford InfoLab; Stanford InfoLab. <http://ilpubs.stanford.edu:8090/422/>.
- Price, Derek De Solla. 1976. "A General Theory of Bibliometric and Other Cumulative Advantage Processes." *Journal of the American Society for Information Science* 27 (5): 292–306. <https://doi.org/10.1002/asi.4630270505>.
- RAPOPORT, ANATOL. 1953. "WHAT IS INFORMATION?" *ETC: A Review of General Semantics* 10 (4): 247–60. <http://www.jstor.org/stable/42581363>.
- Robertson, Stephen. 1977. "The Probability Ranking Principle in IR." *Journal of Documentation* 33 (December): 294–304. <https://doi.org/10.1108/eb026647>.
- Robertson, Stephen, and Karen Sparck-Jones. 1976. "Relevance Weighting of Search Terms." *Journal of the American Society for Information Science* 27 (May): 129–46. <https://doi.org/10.1002/asi.4630270302>.
- Robertson, Stephen, and Hugo Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval* (Hanover, MA, USA) 3 (4): 333–89. <https://doi.org/10.1561/15000000019>.
- Shannon, Claude E. 1948. "A Mathematical Theory of Communication." *Bell System Technical Journal* 27: 379–423 and 623–656.
- Yang, Yiming, and Jan O. Pedersen. 1997. "A Comparative Study on

References

- Feature Selection in Text Categorization.” *Proceedings of the Fourteenth International Conference on Machine Learning* (San Francisco, CA, USA), ICML '97, 412–20. <http://dl.acm.org/citation.cfm?id=645526.657137>.
- Zaragoza, H., N. Craswell, M. Taylor, S. Saria, and S. E. Robertson. 2004. “Microsoft Cambridge at TREC 2004: Web and HARD Track.” *The Thirteenth Text Retrieval Conference (TREC 2004)*, 1–7.
- Zhang, Aston, Zachary C. Lipton, Mu Li, and Alexander J. Smola. 2023. *Dive into Deep Learning*. Cambridge University Press.
- Zipf, George K. 1949. *Human Behaviour and the Principle of Least Effort*. Addison-Wesley.